

C++ and Robotics Interview Tutorial

A complete, self-contained study path for the six phases in `cpp-interview-tracker.html`. Every problem and concept in the tracker has a section here and a compilable, self-testing C++ file under `code/`. Read a chapter, type the solutions yourself, then mark the tracker.

The tutorial assumes you know Python and nothing about C++. Every idea starts with a picture or an everyday story, then a small example worked by hand, then the Python you would have written, then the C++ with every new symbol explained. Every problem starts with the full problem in plain words and a drawn example before any code appears.

Chapters

#	Chapter	Tracker phase	What you get
01	C++ basics	C++ basics	What C++ is, how to compile and run, types, functions, loops, vector, string, maps, pairs, 10 warm-up problems
02	Memory, objects and the STL	C++ interviewers test	Pointers and references drawn as boxes, const, classes and RAII, smart pointers, stack vs heap, the <code>size_t</code> trap, sort/heap/queue/map, 8 array/hash/interval problems
03	Grids and BFS/DFS	Grids and BFS/DFS	Grids, queues, stacks and recursion from zero, BFS and DFS with hand traces, backtracking, grid DP, 10 grid problems, A* built from scratch with tests
04	Graphs and Dijkstra	Graphs and Dijkstra	Graphs as cities and roads, BFS/DFS on graphs, topological sort, union-find, Dijkstra traced by hand, 8 graph problems
05	Heaps, trees, design	Heaps, trees, design	Heaps drawn as trees and arrays, top-k, sliding windows, binary trees, LRU cache as a bookshelf, 8 problems
06	Robotics-specific	Robotics-specific	Kalman filter as blending two guesses, PID as cruise control, rotations and frames with numbers, RANSAC, two premium robot problems, portfolio and CV guidance
07	Interview day	(all)	Read this last: how a live C++ round runs, how to talk while coding, a question bank with short answers, the last week

System design

The [system-design/](#) folder is a separate five-level path, from "what is a system" to full robotics design questions worked end to end. Start it after chapter 03, one level per sitting.

PDF versions

Every chapter and every system design level is also rendered as a PDF under `pdf/`, with the figures at fixed width so nothing wraps. `pdf/tutorial-book.pdf` and `pdf/system-design-book.pdf` are the combined books. To rebuild after editing any markdown file:

```
python3 pdf/build_pdfs.py
```

How to use the code

Every .cpp file under `code/<chapter>/` is standalone. It contains the solution and a `main()` with assert-based tests and prints `OK <name>` when it passes.

```
cd "code"
make           # build and run everything, including the A* GoogleTest suite
make 03        # only chapter 03
make astar     # only the A* project
```

To compile a single file by hand (this is the command you should be comfortable typing):

```
g++ -std=c++17 -Wall -Wextra -O2 01-cpp-basics/01_two_sum.cpp -o two_sum && ./two_sum
```

Eigen files (chapter 06) need the include path:

```
g++ -std=c++17 -Wall -Wextra -O2 -isystem /usr/include/eigen3 06-robotics/03_transforms_eigen.cpp -o t && ./t
```

For debugging, swap `-O2` for `-g -fsanitize=address,undefined`. The sanitizers catch out-of-bounds reads, use-after-free and signed overflow at run time. Interviewers like hearing that you use them.

The study method that actually works

1. **Read the concept part** of a chapter (Part A) slowly, and do every hand trace on paper yourself before reading on. Say each "In your own words" summary from memory.
2. **For each problem, close the tutorial and try it in C++ first**, timed, 25 minutes. Compile with `-Wall -Wextra`. Fix every warning.
3. **Only then read the solution.** Compare. If you needed it, mark the tracker "needed help" and redo the problem three days later from a blank file.
4. **Mark a concept done only when you can explain it to someone else** without notes. The "Check yourself" questions at the end of each chapter are the test.
5. **Type, do not copy-paste.** Muscle memory for `std::vector<std::vector<int>>`, `auto& [k, v]`, `std::priority_queue<..., std::greater<>>` is exactly what a live round tests.

Six-week plan (about 2 hours a day)

Week	Focus	Target on the tracker
1	Chapter 01, all of it	C++ basics 18/18
2	Chapter 02 concepts + problems	C++ interviewers test 19/19
3	Chapter 03 problems 1-10	Grids 10/11
4	A* project (3 days) + Chapter 04 traversal problems	Grids 11/11, Graphs 4/8
5	Chapter 04 shortest paths + Chapter 05	Graphs 8/8, Heaps 8/8
6	Chapter 06 builds + portfolio + Chapter 07 drills	Robotics 9/9, overall "Ready"

If you have less time: weeks 1, 2, 3 and the Dijkstra template from chapter 04 are the minimum that gets you through a typical UK robotics coding round. Chapter 06 is what gets you through the robotics-specific round and what makes your CV stand out.

Readiness standard (same as the tracker)

- All C++ basics and interviewer-tested concepts done.
- At least 80% of problems done, at least 60% of all problems solved unaided in 25 minutes.
- A* and Dijkstra written from scratch without looking anything up.

Directory layout

```
tutorial/          the chapters (markdown)
code/
  Makefile         build + run everything
  01-cpp-basics/   one .cpp per problem, plus 00_*.demo.cpp for concepts
  02-cpp-interviewers-test/
  03-grids-bfs-dfs/
    astar/         standalone CMake project with GoogleTest (portfolio piece)
  04-graphs-dijkstra/
  05-heaps-trees-design/
  06-robotics/     Kalman, PID, Eigen transforms, RANSAC, premium problems
```

01. C++ basics

This chapter takes you from "I have never compiled a C++ program" to "I can write a small C++ solution with vectors, strings, maps and pairs, build it, run it, and read the compiler's complaints". Every later chapter (grids, graphs, trees) assumes exactly this and nothing more. Robots are programmed in C++ because it is fast enough to run a control loop a thousand times a second on a small on-board computer; every ROS node, motor driver and planner you will touch is written in it.

Rough time: 4 days at about 2 hours a day. Day 1 is A0 to c3 (get something compiling, learn what a box is). Day 2 is c4 to c7 plus the demo file. Days 3 and 4 are the ten problems in Part B.

What you need before this chapter: Python. You should be comfortable with lists, dicts, sets, functions and for loops in Python. Nothing else. You also need a terminal (Linux, Mac, or WSL on Windows) where typing `g++ --version` prints a version number.

Part A — The language, from zero

A0. What C++ is and why it feels different

Python is a language you talk to. You type a line, the Python interpreter reads it and does it straight away. If line 40 has a typo, you find out when the program reaches line 40.

C++ is a language you write a letter in. You write the whole program first. Then you hand the file to a translator, the *compiler* (ours is called `g++`). The compiler reads every line, checks it, and produces a brand new file in the computer's own language: an *executable*. Then you run that executable. Two steps, every time: compile, then run.

hello.cpp	----- g++ ----->	hello	----- ./hello ----->	Hello, robot
(text you typed)	(the compiler: reads, checks, translates)	(an executable: a real program the machine runs)		(what you see)

If line 40 has a typo, the compiler refuses to produce the executable and tells you the line number. Nothing runs until the whole file is right. That is annoying for the first week and a relief for the rest of your life.

C++ is typed. In Python you write `x = 5` and later `x = "five"` and nobody minds. In C++ every variable is a *box with a fixed shape*, and you say the shape when you make the box:

```
int x = 5;
```

reads as "make a box shaped for a whole number (`int`), call it `x`, put 5 in it". You cannot put "five" in that box later; the compiler stops you. This feels like paperwork at first. It is also why the compiler can catch so many mistakes before anything runs, and why the finished program is fast: the computer never has to stop and ask "what is this thing?".

The smallest useful program. Five lines that do something:

```
#include <iostream>

int main() {
    std::cout << "Hello, robot\n";
    return 0;
}
```

Line by line:

- `#include <iostream>` : "bring in the toolbox for input and output". It is Python's `import`. `iostream` is what gives you `std::cout`.
- `int main() {` : every C++ program starts running at a function called `main`. The `int` says it hands back a whole number when it finishes. The `{` opens the body.
- `std::cout << "Hello, robot\n";` : `std::cout` is the screen. `<<` means "send this to". `"\n"` is a newline. The `;` ends the statement. Python ends a statement at the end of the line; C++ ends it at the semicolon, so you can split one statement across lines.
- `return 0;` : "finished, no error". Returning 0 from `main` means success.
- `}` closes the body.

Three habits from day one.

1. **std:: in front of library things.** Everything from the standard library lives in a *namespace* called `std`, and you name it in full: `std::cout`, `std::vector`, `std::string`. Python says `math.sqrt`; C++ says `std::sqrt`. Some tutorials add `using namespace std;` at the top to skip this. Do not; it causes name clashes later and every codebase you will work in forbids it.
2. **Braces, not indentation.** Python groups lines by indenting them. C++ groups them with `{ }` and ignores indentation completely. Indent anyway, for the humans reading.
3. **Semicolons.** Every statement ends with `;`. Forgetting one is the first error you will ever get from a compiler, and the next section shows you what it looks like.

In your own words: C++ is compiled: I write the whole file, `g++` checks and translates it into an executable, and then I run that. Every variable is a box with a fixed type that I declare up front. Library names start with `std::`, blocks use braces, statements end with a semicolon.

c8. Compile and run locally with g++ (not only in the browser)

You need to be able to run things before you can learn anything else, so this comes first.

Type the hello program above into a file called `hello.cpp`. In a terminal, in the same folder, run these two lines:

```
g++ -std=c++17 -Wall -Wextra -O2 hello.cpp -o hello
./hello
```

The first line compiles. If it succeeds it prints nothing and creates a file called `hello`. The second line runs that file and prints `Hello, robot`. If the compile fails, no `hello` file is made and `g++` prints why.

What each flag means, one line each:

Flag	Meaning
<code>-std=c++17</code>	Use the 2017 version of the language. Needed for <code>auto</code> [a, b] and other things in this chapter.
<code>-Wall</code>	Turn on the common warnings (unused variable, missing return, signed/unsigned mix).
<code>-Wextra</code>	Turn on a few more. Treat every warning as a bug.
<code>-O2</code>	Optimise. Makes the program fast; without it, timing means nothing.
<code>hello.cpp</code>	The file to compile.
<code>-o hello</code>	Name the output file <code>hello</code> . Without this you get a file called <code>a.out</code> .

For the files in this repo the same command is used, with a throwaway output name:

```
cd "code/01-cpp-basics"
g++ -std=c++17 -Wall -Wextra -O2 01_two_sum.cpp -o /tmp/x && /tmp/x
```

&& means "and if that succeeded, then". So the program only runs if it compiled. Each file prints OK <filename> when all its tests pass, or stops with an assertion failure telling you which line was wrong.

Reading an error message. Break the hello program on purpose: add a variable and forget the semicolon after it.

```
#include <iostream>

int main() {
    int count = 3
    std::cout << count << "\n";
    return 0;
}
```

Save it as `bad.cpp` and compile. `g++` prints:

```
bad.cpp: In function 'int main()':
bad.cpp:5:5: error: expected ',' or ';' before 'std'
   5 |     std::cout << count << "\n";
     |         ^~~
```

How to read it:

- `bad.cpp:5:5` is *file : line : column*. The compiler is pointing at line 5, column 5.
- `error:` is the kind. An `error` means no executable was produced. A `warning` means it was produced but something looks wrong.
- `expected ';' before 'std'` is the message. It found `std` where it expected a `;`.
- The next two lines show you the source line and a `^` under the exact spot.

Notice the compiler points at line 5, but the mistake is at the *end of line 4*: that is where the semicolon belongs. The compiler only knows something is wrong when it reaches the next thing. Rule: look at the line it names, then look at the line above. The fix is `int count = 3;`.

The three messages you will see most often in this chapter:

Message	What it means	Fix
error: 'foo' was not declared in this scope	You used a name the compiler has never seen.	A typo, a missing <code>#include</code> , or you used a variable before making it.
error: 'class std::vector<int>' has no member named 'append'	You called a method that this type does not have.	Look up the C++ name (a vector's is <code>push_back</code>).
warning: comparison of integer expressions of different signedness	You compared an <code>int</code> with something like <code>v.size()</code> .	See the skip paragraph in c1; the habit is <code>static_cast<int>(v.size())</code> .

Two more rules. When there are many errors, fix the *first* one and recompile; later errors are often caused by the first. And a warning is a bug you have not met yet: fix them all.

Feeding input. The files in this repo test themselves with `assert`, so they read no input. When you want a program that reads numbers, this is the pattern:

```
#include <iostream>
#include <vector>

int main() {
    int n;
    std::cin >> n;                // read one number
    std::vector<int> v(n);        // n boxes
    for (int& x : v) std::cin >> x; // fill each box from the input
    int total = 0;
    for (int x : v) total += x;
    std::cout << "sum = " << total << "\n";
}
```

```
    return 0;
}
```

- `std::cin >> n`; reads the next whitespace-separated token from the input into `n`. It is Python's `n = int(input())`, but it does not care about line breaks.
- `std::vector<int> v(n)`; makes a list of `n` zeros (section c4).
- `for (int& x : v)` walks through the list. The `&` matters: it makes `x` be each box, so `std::cin >> x` fills the real box. Without the `&`, `x` would be a copy and `v` would stay all zeros (section c2 explains `&`).

Save as `input.cpp`, compile as before, and give it input either by typing it, by piping it, or from a file:

```
echo "3 10 20 30" | ./input
./input < input.txt
```

Both print `sum = 60`.

A minimal VS Code setup. Put this in `.vscode/tasks.json` in the project folder. Then `Ctrl+Shift+B` compiles and runs whichever `.cpp` file is open, and errors become clickable in the Problems panel.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build & run current file",
      "type": "shell",
      "command": "g++ -std=c++17 -Wall -Wextra -O2 \"${file}\" -o /tmp/x && /tmp/x",
      "group": { "kind": "build", "isDefault": true },
      "presentation": { "reveal": "always", "clear": true },
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

The Python to C++ cheat table. You will meet each of these properly in c1 to c7. This table is here so you have one place to look things up later. How to read it: the left column is the thing you would reach for in Python; the middle is what you type in C++ instead; the right column is the `#include` line you must add at the top of the file to use it. Do not memorise it now. Come back to it every time you think "how do I say X in C++".

Python	C++ (C++17)	Header
<code>list</code>	<code>std::vector<T></code>	<code><vector></code>
<code>dict</code>	<code>std::unordered_map<K, V></code> (<code>std::map</code> if you need sorted keys)	<code><unordered_map></code> / <code><map></code>
<code>set</code>	<code>std::unordered_set<T></code> (<code>std::set</code> if sorted)	<code><unordered_set></code> / <code><set></code>
<code>tuple</code>	<code>std::pair<A, B></code> / <code>std::tuple<...></code>	<code><utility></code> / <code><tuple></code>
<code>str</code>	<code>std::string</code>	<code><string></code>
<code>collections.deque</code>	<code>std::deque<T></code> ; as a queue use <code>std::queue<T></code>	<code><deque></code> / <code><queue></code>
<code>heapq (min-heap)</code>	<code>std::priority_queue<T, std::vector<T>, std::greater<T>></code> (max-heap by default)	<code><queue></code>
<code>list</code> used as a stack	<code>std::stack<T></code> or <code>std::vector<T></code> with <code>back()</code> / <code>pop_back()</code>	<code><stack></code>
<code>None</code>	<code>std::optional<T></code> , or a sentinel like <code>-1</code> / <code>nullptr</code>	<code><optional></code>
<code>len(x)</code>	<code>x.size()</code> (returns an unsigned number, see c1)	(none)
<code>x in container</code>	<code>count(x)</code> for sets and maps; <code>std::find</code> for vectors	<code><algorithm></code>

Python	C++ (C++17)	Header
<code>max(a, b), min(a, b)</code>	<code>std::max(a, b), std::min(a, b)</code> (both arguments must have the same type)	<code><algorithm></code>
<code>max(xs)</code>	<code>*std::max_element(xs.begin(), xs.end())</code>	<code><algorithm></code>
<code>sum(xs)</code>	<code>std::accumulate(xs.begin(), xs.end(), 0LL)</code> (0LL so it sums in 64 bits)	<code><numeric></code>
<code>sorted(xs) / xs.sort()</code>	<code>std::sort(xs.begin(), xs.end())</code>	<code><algorithm></code>
<code>float('inf')</code>	<code>INT_MAX / std::numeric_limits<int>::max() / 1e18</code> for long long	<code><limits> / <limits></code>
<code>abs(x)</code>	<code>std::abs(x)</code>	<code><cstdlib> / <cmath></code>
<code>x // y, x % y</code>	<code>x / y, x % y</code> (rounds toward zero, see c1)	(none)
<code>lambda x: x*2</code>	<code>[](int x) { return x * 2; }; [&]</code> to see local variables	(none)
<code>print(x)</code>	<code>std::cout << x << '\n';</code>	<code><iostream></code>
<code>assert cond</code>	<code>assert(cond);</code>	<code><cassert></code>
<code>def f(xs: list) -> int:</code>	<code>int f(const std::vector<int>& xs)</code>	(none)

(You can skip this on a first read.) While hunting a bug, compile with `g++ -std=c++17 -Wall -Wextra -g -fsanitize=address,undefined file.cpp -o /tmp/x`. `-g` adds line numbers to crash reports; `-fsanitize=address` makes reading past the end of a vector stop the program with a message instead of silently reading garbage; `-fsanitize=undefined` does the same for integer overflow. Drop them (use `-O2`) when you want to measure speed.

In your own words: I compile with `g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o name` and run with `./name`. An error message is `file:line:column` and a sentence; I look at that line and the one above, fix the first error, and recompile. Warnings are bugs.

c1. Variables, types, int/long/double, casting

A type is the shape of a box. In Python a variable is a name tag tied to some object, and you can move the tag to anything. In C++ a variable *is* a box. The box has a fixed shape (its type), chosen when you make it, and only things of that shape fit.

Python:

```
count -----> 5
(a tag tied to
a thing; can be
retied to "five")
```

C++:

```
+-----+
|    5    | count
+-----+
a box shaped for whole numbers (int).
It stays that shape for its whole life.
```

The shapes you need, with their ranges in plain words:

Type	What fits in it	How big	Use it for
<code>int</code>	whole numbers	about minus 2 billion to plus 2 billion (exactly 2,147,483,647 at the top)	counts, indices, most answers
<code>long long</code>	whole numbers, bigger box	about plus or minus 9 quintillion (9 followed by 18 zeros)	sums and products that could pass 2 billion
<code>double</code>	numbers with a decimal point	enormous range, about 15 correct digits	distances, angles, anything Python calls <code>float</code>

Type	What fits in it	How big	Use it for
char	exactly one character	'a', '7', ' '; secretly a small whole number (see c5)	letters inside a string
bool	true or false	just those two	flags, yes/no answers

Making a box looks like Python with the type written in front:

```
int count = 0;           // Python: count = 0
long long total = 0;     // Python: total = 0    (Python ints just grow; C++ ones do not)
double speed = 1.5;      // Python: speed = 1.5
char grade = 'A';        // single quotes for one character; double quotes are for strings
bool done = false;       // lowercase true / false, not True / False
```

Surprise 1: dividing whole numbers gives a whole number. In Python `7 / 2` is 3.5. In C++, if both sides of `/` are `int`, the result is an `int`, and the decimals are thrown away. Python's `//` rounds *down*; C++ rounds *toward zero*. The two agree for positive numbers and disagree for negatives.

```
assert(7 / 2 == 3);       // Python: 7 / 2 == 3.5, 7 // 2 == 3
assert(-7 / 2 == -3);    // Python: -7 // 2 == -4 (down); C++ goes toward zero
assert(7 % 2 == 1);      // remainder, same as Python for positives
assert(-7 % 2 == -1);    // Python: -7 % 2 == 1; C++ keeps the sign of the left number
assert(7.0 / 2 == 3.5);  // one side is a double, so the division keeps decimals
```

The fix when you want 3.5: make one side a `double`, either by writing `7.0` or by casting (below).

Surprise 2: the box can overflow. An `int` holds up to 2,147,483,647. Think of the odometer in a car with only so many digits: add one more mile and it rolls over.

```
2,147,483,647 + 1 -> rolls over to -2,147,483,648    (in an int box)
100,000 * 100,000 = 10,000,000,000    which does not fit in an int at all
```

Python never does this because its integers grow as needed. In C++ the rule is: if a number *might* pass 2 billion, use `long long`, and widen *before* the arithmetic, not after:

```
int a = 100000, b = 100000;
long long product = static_cast<long long>(a) * b; // a is widened first, so the multiply is done in the big box
assert(product == 10000000000LL);
```

Writing `long long bad = a * b;` instead does the multiply in `int` (overflow happens there) and only then copies the wrong number into the big box. The `LL` on `10000000000LL` tells the compiler the number itself is a `long long`.

Casting: telling the compiler to change the box shape. `static_cast<T>(x)` reads as "make me a `T` out of `x`". You just saw it widen an `int` to `long long`. Two more uses:

```
double d = 3.99;
int whole = static_cast<int>(d);           // 3: cuts off the decimals, it does not round
assert(whole == 3);
double half = static_cast<double>(7) / 2;  // 3.5: the 7 became a double before dividing
assert(half == 3.5);
```

C++ also converts on its own in some places, silently: `int x = 3.99;` compiles and stores 3, `bool b = 42;` stores `true`, and `true + true` is 2. `-Wall -Wextra` do not warn about most of these, so read your types.

(You can skip this on a first read.) `v.size()` on a vector does not return an `int`. It returns `size_t`, an *unsigned* type that cannot be negative. Two things follow. Comparing `int i` with `v.size()` gives the "different signedness" warning from `c8`; the habit is to convert once: `int n = static_cast<int>(v.size());` and compare with `n`. And `v.size() - 1` on an empty vector is not `-1`; the unsigned box rolls over the other

way to 18,446,744,073,709,551,615. Cast to `int` before subtracting: `static_cast<int>(v.size()) - 1`. Also, overflow of a signed `int` is officially "undefined behaviour": the compiler is allowed to assume it never happens, so the rolled-over odometer above is what usually happens, not what is guaranteed.

Try it: in `00_basics_demo.cpp`, change `assert(-7 / 2 == -3);` to `assert(-7 / 2 == -4);`. Predict what the program prints before you compile and run it.

In your own words: A type is the fixed shape of a variable's box. `int` holds up to about 2 billion, `long long` up to about 9 quintillion, `double` holds decimals. Dividing two ints throws away the decimals and rounds toward zero. If a result might pass 2 billion I use `long long` and widen with `static_cast` before the arithmetic.

c2. Functions, pass by value vs reference (`int&`)

The shape of a function. Python:

```
def add(a, b):
    return a + b
```

C++:

```
int add(int a, int b) {
    return a + b;
}
```

Read it as: "a function that hands back an `int`, called `add`, taking two `int` boxes `a` and `b`". The return type comes first. Every parameter has a type. A function that hands back nothing says `void` where the `int` is. Functions must be written *above* the place they are used (or declared above; for now, just put helpers above `main`).

Two boxes. When you call `add(2, 3)`, what exactly does the function get? In C++ there are two answers, and you choose with one character.

By value: `void f(int x)`

```
caller's box      f's box
+-----+ copy  +-----+
|  5  | -----> |  5  |
+-----+      +-----+
   n              x
```

f does `x += 1`: `n` is still 5

By reference: `void f(int& x)`

```
caller's box
+-----+
|  5  | <--- x is a second name
+-----+    tied to this same box
   n      x
```

f does `x += 1`: `n` is now 6

By value (`int x`): the function gets a **copy** in a brand new box. Whatever it does to `x`, the caller's `n` never changes. This is the default.

By reference (`int& x`): the `&` means "do not copy; let `x` be another name for the caller's box". Changes to `x` are changes to `n`.

```
void addOneCopy(int x) { x += 1; }    // works on a private copy
void addOneRef(int& x) { x += 1; }    // works on the caller's own box
```

```
int n = 5;
addOneCopy(n);
assert(n == 5);    // the copy was changed, n was not
addOneRef(n);
assert(n == 6);    // x was another name for n
```

const& means "look but do not touch". Copying an `int` is nothing. Copying a vector of a million numbers is a million copies, every call. So for anything bigger than a number, you pass by reference to avoid the copy, and you add `const` to promise you will not change it. The compiler enforces the promise: try to modify a `const&` parameter and you get an error.

```
int sum(const std::vector<int>& v) {    // no copy is made; v cannot be changed inside
    int s = 0;
    for (int x : v) s += x;
    return s;
}
```

The three forms, side by side:

You write	Copied?	Function can change the caller's data?	Use for
<code>int x / char c / double d</code>	yes, but it is tiny	no	numbers, characters, bools
<code>std::vector<int>& v</code>	no	yes	when changing the caller's data is the point (sort it in place, fill it in)
<code>const std::vector<int>& v</code>	no	no	anything big that you only read: vectors, strings, maps. This is the default for big things

The Python intuition, and where it breaks. In Python, a function that gets a list can change the caller's list, because both names point at the same object:

```
def f(xs):
    xs.append(99)

a = [1, 2, 3]
f(a)
print(len(a))    # 4: the caller's list changed
```

In C++ that is only true if you write the `&`:

```
void byValue(std::vector<int> v) { v.push_back(99); }    // v is a full copy; the caller sees nothing
void byRef(std::vector<int>& v) { v.push_back(99); }    // v is the caller's vector
```

```
std::vector<int> a = {1, 2, 3};
byValue(a);
assert(a.size() == 3);    // unchanged, unlike Python
byRef(a);
assert(a.size() == 4);    // changed, like Python
```

It breaks the other way too. In Python, a function can never change the caller's *integer*: `def g(x): x += 1` leaves the caller's `x` alone. In C++, `void g(int& x) { x += 1; }` does change it. So in C++ you never guess from the type of the thing; you read the parameter declaration. `&` means shared, no `&` means copy.

(You can skip this on a first read.) Returning a vector *from* a function by value (`std::vector<int> makeSquares(int n)`) is fine and does not copy in practice: the compiler hands the caller the same buffer. What you must never do is return a reference to a local variable; the local is destroyed when the function ends and the reference points at nothing.

Try it: in `00_basics_demo.cpp`, delete the `&` in `void byRef(std::vector<int>& v)`. Predict which `assert` fails before you compile and run.

In your own words: A parameter without & is a copy in a new box; the caller never sees changes. A parameter with & is a second name for the caller's box. For anything bigger than a number I write `const T&` so there is no copy and I cannot change it, and plain `T&` only when changing the caller's data is the whole point.

c3. if/for/while, range-for, auto

Control flow is the Python you know with braces and parentheses. Side by side:

```
if x > 0 and x < 10:
    print("small")
elif x == 0:
    print("zero")
else:
    print("other")
```

```
if (x > 0 && x < 10) {
    std::cout << "small\n";
} else if (x == 0) {
    std::cout << "zero\n";
} else {
    std::cout << "other\n";
}
```

The differences: the condition must be in (); and is &&, or is ||, not is !; the body is in { }; and there is no chained comparison, so Python's `0 < x < 10` must be written `x > 0 && x < 10`. `while` is the same story:

```
int lo = 0, hi = 10;
while (lo < hi) {
    ++lo;
}
```

The three kinds of for loop.

1. **Counting.** Python `for i in range(n):` is

```
cpp for (int i = 0; i < n; ++i) { // use i }
```

Three parts separated by `;`: *start* (`int i = 0`), *keep going while* (`i < n`), *step after each round* (`++i`, which is `i += 1`). To count down: `for (int i = n - 1; i >= 0; --i)`.

1. **Range-for by copy.** Python `for x in v:` is

```
cpp for (int x : v) { // x is a copy of each element in turn }
```

Read `:` as "in". Because `x` is a copy, changing `x` does not change `v`.

1. **Range-for by reference.** Add & and `x` becomes each element itself:

```
cpp for (int& x : v) { x *= 2; // this changes v }
```

`break` and `continue` work exactly as in Python.

auto means "you work out the type". Instead of writing the type of a box, you can write `auto` and the compiler looks at the right-hand side and picks the shape:

```
auto n = v.size();           // n gets whatever type size() returns
auto it = seen.find(5);      // it gets the iterator type, which is long to spell out
```

Use it when the type is long or boring to type. Do not use it when the type carries meaning you would otherwise state (`int count = 0;` is clearer than `auto count = 0;`).

auto versus auto&: copy versus alias. This is the trap that bites everyone once. In a range-for, auto gives you a *copy* of each element, auto& gives you the element itself, exactly like `int` versus `int&` above. With a grid (a vector of rows) it matters a lot:

grid, the real rows <pre> +---+---+---+ 0 0 0 row 0 +---+---+---+ 0 0 0 row 1 +---+---+---+ </pre>	for (auto row : grid) row is a COPY of the row: <pre> +---+---+---+ 0 0 0 (new boxes) +---+---+---+ </pre> row[0] = 1 changes the copy, then the copy is thrown away.	for (auto& row : grid) row is another name for the real row. No new boxes. row[0] = 1 changes grid.
---	--	--

```

std::vector<std::vector<int>> grid(2, std::vector<int>(3, 0));
for (auto row : grid) row[0] = 1;      // each row is copied; the copies are changed and thrown away
assert(grid[0][0] == 0);
for (auto& row : grid) row[0] = 1;     // row IS the grid's row
assert(grid[0][0] == 1);

```

The rule: `const auto&` when you only read, `auto&` when you change, plain `auto` only for small things like `int` and `char`. Written out: `for (const auto& row : grid)` to read rows.

The one-line if. Python's `-1 if x < 0 else 1` is `(x < 0) ? -1 : 1`. Read `?` as "then" and `:` as "else".

(You can skip this on a first read.) There is no `for ... else` in C++; use a `bool found` flag or put the loop in a function and `return` from inside it. There is also no way to `break` out of two nested loops at once; again, put them in a function and `return`.

In your own words: `if`, `while` and the counting `for` are Python with parentheses and braces. `for (int x : v)` walks a container by copy, `for (int& x : v)` by reference. `auto` asks the compiler to pick the type, and `auto&` in a loop means "the real element, not a copy".

c4. `std::vector` — `push_back`, `size`, indexing, 2D vectors

A `std::vector` is a Python list that holds one type. It is a row of boxes, all the same shape, numbered from 0, that can grow at the end.

```

std::vector<int> v = {4, 8, 15};

index:      0      1      2
v :         +-----+-----+-----+
           |  4  |  8  | 15  |         v.size() == 3
           +-----+-----+-----+

v.push_back(16);

v :         +-----+-----+-----+-----+
           |  4  |  8  | 15  | 16  |         v.size() == 4
           +-----+-----+-----+-----+

```

`std::vector<int>` reads as "vector of int". The thing in the angle brackets is the shape of every box. `std::vector<std::string>` holds strings, `std::vector<bool>` holds flags. You need `#include <vector>` at the top of the file.

Every common operation, side by side with Python:

Python	C++	Note
<code>v = []</code>	<code>std::vector<int> v;</code>	empty
<code>v = [4, 8, 15]</code>	<code>std::vector<int> v = {4, 8, 15};</code>	

Python	C++	Note
<code>v = [0] * n</code>	<code>std::vector<int> v(n, 0);</code>	n boxes, each holding 0. Round brackets.
<code>v.append(x)</code>	<code>v.push_back(x);</code>	
<code>v.pop()</code>	<code>int x = v.back(); v.pop_back();</code>	two steps: look at the last one, then remove it. <code>pop_back()</code> returns nothing
<code>v[-1]</code>	<code>v.back()</code>	no negative indexing in C++
<code>v[0]</code>	<code>v.front()</code> or <code>v[0]</code>	
<code>len(v)</code>	<code>v.size()</code>	unsigned, see c1
<code>not v</code>	<code>v.empty()</code>	
<code>v[i]</code>	<code>v[i]</code>	no bounds check: <code>v[100]</code> on a 3-element vector does not raise, it reads garbage or crashes
<code>v[i] (checked)</code>	<code>v.at(i)</code>	throws an exception if out of range; use while debugging
<code>v.sort()</code>	<code>std::sort(v.begin(), v.end());</code>	needs <code>#include <algorithm></code>
<code>v[::-1] (in place)</code>	<code>std::reverse(v.begin(), v.end());</code>	<code><algorithm></code>
<code>x in v</code>	<code>std::find(v.begin(), v.end(), x) != v.end()</code>	<code><algorithm></code> ; a linear scan, like Python
<code>v[1:3]</code>	<code>std::vector<int>(v.begin() + 1, v.begin() + 3)</code>	a copy of positions 1 and 2
<code>v.clear()</code>	<code>v.clear();</code>	

`v.begin()` and `v.end()` are "the start" and "one past the last"; the algorithm functions take that pair to mean "the whole vector". Just copy the pattern.

```
std::vector<int> v;
v.push_back(4);
v.push_back(8);
v.push_back(15);
assert(v.size() == 3);
assert(v[0] == 4 && v.back() == 15);
int last = v.back();
v.pop_back();
assert(last == 15 && v.size() == 2);
std::vector<int> zeros(5, 0);           // five boxes of 0
assert(zeros.size() == 5 && zeros[4] == 0);
std::sort(v.begin(), v.end());
assert(v == std::vector<int>({4, 8})); // you can compare whole vectors with ==
```

The bracket trap. `std::vector<int> a(5, 0)` with round brackets means "5 boxes of 0". `std::vector<int> b{5, 0}` with curly brackets means "two boxes, holding 5 and 0". Use round brackets for (count, value) and curly for a literal list.

A 2D vector is a vector of rows. A grid with 3 rows and 4 columns is a vector holding 3 vectors, each holding 4 ints:

```
std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));
```

	c=0	c=1	c=2	c=3	
	+-----+	+-----+	+-----+	+-----+	
r=0	0	0	0	0	<- grid[0], a vector<int> of size 4
	+-----+	+-----+	+-----+	+-----+	
r=1	0	0	0	0	<- grid[1]

```

r=2      +-----+-----+-----+-----+
         | 0  | 0  | 0  | 0  |  <- grid[2]
         +-----+-----+-----+-----+

grid[1][2] = 7;   row 1, then column 2

```

That construction line, word by word:

- `std::vector<std::vector<int>>` : the type. "A vector whose boxes are each a vector of int."
- `grid` : the name.
- `(3, ...)` : make 3 boxes (3 rows), and fill each one with a copy of ...
- `std::vector<int>(4, 0)` : ... a vector of 4 ints, each 0. That is one row.

So it is Python's `grid = [[0] * 4 for _ in range(3)]`. Getting the size back:

```

std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));
grid[1][2] = 7;
int rows = static_cast<int>(grid.size());           // 3
int cols = static_cast<int>(grid[0].size());        // 4, the length of the first row
assert(rows == 3 && cols == 4 && grid[1][2] == 7);

```

`grid[r][c]` means row `r`, then column `c`, exactly as Python's list of lists. Row first.

(You can skip this on a first read.) `v.reserve(1000)` tells the vector to get room for 1000 elements now, so the next 1000 `push_backs` do not have to grow the storage; `v.size()` is still 0 after it. Each row of a 2D vector is its own separate block of memory, which is fine for everything in this tutorial.

Try it: in `00_basics_demo.cpp`, change `v.at(100)` to `v[100]` inside the `try` block. Predict what happens to `assert(throw)`. Then run it: either the program crashes, or `v[100]` quietly reads garbage, nothing is thrown, and the `assert` fails. `[]` does not check.

In your own words: `std::vector<T>` is a list where every element has type `T`. `push_back` appends, `size()` is the length, `[]` indexes from 0 with no bounds check. `std::vector<int> v(n, 0)` is `n` zeros, and `std::vector<std::vector<int>> g(rows, std::vector<int>(cols, 0))` is a grid of zeros indexed `g[r][c]`.

c5. `std::string` — `substr`, `find`, `compare`, `to_string`/`stoi`

A `std::string` is a vector of characters with some extras. Unlike a Python string, you can change it in place: `s[0] = 'H'` works, `s += "!"` appends. You need `#include <string>`.

```

std::string s = "hello";

index:    0      1      2      3      4
s :       +-----+-----+-----+-----+
         | 'h' | 'e' | 'l' | 'l' | 'o' |      s.size() == 5
         +-----+-----+-----+-----+

```

Python	C++	Note
<code>s = "hello"</code>	<code>std::string s = "hello";</code>	double quotes for strings, single for one <code>char</code>
<code>len(s)</code>	<code>s.size()</code>	
<code>s[i]</code>	<code>s[i]</code>	gives a <code>char</code> ; no negative indexing
<code>s + t</code>	<code>s + t</code>	
<code>s += "x"</code>	<code>s += "x";</code> or <code>s.push_back('x');</code>	cheap, unlike Python where strings are immutable
<code>s[2:5]</code>	<code>s.substr(2, 3)</code>	(start, length) , not (start, end): 3 characters starting at 2
<code>s[6:]</code>	<code>s.substr(6)</code>	from 6 to the end

Python	C++	Note
<code>s.find("lo")</code>	<code>s.find("lo")</code>	returns the index, or <code>std::string::npos</code> if absent (Python returns -1)
<code>"lo" in s</code>	<code>s.find("lo") != std::string::npos</code>	
<code>s == t,</code> <code>s < t</code>	<code>s == t, s < t</code>	dictionary order, same as Python
<code>str(42)</code>	<code>std::to_string(42)</code>	gives "42"
<code>int("42")</code>	<code>std::stoi("42")</code>	"string to int"; <code>std::stod</code> for a double, <code>std::stoll</code> for long long
<code>s[::-1]</code>	<code>std::string r(s.rbegin(), s.rend());</code>	a reversed copy; or <code>std::reverse(s.begin(), s.end())</code> in place
<code>for ch in s:</code>	<code>for (char ch : s)</code>	
<code>s.split()</code>	no one-liner; see the skip paragraph	

```
std::string s = "hello world";
assert(s.size() == 11);
assert(s.substr(6) == "world");
assert(s.substr(0, 5) == "hello");           // position 0, length 5
assert(s.find("wor") == 6);
assert(s.find('z') == std::string::npos);    // not found
assert(std::to_string(42) == "42");
assert(std::stoi("42") + 1 == 43);
s += "!";
assert(s.back() == '!');
std::string r(s.rbegin(), s.rend());
assert(r == "!dlrow olleh");
```

The '0' trick: a char is secretly a number. Every character is stored as a number, its ASCII code. The digits and the lowercase letters are stored in order:

character:	'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'
its number:	48	49	50	51	52	53	54	55	56	57
character:	'a'	'b'	'c'	'd'	...	'z'				
its number:	97	98	99	100	...	122				

So subtracting characters subtracts their numbers, and that turns a digit character into the digit, or a letter into its position in the alphabet:

```
'7' - '0' = 55 - 48 = 7           the digit seven, as a number
'c' - 'a' = 99 - 97 = 2           c is the 3rd letter, position 2 from 0
'a' + 2   = 97 + 2 = 99   which is 'c' (going back the other way)
```

```
char c = '7';
assert(c - '0' == 7);
assert('c' - 'a' == 2);
assert(static_cast<char>('a' + 2) == 'c');    // the addition gives an int; cast it back to char
int count[26] = {};                          // 26 boxes of 0, one per letter; the {} zeroes them
for (char ch : std::string("banana")) ++count[ch - 'a'];
assert(count['a' - 'a'] == 3 && count['n' - 'a'] == 2);
```

That last pattern, "26 boxes indexed by `ch - 'a'`", is how you count letters without a dictionary. You will use it in Valid Anagram below. Python's `ord(ch) - ord('a')` is the same idea with the numbers made visible.

The find trap. `s.find('z')` returns `std::string::npos` when the character is absent. `npos` is a huge unsigned number, not `-1`, so `if (s.find('z') >= 0)` is always true. Always compare with `npos`.

(You can skip this on a first read.) Splitting on whitespace uses a string stream: `std::istringstream in(s); std::string word; while (in >> word) words.push_back(word);` with `#include <sstream>`. To split on a specific character use `while (std::getline(in, word, ','))`. `std::isdigit(c)`, `std::isalpha(c)` and `std::tolower(c)` live in `<cctype>`.

In your own words: `std::string` is a changeable vector of `char`. `substr` takes a start and a *length*. `find` returns `npos` when absent, so I compare with `npos`, not `-1`. A `char` is a small number, so `ch - '0'` is the digit and `ch - 'a'` is the letter's position.

c6. `std::unordered_map` and `unordered_set`

An `unordered_map` is a Python dict. An `unordered_set` is a Python set. Both are hash tables: looking something up takes the same short time no matter how many things are inside. Both need to know the type of what they hold. Headers: `<unordered_map>` and `<unordered_set>`.

```
std::unordered_map<std::string, int> ages;      key: string,  value: int
```

```

      key      ->  value
+-----+ +-----+
| "ann" | -> | 30 |
+-----+ +-----+
| "bob" | -> | 25 |
+-----+ +-----+
```

Python	C++	Note
<code>d = {}</code>	<code>std::unordered_map<std::string, int> d;</code>	
<code>d[k] = v</code>	<code>d[k] = v;</code>	
<code>d[k]</code> (read)	<code>d[k]</code>	inserts k with value 0 if absent , see the trap
<code>k in d</code>	<code>d.count(k)</code>	gives 0 or 1, never inserts
<code>d.get(k)</code>	<code>auto it = d.find(k); if (it != d.end()) use(it->second);</code>	<code>find</code> gives an iterator; <code>d.end()</code> means "not there"
<code>d[k] += 1</code> on a <code>defaultdict(int)</code>	<code>d[k] += 1;</code>	the auto-insert is exactly what you want here
<code>del d[k]</code>	<code>d.erase(k);</code>	
<code>for k, v in d.items():</code>	<code>for (const auto& [k, v] : d)</code>	see below
<code>len(d)</code>	<code>d.size()</code>	
<code>s = set()</code>	<code>std::unordered_set<int> s;</code>	
<code>s.add(x)</code>	<code>s.insert(x);</code>	
<code>x in s</code>	<code>s.count(x)</code>	0 or 1
<code>s.remove(x)</code>	<code>s.erase(x);</code>	

Order of iteration is arbitrary in both languages (Python dicts keep insertion order since 3.7; C++ unordered containers do not).

The [] trap, drawn. In Python, reading `d["bob"]` when "bob" is missing raises `KeyError`. In C++, `d["bob"]` when "bob" is missing **creates** the entry with value 0 and hands it back. Reading looks harmless and silently changes the map:

```

before:    ages = { "ann" -> 30 }                ages.size() == 1

           if (ages["bob"] > 18) { ... }    <- just a look, surely?

after:     ages = { "ann" -> 30,  "bob" -> 0 }    ages.size() == 2
           ^^^^^^^^^^ invented by the [] lookup

```

```

std::unordered_map<std::string, int> ages;
ages["ann"] = 30;
assert(ages.size() == 1);
assert(ages.count("bob") == 0);           // count never inserts
int bobsAge = ages["bob"];                // this DOES insert bob -> 0
assert(bobsAge == 0 && ages.size() == 2);

```

So: use `[]` when you want to write or you want the auto-insert (counting). Use `count` or `find` when you only want to know whether something is there. In Two Sum below, using `[]` for the lookup would make you "find" a number you never stored.

Iterating with `auto& [k, v]`. Each entry of a map is a pair (c7) of key and value. The cleanest loop names both:

```

std::unordered_map<std::string, int> freq;
for (char ch : std::string("banana")) freq[std::string(1, ch)] += 1;
int total = 0;
for (const auto& [letter, n] : freq) { // letter is the key, n is the value
    assert(letter.size() == 1);
    total += n;
}
assert(total == 6 && freq["a"] == 3 && freq["n"] == 2 && freq["b"] == 1);

```

`const auto&` so each entry is looked at, not copied. Without the `&`, every pair is copied on every round of the loop.

insert tells you whether it was new. For a set, `s.insert(x)` returns a pair; its `.second` is `true` if `x` was actually added and `false` if it was already there. That saves a separate `count` call:

```

std::unordered_set<int> seen;
assert(seen.insert(5).second == true);    // 5 was new
assert(seen.insert(5).second == false);   // 5 was already there
assert(seen.size() == 1);

```

(You can skip this on a first read.) `std::pair<int,int>` cannot be a key of an unordered container out of the box (there is no built-in hash for it; the compile error is a page of template text mentioning `std::hash`). For grid coordinates use a `std::vector<std::vector<bool>>` `visited` instead, or encode `r * cols + c` into one `int`. `std::map` and `std::set` are the sorted cousins: a bit slower, but iteration comes out in key order.

In your own words: `unordered_map` is a dict and `unordered_set` is a set. `d[k]` inserts a default when `k` is missing, so to test membership I use `count` or `find`. I iterate with `for (const auto& [k, v] : d)`. `s.insert(x).second` tells me whether `x` was new.

c7. `std::pair`, `std::tuple`, structured bindings

A `std::pair` is a tuple with exactly two slots, and the slots have fixed types. Python's `(3, "cat")` becomes:

```

std::pair<int, std::string> p = {3, "cat"};

+-----+-----+
|   3   | "cat" |

```

```
+-----+-----+
  .first  .second
```

The slots are called `.first` and `.second`, not `[0]` and `[1]`. Header: `<utility>` (but `<vector>` and `<map>` already pull it in).

```
std::pair<int, std::string> p = {3, "cat"};
assert(p.first == 3 && p.second == "cat");
std::pair<int, int> q = std::make_pair(1, 2);    // make_pair works out the types for you
assert(q.first == 1 && q.second == 2);
std::vector<std::pair<int, int>> edges;
edges.push_back({0, 1});                       // {a, b} builds a pair where one is expected
edges.emplace_back(1, 2);                       // same thing, built in place
assert(edges.size() == 2 && edges[1].second == 2);
```

Comparing pairs: first slot, then second. Pairs compare like words in a dictionary: compare `.first`; only if those are equal, compare `.second`. This is what makes a vector of pairs sort sensibly with no extra work, and why a queue of (priority, item) pairs orders by priority.

```
(1, 5) < (2, 0)    because 1 < 2, the second slots are never looked at
(1, 5) < (1, 7)    first slots tie, then 5 < 7
(2, 0) == (2, 0)
```

```
assert(std::make_pair(1, 5) < std::make_pair(2, 0));
assert(std::make_pair(1, 5) < std::make_pair(1, 7));
assert(std::make_pair(2, 0) == std::make_pair(2, 0));
std::vector<std::pair<int, int>> pts = {{2, 0}, {1, 7}, {1, 5}};
std::sort(pts.begin(), pts.end());
assert(pts[0] == std::make_pair(1, 5) && pts[2] == std::make_pair(2, 0));
```

A `std::tuple` is the same with any number of slots. `std::tuple<int, int, double> t = {1, 2, 3.5};` and you read slot `i` with `std::get<i>(t)`, where `i` must be a literal number. Header `<tuple>`. You will rarely need one; a small struct with named fields is usually clearer.

Structured bindings: Python's `a, b = p`. With `auto [a, b] = p`; C++ makes two new variables `a` and `b` holding the two slots. It works on pairs, tuples and simple structs, and it is what makes map loops readable:

```
std::pair<int, int> cell = {4, 7};
auto [r, c] = cell;           // r == 4, c == 7, two new variables
assert(r == 4 && c == 7);
std::tuple<int, int, double> t = {1, 2, 3.5};
assert(std::get<2>(t) == 3.5);
auto [x, y, w] = t;
assert(x == 1 && y == 2 && w == 3.5);
std::vector<std::pair<int, int>> pairs = {{1, 2}, {3, 4}};
int sum = 0;
for (const auto& [a, b] : pairs) sum += a * b;    // Python: for a, b in pairs
assert(sum == 14);
```

Python	C++
<code>t = (1, 2)</code>	<code>std::pair<int, int> t = {1, 2};</code> or <code>std::make_pair(1, 2)</code>
<code>a, b = t</code>	<code>auto [a, b] = t;</code>
<code>for a, b in pairs:</code>	<code>for (const auto& [a, b] : pairs)</code>
<code>t[0], t[1]</code>	<code>t.first, t.second</code>
<code>sorted(pairs)</code>	<code>std::sort(pairs.begin(), pairs.end())</code> (same first-then-second order)

(You can skip this on a first read.) A structured binding always declares new variables; you cannot bind into variables that already exist (that is `std::tie(a, b) = p;`), and you cannot skip a slot (`auto [_, b]` still declares `_`). `std::make_pair(1, "x")` makes the second slot a `const char*`, not a `std::string`; when you want a string, write the type out.

In your own words: A pair is a two-slot tuple with `.first` and `.second`. Pairs compare by the first slot, then the second, so vectors of pairs sort with no extra code. `auto [a, b] = p;` unpacks a pair into two named variables, and `for (const auto& [k, v] : map)` walks a map.

A9. The demo file

Every surprise from c1 to c7 is written down as an `assert` in one file, so you can read the numbers instead of taking my word for it. Compile and run it:

```
g++ -std=c++17 -Wall -Wextra -O2 00_basics_demo.cpp -o /tmp/x && /tmp/x
```

```
// 00_basics_demo.cpp – the C++ traps that bite Python developers first.
// Every block asserts what actually happens, so you can read the numbers
// rather than take my word for it.
#include <cassert>
#include <climits>
#include <cstdint>
#include <iostream>
#include <string>
#include <vector>

// ---- pass by value vs by reference -----
void byValue(std::vector<int> v) { v.push_back(99); } // copies the vector
void byRef(std::vector<int>& v) { v.push_back(99); } // aliases the caller's
int sumConstRef(const std::vector<int>& v) { // no copy, can't modify
    int s = 0;
    for (int x : v) s += x;
    return s;
}

// Returning a vector by value is fine: the compiler moves/elides the copy.
std::vector<int> makeSquares(int n) {
    std::vector<int> out;
    out.reserve(n);
    for (int i = 0; i < n; ++i) out.push_back(i * i);
    return out;
}

int main() {
    // ---- integer division and modulo -----
    assert(7 / 2 == 3); // Python: 7 / 2 == 3.5, 7 // 2 == 3
    assert(-7 / 2 == -3); // truncates toward zero; Python: -7 // 2 == -4
    assert(-7 % 2 == -1); // sign follows the dividend; Python: -7 % 2 == 1
    assert(7.0 / 2 == 3.5); // one double operand makes it floating division
    assert(static_cast<double>(7) / 2 == 3.5);

    // ---- int overflow (int is 32-bit here; Python ints are unbounded) ----
    int big = INT_MAX; // 2147483647
    long long safe = static_cast<long long>(big) + 1; // widen BEFORE adding
    assert(safe == 2147483648LL);
    // `big + 1` as int would be signed overflow = undefined behaviour.
    // -fsanitize=undefined reports it at runtime; -O2 may "optimise" it away.
    int a = 100000, b = 100000;
    long long product = static_cast<long long>(a) * b; // 1e10 does not fit int
    assert(product == 10000000000LL);

    // ---- size_t subtraction -----
    std::vector<int> empty;
    // empty.size() - 1 is size_t and wraps to 18446744073709551615, not -1.
```

```

assert(empty.size() - 1 > 1000000);
// The idiom: cast to int first, then subtract.
int lastIdx = static_cast<int>(empty.size()) - 1;
assert(lastIdx == -1);
// Loop that would run "forever" on an empty vector: for (size_t i = 0; i <= v.size() - 1; ++i)
// Loop that is safe: for (int i = 0; i < (int)v.size(); ++i)

// ---- char arithmetic ----
char c = '7';
assert(c - '0' == 7); // char is a small integer; '7' - '0' == 7
assert('c' - 'a' == 2); // index into a 26-bucket array
assert(static_cast<char>('a' + 2) == 'c');
int count[26] = {}; // zero-initialised
for (char ch : std::string("banana")) ++count[ch - 'a'];
assert(count['a' - 'a'] == 3 && count['n' - 'a'] == 2);

// ---- implicit conversions ----
double d = 3.99;
int truncated = static_cast<int>(d); // 3, not 4 - truncation, not rounding
assert(truncated == 3);
int fromBool = true + true; // bool promotes to int
assert(fromBool == 2);
bool fromInt = 42; // any non-zero -> true
assert(fromInt);

// ---- pass by value vs by reference ----
std::vector<int> v = {1, 2, 3};
byValue(v);
assert(v.size() == 3); // caller unchanged (Python would see 4)
byRef(v);
assert(v.size() == 4); // caller sees the push
assert(sumConstRef(v) == 105);
std::vector<int> sq = makeSquares(4);
assert(sq == std::vector<int>({0, 1, 4, 9}));

// ---- auto copy vs auto& ----
std::vector<std::vector<int>> grid(2, std::vector<int>(3, 0));
for (auto row : grid) row[0] = 1; // `row` is a COPY; grid untouched
assert(grid[0][0] == 0 && grid[1][0] == 0);
for (auto& row : grid) row[0] = 1; // reference; grid modified
assert(grid[0][0] == 1 && grid[1][0] == 1);
for (const auto& row : grid) assert(row.size() == 3); // read-only, no copy

// ---- operator[] does not bounds-check ----
// v[100] compiles and is undefined behaviour. v.at(100) throws
// std::out_of_range. Use .at() while debugging, [] once correct.
bool threw = false;
try { v.at(100); } catch (const std::out_of_range&) { threw = true; }
assert(threw);

std::cout << "OK 00_basics_demo.cpp\n";
return 0;
}

```

What to look at:

- The first block is c1: integer division, `-7 / 2`, and `7.0 / 2`. Each `assert` states what C++ does and the comment states what Python does.
- The overflow block widens with `static_cast<long long>` before the `+ 1` and the `* b`. `INT_MAX` (from `<climits>`) is 2,147,483,647, the top of the `int` box.
- The `size_t` block is the c1 skip paragraph made concrete: `empty.size() - 1` is a huge number, and the fix is to cast to `int` first.
- The `char` block is the `'0'` trick from c5, including the 26-box letter count.
- `byValue`, `byRef` and `sumConstRef` at the top are the three parameter forms from c2, and the asserts in `main` show which one changed the caller's vector.

- The `auto` versus `auto&` block is the c3 drawing as code: the first loop changes copies and the grid is untouched.
- The last block shows `.at()` throwing where `[]` would silently misbehave. `try / catch` is Python's `try / except`; you do not need it anywhere else in this chapter.

Then break it. Change one number in an `assert`, predict the message, and run it. An assertion failure prints the file, the line, and the condition that was false; get used to reading that.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution with every new thing explained, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest. You already know these problems from Python; the point is to produce the C++ without stalling on syntax.

1. Two Sum (LeetCode 1, easy)

The problem in plain words. You are given a list of whole numbers `nums` and a number `target`. Find two *different positions* in the list whose numbers add up to `target`, and return those two positions. There is exactly one answer. You may not use the same position twice, but two different positions may hold the same number.

```
nums = [ 2,  7, 11, 15 ]      target = 9
index   0   1   2   3

2 + 7 = 9, at positions 0 and 1 -> answer [0, 1]

nums = [3, 2, 4], target = 6   -> 2 + 4 at positions 1 and 2 -> [1, 2]
                                   (not [0, 0]: position 0 used twice is not allowed)
nums = [3, 3], target = 6      -> [0, 1] (same number, different positions: fine)
```

The list has between 2 and 10,000 numbers, which can be negative.

By hand. Walk along the list. At each number, ask "which number would I need to go with this one?" That is `target` minus the current number. Then ask "have I already walked past that number?" If yes, you have your pair. If no, write the current number down (with its position) so later numbers can find it, and move on.

The idea. Keep a dictionary from *number seen* to *its position*. For each position `i`: compute `need = target - nums[i]`; if `need` is in the dictionary, return its stored position and `i`; otherwise store `nums[i] -> i`. Storing *after* checking is what stops a position from pairing with itself.

The Python you would have written.

```
def twoSum(nums, target):
    seen = {} # value -> index
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
```

In C++. This is the full file. `main()` at the bottom runs the tests.

```

// LeetCode 1. Two Sum
#include <cassert>
#include <iostream>
#include <unordered_map>
#include <vector>

// One pass: for each x, ask "have I already seen target - x?". If not, remember x.
std::vector<int> twoSum(std::vector<int>& nums, int target) {
    std::unordered_map<int, int> seen; // value -> index
    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        int need = target - nums[i];
        auto it = seen.find(need); // find, not [] - [] would insert need with index 0
        if (it != seen.end()) return {it->second, i};
        seen[nums[i]] = i; // insert AFTER checking so we never pair i with itself
    }
    return {}; // LeetCode guarantees a solution; keep the compiler happy
}

int main() {
    std::vector<int> a = {2, 7, 11, 15};
    assert(twoSum(a, 9) == std::vector<int>({0, 1}));

    std::vector<int> b = {3, 2, 4};
    assert(twoSum(b, 6) == std::vector<int>({1, 2}));

    std::vector<int> c = {3, 3}; // duplicate values: must not reuse index
    assert(twoSum(c, 6) == std::vector<int>({0, 1}));

    std::vector<int> d = {-1, -2, -3, -4, -5}; // negatives
    assert(twoSum(d, -8) == std::vector<int>({2, 4}));

    std::vector<int> e = {1}; // no answer -> empty
    assert(twoSum(e, 2).empty());

    std::cout << "OK 01_two_sum.cpp\n";
    return 0;
}

```

What is new:

- `std::vector<int> twoSum(std::vector<int>& nums, int target)` : returns a vector of ints (the two positions) and takes the list by reference (c2) plus an `int`. This is LeetCode's exact signature.
- `std::unordered_map<int, int> seen;` : Python's `seen = {}` with the types stated: keys are ints, values are ints (c6).
- `for (int i = 0; i < static_cast<int>(nums.size()); ++i)` : Python's `for i in range(len(nums))`. The cast is the c1 habit so `int i` and `size()` compare without a warning.
- `auto it = seen.find(need);` and `if (it != seen.end())` : Python's `if need in seen`. `find` hands back an *iterator*, a thing that points at the entry or at `end()` meaning "not there". `it->second` is the value stored (the position). We use `find` and not `seen[need]` because `[]` would insert `need` with position 0 (the c6 trap) and we would "find" a number we never saw.
- `return {it->second, i};` : builds the two-element vector in place. Python's `return [seen[need], i]`.
- `return {};` : an empty vector. Needed because a function that promises to return something must do so on every path, or the compiler warns.

Walk through the example. `nums = [2, 7, 11, 15]`, `target = 9`, `seen` empty. `i = 0`: `need = 9 - 2 = 7`; not in `seen`; store `2 -> 0`. `i = 1`: `need = 9 - 7 = 2`; `find(2)` hits, `it->second` is 0; return `{0, 1}`. Done in two steps.

Complexity. One pass over the list, one dictionary lookup per element: time proportional to the length, $O(n)$. The dictionary can hold every number, so memory is $O(n)$ too.

Robotics. Matching two sensor streams by timestamp is this pattern: index one stream by its stamp in a hash map, then walk the other stream once asking "is there a sample at (or near) the stamp I need?". Nested loops over both streams would be far too slow at 100 Hz.

Follow-ups you may get. - *The list is sorted; can you use no extra memory?* Two pointers, one at each end, moving inward: if the sum is too small move the left one right, if too big move the right one left. $O(n)$ time, no map. - *Return all pairs, not just one?* Store counts in the map instead of positions, and handle $x ==$ need separately. - *Why `unordered_map` and not `map`?* `map` keeps keys sorted, which costs $O(\log n)$ per lookup; we do not need order, so the hash map's $O(1)$ wins.

2. Contains Duplicate (LeetCode 217, easy)

The problem in plain words. Given a list of whole numbers, return `true` if any number appears two or more times, otherwise `false`.

```
[1, 2, 3, 1]  -> 1 appears twice      -> true
[1, 2, 3, 4]  -> every number once   -> false
[]            -> nothing can repeat   -> false
[42]         -> one number, once     -> false
```

Up to 100,000 numbers, which can be negative.

By hand. Read the numbers one at a time and keep a list of the ones you have already seen. If the number you are reading is already on your list, stop: duplicate. If you get to the end, no duplicate. Keeping the list as a set makes "is it on my list" instant.

The idea. A set seen. For each number: if it is in seen, return `true`; else add it. After the loop return `false`. In C++ the "is it in, and if not add it" pair is one call: `insert` reports whether the number was new.

The Python you would have written.

```
def containsDuplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

In C++.

```
// LeetCode 217. Contains Duplicate
#include <algorithm>
#include <cassert>
#include <iostream>
#include <unordered_set>
#include <vector>

// Hash set:  $O(n)$  time,  $O(n)$  space. insert() returns {iterator, inserted?}.
bool containsDuplicate(std::vector<int>& nums) {
    std::unordered_set<int> seen;
    seen.reserve(nums.size() * 2); // avoid rehashing; optional but interviewers like it
    for (int x : nums) {
        if (!seen.insert(x).second) return true;
    }
    return false;
}

// Alternative: sort and compare neighbours.  $O(n \log n)$  time,  $O(1)$  extra space.
bool containsDuplicateSort(std::vector<int> nums) { // by value: we may reorder it
    std::sort(nums.begin(), nums.end());
    return std::adjacent_find(nums.begin(), nums.end()) != nums.end();
}
```

```

}

int main() {
    std::vector<int> a = {1, 2, 3, 1};
    assert(containsDuplicate(a) == true);
    assert(containsDuplicateSort(a) == true);

    std::vector<int> b = {1, 2, 3, 4};
    assert(containsDuplicate(b) == false);
    assert(containsDuplicateSort(b) == false);

    std::vector<int> c = {1, 1, 1, 3, 3, 4, 3, 2, 4, 2};
    assert(containsDuplicate(c) == true);

    std::vector<int> d = {}; // empty
    assert(containsDuplicate(d) == false);
    std::vector<int> e = {42}; // single element
    assert(containsDuplicate(e) == false);

    std::cout << "OK 02_contains_duplicate.cpp\n";
    return 0;
}

```

What is new:

- `std::unordered_set<int> seen;` : Python's `seen = set()` for ints (c6).
- `seen.reserve(nums.size() * 2);` : tell the set to make room up front so it does not have to grow several times. Optional; the code is correct without it.
- `if (!seen.insert(x).second) return true;` : `insert` returns a pair whose `.second` is true if `x` was new. `!` flips it: "if it was *not* new, we found a duplicate". This is Python's `if x in seen: return True` and `seen.add(x)` in one line.
- The second function, `containsDuplicateSort`, is the other way to do it: sort the list, then a duplicate must be next to its twin. `std::adjacent_find` walks the sorted list looking for two equal neighbours and returns `end()` if there are none. It takes `nums` by *value* (no `&`, `c2`) on purpose: sorting reorders the list, and the copy keeps the caller's order intact.

Walk through the example. `[1, 2, 3, 1]`: insert 1, new. Insert 2, new. Insert 3, new. Insert 1, `.second` is false, return true. For `[1, 2, 3, 4]` all four inserts are new and the loop ends; return false.

Complexity. Hash set version: one pass, $O(n)$ time and $O(n)$ memory. Sort version: $O(n \log n)$ time, but only the copy for memory.

Robotics. Checking that a list of IDs has no repeats is routine: the landmark IDs a SLAM keyframe observed, the joint names in a robot description file, the node names on a ROS graph. All of them are "insert into a set, watch `.second`".

Follow-ups you may get. - *The values are all between 0 and n ?* Use a `std::vector<bool> seen(n + 1)` indexed by the value: no hashing at all. - *A duplicate within distance k of itself (LeetCode 219)?* Keep a sliding-window set: erase `nums[i - k]` as `i` advances so the set only holds the last k numbers. - *Which version would you choose?* The set for speed and clarity; the sort if memory is tight and reordering is allowed.

3. Valid Anagram (LeetCode 242, easy)

The problem in plain words. Two strings `s` and `t` of lowercase letters. Return true if `t` is a rearrangement of the letters of `s` (same letters, same counts, any order), otherwise false.

<code>s = "anagram"</code>	<code>t = "nagaram"</code>	-> same letters: a a a g m n r	-> true
<code>s = "rat"</code>	<code>t = "car"</code>	-> rat has t, car has c	-> false
<code>s = ""</code>	<code>t = ""</code>	-> nothing versus nothing	-> true
<code>s = "a"</code>	<code>t = "ab"</code>	-> different lengths	-> false

Up to 50,000 letters each, all lowercase a to z.

By hand. Count how many of each letter `s` has, count the same for `t`, and compare the two tallies. On paper you would make a table with 26 rows. A quicker version: go through both strings together, adding one to a letter's row for `s` and taking one away for `t`; if every row ends at zero they match.

The idea. Lengths differ: `false`. Otherwise 26 counters, all zero. For each position `i`, add 1 at `s[i]`'s counter and subtract 1 at `t[i]`'s. Any counter not zero at the end: `false`. The counter for a letter is found with the `'0'` trick from c5: `ch - 'a'` is 0 for a, 1 for b, and so on.

The Python you would have written.

```
def isAnagram(s, t):
    if len(s) != len(t):
        return False
    count = [0] * 26
    for a, b in zip(s, t):
        count[ord(a) - ord('a')] += 1
        count[ord(b) - ord('a')] -= 1
    return all(c == 0 for c in count)
```

In C++.

```
// LeetCode 242. Valid Anagram
#include <cassert>
#include <iostream>
#include <string>

// Lowercase letters only -> a fixed 26-slot count array beats a hash map.
// +1 for s, -1 for t; anagram iff every slot ends at zero.
bool isAnagram(std::string s, std::string t) {
    if (s.size() != t.size()) return false;
    int count[26] = {}; // {} zero-initialises; plain int count[26]; does not
    for (size_t i = 0; i < s.size(); ++i) {
        ++count[s[i] - 'a'];
        --count[t[i] - 'a'];
    }
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}

int main() {
    assert(isAnagram("anagram", "nagaram") == true);
    assert(isAnagram("rat", "car") == false);
    assert(isAnagram("", "") == true); // empty strings
    assert(isAnagram("a", "ab") == false); // length mismatch
    assert(isAnagram("aacc", "ccac") == false); // same letters, different multiset
    std::cout << "OK 03_valid_anagram.cpp\n";
    return 0;
}
```

What is new:

- `int count[26] = {};` : 26 boxes of `int`, all zero. This is a plain C-style array, a vector with a size fixed at compile time. The `= {}` is what zeroes it; `int count[26];` alone leaves garbage inside, unlike Python's `[0] * 26`.
- `for (size_t i = 0; i < s.size(); ++i)` : `size_t` is the type `size()` returns (c1 skip paragraph), so using it for `i` avoids the signedness warning without a cast. Both are fine.
- `++count[s[i] - 'a'];` : `s[i]` is a `char`; `s[i] - 'a'` is its position in the alphabet (c5); `++` adds one to that box. Python's `count[ord(a) - ord('a')] += 1`.
- `for (int c : count)` : range-for works on plain arrays too.

Walk through "rat" and "car". Lengths both 3. $i = 0$: +1 at r, -1 at c. $i = 1$: +1 at a, -1 at a (back to 0). $i = 2$: +1 at t, -1 at r (back to 0). Final non-zero boxes: c is -1, t is +1. Return false. For "anagram" / "nagaram" every box returns to 0 and the answer is true.

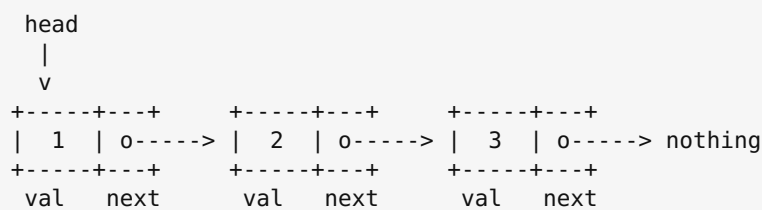
Complexity. One pass over the strings and one over the 26 boxes: $O(n)$ time. Memory is 26 ints no matter how long the strings are: $O(1)$.

Robotics. A fixed-size array indexed by `value - base` is a histogram without a map: counting how many cells of a costmap hold each of 256 cost values, or binning laser intensities. Same `[c - 'a']` arithmetic with a different alphabet.

Follow-ups you may get. - What if the strings can hold any character, not just lowercase? Swap the 26-box array for an `unordered_map<char, int>`; the logic is identical. - Group a list of words into anagram groups (LeetCode 49)? Use the sorted word, or the 26 counts turned into a string, as a map key, and collect words under it. - What breaks if a capital letter sneaks in? 'A' - 'a' is negative, so the index is off the front of the array: undefined behaviour. The constraint promises lowercase; check it if you cannot trust the input.

4. Reverse Linked List (LeetCode 206, easy)

First, what a linked list is. A vector keeps its boxes side by side. A linked list keeps its boxes anywhere in memory, and each box carries an *arrow* to the next box. The whole list is reached by holding the arrow to the first box (the *head*). The last box's arrow points at nothing.



What a pointer is, for now. A pointer is an arrow to a box. Chapter 02 goes deeper; for this chapter you need three things:

- `ListNode*` (with the star) is the type "arrow to a `ListNode` box". `ListNode* head` is a box that holds an arrow, not a node.
- `->` follows the arrow and picks a field. `head->val` is "follow head, read val": 1 in the picture. `head->next` is "follow head, read its arrow": the arrow to the 2 box. Python's `head.val` and `head.next`.
- `nullptr` is "no arrow", the arrow pointing at nothing. Python's `None`. Following it is a crash, so you always check `!= nullptr` first.

The node itself is a `struct`, a box with named fields, which is Python's class without methods:

```

struct ListNode {
    int val;           // the number stored in this box
    ListNode* next;    // the arrow to the next box (nullptr at the end)
};

```

Making boxes and hooking them together:

```

ListNode* a = new ListNode{1, nullptr}; // make a box holding 1; a is the arrow to it
ListNode* b = new ListNode{2, nullptr}; // another box holding 2
a->next = b;                             // the arrow inside a now points at b
assert(a->next->val == 2);                 // follow a, follow its next, read val
assert(b->next == nullptr);               // b points at nothing
delete b;                                 // give the boxes back when done (the files do this in deleteList)
delete a;

```

`new` makes a box that outlives the function it was made in, and hands back the arrow to it. `delete` throws the box away. LeetCode does that for you; the test files here do it in a helper.

The problem in plain words. Given the head of a linked list, reverse it so the arrows point the other way, and return the new head (the box that used to be last).

```

before:  1 -> 2 -> 3 -> 4 -> 5 -> nothing    head is the 1 box
after:    5 -> 4 -> 3 -> 2 -> 1 -> nothing    head is the 5 box

one node: 7 -> nothing      stays 7 -> nothing
empty list: nothing        stays nothing

```

Up to 5,000 nodes.

By hand. Walk along the list one box at a time. At each box, turn its arrow around so it points at the box you just came from. The catch: the moment you turn the arrow, you lose the way forward. So before turning it, write down where it pointed. Three pieces of paper: *prev* (the box behind me), *cur* (the box I am on), *next* (where I was about to go).

```

start:      prev = nothing    cur = 1        1 -> 2 -> 3

step 1:      next = 2          turn 1's arrow: 1 -> nothing    prev = 1, cur = 2
step 2:      next = 3          turn 2's arrow: 2 -> 1          prev = 2, cur = 3
step 3:      next = nothing    turn 3's arrow: 3 -> 2          prev = 3, cur = nothing

cur is nothing: stop. prev (the 3 box) is the new head.    3 -> 2 -> 1 -> nothing

```

The idea. `prev = null`, `cur = head`. While `cur` is not null: save `next = cur.next`; point `cur.next` at `prev`; move both forward (`prev = cur`, `cur = next`). Return `prev`.

The Python you would have written.

```

def reverseList(head):
    prev = None
    cur = head
    while cur is not None:
        nxt = cur.next      # save before we overwrite
        cur.next = prev
        prev = cur
        cur = nxt
    return prev

```

In C++.

```

// LeetCode 206. Reverse Linked List
#include <cassert>
#include <iostream>
#include <vector>

// LeetCode's definition, verbatim.
struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* next) : val(x), next(next) {}
};

// Iterative: walk the list, flipping each `next` to point backwards.
// Three pointers: prev (already reversed), cur (being flipped), next (saved before we overwrite).
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur != nullptr) {
        ListNode* next = cur->next; // save before we lose it
        cur->next = prev;
        prev = cur;
        cur = next;
    }
}

```

```

    }
    return prev; // prev is the new head (nullptr for an empty list)
}

// Recursive: reverse everything after head, then hook head onto the tail.
// O(n) stack depth - mention that to the interviewer.
ListNode* reverseListRecursive(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;
    ListNode* newHead = reverseListRecursive(head->next);
    head->next->next = head; // the node after us now points back at us
    head->next = nullptr; // we become the tail (until a caller re-hooks us)
    return newHead;
}

// ---- test helpers -----
ListNode* buildList(const std::vector<int>& vals) {
    ListNode dummy;
    ListNode* tail = &dummy;
    for (int v : vals) {
        tail->next = new ListNode(v);
        tail = tail->next;
    }
    return dummy.next;
}

std::vector<int> toVector(ListNode* head) {
    std::vector<int> out;
    for (; head != nullptr; head = head->next) out.push_back(head->val);
    return out;
}

void deleteList(ListNode* head) {
    while (head != nullptr) {
        ListNode* next = head->next;
        delete head;
        head = next;
    }
}

int main() {
    {
        ListNode* h = buildList({1, 2, 3, 4, 5});
        h = reverseList(h);
        assert(toVector(h) == std::vector<int>({5, 4, 3, 2, 1}));
        h = reverseListRecursive(h); // reverse back
        assert(toVector(h) == std::vector<int>({1, 2, 3, 4, 5}));
        deleteList(h);
    }
    {
        ListNode* h = buildList({1, 2});
        h = reverseListRecursive(h);
        assert(toVector(h) == std::vector<int>({2, 1}));
        deleteList(h);
    }
    {
        ListNode* h = buildList({7}); // single node
        assert(toVector(reverseList(h)) == std::vector<int>({7}));
        deleteList(h);
    }
    assert(reverseList(nullptr) == nullptr); // empty list
    assert(reverseListRecursive(nullptr) == nullptr);

    std::cout << "OK 04_reverse_linked_list.cpp\n";
    return 0;
}

```

What is new:

- The struct `ListNode` at the top is LeetCode's, with three *constructors* (the lines starting `ListNode(...)`): ways to make a node with no value, with a value, or with a value and a next arrow. : `val(x), next(nullptr)` sets the fields. You will always be given this struct; you do not write it.
- `ListNode* reverseList(ListNode* head)` : takes an arrow, returns an arrow.
- `ListNode* prev = nullptr;` : Python's `prev = None`.
- `while (cur != nullptr)` : Python's `while cur is not None`.
- `ListNode* next = cur->next;` : Python's `nxt = cur.next`. -> follows the arrow.
- `reverseListRecursive` is the second way: reverse everything after head, then hook head on at the end. `head->next->next = head` reads "the box after me now points back at me". It uses one stack frame per node, so for 5,000 nodes it is fine and for a million it is not.
- `buildList`, `toVector`, `deleteList` are helpers for the tests only: make a list from a vector, read a list into a vector, throw the boxes away.

Walk through 1 -> 2 -> 3. `prev = null, cur = 1`. Round 1: `next = 2; 1.next = null; prev = 1; cur = 2`. Round 2: `next = 3; 2.next = 1; prev = 2; cur = 3`. Round 3: `next = null; 3.next = 2; prev = 3; cur = null`. Loop ends. Return `prev`, the 3 box: `3 -> 2 -> 1`.

Complexity. Visits each box once: $O(n)$ time. The loop version uses three arrows no matter how long the list is: $O(1)$ memory. The recursive version uses $O(n)$ stack.

Robotics. Following `next` arrows box by box is how you walk any linked structure: climbing a transform tree from a wheel frame up through `parent` links to the map frame, or walking a chain of message buffers in a zero-copy transport. The discipline "save the arrow before you overwrite it" is the entire skill.

Follow-ups you may get. - *What if you forget to save `next`?* After `cur->next = prev` the rest of the list is unreachable; you reverse one box and lose the rest. - *Reverse only positions m to n (LeetCode 92)?* Walk to box $m-1$, run this same loop for $n-m+1$ boxes, then reconnect the two ends. - *Why does the caller have to write `h = reverseList(h)`?* After reversing, the old head is the tail. If the caller keeps using its old arrow it sees a one-box list.

5. Merge Two Sorted Lists (LeetCode 21, easy)

The problem in plain words. Two linked lists, each already sorted from small to large. Join them into one sorted list by re-pointing the existing boxes (do not make new ones) and return the head of the result.

```
list1:  1 -> 2 -> 4
list2:  1 -> 3 -> 4
result: 1 -> 1 -> 2 -> 3 -> 4 -> 4

list1 empty, list2 = 0      -> 0
both empty                -> empty
list1 = 5 -> 6 -> 7, list2 = 1 -> 2      -> 1 -> 2 -> 5 -> 6 -> 7
```

Each list has up to 50 nodes.

By hand. Put a finger on the front of each list. Compare the two boxes under your fingers, take the smaller one, attach it to the end of the result, and move that finger forward. Repeat until one list runs out. Then attach the whole rest of the other list in one go (it is already sorted).

The awkward part is "attach to the end of the result" when the result is still empty: there is no end yet. The trick is a *dummy* box: a fake first box that holds nothing useful. You always attach after it, and at the end the real answer is whatever comes after the dummy.

```
dummy -> nothing           tail = dummy
compare 1 (list1) and 1 (list2): take list1's 1  dummy -> 1           tail = that 1
compare 2 and 1: take list2's 1                  dummy -> 1 -> 1         tail = second 1
compare 2 and 3: take 2                          dummy -> 1 -> 1 -> 2
```

```

compare 4 and 3: take 3                dummy -> 1 -> 1 -> 2 -> 3
compare 4 and 4: take list1's 4 (ties go to list1) dummy -> 1 -> 1 -> 2 -> 3 -> 4
list1 is empty: attach the rest of list2 (4)    dummy -> 1 -> 1 -> 2 -> 3 -> 4 -> 4
answer: whatever dummy points at

```

The idea. `dummy` node, `tail = dummy`. While both lists are non-empty: hang the smaller head on `tail.next`, advance that list, advance `tail`. Then `tail.next = whichever list is left`. Return `dummy.next`.

The Python you would have written.

```

def mergeTwoLists(l1, l2):
    dummy = ListNode()           # a fake first node so there is always a tail
    tail = dummy
    while l1 is not None and l2 is not None:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 is not None else l2
    return dummy.next

```

In C++.

```

// LeetCode 21. Merge Two Sorted Lists
#include <cassert>
#include <iostream>
#include <vector>

struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* next) : val(x), next(next) {}
};

// Dummy-head idiom: a stack-allocated sentinel node means "append to tail"
// never has to special-case the empty result. Return dummy.next.
ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    ListNode dummy;           // lives on the stack, never returned itself
    ListNode* tail = &dummy;
    while (list1 != nullptr && list2 != nullptr) {
        if (list1->val <= list2->val) { // <= keeps the merge stable
            tail->next = list1;
            list1 = list1->next;
        } else {
            tail->next = list2;
            list2 = list2->next;
        }
        tail = tail->next;
    }
    tail->next = (list1 != nullptr) ? list1 : list2; // splice the leftover run
    return dummy.next;
}

// ---- test helpers -----
ListNode* buildList(const std::vector<int>& vals) {
    ListNode dummy;
    ListNode* tail = &dummy;
    for (int v : vals) {
        tail->next = new ListNode(v);
        tail = tail->next;
    }
    return dummy.next;
}

```

```

}
std::vector<int> toVector(ListNode* head) {
    std::vector<int> out;
    for (; head != nullptr; head = head->next) out.push_back(head->val);
    return out;
}
void deleteList(ListNode* head) {
    while (head != nullptr) {
        ListNode* next = head->next;
        delete head;
        head = next;
    }
}

int main() {
    {
        ListNode* m = mergeTwoLists(buildList({1, 2, 4}), buildList({1, 3, 4}));
        assert(toVector(m) == std::vector<int>({1, 1, 2, 3, 4, 4}));
        deleteList(m);
    }
    assert(mergeTwoLists(nullptr, nullptr) == nullptr);           // both empty
    {
        ListNode* m = mergeTwoLists(nullptr, buildList({0}));    // one empty
        assert(toVector(m) == std::vector<int>({0}));
        deleteList(m);
    }
    {
        ListNode* m = mergeTwoLists(buildList({5, 6, 7}), buildList({1, 2})); // no interleaving
        assert(toVector(m) == std::vector<int>({1, 2, 5, 6, 7}));
        deleteList(m);
    }
    std::cout << "OK 05_merge_two_sorted_lists.cpp\n";
    return 0;
}

```

What is new:

- `ListNode dummy;` : a node made *without new*. It is an ordinary local box (no star in the type) that disappears when the function ends. That is fine because we never return the dummy itself, only what comes after it.
- `ListNode* tail = &dummy;` : `&` in front of a variable means "the arrow to this box". So `tail` is an arrow pointing at `dummy`. (This `&` is a different use from the `&` in a parameter type, `c2`; both mean "refer to the real thing rather than copy it".)
- `list1->val <= list2->val` : follow each arrow, compare the numbers. `<=` rather than `<` means ties take from `list1`, which keeps equal elements in their original order.
- `tail->next = (list1 != nullptr) ? list1 : list2;` : the one-line if from `c3`. "If `list1` still has boxes, attach `list1`, else attach `list2`."
- `return dummy.next;` : `.` not `->`, because `dummy` is a box, not an arrow to a box. Python's `return dummy.next`.

Walk through the example. Follow the hand trace above; the code does exactly those steps. After the loop `list1` is `nullptr` and `list2` is the last 4, so the ternary attaches `list2`. `dummy.next` is the first 1, which is returned.

Complexity. Each box is looked at once: $O(n + m)$ time for lists of length n and m . No new boxes are made: $O(1)$ extra memory.

Robotics. Merging two time-sorted streams is message synchronisation: combining a lidar scan stream and an odometry stream into one timeline ordered by stamp is this loop with `stamp` in place of `val`. The dummy-head trick reappears any time you build a linked result box by box.

Follow-ups you may get. - *Merge k sorted lists (LeetCode 23)?* Keep the current head of every list in a min-heap keyed by value; pop the smallest, push its successor. $O(N \log k)$. - *Why not return &dummy?* dummy is destroyed when the function returns; the arrow would point at a dead box. - *Recursive version?* The smaller head's next is `merge(rest of that list, other list)`. Same time, $O(n + m)$ stack.

6. Maximum Subarray (LeetCode 53, medium)

The problem in plain words. Given a list of whole numbers (some negative), find the *contiguous* run of one or more neighbouring numbers whose sum is largest, and return that sum. You do not have to return the run itself.

```
nums = [ -2,  1, -3,  4, -1,  2,  1, -5,  4 ]
           ^^^^^^^^^^^^^^^^^
           4 + (-1) + 2 + 1 = 6      -> answer 6

[1]                -> 1      (the only run)
[5, 4, -1, 7, 8] -> 23      (the whole list)
[-3, -1, -2]      -> -1      (all negative: the best run is the single number -1)
```

Up to 100,000 numbers, each between -10,000 and 10,000.

By hand. Walk left to right, keeping a running sum of "the best run that ends right here". At each number you have two choices: extend the run you were carrying (add this number to it) or throw that run away and start a new one at this number. Starting fresh is better exactly when the run you were carrying had gone negative, because a negative prefix only drags you down. Keep the best running sum you ever saw.

number:	-2	1	-3	4	-1	2	1	-5	4	
run here:	-2	1	-2	4	3	5	6	1	5	
best so far:	-2	1	1	4	4	5	6	6	6	-> 6

"run here" for 1 is $\max(1, -2 + 1) = 1$: start fresh. For 4 it is $\max(4, -2 + 4) = 4$: fresh again. For -1 it is $\max(-1, 4 + -1) = 3$: extend.

The idea. `cur = best = nums[0]`. For each later number `x`: `cur = max(x, cur + x)`; `best = max(best, cur)`. Return `best`. Starting `best` at `nums[0]`, not at 0, is what makes the all-negative case right. This is known as Kadane's algorithm.

The Python you would have written.

```
def maxSubArray(nums):
    best = nums[0]
    cur = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)      # extend the run, or start again at x
        best = max(best, cur)
    return best
```

In C++.

```
// LeetCode 53. Maximum Subarray (Kadane)
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Kadane: best sum ending at i is either "extend the previous run" or "start fresh at i".
// Track the best of those over the whole array.
int maxSubArray(std::vector<int>& nums) {
    int best = nums[0];           // NOT 0 – an all-negative array must return its largest element
    int cur = nums[0];
    for (size_t i = 1; i < nums.size(); ++i) {
```

```

        cur = std::max(nums[i], cur + nums[i]);
        best = std::max(best, cur);
    }
    return best;
}

int main() {
    std::vector<int> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    assert(maxSubArray(a) == 6); // [4,-1,2,1]

    std::vector<int> b = {1};
    assert(maxSubArray(b) == 1); // single element

    std::vector<int> c = {5, 4, -1, 7, 8};
    assert(maxSubArray(c) == 23); // whole array

    std::vector<int> d = {-3, -1, -2};
    assert(maxSubArray(d) == -1); // all negative: initialising best=0 would give 0

    std::cout << "OK 06_maximum_subarray.cpp\n";
    return 0;
}

```

What is new:

- `#include <algorithm>` gives `std::max` and `std::min`.
- `for (size_t i = 1; i < nums.size(); ++i)` : Python's `for x in nums[1:]` written with an index, starting from 1.
- `cur = std::max(nums[i], cur + nums[i]);` : Python's `max(x, cur + x)`. Both arguments must have the same type (int and int here).

Walk through the example. See the table above. `cur` and `best` start at -2. Then `x = 1`: `cur = max(1, -1) = 1`, `best = 1`. `x = -3`: `cur = max(-3, -2) = -2`, `best` stays 1. `x = 4`: `cur = 4`, `best = 4`. `x = -1`: `cur = 3`. `x = 2`: `cur = 5`, `best = 5`. `x = 1`: `cur = 6`, `best = 6`. `x = -5`: `cur = 1`. `x = 4`: `cur = 5`. Return 6.

Complexity. One pass, two variables: $O(n)$ time, $O(1)$ memory.

Robotics. "The strongest contiguous stretch" of a 1D signal: the longest run of a laser scan above a reflectance threshold, or the window of accelerometer samples with the most energy when detecting a bump. Same running-sum trick.

Follow-ups you may get. - *Why not start `best` at 0?* Then `[-3, -1, -2]` would return 0, but an empty run is not allowed; the answer must be -1. - *Return the start and end positions too?* Remember where the current run started (reset it whenever you start fresh) and copy those positions whenever `best` improves. - *Could the sum overflow an `int`?* 100,000 numbers of at most 10,000 is at most one billion, which fits. With bigger limits, use `long long`.

7. Move Zeroes (LeetCode 283, easy)

The problem in plain words. Given a list of whole numbers, move every 0 to the end of the list, keeping the other numbers in their original order. Do it *in place*: change the given list, do not build a new one, and return nothing.

```

[0, 1, 0, 3, 12]  ->  [1, 3, 12, 0, 0]
[0]               ->  [0]
[1, 2, 3]         ->  [1, 2, 3]      (nothing to do)
[0, 0, 1]         ->  [1, 0, 0]

```

Up to 10,000 numbers.

By hand. Keep a marker for "the next free slot at the front", starting at position 0. Walk through the list; each time you meet a non-zero, swap it into the free slot and move the marker one to the right. Zeros are never picked up, so they naturally end up in the slots after the marker.

read ->	0	1	0	3	12	write = 0
read=0:	0 is zero, skip					write = 0
read=1:	1 non-zero, swap slots 0,1					[1, 0, 0, 3, 12] write = 1
read=2:	0 is zero, skip					write = 1
read=3:	3 non-zero, swap slots 1,3					[1, 3, 0, 0, 12] write = 2
read=4:	12 non-zero, swap slots 2,4					[1, 3, 12, 0, 0] write = 3

The idea. Two indices: `read` scans every position, `write` is the next slot for a non-zero. When `nums[read]` is non-zero, swap `nums[write]` and `nums[read]`, then `write += 1`.

The Python you would have written.

```
def moveZeroes(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
```

In C++.

```
// LeetCode 283. Move Zeroes
#include <cassert>
#include <iostream>
#include <utility>
#include <vector>

// Two pointers: `write` is the slot for the next non-zero. Swapping (instead of
// overwriting then zero-filling) keeps it a single pass and preserves order.
void moveZeroes(std::vector<int>& nums) {
    int write = 0;
    for (int read = 0; read < static_cast<int>(nums.size()); ++read) {
        if (nums[read] != 0) {
            std::swap(nums[write], nums[read]);    // no-op when write == read
            ++write;
        }
    }
}

int main() {
    std::vector<int> a = {0, 1, 0, 3, 12};
    moveZeroes(a);
    assert(a == std::vector<int>({1, 3, 12, 0, 0}));

    std::vector<int> b = {0};
    moveZeroes(b);
    assert(b == std::vector<int>({0}));

    std::vector<int> c = {};                                // empty
    moveZeroes(c);
    assert(c.empty());

    std::vector<int> d = {1, 2, 3};                          // no zeroes: order untouched
    moveZeroes(d);
    assert(d == std::vector<int>({1, 2, 3}));

    std::vector<int> e = {0, 0, 1};
    moveZeroes(e);
    assert(e == std::vector<int>({1, 0, 0}));

    std::cout << "OK 07_move_zeroes.cpp\n";
```

```
    return 0;
}
```

What is new:

- `void moveZeroes(std::vector<int>& nums)` : returns nothing (void) and takes the list *by reference*. The `&` is the whole problem: without it the function would sort a copy and the caller's list would not change (c2).
- `std::swap(nums[write], nums[read]);` : Python's `a, b = b, a`. Needs `#include <utility>`. When `write == read` it swaps a box with itself, which is harmless.

Walk through the example. See the table above; the code does exactly those swaps. The function ends with `nums` equal to `[1, 3, 12, 0, 0]` and returns nothing.

Complexity. One pass, one swap per non-zero: $O(n)$ time, $O(1)$ memory.

Robotics. Compacting a buffer in place is how you drop bad points from a point cloud (NaN ranges, zero-intensity returns) without allocating a second cloud: the same read/write sweep, usually spelled `std::remove_if` followed by `erase`.

Follow-ups you may get. - *Fewer writes?* Copy each non-zero forward to `nums[write]` (no swap), then fill from `write` to the end with zeros. Fewer stores when zeros are rare. - *A one-liner from the library?* `std::stable_partition(nums.begin(), nums.end(), [](int x) { return x != 0; });` keeps order and moves the zeros to the back. - *What if the function took `std::vector<int> nums` without `&`?* It compiles, runs, and the caller's vector is unchanged. The tests in `main` would fail.

8. Best Time to Buy and Sell Stock (LeetCode 121, easy)

The problem in plain words. A list of prices, one per day. You may buy on one day and sell on a *later* day, once. Return the biggest profit you can make. If no profit is possible, return 0 (you simply do not trade).

```
prices = [7, 1, 5, 3, 6, 4]
day      0  1  2  3  4  5

buy on day 1 at 1, sell on day 4 at 6 -> profit 5      -> answer 5
(buying at 1 and selling at 7 is not allowed: day 0 is before day 1)

[7, 6, 4, 3, 1] -> prices only fall -> 0
[5]             -> one day, cannot sell after buying -> 0
[2, 4, 1]       -> buy 2, sell 4      -> 2 (the later 1 is cheaper but nothing comes after it)
```

Up to 100,000 days.

By hand. Walk through the days. Keep two notes: the cheapest price you have seen so far, and the best profit so far. On each day, "sell today after buying at the cheapest price so far" is today's price minus that minimum; if that beats the best, write it down. Then, if today is cheaper than the minimum, update the minimum.

day:	0	1	2	3	4	5	
price:	7	1	5	3	6	4	
min so far:	7	1	1	1	1	1	
best:	0	0	4	4	5	5	-> 5

The idea. `minPrice = prices[0]`, `best = 0`. For each later price `p`: `best = max(best, p - minPrice)` and then `minPrice = min(minPrice, p)`. The order matters: update `best` first, so you never "sell" on the same day you set the minimum.

The Python you would have written.

```
def maxProfit(prices):
    min_price = prices[0]
    best = 0
```

```

for p in prices[1:]:
    best = max(best, p - min_price)
    min_price = min(min_price, p)
return best

```

In C++.

```

// LeetCode 121. Best Time to Buy and Sell Stock
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// One pass: the best sale at day i uses the cheapest buy seen so far.
// Track minPrice and the best profit; never sell before you buy.
int maxProfit(std::vector<int>& prices) {
    int minPrice = prices[0];
    int best = 0; // "no transaction" is always allowed
    for (size_t i = 1; i < prices.size(); ++i) {
        best = std::max(best, prices[i] - minPrice);
        minPrice = std::min(minPrice, prices[i]);
    }
    return best;
}

int main() {
    std::vector<int> a = {7, 1, 5, 3, 6, 4};
    assert(maxProfit(a) == 5); // buy 1, sell 6

    std::vector<int> b = {7, 6, 4, 3, 1};
    assert(maxProfit(b) == 0); // monotone decreasing: never buy

    std::vector<int> c = {5};
    assert(maxProfit(c) == 0); // single day

    std::vector<int> d = {2, 4, 1};
    assert(maxProfit(d) == 2); // the later min (1) must not be paired with earlier 4

    std::cout << "OK 08_best_time_to_buy_and_sell_stock.cpp\n";
    return 0;
}

```

What is new: nothing beyond problem 6. `std::max` and `std::min` from `<algorithm>`, an index loop from 1, and `prices[i] - minPrice` as the sell-today profit.

Walk through the example. `minPrice = 7`, `best = 0`. Day 1, price 1: profit `1 - 7 = -6`, `best` stays 0; `min` becomes 1. Day 2, price 5: profit 4, `best = 4`. Day 3, price 3: profit 2, no change. Day 4, price 6: profit 5, `best = 5`. Day 5, price 4: profit 3, no change. Return 5.

Complexity. One pass, two variables: $O(n)$ time, $O(1)$ memory.

Robotics. "Current value minus the running minimum" computed online is the largest rise since the lowest point so far: the worst altitude gain after a dip in a barometer stream, or the largest drift excursion in an odometry error signal, without storing the history.

Follow-ups you may get. - Allowed to buy and sell many times (LeetCode 122)? Add up every positive day-to-day difference. - Return the buy day and sell day? Remember the index of `minPrice`, and copy it (plus today's index) whenever `best` improves. - Why not two nested loops trying every pair? Correct but $O(n^2)$; 100,000 days is ten billion pairs.

9. Valid Parentheses (LeetCode 20, easy)

The problem in plain words. A string made only of the six characters `(, ), [, ], {, }`. Return `true` if every opening bracket is closed by the matching kind of bracket, in the right order (the most recently opened bracket must be the first one closed). Otherwise `false`.

```
"()"      -> true
"()[{}]"  -> true
"([])"    -> true      [ ] is closed inside ( )
"()"      -> false      ( is closed by ]
"([])"    -> false      [ is still open when ) arrives
"("       -> false      never closed
""        -> true       nothing to check
```

Up to 10,000 characters.

By hand. Read left to right and keep a pile of "brackets still open". An opener goes on top of the pile. A closer must match the bracket on top of the pile; if it does, take that opener off the pile; if it does not (or the pile is empty), the string is bad. At the end the pile must be empty.

```
"([])":  ( -> pile: (
          [ -> pile: ( [
          ] -> top is [, matches, remove -> pile: (
          ) -> top is (, matches, remove -> pile: empty -> true

"{}":    ( -> pile: (
          ] -> top is (, does not match ] -> false
```

The idea. A stack is a pile of plates: you put a plate on top and you take from the top, so the last one on is the first one off. For each character: if it is a closer, check the top of the stack is the matching opener and pop it, else return `false`; if it is an opener, push it. Return whether the stack is empty at the end.

The Python you would have written.

```
def isValid(s):
    close_to_open = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in s:
        if ch in close_to_open:           # a closer
            if not stack or stack[-1] != close_to_open[ch]:
                return False
            stack.pop()
        else:                             # an opener
            stack.append(ch)
    return not stack
```

In C++.

```
// LeetCode 20. Valid Parentheses
#include <cassert>
#include <iostream>
#include <stack>
#include <string>
#include <unordered_map>

// Stack of pending openers. A closer must match the most recent opener.
bool isValid(std::string s) {
    static const std::unordered_map<char, char> closeToOpen = {
        {')', '('}, {'}', '{'}, {']', '['}
    };
    std::stack<char> st;
    for (char c : s) {
        auto it = closeToOpen.find(c);
        if (it == closeToOpen.end()) {
            st.push(c);           // opener
        } else {
            st.pop();
        }
    }
    return st.empty();
}
```

```

        if (st.empty() || st.top() != it->second) return false;
        st.pop(); // std::stack::pop() returns void – read top() first
    }
}
return st.empty(); // leftover openers mean unbalanced
}

int main() {
    assert(isValid("(") == true);
    assert(isValid("()[{}]" == true);
    assert(isValid("]") == false);
    assert(isValid("([])" == true);
    assert(isValid("") == true); // empty string is valid
    assert(isValid("(") == false); // unmatched opener
    assert(isValid(")") == false); // closer with empty stack
    assert(isValid("([])" == false); // interleaved
    std::cout << "OK 09_valid_parentheses.cpp\n";
    return 0;
}

```

What is new:

- `static const std::unordered_map<char, char> closeToOpen = {...};` : the same dictionary as Python, from closing character to opening character. `const` because it never changes; `static` so it is built once, the first time the function runs, not on every call.
- `std::stack<char> st;` : a stack of characters, from `#include <stack>`. `push` puts one on top, `top()` looks at the top, `pop()` removes the top. `pop()` returns nothing, so you always `top()` first if you need the value. Python's list with `append` and `pop()`.
- `auto it = closeToOpen.find(c); if (it == closeToOpen.end())` : Python's `if ch in close_to_open` turned around: `end()` means "not a closer, so an opener".
- `if (st.empty() || st.top() != it->second) return false;` : the two ways a closer can fail: nothing to close, or the wrong thing on top. `it->second` is the matching opener.
- `return st.empty();` : Python's `return not stack`.

Walk through "([])". (: push, stack (. [: push, stack ([.) : it is a closer whose opener is (; top is [; mismatch; return false.

Complexity. One pass, each character pushed and popped at most once: $O(n)$ time, $O(n)$ memory for the stack.

Robotics. A stack of pending openers is a tiny parser. Checking that `<link>` and `<joint>` tags nest properly in a URDF, or that a behaviour-tree XML has matching begin and end markers, is this loop with tag names instead of brackets.

Follow-ups you may get. - *Only one kind of bracket?* A counter is enough: add one for (, subtract for), fail if it ever goes negative, succeed if it ends at 0. - *Minimum insertions to make it valid?* Same stack; count closers that find nothing to match, then add the openers left on the stack. - *What happens if you call `st.top()` on an empty stack?* Undefined behaviour, usually a crash. That is why `st.empty()` is checked first.

10. Binary Search (LeetCode 704, easy)

The problem in plain words. A list of whole numbers sorted from small to large, and a `target`. Return the position of `target` in the list, or -1 if it is not there. Do it in $O(\log n)$ time, meaning: halve the amount you still have to look at with every step.

```

nums   = [ -1,  0,  3,  5,  9, 12 ]    target = 9    ->  4
index  =  0   1   2   3   4   5

same list, target = 2                    -> -1 (not present)

```

```
target = -1 -> 0      target = 12 -> 5      (the ends)
[]          -> -1     [5], target 5 -> 0
```

Up to 10,000 numbers, all different.

By hand. Think of finding a word in a dictionary. Open in the middle. If the word you want comes later, ignore the whole first half; if earlier, ignore the whole second half. Open the middle of what is left. Each look throws away half of what remains, so 10,000 entries take about 14 looks.

```
target 9:
lo=0 hi=5 mid=2 nums[2]=3 < 9 -> 9 is to the right: lo = 3
lo=3 hi=5 mid=4 nums[4]=9 == 9 -> found at 4

target 2:
lo=0 hi=5 mid=2 nums[2]=3 > 2 -> 2 is to the left: hi = 1
lo=0 hi=1 mid=0 nums[0]=-1 < 2 -> right: lo = 1
lo=1 hi=1 mid=1 nums[1]=0 < 2 -> right: lo = 2
lo=2 hi=1 lo > hi: nothing left -> -1
```

The idea. `lo = 0`, `hi = n - 1`, the range still possible (both ends included). While `lo <= hi`: `mid` is the middle; if `nums[mid]` is the target return `mid`; if smaller, the target is right of `mid` so `lo = mid + 1`; if larger, `hi = mid - 1`. If the loop ends, `-1`.

The Python you would have written.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

In C++.

```
// LeetCode 704. Binary Search
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Classic closed interval [lo, hi]. Loop while lo <= hi.
int search(std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size()) - 1; // cast first: size() - 1 on empty wraps
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // (lo + hi) / 2 can overflow int
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

// The lower_bound idiom: first index whose value is >= target, in [0, n].
// Half-open [lo, hi); loop while lo < hi. This is what you actually reuse in
// "first true" / "insert position" / "count of elements < x" questions.
int lowerBound(const std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size());
    while (lo < hi) {
```

```

    int mid = lo + (hi - lo) / 2;
    if (nums[mid] < target) lo = mid + 1;
    else hi = mid;
}
return lo;
}

// upper_bound: first index whose value is > target. Only the comparison changes.
int upperBound(const std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

int main() {
    std::vector<int> a = {-1, 0, 3, 5, 9, 12};
    assert(search(a, 9) == 4);
    assert(search(a, 2) == -1);
    assert(search(a, -1) == 0);           // first element
    assert(search(a, 12) == 5);          // last element

    std::vector<int> empty;
    assert(search(empty, 1) == -1);       // empty: hi = -1, loop never runs
    std::vector<int> one = {5};
    assert(search(one, 5) == 0);
    assert(search(one, 4) == -1);

    // lower/upper bound vs the STL versions
    std::vector<int> b = {1, 2, 2, 2, 5, 7};
    assert(lowerBound(b, 2) == 1);
    assert(upperBound(b, 2) == 4);
    assert(upperBound(b, 2) - lowerBound(b, 2) == 3); // count of 2s
    assert(lowerBound(b, 3) == 4);                     // insert position for 3
    assert(lowerBound(b, 100) == 6);                    // == size when nothing is >= target
    assert(lowerBound(b, 0) == 0);
    assert(std::lower_bound(b.begin(), b.end(), 2) - b.begin() == 1);
    assert(std::upper_bound(b.begin(), b.end(), 2) - b.begin() == 4);
    assert(std::binary_search(b.begin(), b.end(), 5) == true);

    std::cout << "OK 10_binary_search.cpp\n";
    return 0;
}

```

What is new:

- `int hi = static_cast<int>(nums.size()) - 1;` : cast *before* subtracting. On an empty list `nums.size() - 1` would roll over to a huge number (c1 skip paragraph); casting first gives -1, so `lo <= hi` is false and the loop never runs.
- `int mid = lo + (hi - lo) / 2;` : Python's `(lo + hi) // 2`, rearranged so `lo + hi` is never computed. With both near 2 billion, `lo + hi` would overflow an `int`; `hi - lo` cannot.
- `lowerBound` and `upperBound` are a second shape of the same search that you will reuse constantly: instead of "where is target", they answer "where is the first number that is `>= target`" (or `> target`). They use a half-open range, `hi = n` and `while (lo < hi)`, and return `n` if no such number exists. The `main` checks them against the library's `std::lower_bound` and `std::upper_bound`, which do the same thing on any sorted vector.

Walk through target 9. `lo = 0, hi = 5. mid = 0 + 5 / 2 = 2, nums[2] = 3 < 9, lo = 3. mid = 3 + 2 / 2 = 4, nums[4] = 9, return 4.` Two looks for six numbers.

Complexity. The range halves each step: $O(\log n)$ time. Three ints: $O(1)$ memory.

Robotics. `lower_bound` on a sorted vector of timestamps finds the pose recorded just before a lidar scan's stamp, so you can interpolate. A trajectory follower uses the same call to find which segment of a time-stamped path the current time falls in. You will type this loop from memory many times.

Follow-ups you may get. - *First and last position of a value that may repeat (LeetCode 34)?*

`lowerBound(target)` and `upperBound(target) - 1`. - *Why not $(lo + hi) / 2$? It overflows when `lo + hi` passes 2,147,483,647.* `lo + (hi - lo) / 2` gives the same `mid` safely. - *What goes wrong if you mix the two shapes?* `while (lo <= hi)` with `hi = n` reads past the end; `while (lo < hi)` with `hi = mid - 1` can skip the answer. Pick one shape and stick to it.

Part C — Check yourself

Answer each one in plain words, without looking, before you mark this chapter done.

1. What are the two steps to get a C++ program running, and what does `g++` produce?
2. What is `-7 / 2` in C++, and what is `-7 // 2` in Python? Why are they different?
3. `int a = 100000, b = 100000; long long p = a * b;` gives the wrong answer. Why, and what is the one-word fix in the middle of that line?
4. Draw the two boxes for `void f(int x)` and `void f(int& x)`. Which one lets `f` change the caller's variable?
5. When would you write `const std::vector<int>&` as a parameter instead of `std::vector<int>?`
6. `for (auto row : grid) row[0] = 1;` leaves `grid` unchanged. Why? What is the fix?
7. Write the line that makes a grid of 3 rows and 4 columns, all zero, and explain each part of it.
8. What is the difference between `std::vector<int> v(5, 0)` and `std::vector<int> v{5, 0}?`
9. What does `s.substr(2, 3)` give for `s = "hello world"`, and what are its two arguments?
10. What does `s.find('z')` return when there is no `z`, and why is `if (s.find('z') >= 0)` always true?
11. Why is `'7' - '0'` equal to 7? What number is `'a'`?
12. `ages["bob"]` when there is no `"bob"` in the map: what happens? How do you check for a key without that side effect?
13. In a linked list, what do `ListNode*`, `->` and `nullptr` mean? What does `head->next->val` read?
14. In Reverse Linked List, why must you save `cur->next` before writing `cur->next = prev?`
15. Write `mid` for binary search so it cannot overflow, and say what `static_cast<int>(nums.size()) - 1` protects against.

02. The C++ everyone asks about: memory, objects, and the STL

This chapter is about a small set of C++ ideas that show up in every piece of robot code: where a variable lives, who is allowed to change it, who owns it and who throws it away, and the handful of standard containers you reach for every day. On a robot these are not book knowledge. A variable that dies one line too early is a crash in the middle of a corridor, and a loop that never ends is a motor that never stops. Part A teaches each idea with a drawing first. Part B is eight array, interval and hashing problems where you use those ideas for real.

Rough time: 4 days at about 2 hours a day. Day 1 is Part A only; do not start the problems until you can draw the memory pictures in m1, m4 and m5 from memory.

What you need before this chapter (all in chapter 01): variables and types, a function that takes `int&`, `std::vector`, `std::string`, `std::unordered_map`, `std::pair`, `auto [a, b] = ...`, and how to compile with `g++`. If any of those look foreign, go back to chapter 01 first.

Compile anything in this chapter with:

```
g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o /tmp/x && /tmp/x
```

Part A — Memory, objects, and the STL, from zero

m1. Pointers vs references; what `++a` inside `void f(int a)` does

The picture. The computer's memory is a long row of numbered boxes. The number on a box is its *address*. A variable is a box with a name tag on it.

address:	100	101	102	103	104
	5				
name:	a				

`int a = 5;` made the box at address 100, wrote 5 in it, and stuck the name tag `a` on it. (Real addresses are big numbers like 140732920127452, and a real `int` takes four boxes. Small numbers and one box per value are fine for thinking.)

A pointer is a box that holds another box's number. `int* p = &a;` makes a new box, and writes *the address of a* into it.

address:	100	101	102	
	5		100	
name:	a		p	

`p` holds the number 100, which is where `a` lives

`*p = 6;` means "go to the box whose number is in `p`, and write 6 there". Since `p` holds 100, that is box `a`:

address:	100	101	102	
	6		100	
name:	a		p	

`a` changed, `p` did not

A reference is a second name tag stuck on the same box. `int& r = a;` makes no new box at all:

```
address:    100
           +-----+
           |   6   |           one box, two name tags: a and r
           +-----+
name:      a, r
```

After that, `r = 7` and `a = 7` do exactly the same thing, because they name the same box.

The symbols, one by one.

- `&a` when `a` is a variable: "the address of `a`", the number written on `a`'s box.
- `int*` in a type: "a pointer to an `int`", a box that holds an address.
- `*p` when `p` is a pointer: "go to that address". Reading `*p` reads the box `p` points at; writing `*p = 6` writes into it.
- `int&` in a type: "a reference to an `int`", a second name for an existing box. You use `r` exactly like `a`, no star needed.
- `p->x` when `p` points at an object with a field `x`: short for `(*p).x`, "go to the object and take its `x`".
- `nullptr`: the special address that means "points at nothing". A pointer can hold it; a reference cannot. Before you use `*p` on a pointer that might be `nullptr`, check `if (p)`.

All of that in code:

```
int a = 5;
int* p = &a;    // p holds a's address
*p = 6;         // a is now 6
int& r = a;     // r is another name for a
r = 7;          // a is now 7, and so is *p
int* nothing = nullptr;
if (nothing) { *nothing = 1; } // skipped: nothing points at nothing
```

The ++a question. This is the one everybody gets asked and half of people get wrong:

```
void f(int a) { ++a; }

int x = 1;
f(x);
// what is x now?
```

The answer is `1`. Draw it. When `f(x)` is called, `int a` (no `&`, no `*`) means "make a *new* box called `a` and copy the value of `x` into it":

caller's boxes			f's own boxes (made when f starts, thrown away when f returns)	
x:	+-----+ 1 +-----+	copy 1 -->	a:	+-----+ 1 +-----+
				++a makes this box 2
				...then f returns and this box is thrown away
x:	1 unchanged			

`++a` adds one to *f*'s private copy. Nothing ever touches `x`'s box. Then `f` returns and the copy is gone.

If you want the caller to see the change, say so in the signature:

```
void g(int& a) { ++a; } // a is a second name tag on the caller's box
void h(int* a) { if (a) ++*a; } // a holds the caller's box number; ++*a goes there and adds one

int x = 1;
f(x); // x == 1 copy
g(x); // x == 2 reference: g's a IS x
```

```
h(&x);    // x == 3    pointer: we hand over x's address with &x
h(nullptr); // safe, h checks before using
```

The Python you know. In Python `def f(a): a += 1` on an int does not change the caller either, but `def f(lst): lst.append(1)` does. In Python the difference depends on what *kind* of object is passed. In C++ the difference is written in the function's signature: `int` copies, `int&` aliases, `int*` passes an address. You can read the signature and know, without looking at the type of the thing.

Pointer or reference? Three rules that separate them:

1. A reference must be given a box the moment it is born (`int& r;` on its own is an error). A pointer can be born empty (`int* p = nullptr;`).
2. A reference can never be moved to a different box. `r = y;` does *not* re-stick the name tag on y's box; it copies y's value into the box r already names. A pointer can be re-aimed any time: `p = &y;`.
3. A reference can never be "nothing", so you never have to check it. A pointer can be `nullptr`, so you must.

So: use a reference when the thing must exist (most function parameters). Use a pointer when "there is nothing here" is a real possibility, or when you need to re-aim it later.

Try it. In `00_memory_demo.cpp`, change the line `byRef(x);` to `byValue(x);` and predict which `assert` fails before you compile.

In your own words: "A pointer is a box holding another box's address; `&` gets an address, `*` follows one. A reference is a second name for the same box, so it can never be empty or moved. `++a` inside `void f(int a)` changes a private copy; the caller sees nothing unless the parameter is `int&` or `int*`."

m2. const and const-correctness (const&, const methods)

The picture. `const` is a sticker that says "look but do not touch". You can stick it on a box, on a name tag, on a pointer, or on a button of a class.

```
limit:    +-----+
          | 10   | [const]    reading limit is fine; limit = 11 is a compile error
          +-----+
```

`const int limit = 10;` makes a box you can read but never write. The compiler refuses the write *before the program runs*, which is far better than finding out on the robot.

const T& parameters: a name tag with a sticker. Take a function that counts letters in a string. Two ways to write it:

```
std::size_t countA(std::string s)           // copies the WHOLE string every call
std::size_t countA(const std::string& s)    // a second name tag on the caller's string, read-only
```

Draw the second one:

```
caller's box                inside countA
text:    +-----+
          | "banana" | <-- s is a name tag on this same box, with a [const] sticker:
          +-----+                countA can read every letter, but s[0] = 'x' will not compile
```

No copy is made, so it is fast even for a string of a million characters, and the sticker tells whoever reads the signature "this function will not change your string". That is why almost every function in this tutorial takes big things (`std::string`, `std::vector`, matrices) as `const T&`. For tiny things like `int` and `double`, just copy them; the copy is one box.

```
std::size_t countA(const std::string& s) {
    std::size_t n = 0;
```

```

    for (char c : s) if (c == 'a') ++n;
    return n;
}
// countA("banana") == 3

```

const methods: a sticker on a button. A class (m3 explains classes properly) is a box with compartments and buttons. Writing `const` after a method's parameter list promises "pressing this button does not change any compartment":

```

class Pose2D {
public:
    Pose2D(double x, double y) : x_(x), y_(y) {}
    double x() const { return x_; }           // const button: only reads x_
    void translate(double dx) { x_ += dx; }    // ordinary button: changes x_
private:
    double x_, y_;
};

const Pose2D home(0.0, 0.0); // a Pose2D with a [const] sticker on the whole box
double hx = home.x();       // fine: x() promised not to change anything
// home.translate(1.0);     // compile error: translate() made no such promise

```

The rule that follows: on a `const` object, or through a `const&`, you can *only* press `const` buttons. So every method that just reads (`x()`, `size()`, `empty()`, `getMin()`) must be marked `const`, or nobody can call it on a `const Pose2D&`. When that error hits you, the fix is to add `const` to the method, never to remove `const` from the parameter.

const before the star vs after the star. With pointers there are two things you could protect: the box `p` points at, or `p` itself.

<pre>const int* p = &a;</pre>		"pointer to a const int" <code>p</code> may be re-aimed, but you cannot write through it
<pre> p +-----+ 100 -----> +-----+ </pre>	<pre> a +-----+ 5 +-----+ </pre>	[const via p] <code>*p = 7</code> is an error; <code>p = &b</code> is fine
<pre>int* const p = &a;</pre>		"const pointer to an int" you may write through <code>p</code>, but <code>p</code> is glued to box <code>a</code>
<pre> p +-----+ 100 ==glued==> +-----+ </pre>	<pre> a +-----+ 5 +-----+ </pre>	<code>*p = 7</code> is fine; <code>p = &b</code> is an error

The trick to read any of these: read the declaration from right to left. `const int* p` reads "p is a pointer to an int that is const". `int* const p` reads "p is a const pointer to an int".

```

int a = 5, b = 9;
const int* ptrToConst = &a; // cannot do *ptrToConst = 7; can do ptrToConst = &b
int* const constPtr = &a;  // can do *constPtr = 7; cannot do constPtr = &b
*constPtr = 7;             // a is now 7
ptrToConst = &b;           // now looks at b

```

The Python you know. Python has no `const`. You rely on convention: a function "should not" change the list you pass it. C++ makes the promise part of the type, and the compiler checks it.

(You can skip this on a first read.) Sometimes a `const` method needs to update one compartment that is not really part of the object's state, such as a cache hit counter or a mutex. Mark that one member `mutable` and the compiler allows it inside `const` methods. `00_memory_demo.cpp` has a `hits()` method that does exactly this.


```

time --->
{   Door a("A")   Door b("B")   log.push_back("work")   }
    |             |             |
    a is made     b is made     the block ends:
    "open A"      "open B"      b is thrown away -> "close B"
                                then a           -> "close A"

```

Two things to notice. The destructors ran without anyone calling them. And they ran in *reverse* order: the last thing made is the first thing thrown away, like taking plates off a pile.

RAII: the library book that returns itself. Imagine a library where every book you pick up walks back to its shelf by itself the moment you leave the room. You could never forget to return one. That is RAII (the name is "Resource Acquisition Is Initialisation", which nobody finds helpful; think "the destructor cleans up"). The pattern is: take the resource in the constructor, give it back in the destructor. Then the resource is always returned, on every way out of the block, including an early `return` and an exception.

You already use RAII without knowing it:

- `std::vector` takes memory in its constructor and gives it back in its destructor.
- `std::ifstream` opens the file in its constructor and closes it in its destructor.
- `std::lock_guard` locks a mutex in its constructor and unlocks it in its destructor.
- The smart pointers in m4 delete the object they own in their destructor.

The Python you know. `__init__` is the constructor. Python has `__del__` but you cannot rely on *when* it runs, so Python invented with `open(...)` as `f`: to get guaranteed cleanup. In C++, every object is inside an invisible with block: the cleanup happens at the `}`.

Rule of zero, in plain words. If every compartment of your class is something that already cleans up after itself (`std::string`, `std::vector`, `std::unique_ptr`, plain numbers), then do not write a destructor at all. The compiler writes one that throws away each compartment in turn, and it is correct. Most classes you write should have zero of: destructor, copy constructor, copy assignment, move constructor, move assignment. Only a class that holds a raw resource (a file handle from a C library, a raw `new`) needs to write them, and then it needs all five.

(You can skip this on a first read.) If a class has any `virtual` method and might be deleted through a pointer to its base class (a `std::unique_ptr<Sensor>` that really holds a `Lidar`), the base class destructor must be declared `virtual ~Sensor() = default;`. Otherwise only the base part is thrown away and the derived part leaks silently. -Wall -Wextra does not warn about this. `00_memory_demo.cpp` shows the correct form.

Try it. In `00_memory_demo.cpp`, add a third `ScopeLog c("C", log);` after `b` and write down the seven log entries you expect, in order, before running it.

In your own words: "A class is a box with compartments and buttons. The constructor runs when the box is made and fills the compartments through the `init` list; the destructor runs by itself when the box goes out of scope, in reverse order of creation. RAII means the destructor returns whatever the constructor took, so cleanup cannot be forgotten. If all my members clean up after themselves, I write no destructor: rule of zero."

m4. `std::unique_ptr` vs `std::shared_ptr`, when to use each

The picture: who owns the toy. An object on the heap (m5 explains the heap; for now: an object that lives outside any function's boxes) is a toy on the floor. Somebody has to be responsible for putting it in the bin when it is no longer needed. C++ makes you say who.

`std::unique_ptr`: one owner. Exactly one child holds the toy. When that child leaves the room (the owner goes out of scope), the toy goes in the bin, automatically, by the destructor (m3). You cannot photocopy a `unique_ptr`; you can only *hand it over* with `std::move`, after which the old holder's hands are empty (`nullptr`).

auto u = std::make_unique<Toy>();	u ---> [Toy]	u owns it
auto v = std::move(u);	u ---> (nothing)	handed over
	v ---> [Toy]	v owns it now
} v leaves the room	[Toy] goes in the bin	

std::shared_ptr: a counter of owners. The toy has a counter sticker: how many children are holding it right now. Every new holder adds one, every leaver subtracts one, and when the counter hits zero the toy goes in the bin.

auto s1 = std::make_shared<Toy>();	[Toy count 1]	held by: s1	
{			
auto s2 = s1;	[Toy count 2]	held by: s1 s2	
}	[Toy count 1]	held by: s1	s2 left the room
s1.reset();	[Toy count 0]	-> bin	s1 let go

std::weak_ptr: may look, does not count. A child who is allowed to look at the toy but is not holding it, so they do not keep it out of the bin. Before using it they must ask "is it still there?" with `w.lock()`, which gives a `shared_ptr` (count goes up by one while they use it) or `nullptr` if the toy is gone. `w.expired()` is the yes/no version of the same question.

All of it in code:

```
#include <memory>

struct Toy { int id = 0; };

std::unique_ptr<Toy> u = std::make_unique<Toy>(); // build the toy and hand me the only owner
u->id = 7; // -> works through smart pointers too
std::unique_ptr<Toy> v = std::move(u); // v owns it now; u == nullptr
// v is thrown away at the end of its block, and the Toy with it

std::shared_ptr<Toy> s1 = std::make_shared<Toy>(); // count 1
{
    std::shared_ptr<Toy> s2 = s1; // count 2
} // s2 gone: count 1
std::weak_ptr<Toy> w = s1; // looks, does not count
bool alive = !w.expired(); // true
s1.reset(); // count 0: Toy is deleted
alive = !w.expired(); // false: w knows it is gone
```

The symbols.

- `std::make_unique<Toy>()` and `std::make_shared<Toy>()`: build a `Toy` on the heap and give you the owner in one step. The arguments in the brackets go to `Toy`'s constructor. You never write `new` yourself.
- `std::move(u)`: "I am done with u, take what it holds". After it, `u` is empty.
- `u->id`: same arrow as a raw pointer (`m1`).
- `u.get()`: the raw address inside, for lending to a function that only *uses* the toy.
- `s1.use_count()`: read the counter. `s1.reset()`: let go.

Which one? `unique_ptr` is the default. It costs exactly the same as a raw pointer, and the ownership is obvious from reading the code. Reach for `shared_ptr` only when you honestly cannot say at compile time who will be the last one using the object: a point cloud handed to three processing stages that finish in any order, a message fanned out to several subscribers. `weak_ptr` is for a child that points back at its parent, so the two do not keep each other alive forever.

Borrowing is fine. A function that only uses the toy and never bins it should take a plain reference (`const Toy&`) or a raw pointer (`Toy*`). Raw pointers are not evil; raw *owning* pointers are, because nothing bins the toy for you. Lend with `u.get()` or `*u`.

The Python you know. Every Python variable is a `shared_ptr`: the object lives while anything refers to it, and a counter decides when it dies. C++ gives you that with `shared_ptr`, but asks you to prefer the cheaper and clearer single owner when you can.

Robotics. A sensor driver object is owned by exactly one node (`unique_ptr`), and its callbacks get a reference to it. In ROS 2 a node is usually a `std::shared_ptr<rcpp::Node>` because both the executor and your code need to keep it alive, and a subscription receives each message as `std::shared_ptr<const Msg>` so one buffer fans out to many callbacks with no copying.

(You can skip this on a first read.) Two traps. Making two `shared_ptr`s from the *same raw pointer* (`std::shared_ptr<T> a(p); std::shared_ptr<T> b(p);`) creates two separate counters, and the toy is binned twice. And using `u` after `std::move(u)` dereferences a `nullptr`. Also, every copy of a `shared_ptr` is an atomic add on the counter, cheap but not free.

Try it. In `00_memory_demo.cpp`, add a second `std::shared_ptr<Sensor> s3 = s1;` inside the inner block and predict the two `use_count()` values.

In your own words: "`unique_ptr` means one owner; it is handed over with `std::move` and the object is deleted when the owner goes out of scope. `shared_ptr` keeps a count of owners and deletes at zero; `weak_ptr` watches without counting. I create both with `make_unique` and `make_shared`, use `unique_ptr` by default, and lend the object to functions as a reference or raw pointer."

m5. Stack vs heap, new/delete and why you avoid them

The picture. There are two places a box can live.

The stack is a pile of trays. Every time a function is called, a tray is put on top of the pile with that function's local boxes on it. When the function returns, the tray is lifted off and every box on it is gone. No bookkeeping, no cleanup, and very fast: putting on a tray is one subtraction.

main() calls f()	f() calls g()	g() returns	f() returns
+-----+	+-----+	+-----+	+-----+
f: a	g: y	f: a	
main: x	f: a	main: x	main: x
+-----+	+-----+	+-----+	+-----+
	g's tray on top	g's tray taken off	f's tray taken off

Every `int a` or `Pose2D p` you declare inside a function lives on its tray.

The heap is a big warehouse. When you need a box that must outlive the current function, or whose size you only know while the program runs (a vector of however many points the lidar returned), you ask the warehouse for a shelf. The old way was `new`: it hands you a ticket (a pointer) to the shelf. When you are done you must hand the ticket back with `delete`. Forget, and the shelf stays taken forever: a *memory leak*. Hand the same ticket back twice and the warehouse corrupts its records: a *double free*.

```
int* raw = new int(5);    // ask the warehouse for one int, write 5 in it; raw is the ticket
// ... use *raw ...
delete raw;               // hand the ticket back. Forget this and the shelf is lost forever.
raw = nullptr;            // the ticket is void now; make that visible
```

Why returning a reference to a local is wrong. The tray is gone.

```
// int& bad() { int x = 1; return x; }    // never write this
```

during bad()	after bad() returns
+-----+	+-----+
bad: x=1 <-- ref	<-- the reference still points here,

```
| main |
+-----+
```

```
| main |
+-----+
```

but the tray is gone. Reading it gives garbage or a crash.

g++ -Wall warns about this one ("reference to local variable returned"). Returning by *value* is fine and is what you should do:

```
std::vector<int> good() {
    std::vector<int> v = {1, 2, 3};
    return v;           // handed to the caller, no copy in practice, nothing leaks
}
```

Where a vector really lives. This is the picture that makes everything click. A `std::vector<int> v(1000, 7)` declared in a function puts a *small* object on the tray (a pointer, a size, a capacity) and the 1000 ints on a shelf in the warehouse. The vector's destructor (m3) returns the shelf the moment the tray is lifted.

stack tray		heap warehouse
+-----+		+-----+ ... +-----+
v: ptr ----->		7 7 7 7 ... 7 1000 ints
size = 1000		+-----+ ... +-----+
capacity = 1000		returned to the warehouse by ~vector()
+-----+		

```
{
    std::vector<int> big(1000, 7); // 1000 ints on a shelf in the warehouse
    // big[999] == 7
}
// shelf handed back here, automatically
```

Why we let vector and unique_ptr do new and delete for us. Every *new* needs exactly one *delete* on every path out of the function: the normal end, each early *return*, and any exception. Humans get that wrong. `std::vector`, `std::string`, `std::make_unique` and `std::make_shared` do the *new* in their constructor and the *delete* in their destructor, and the destructor runs on every path (m3). So in modern C++ you almost never type *new* or *delete*. When you see them, it is old code, or a library with its own rules (Qt widgets), or embedded code that allocates everything at start-up and never frees.

The Python you know. Every Python object is on the heap and the garbage collector is the warehouse clerk who eventually notices nobody is using a shelf. C++ has no clerk. Destructors are the clerk, and they work because they run at a known moment.

(You can skip this on a first read.) Asking the warehouse for a shelf takes tens to hundreds of nanoseconds and can take a lock, and the time is not predictable. In a hard real-time control loop you therefore avoid *any* heap allocation after start-up: *reserve()* your vectors once, then only *push_back* within that capacity. Also, memory from *new[]* must be freed with *delete[]*, not *delete*; one more reason to leave it to `std::vector`.

Try it. In `00_memory_demo.cpp`, uncomment `int& bad()` and compile with `-Wall`. Read the warning.

In your own words: "Locals live on the stack, a pile of trays that are thrown away when the function returns, so a reference to a local is pointing at a tray that no longer exists. Big or long-lived data lives on the heap, which must be given back. I never write *new* and *delete* myself because *vector* and *unique_ptr* do them in a constructor and destructor that run on every path."

m6. size_t is unsigned: why while(--n >= 0) loops forever

The picture. `v.size()` does not give you an `int`. It gives a `std::size_t`, a number that *cannot be negative*. Think of a car's odometer with 20 digits and no minus sign. Drive forward and it counts up. Now imagine driving *backwards* from 0. There is no -1 to show, so the odometer rolls round to all nines... in binary, to the biggest number it can hold:

```
size_t n = 0;
--n;
```

```
...  <-  3  <-  2  <-  1  <-  0  <-  18446744073709551615  <-  18446744073709551614  <-  ...
                ^
            subtract 1 from 0 and you land at the far right end, not at -1
```

That number, 18446744073709551615, is `SIZE_MAX`, the largest `size_t`. It is roughly 18 quintillion.

The loop that never ends. Now read this with the odometer in mind:

```
std::vector<int> v = {1, 2, 3};
std::size_t n = v.size();
// while (--n >= 0) { use v[n]; }  <-- do not run this
```

`n` is 3, then 2, 1, 0 and the body runs for each: fine so far. Then `--n` from 0 rolls round to 18446744073709551615. Is that `>= 0`? Yes. Every `size_t` is `>= 0`; there is no other kind. So the loop never stops, and `v[18446744073709551615]` reads memory that is not yours and the program crashes. `-Wextra` does warn ("comparison of unsigned expression in `>= 0` is always true"), which is one of the reasons this tutorial always compiles with it.

The same odometer bites in `v.size() - 1` when `v` is empty:

```
std::vector<int> empty;
std::size_t wrapped = empty.size() - 1;    // 0 - 1 rolls round to 18446744073709551615
// for (std::size_t i = 0; i < empty.size() - 1; ++i) would run "forever" (it crashes first)
```

Two safe ways to loop backwards.

```
std::vector<int> v = {1, 2, 3};
int sum = 0;

// Way 1: turn the size into an int once, then use ordinary ints. This is the tutorial's habit.
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) sum += v[i];
// sum == 6

// Way 2: test first, then subtract. Read "i-- > 0" as "while i is still above 0, step down".
sum = 0;
for (std::size_t i = v.size(); i-- > 0;) sum += v[i];
// sum == 6
```

Way 2 works because the comparison `i > 0` happens *before* the `--`. When `i` is 1 the test passes, `i` becomes 0, the body sees `v[0]`. When `i` is 0 the test fails and the loop ends before any roll-round can happen.

The rule this whole tutorial follows. At the top of a function write `const int n = static_cast<int>(v.size());` and use `int` for every index after that. `static_cast<int>(...)` is C++ for "convert this to an int, and yes I mean it". Problem sizes on LeetCode fit in an `int` comfortably. And never write `size() - 1` without first checking `empty()`.

The Python you know. Python integers never roll round; `0 - 1` is `-1` and `range(len(v) - 1, -1, -1)` just works. That is why this trap only appears once you write C++.

(You can skip this on a first read.) Comparing an `int` with a `size_t`, as in `for (int i = 0; i < v.size(); ++i)`, compiles but warns under `-Wall -Wextra (-Wsign-compare)`, because the `int` is silently converted to unsigned first: a negative `i` becomes enormous and the comparison is wrong. Casting the size once to `int` avoids the warning and the surprise.

Try it. In `00_memory_demo.cpp`, change `int n = 3` in the last block to `std::size_t n = 3` and compile with `-Wall -Wextra`. Read the warning, then put it back. Do not run it.

In your own words: "`size()` returns an unsigned `size_t`, and subtracting one from zero rolls round to the biggest possible value instead of going negative, so `--n >= 0` is always true. I cast the size to `int` once at the top and loop with ints, or I use `for (size_t i = n; i-- > 0;)`, and I check for empty before writing `size() - 1`."

The memory demo file

Everything in m1 to m6 is in one file, as small asserts you can step through:

```
// 00_memory_demo.cpp – pointers vs references, const, RAII, unique_ptr/shared_ptr,
// stack vs heap, and the size_t trap (shown safely).
#include <cassert>
#include <cstdint>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ----- m1: pointers vs references -----
void byValue(int a) { ++a; }           // ++a touches a private copy; caller sees nothing
void byRef(int& a) { ++a; }           // alias of the caller's variable
void byPtr(int* a) { if (a) ++*a; }   // may be null, must be checked; *a is the pointee

// ----- m2: const-correctness -----
class Pose2D {
public:
    Pose2D(double x, double y) : x_(x), y_(y) {}           // member init list
    double x() const { return x_; }                       // const method: callable on a const Pose2D
    void translate(double dx) { x_ += dx; }                // non-const: mutates
    std::size_t hits() const { ++cacheHits_; return cacheHits_; } // mutable: bookkeeping in a const method
private:
    double x_, y_;
    mutable std::size_t cacheHits_ = 0;
};

// const T& – read-only view, no copy. Passing std::string by value would copy the buffer.
std::size_t countA(const std::string& s) {
    std::size_t n = 0;
    for (char c : s) if (c == 'a') ++n;
    return n;
}

// ----- m3: RAII -----
// The destructor runs when the object leaves scope, in every exit path (return, exception).
struct ScopeLog {
    ScopeLog(std::string name, std::vector<std::string>& log) : name_(std::move(name)), log_(log) {
        log_.push_back("open " + name_);
    }
    ~ScopeLog() { log_.push_back("close " + name_); }
    ScopeLog(const ScopeLog&) = delete; // a resource handle should not be copied
    ScopeLog& operator=(const ScopeLog&) = delete;
private:
    std::string name_;
    std::vector<std::string>& log_;
};

// ----- m4: ownership with smart pointers -----
struct Sensor {
    virtual ~Sensor() = default; // virtual: deleting through a base pointer must run the derived dtor
    virtual std::string name() const = 0;
};
struct Lidar : Sensor {
    std::string name() const override { return "lidar"; }
};

// Non-owning observer: a raw pointer is fine when the callee never deletes.
std::string describe(const Sensor* s) { return s ? s->name() : "none"; }

// ----- m5: stack vs heap -----
// Returning a reference to a local is a dangling reference – never do this.
// int& bad() { int x = 1; return x; } // g++ -Wall warns: reference to local variable
std::vector<int> good() { std::vector<int> v = {1, 2, 3}; return v; } // moved/elided, no copy, no leak

int main() {
    // m1
    int x = 1;
    byValue(x); assert(x == 1);
    byRef(x);   assert(x == 2);
    byPtr(&x);  assert(x == 3);
    byPtr(nullptr); // safe because byPtr checks
    int* p = &x; *p = 10; assert(x == 10);
    int& r = x; r = 11; assert(x == 11); // r cannot be reseated: "r = y" would assign into x
    Pose2D pose(1.0, 2.0);
    Pose2D* pp = &pose;
}
```

```

assert(pp->x() == 1.0);           // -> is (*pp).x()

// m2
const Pose2D cp(5.0, 0.0);
assert(cp.x() == 5.0);           // OK: x() is const
// cp.translate(1.0);           // compile error: non-const method on const object
assert(cp.hits() == 1);         // mutable member changed inside a const method
assert(countA("banana") == 3);
const int* ptrToConst = &x;     // cannot write *ptrToConst; can repoint
int* const constPtr = &x;       // can write *constPtr; cannot repoint
*constPtr = 12; assert(*ptrToConst == 12);

// m3
std::vector<std::string> log;
{
    ScopeLog a("A", log);
    ScopeLog b("B", log);
    log.push_back("work");
}
// destroyed in reverse order: B then A
assert((log == std::vector<std::string>{"open A", "open B", "work", "close B", "close A"}));

// m4
std::unique_ptr<Sensor> owner = std::make_unique<Lidar>(); // sole owner, zero overhead
assert(describe(owner.get()) == "lidar");                 // lend a non-owning view
std::unique_ptr<Sensor> other = std::move(owner);          // ownership transferred, not copied
assert(owner == nullptr && other->name() == "lidar");

std::shared_ptr<Sensor> s1 = std::make_shared<Lidar>();
{ std::shared_ptr<Sensor> s2 = s1; assert(s1.use_count() == 2); }
assert(s1.use_count() == 1);                               // s2 released its share
std::weak_ptr<Sensor> w = s1;                               // observes without owning
assert(!w.expired());
s1.reset();
assert(w.expired());                                       // weak_ptr knows the object is gone

// m5
assert(good().size() == 3);
{
    std::vector<int> heapBacked(1000, 7); // the vector's buffer is on the heap, freed by its dtor
    assert(heapBacked[999] == 7);
}
int* raw = new int(5); // legacy style: you now own a delete
assert(*raw == 5);
delete raw;           // forget this (or throw before it) and you leak
raw = nullptr;

// m6: size_t is unsigned
std::vector<int> empty;
std::size_t wrapped = empty.size() - 1; // 0 - 1 wraps to SIZE_MAX, not -1
assert(wrapped == static_cast<std::size_t>(-1));
// Safe backwards loops:
std::vector<int> v = {1, 2, 3};
int sum = 0;
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) sum += v[i]; // idiom 1: cast once
assert(sum == 6);
sum = 0;
for (std::size_t i = v.size(); i-- > 0;) sum += v[static_cast<int>(i)]; // idiom 2: test-then-decrement
assert(sum == 6);
// while (--n >= 0) with std::size_t n would never be false; with int n it is fine:
int n = 3, ticks = 0;
while (--n >= 0) ++ticks;
assert(ticks == 3);

std::cout << "OK 00_memory_demo.cpp\n";
return 0;
}

```

What to look at:

- `byValue`, `byRef`, `byPtr` at the top, and the three calls in `main()` that leave `x` at 1, 2, 3. That is `m1`'s drawing as code.
- `Pose2D`: the `const` method `x()`, the non-const `translate()`, and the commented-out line `cp.translate(1.0)` that would not compile. Also the mutable counter from the skip paragraph.
- `ScopeLog` and the `log` vector: the timeline from `m3`. Check the order "close B" then "close A".
- `Sensor` and `Lidar`: `unique_ptr` handed over with `std::move`, `shared_ptr` counting up and down inside a block, `weak_ptr` noticing when the object is gone.

- `good()` returning a vector by value, `heapBacked` living on a heap shelf, and the one `new/delete` pair in the file, there only to show what you are avoiding.
- The `size_t` block at the end: the rolled-round value, both safe backwards loops, and the `while (--n >= 0)` loop that is safe only because `n` is an `int`.

s1. `std::sort` with custom comparator / lambda

What a lambda is. A lambda is a small function without a name, written right where you need it. Instead of

```
bool bigger(int a, int b) { return a > b; }
// ... somewhere far away ...
std::sort(v.begin(), v.end(), bigger);
```

you write the function inline:

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

The shape is `[captures](parameters) { body }`. The square brackets are the strange part; the rest is an ordinary function with the name and the return type left out (the compiler works out the return type from the `return` line).

The square brackets: what does the lambda get to see? A lambda can use variables from the function around it, but you must say *how*:

- `[]` : the lambda uses nothing from outside. Most comparators are like this.
- `[k]` : the lambda gets its *own copy* of `k`, taken at the moment the lambda is written.
- `[&k]` : the lambda gets a *name tag on the real k* (a reference, `m1`).
- `[=]` : copy everything from outside that the body mentions. `[&]` : reference everything.

Draw the difference:

```
int k = 2;
auto timesK = [k](int a) { return a * k; }; // copies k: timesK has its own box holding 2
auto bumpK = [&k]() { ++k; };               // refers to k: bumpK's k IS main's k
bumpK();                                   // main's k is now 3
int r = timesK(3);                         // 6, not 9: timesK still holds its copy of 2
```

main's boxes +-----+ k 3 <-- bumpK writes here +-----+	timesK's private box +-----+ 2 copied when +-----+ the lambda was written	bumpK (no box of its own; just a name tag on main's k)
---	---	---

`auto` in front of a lambda is required: the lambda's type has no name you could write. When a comparator needs to look at a big vector from outside, capture it by reference (`[&cost]`) so the vector is not copied into the lambda.

What the comparator must answer. `std::sort` calls your lambda with two elements and expects the answer to one question: "*does a go before b?*" True means "a first". So `a < b` sorts ascending, `a > b` sorts descending.

Sort by the second field. This is the case you meet most: a list of pairs, sort by the number.

```
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
```

```
std::vector<std::pair<std::string, int>> people = {{"ann", 31}, {"bob", 25}, {"cy", 40}};
std::sort(people.begin(), people.end(),
    [](const auto& a, const auto& b) { return a.second < b.second; });
// people is now: bob 25, ann 31, cy 40
```

- `people.begin(), people.end()`: sort the whole vector (s4 explains what these are).
- `const auto& a`: "a read-only name tag on one element, whatever type it is". No copy.
- `a.second`: the number in the pair.

Without a comparator, `std::sort(people.begin(), people.end())` sorts pairs by first and, when those tie, by second. That is often exactly right, for example to sort (row, col) points.

The Python you know. `sorted(people, key=lambda p: p[1])`. Python's key gives back "the thing to sort by". C++'s comparator instead answers "is a before b?". Same idea, different question, and the C++ form lets you say "score descending, then name ascending" in one function.

The rule that keeps sort from crashing. The comparator must say "is a *before* b", never "is a before or equal to b". Write `<`, never `<=`. If you write `<=`, then asked "is a before a?" your lambda says yes, which is nonsense, and `std::sort` is allowed to walk off the end of the array. (The proper name for this rule is *strict weak ordering*. Remember it as: strictly before.)

(You can skip this on a first read.) `std::sort` is $O(n \log n)$ and may reorder elements that compare equal. If equal elements must keep their original order, use `std::stable_sort`. To sort by several keys in one line, compare `std::tie(a.score, a.frame) < std::tie(b.score, b.frame)`; see `00_stl_demo.cpp` for a descending-then-ascending version.

Try it. In `00_stl_demo.cpp`, change the descending comparator to `a >= b` and run it. It may work, may crash, may print garbage: that is what "allowed to do anything" looks like. Put it back.

In your own words: "A lambda is a function without a name, written where it is used; the square brackets say which outside variables it copies or refers to. A sort comparator answers 'does a go before b?' with a strict `<`, never `<=`. To sort pairs by the second field I compare `a.second < b.second`."

s2. `std::priority_queue` (min-heap via `greater<>`)

The picture. A priority queue is a pile where the biggest thing is always on top. You throw items onto the pile in any order; whenever you look, the top is the largest, and when you take the top off, the next largest rises to take its place.

push 3, then 1, then 4:

```
top -> [ 4 ]      top() is 4
        [ 3 ]      pop() removes the 4; now 3 is on top
        [ 1 ]
```

(Inside it is a clever tree called a heap, arranged so that push and pop each cost about $\log n$ steps and looking at the top costs one. You never see the tree; you only touch the top.)

```
#include <queue>

std::priority_queue<int> maxHeap;
for (int e : {3, 1, 4}) maxHeap.push(e);
// maxHeap.top() == 4
maxHeap.pop();      // returns nothing; look with top() first if you need the value
// maxHeap.top() == 3
```

Smallest on top: `std::greater<>`. Most robotics uses (Dijkstra, A*, "k closest") want the *smallest* on top. You get that by telling the pile to rank items backwards, using `std::greater<int>` as the third piece of the type:

```
#include <functional>    // std::greater

std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
for (int e : {3, 1, 4}) minHeap.push(e);
// minHeap.top() == 1
minHeap.pop();
// minHeap.top() == 3
```

Three things inside the angle brackets, in this order: the element type, the container the pile is built on (always `std::vector<...>`), and the comparison. `std::greater<int>` means "treat a as lower priority than b when $a > b$ ", so big numbers sink and the smallest floats to the top. If you forget the middle `std::vector<int>` it does not compile.

A pile of pairs sorts by the first element. Pairs compare by `first`, and only on a tie by `second`. So a pile of (distance, node) pairs with `std::greater` keeps the *closest node* on top. That is exactly the frontier of Dijkstra's algorithm (chapter 04).

```
using DistNode = std::pair<double, int>;    // (distance, node id)
std::priority_queue<DistNode, std::vector<DistNode>, std::greater<DistNode>> frontier;
frontier.push({2.5, 7});
frontier.push({0.5, 3});
frontier.push({1.0, 9});
// frontier.top() is {0.5, 3}: the smallest distance, and .second gives node 3
```

- `using DistNode = std::pair<double, int>;` gives a long type a short name, like a Python variable that holds a type.
- `{0.5, 3}` builds the pair on the spot.

The Python you know. `heapq` is always a min-heap on a list; to get a max-heap you push `-x`. C++ is the other way round: max by default, `std::greater` for min. Both have no "peek and pop in one call": in C++ read `top()`, then `pop()`.

(You can skip this on a first read.) For a custom ordering (a struct with a score), write a lambda and pass its type with `decltype(cmp)` and the lambda itself to the constructor: `std::priority_queue<T, std::vector<T>, decltype(cmp)> pq(cmp);`. The lambda's meaning is inverted compared with `std::sort`: returning true means "a sits *below* b". Also, there is no way to change an item's priority once it is in the pile; Dijkstra pushes a fresh entry instead and skips stale ones when they surface.

Try it. In `00_stl_demo.cpp`, push `{0.5, 1}` into `frontier` as well and predict which node is on top (hint: equal `first`, so `second` decides).

In your own words: "A priority queue is a pile with the biggest on top; push and pop are $\log n$, top is constant. For smallest-on-top I write `std::priority_queue<T, std::vector<T>, std::greater<T>>`. A pile of pairs ranks by the first element, so (distance, node) gives me the closest node on top."

s3. `std::deque`, `std::queue`, `std::stack`

The picture. You met two of these in chapter 03, A1b: a **queue** is the line at a shop (join at the back, served from the front) and a **stack** is a pile of plates (put on top, take from the top). The third, a **deque** ("double-ended queue", said "deck"), is a line where people may join or leave at *either* end, and where you may also point at the fifth person in line without walking there.

queue:	front [1 2 3] back	push at the back, pop from the front
stack:	[3] <- top	push on top, pop from the top
	[2]	
	[1]	
deque:	front [0 1 2] back	push or pop at both ends, plus <code>w[1]</code>

The table. Every operation you will use:

	<code>std::stack<T></code>	<code>std::queue<T></code>	<code>std::deque<T></code>
add	<code>push(x)</code> on top	<code>push(x)</code> at the back	<code>push_back(x)</code> , <code>push_front(x)</code>
remove	<code>pop()</code> from top	<code>pop()</code> from front	<code>pop_back()</code> , <code>pop_front()</code>
look	<code>top()</code>	<code>front()</code> , <code>back()</code>	<code>front()</code> , <code>back()</code> , <code>w[i]</code>
how many / any	<code>size()</code> , <code>empty()</code>	<code>size()</code> , <code>empty()</code>	<code>size()</code> , <code>empty()</code>
Python	<code>list.append</code> / <code>list.pop()</code>	<code>deque.append</code> / <code>deque.popleft</code>	<code>collections.deque</code>

```
#include <deque>
#include <queue>
#include <stack>

std::queue<int> q;
q.push(1); q.push(2);
// q.front() == 1
q.pop(); // returns nothing: read front() first if you need it
// q.front() == 2, q.size() == 1

std::stack<int> st;
st.push(1); st.push(2);
// st.top() == 2
st.pop();
// st.top() == 1

std::deque<int> w;
w.push_back(1); w.push_back(2); w.push_front(0);
// w.front() == 0, w.back() == 2, w[1] == 1
w.pop_front(); w.pop_back();
// w.size() == 1, w[0] == 1
```

Two habits to build. First, `pop()` gives nothing back on all three (and on `priority_queue`). Look with `front()` or `top()`, then `pop()`. Second, looking at an empty one (`q.front()` when `q.empty()`) is not an error you can catch; it reads garbage or crashes. Check `empty()` first.

Which one when? `queue` for BFS (chapter 03). `stack` for DFS without recursion, matching brackets, and Min Stack (problem 8 below). `deque` when you need both ends, which is the sliding window maximum in chapter 05.

(You can skip this on a first read.) `std::queue` and `std::stack` are thin wrappers around a `std::deque` that hide the operations you should not use; that is why `queue` has no `q[i]` and no way to loop over it. A `deque` is stored as a chain of fixed-size chunks, which is how it manages constant-time work at both ends *and* `w[i]`.

In your own words: "queue is a shop line: push at the back, pop from the front. stack is a pile of plates: push and pop at the top. deque allows both ends and indexing. `pop()` returns nothing, so I read `front()` or `top()` first, and I check `empty()` before I look."

s4. `std::map` vs `unordered_map`, iterators

The picture: a phone book and a cupboard of buckets. Both containers store key -> value pairs. They differ in *how* they keep them.

<code>std::map<int, std::string></code> (a phone book, sorted by key)	<code>std::unordered_map<int, std::string></code> (a cupboard of buckets)
10 -> "a"	bucket 0: (30, "c")
20 -> "b"	bucket 1: (empty)
30 -> "c"	bucket 2: (10, "a") (20, "b")
walk it: 10, 20, 30, always in order	walk it: some order, never rely on it

find 20: open the middle, log n steps
needs: a < on the key

find 20: hash(20) says "bucket 2", one step
needs: a hash and == on the key

Use `unordered_map` for plain counting and looking up (Two Sum, Top K Frequent). Use `map` when you need the keys in order, or need "the first key at or above x": timestamps, sorted events, interval bookkeeping. Python's `dict` is a hash table like `unordered_map`; there is no built-in sorted dict.

What an iterator is. An iterator is a *bookmark* into a container. It marks one element; you can read the element through it, and move the bookmark forward with `++`.

```
elements:  [10:"a"]  [20:"b"]  [30:"c"]  (nothing here)
             ^                ^
           begin()            end()
```

`begin()` is a bookmark on the first element.

`end()` is a bookmark one past the last: a marker that means "you have run out".

Never read through `end()`. You only compare against it.

For a `map`, `it->first` is the key on the bookmarked page and `it->second` is the value (the arrow from `m1`: the bookmark behaves like a pointer to a pair). `auto` is your friend; iterator types have very long names.

```
#include <map>
#include <string>
#include <unordered_map>

std::map<int, std::string> timeline = {{30, "c"}, {10, "a"}, {20, "b"}};

std::string order;
for (const auto& [t, name] : timeline) order += name; // visits 10, 20, 30: order == "abc"

auto it = timeline.lower_bound(15); // bookmark on the first key >= 15, which is 20
// it->first == 20, it->second == "b"
bool found = timeline.lower_bound(99) != timeline.end(); // false: no key >= 99
```

- `for (const auto& [t, name] : timeline)` is the range-for from chapter 01; each element is a pair and `[t, name]` unpacks it, like Python's `for t, name in d.items()`.
- `lower_bound(x)`: the first key that is `>= x`. Only `map` has it; the cupboard is not sorted.
- Comparing with `end()` is how you ask "did you find anything?".

The erase-while-looping trap. Suppose you walk the phone book and tear out the page for key 20.

```
[10]  [20]  [30]
      ^ e           e is the bookmark on 20

timeline.erase(e);           the page is torn out...

[10]  [30]
      ? e                 ...and e is a bookmark on a page that no longer exists.
                          ++e now reads a torn-out page: garbage or a crash.
```

The fix: `erase` hands you back a fresh bookmark on the *next* page. Use it.

```
for (auto e = timeline.begin(); e != timeline.end(); e = timeline.erase(e)) { // note: no ++e up here
    if (e->first == 20) e = timeline.erase(e); // erase gives the bookmark on 30
    else ++e; // only step when we did not erase
}
// timeline now holds 10 and 30
```

`m[key]` inserts. In both containers, `counts["z"]` on a missing key *creates* the entry with a default value (0 for `int`) and returns it. That is perfect for counting and terrible for a read-only check, because the check quietly grows the map. For "is it there?" use `find` or `count`.

```
std::unordered_map<std::string, int> counts;
for (const char* w : {"a", "b", "a"}) ++counts[w];    // "a" -> 2, "b" -> 1
// counts.size() == 2
bool hasZ = counts.find("z") != counts.end();        // false, and nothing was inserted
int zCount = counts.count("z");                      // 0
```

- `++counts[w]`: look up (or create at 0), then add one. This is Python's `counts[w] = counts.get(w, 0) + 1`, or Counter.
- `counts.find("z")` gives a bookmark, or `end()` if absent. It never inserts.

(You can skip this on a first read.) Bookmark arithmetic like `it + 2` or `it2 - it1` works only on containers stored as one block: `vector`, `deque`, `array`. On a map use `std::next(it)` and `std::prev(it)`. Bookmarks can also go stale without you erasing anything: `push_back` on a `vector` may move the whole block to a bigger shelf (m5), after which every bookmark into it is dead. Take bookmarks late and use them quickly.

Try it. In `00_stl_demo.cpp`, change `counts.find("z") == counts.end()` to `counts["z"] == 0` and predict what `counts.size()` becomes.

In your own words: "map is a sorted phone book, log n per operation, with `lower_bound` for 'first key at or above x'. `unordered_map` is a cupboard of buckets, constant time on average, no order. An iterator is a bookmark; `end()` is the one-past-the-end marker I compare against but never read. `m[key]` inserts if missing, so I use `find` for read-only lookups, and when erasing in a loop I use the bookmark that `erase` returns."

s5. `std::array`, `std::string_view` basics

`std::array`: a vector whose size is fixed forever. The size is written into the type, `std::array<double, 3>`, and can never change: no `push_back`, no `resize`. In return there is no heap shelf at all (m5): the elements sit right on the stack tray, and the whole thing is as cheap as three plain doubles. Use it when the size is known while you write the program: an xyz point, a 4x4 transform, the four neighbour offsets, a tiny fixed grid.

```
std::array<double, 3> xyz      stack tray: [ 1.0 | 2.0 | 3.0 ]    no pointer, no warehouse
std::vector<double> v         stack tray: [ ptr | size | cap ] --> warehouse: [ 1.0 | 2.0 | 3.0 ]
```

```
#include <array>

std::array<double, 3> xyz = {1.0, 2.0, 3.0};
// xyz.size() == 3, xyz[2] == 3.0
std::array<std::array<int, 3>, 3> grid{};    // a 3x3 grid, all zeros: the {} does the zeroing
grid[1][1] = 9;
// grid[1][1] == 9, grid[0][0] == 0
```

Everything you know from `vector` still works: `size()`, `[i]`, range-for, `begin()/end()`.

`std::string_view`: a window onto someone else's string. A `string_view` does not own any characters. It is two numbers: where the characters start, and how many there are. Making one copies nothing; cutting a piece out of one (`substr`) copies nothing either.

```
std::string topic = "/sensors/lidar/points";    topic owns 21 characters on a heap shelf

std::string_view sv = topic;                    sv = [ start: topic's first char | length: 21 ]
std::string_view part = sv.substr(9, 5);        part = [ start: the 'l' of lidar | length: 5 ] -> "lidar"

/ s e n s o r s / l i d a r / p o i n t s
^           ^-----^
sv starts here    part is this window
```

```
#include <string>
#include <string_view>

bool startsWith(std::string_view s, std::string_view prefix) {
    return s.substr(0, prefix.size()) == prefix;
}

std::string topic = "/sensors/lidar/points";
std::string_view sv = topic;           // a window onto topic; nothing copied
// startsWith(sv, "/sensors") == true
// sv.substr(9, 5) == "lidar"          // another window, still nothing copied
// startsWith("literal", "lit") == true // a quoted literal becomes a string_view for free
```

Taking `std::string_view` as a parameter (instead of `const std::string&`) means callers with a `std::string`, a quoted literal, or a `char*` all work without building a temporary string.

Why the window goes dark. The window shows characters that belong to someone else. If that someone dies, the window is looking at a returned warehouse shelf (m5).

```
// std::string_view dark = std::string("temp"); // the temporary string dies at the ';'
//                                           // dark now looks at freed memory. Never do this.
```

Same trap in disguise: returning a `string_view` onto a local `std::string`, or storing a `string_view` in a struct that outlives the string. The rule: only use a `string_view` while you can point at the `std::string` (or literal) that owns the characters and know it is still alive. Literals live for the whole program, so views of them are always safe.

The Python you know. Python strings are immutable and every slice `s[9:14]` copies. A `string_view` slice is free but borrows. `std::array` is a tuple of fixed length that you can still write into.

Try it. In `00_stl_demo.cpp`, change `sv.substr(9, 5)` to `sv.substr(1, 7)` and predict the string before you run it.

In your own words: "`std::array` is a vector whose size is part of the type and never changes, with no heap allocation; I use it for small fixed-size things like points and direction tables. `string_view` is a window onto characters someone else owns: no copies, free slicing, but it must not outlive the string it looks at."

The STL demo file

Everything in `s1` to `s5`, as asserts:

```
// 00_stl_demo.cpp - sort with lambdas, priority_queue, deque/queue/stack, map vs unordered_map,
// std::array and std::string_view.
#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <deque>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <queue>
#include <stack>
#include <string>
#include <string_view>
#include <unordered_map>
#include <utility>
#include <vector>

struct Detection {
    std::string label;
    double score;
    int frame;
};

// string_view: a (pointer, length) view of characters someone else owns. No copy, no allocation.
bool startsWith(std::string_view s, std::string_view prefix) {
```

```

    return s.substr(0, prefix.size()) == prefix;
}

int main() {
    // ----- s1: std::sort with custom comparator / lambda -----
    std::vector<int> v = {5, 2, 9, 1};
    std::sort(v.begin(), v.end()); // ascending, uses operator<
    assert((v == std::vector<int>{1, 2, 5, 9}));
    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // descending; comparator = "a goes before b?"
    assert((v == std::vector<int>{9, 5, 2, 1}));

    // pairs sort lexicographically out of the box: by first, then second
    std::vector<std::pair<int, int>> pts = {{2, 1}, {1, 5}, {1, 2}};
    std::sort(pts.begin(), pts.end());
    assert((pts == std::vector<std::pair<int, int>>{{1, 2}, {1, 5}, {2, 1}}));

    // structs: multiple keys – score descending, then frame ascending. std::tie makes this one line.
    std::vector<Detection> dets = {{{"car", 0.9, 3}, {"person", 0.9, 1}, {"bike", 0.5, 2}};
    std::stable_sort(dets.begin(), dets.end(), [](const Detection& a, const Detection& b) {
        return std::tie(b.score, a.frame) < std::tie(a.score, b.frame);
    });
    assert(dets[0].label == "person" && dets[1].label == "car" && dets[2].label == "bike");

    // argsort: sort indices by the values they point at – the lambda captures `cost` by reference
    std::vector<double> cost = {3.5, 0.2, 1.7};
    std::vector<int> idx(cost.size());
    std::iota(idx.begin(), idx.end(), 0);
    std::sort(idx.begin(), idx.end(), [&cost](int a, int b) { return cost[a] < cost[b]; });
    assert((idx == std::vector<int>{1, 2, 0}));

    // captures: [=] copies, [&] references, [k] copies just k
    int k = 2;
    auto timesK = [k](int a) { return a * k; };
    auto bumpK = [&k]() { ++k; };
    bumpK();
    assert(timesK(3) == 6 && k == 3); // timesK captured k when it was 2

    // ----- s2: std::priority_queue -----
    std::priority_queue<int> maxHeap;
    for (int e : {3, 1, 4}) maxHeap.push(e);
    assert(maxHeap.top() == 4);

    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap; // min-heap via greater<>
    for (int e : {3, 1, 4}) minHeap.push(e);
    assert(minHeap.top() == 1);
    minHeap.pop(); // pop() returns void
    assert(minHeap.top() == 3);

    // min-heap of (distance, node) – Dijkstra's frontier. pair compares by first, so distance wins.
    using DistNode = std::pair<double, int>;
    std::priority_queue<DistNode, std::vector<DistNode>, std::greater<DistNode>> frontier;
    frontier.push({2.5, 7}); frontier.push({0.5, 3}); frontier.push({1.0, 9});
    assert(frontier.top().second == 3);

    // custom comparator: lambda + decltype, pass the lambda to the constructor
    auto byScoreAsc = [](const Detection& a, const Detection& b) { return a.score > b.score; }; // "a is lower priority than b"
    std::priority_queue<Detection, std::vector<Detection>, decltype(byScoreAsc)> lowScoreFirst(byScoreAsc);
    for (const auto& d : dets) lowScoreFirst.push(d);
    assert(lowScoreFirst.top().label == "bike");

    // ----- s3: std::deque, std::queue, std::stack -----
    std::queue<int> q; // adapter over deque: push back, pop front (BFS)
    q.push(1); q.push(2);
    assert(q.front() == 1);
    q.pop(); // void – read front() first if you need the value
    assert(q.front() == 2 && q.size() == 1);

    std::stack<int> st; // adapter over deque: push/pop/top at the back (DFS, parentheses)
    st.push(1); st.push(2);
    assert(st.top() == 2);
    st.pop();
    assert(st.top() == 1);

    std::deque<int> window; // both ends O(1), plus random access – sliding-window max
    window.push_back(1); window.push_back(2); window.push_front(0);
    assert(window.front() == 0 && window.back() == 2 && window[1] == 1);
    window.pop_front(); window.pop_back();
    assert(window.size() == 1 && window[0] == 1);

    // ----- s4: std::map vs unordered_map, iterators -----
    std::map<int, std::string> timeline = {{30, "c"}, {10, "a"}, {20, "b"}}; // red-black tree, key order
    std::string order;
    for (const auto& [t, name] : timeline) order += name; // iterates 10, 20, 30
    assert(order == "abc");
    auto it = timeline.lower_bound(15); // first key >= 15 – O(log n), only on ordered containers
    assert(it != timeline.end() && it->first == 20);
    assert(timeline.lower_bound(99) == timeline.end());
    assert(timeline.rbegin()->first == 30); // reverse iterator: largest key

```

```

// erase-while-iterating: erase() returns the next valid iterator; ++it after erase is a bug
for (auto e = timeline.begin(); e != timeline.end(); ) {
    if (e->first == 20) e = timeline.erase(e);
    else ++e;
}
assert(timeline.size() == 2 && timeline.count(20) == 0);

std::unordered_map<std::string, int> counts; // hash table, no order, O(1) average
for (const char* w : {"a", "b", "a"}) ++counts[w]; // operator[] inserts 0 if missing
assert(counts["a"] == 2 && counts.size() == 2);
assert(counts.find("z") == counts.end()); // find() does not insert; operator[] would
assert(counts.count("z") == 0 && counts.size() == 2);

// iterator arithmetic (it + 2, it - begin) only on random-access containers (vector, deque, array)
std::vector<int> w = {4, 5, 6};
auto mid = w.begin() + 1;
assert(*mid == 5 && mid - w.begin() == 1);
assert(std::next(timeline.begin()) == std::prev(timeline.end())); // for map, use std::next/std::prev

// ----- s5: std::array, std::string_view -----
std::array<double, 3> xyz = {1.0, 2.0, 3.0}; // size is part of the type, lives on the stack
assert(xyz.size() == 3 && xyz[2] == 3.0);
std::array<std::array<int, 3>, 3> grid{}; // fixed 3x3, zero-initialised: a tiny costmap
grid[1][1] = 9;
assert(grid[1][1] == 9 && grid[0][0] == 0);

std::string topic = "/sensors/lidar/points";
std::string_view sv = topic; // views topic's buffer; topic must outlive sv
assert(startsWith(sv, "/sensors"));
assert(sv.substr(9, 5) == "lidar"); // substr on a view is O(1): no allocation
assert(startsWith("literal", "lit")); // literals convert too

std::cout << "OK 00_stl_demo.cpp\n";
return 0;
}

```

What to look at:

- The four `std::sort` calls at the top: no comparator, a descending lambda, pairs sorting themselves, and the `argsort` that captures `cost` by reference with `[&cost]`.
- `timesK` and `bumpK`: the copy-versus-reference capture drawing from `s1`, as an assert.
- `maxHeap`, `minHeap` and `frontier`: the three pile shapes from `s2`. Note `pop()` returns nothing.
- The `queue`, `stack` and `deque` block: the table from `s3`, one line per cell.
- `timeline`: range-for in key order, `lower_bound`, `rbegin()` for the largest key, and the erase-while-looping loop written the safe way.
- `counts`: `++counts[w]` inserting, and `find` not inserting.
- `xyz`, `grid`, `sv`: the array and the string window from `s5`.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution with every new construct explained, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Product of Array Except Self (LeetCode 238, medium)

The problem in plain words. You are given a list of integers `nums`. Build a new list `answer` of the same length where `answer[i]` is the product of every number in `nums` except `nums[i]`. You are not allowed to use division, and the whole thing must take time proportional to the length of the list.

index:	0	1	2	3	
nums:	1	2	3	4	
answer:	24	12	8	6	answer[1] = 1 * 3 * 4 = 12 (everything but the 2)

Edge cases: a zero in the list makes every other slot 0, and the zero's own slot gets the product of the rest ([1, 1, 0, -3, 3] gives [0, 0, 9, 0, 0]). Two zeros make everything 0. The list has at least 2 numbers and at most 100,000, and the problem promises every product fits in a 32-bit `int`.

By hand. For each slot, cover it with your finger and multiply the numbers on the left of your finger by the numbers on the right. For slot 1 that is 1 on the left and $3 * 4 = 12$ on the right: 12. Doing that for every slot is slow (each slot re-multiplies almost the whole list), but it shows the shape: *left product times right product*. So compute all the left products in one sweep, all the right products in another sweep, and multiply.

nums:	1	2	3	4	
left of i:	1	1	2	6	nothing to the left of slot 0 counts as 1
right of i:	24	12	4	1	nothing to the right of slot 3 counts as 1
multiply:	24	12	8	6	

The idea. 1. Pass 1, left to right: `out[i] = out[i-1] * nums[i-1]`, with `out[0] = 1`. After this, `out` holds the left products. 2. Pass 2, right to left, carrying a running suffix that starts at 1: multiply `out[i]` by suffix, then fold `nums[i]` into suffix. 3. The output list is reused as the left-product list, so the only extra memory is suffix.

The Python you would have written.

```
def productExceptSelf(nums):
    n = len(nums)
    out = [1] * n
    for i in range(1, n):
        out[i] = out[i - 1] * nums[i - 1]    # product of everything left of i
    suffix = 1
    for i in range(n - 1, -1, -1):
        out[i] *= suffix                    # times product of everything right of i
        suffix *= nums[i]
    return out
```

In C++.

```
// 01_product_except_self.cpp - LeetCode 238. Prefix/suffix products, no division, O(1) extra space.
#include <cassert>
#include <iostream>
#include <vector>

std::vector<int> productExceptSelf(std::vector<int>& nums) {
    const int n = static_cast<int>(nums.size());
    std::vector<int> out(n, 1);
    // Pass 1: out[i] = product of everything to the left of i.
    for (int i = 1; i < n; ++i) out[i] = out[i - 1] * nums[i - 1];
    // Pass 2: walk right-to-left carrying the running product of everything to the right.
    // The output array is reused as the prefix array, so the only extra space is `suffix`.
    int suffix = 1;
    for (int i = n - 1; i >= 0; --i) {
        out[i] *= suffix;
        suffix *= nums[i];
    }
    return out;
}

int main() {
    std::vector<int> a = {1, 2, 3, 4};
    assert((productExceptSelf(a) == std::vector<int>{24, 12, 8, 6}));

    std::vector<int> b = {-1, 1, 0, -3, 3};    // a zero: every other slot becomes 0, the zero's slot gets the rest
    assert((productExceptSelf(b) == std::vector<int>{0, 0, 9, 0, 0}));

    std::vector<int> c = {0, 0};               // two zeros: everything is 0
    assert((productExceptSelf(c) == std::vector<int>{0, 0}));
}
```

```

std::vector<int> d = {5, 7}; // minimum size per the problem (n >= 2)
assert((productExceptSelf(d) == std::vector<int>{7, 5}));

std::cout << "OK 01_product_except_self.cpp\n";
return 0;
}

```

What is new:

- `const int n = static_cast<int>(nums.size());` is Python's `n = len(nums)`, with the size turned into an `int` once (m6) so the backwards loop below is safe.
- `std::vector<int> out(n, 1);` is `out = [1] * n`: "n copies of 1".
- `for (int i = n - 1; i >= 0; --i)` is `for i in range(n - 1, -1, -1)`. It is safe *only* because `i` is an `int`; with `size_t` it would be the odometer loop from m6.
- `out[i] *= suffix;` is the same as Python. `return out;` hands the vector to the caller by value (m5); nothing is copied in practice.

Walk through the example. `nums = [1, 2, 3, 4]`, `n = 4`, `out = [1, 1, 1, 1]`.

Pass 1: `i=1: out[1] = out[0] * nums[0] = 1 * 1 = 1`. `i=2: out[2] = 1 * 2 = 2`. `i=3: out[3] = 2 * 3 = 6`. Now `out = [1, 1, 2, 6]`, the left products.

Pass 2, suffix = 1: `i=3: out[3] = 6 * 1 = 6`, suffix = 4. `i=2: out[2] = 2 * 4 = 8`, suffix = 12. `i=1: out[1] = 1 * 12 = 12`, suffix = 24. `i=0: out[0] = 1 * 24 = 24`. Result `[24, 12, 8, 6]`.

Complexity. Two passes over the list: time proportional to n , $O(n)$. Extra memory is one `int`, $O(1)$, not counting the output you were asked to produce.

Robotics. Running products and running sums are the same trick as an *integral image* or a cumulative-sum row of a costmap: precompute once in a sweep, and then any "everything except this cell" or "sum of this window" question is answered in constant time. Sensor pipelines use this to compute windowed statistics over a stream without re-reading the window each tick.

Follow-ups you may get. - *Why not multiply everything and divide by `nums[i]`?* The problem forbids division, and a zero breaks it anyway. With division allowed you would count the zeros: none, divide; exactly one, only that slot is non-zero; two or more, all zeros. - *What about overflow?* The problem promises the products fit in 32 bits. In real code use `long long` or check. - *Can you do it with only the left-to-right pass?* No: slot `i` needs numbers to its right, which a single left-to-right pass has not seen yet. Two passes are the minimum.

2. Merge Intervals (LeetCode 56, medium)

The problem in plain words. You are given a list of intervals, each a pair `[start, end]` meaning "from start to end, inclusive". Some of them overlap. Merge every group of overlapping intervals into one, and return the merged list. Intervals that only *touch* (`[1, 4]` and `[4, 5]`) count as overlapping.

```

number line:  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[1, 3]          [=====]
[2, 6]          [=====]
[8, 10]         [=====]
[15, 18]        [=====]

                these two overlap -> [1, 6]
                alone              -> [8, 10]
                                   [=====] -> [15, 18]

answer: [[1, 6], [8, 10], [15, 18]]

```

Edge cases: the input may be in any order; an interval may be entirely inside another (`[1, 4]` and `[2, 3]` merge to `[1, 4]`, *not* `[1, 3]`); an empty list gives an empty list; a single interval like `[5, 5]` stays as it is. Up to 10,000 intervals.

By hand. Sort the intervals by their start. Then walk them left to right keeping "the interval I am currently building". If the next one starts at or before the end of what I am building, it overlaps: stretch my end to whichever end is bigger. If it starts after my end, there is a gap: put what I built into the answer and start building from this new one.

The idea. 1. Sort by start (a lambda comparator, `s1`). 2. Put the first interval into `out`. 3. For each interval: if its start is $\leq$ the end of `out.back()`, set that end to `max(current end, this end)`. Otherwise push the interval as a new group.

The Python you would have written.

```
def merge(intervals):
    intervals.sort(key=lambda iv: iv[0])
    out = []
    for start, end in intervals:
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)    # overlap: stretch the last group
        else:
            out.append([start, end])             # gap: start a new group
    return out
```

In C++.

```
// 02_merge_intervals.cpp - LeetCode 56. Sort by start, then sweep and extend the last merged interval.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>>> merge(std::vector<std::vector<int>>& intervals) {
    if (intervals.empty()) return {};
    // vector<int> already compares lexicographically, so this sorts by start (then end).
    // Being explicit with a lambda is what interviewers expect to see.
    std::sort(intervals.begin(), intervals.end(),
        [](const std::vector<int>& a, const std::vector<int>& b) { return a[0] < b[0]; });
    std::vector<std::vector<int>>> out;
    out.push_back(intervals[0]);
    for (const auto& iv : intervals) {
        std::vector<int>& last = out.back(); // reference: we mutate in place
        if (iv[0] <= last[1]) last[1] = std::max(last[1], iv[1]); // overlap (touching counts): extend
        else out.push_back(iv); // gap: start a new interval
    }
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>>;
    VV a = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};
    assert((merge(a) == VV{{1, 6}, {8, 10}, {15, 18}}));

    VV b = {{1, 4}, {4, 5}}; // touching endpoints merge
    assert((merge(b) == VV{{1, 5}}));

    VV c = {{1, 4}, {2, 3}}; // fully contained: end must be max, not the later interval's end
    assert((merge(c) == VV{{1, 4}}));

    VV d = {};
    assert(merge(d).empty());

    VV e = {{5, 5}}; // single degenerate interval
    assert((merge(e) == VV{{5, 5}}));

    std::cout << "OK 02_merge_intervals.cpp\n";
    return 0;
}
```

What is new:

- `std::sort(intervals.begin(), intervals.end(), [](const std::vector<int>& a, const std::vector<int>& b) { return a[0] < b[0]; });` is `intervals.sort(key=lambda iv: iv[0])`. The lambda

answers "does a go before b?" by comparing starts (s1). Each interval here is a `std::vector<int>` of two numbers, matching LeetCode's signature.

- `if (intervals.empty()) return {};` returns an empty vector; Python did this with the `if out` check. The C++ version pushes `intervals[0]` first, so the loop's first step compares the first interval with itself, which harmlessly does nothing.
- `std::vector<int>& last = out.back();` is `out[-1]`, but as a **reference** (m1). This is the line people get wrong: without the `&`, `last` is a copy, `last[1] = ...` changes the copy, and the answer never grows.
- `for (const auto& iv : intervals)` is `for start, end in intervals`, reading each interval through a const reference (m2) so nothing is copied.
- `std::max(last[1], iv[1])` is `max(...)`. `#include <algorithm>` brings in `std::sort` and `std::max`.

Walk through the example. After sorting (already sorted here): `[[1,3],[2,6],[8,10],[15,18]]`. `out = [[1,3]]`.

- `[1,3]: 1 <= 3`, overlap; `last[1] = max(3, 3) = 3`. Nothing changes.
- `[2,6]: 2 <= 3`, overlap; `last[1] = max(3, 6) = 6`. `out = [[1,6]]`.
- `[8,10]: 8 <= 6?` No, gap; push. `out = [[1,6],[8,10]]`.
- `[15,18]: 15 <= 10?` No; push. `out = [[1,6],[8,10],[15,18]]`.

Complexity. Sorting dominates: $O(n \log n)$ time. The output takes $O(n)$ memory.

Robotics. A sensor-fusion node merges the time windows during which each sensor was valid into one "trusted" timeline. A coverage planner merges the swept segments along a row into the spans already cleaned. A 1D laser scan's free and occupied runs are merged the same way.

Follow-ups you may get. - *Why does sorting by start make one pass enough?* Once sorted, any interval that overlaps some group must overlap the group's most recent extension, so you only ever compare with `out.back()`. - *What if intervals arrive one at a time?* That is Insert Interval, the next problem. For a stream, keep a `std::map<int, int>` keyed by start and merge on insert. - *What if `[2,3]` came after `[1,4]`?* The `max` handles it: the end stays 4. Using the incoming end instead of `max` is the classic bug.

3. Insert Interval (LeetCode 57, medium)

The problem in plain words. You are given a list of intervals that is *already sorted by start and has no overlaps*, plus one new interval. Insert the new interval so the list is still sorted and still has no overlaps, merging as needed. Return the new list.

existing:	[1,2]	[3,5]	[6,7]	[8,10]		[12,16]
new:			[4-----8]			
phase 1:	[1,2] ends before 4				-> copy as is	
phase 2:	[3,5], [6,7], [8,10] touch new				-> swallow them: new grows to [3,10]	
phase 3:	[12,16] starts after 10				-> copy as is	
answer:	[1,2]	[3,10]	[12,16]			

Edge cases: the existing list may be empty (answer is just the new interval); the new interval may go before everything, after everything, or swallow everything; touching counts as overlap. Up to 10,000 intervals.

By hand. Walk the sorted list left to right. At first the intervals end before the new one even starts: copy them. Then you hit intervals that overlap the new one: do not copy them, instead widen the new interval to cover each of them. Once you reach an interval that starts after the new one's (possibly widened) end, put the new interval down and copy the rest.

The idea. One pass in three phases, with one index `i`: 1. While `intervals[i].end < new.start`: copy. 2. While `intervals[i].start <= new.end`: `new.start = min(...)`, `new.end = max(...)`, step. 3. Push new, then copy everything left.

The Python you would have written.

```
def insert(intervals, new):
    out = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < new[0]:           # phase 1: ends before new starts
        out.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= new[1]:         # phase 2: overlaps new
        new[0] = min(new[0], intervals[i][0])
        new[1] = max(new[1], intervals[i][1])
        i += 1
    out.append(new)
    while i < n:                                       # phase 3: the rest
        out.append(intervals[i]); i += 1
    return out
```

In C++.

```
// 03_insert_interval.cpp - LeetCode 57. Input is sorted and non-overlapping; one linear pass in three phases.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>>> insert(std::vector<std::vector<int>>>& intervals, std::vector<int>& newInterval) {
    std::vector<std::vector<int>>> out;
    out.reserve(intervals.size() + 1);
    const int n = static_cast<int>(intervals.size());
    int i = 0;
    // Phase 1: everything that ends before the new interval starts is untouched.
    while (i < n && intervals[i][1] < newInterval[0]) out.push_back(intervals[i++]);
    // Phase 2: everything that overlaps gets absorbed into newInterval.
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = std::min(newInterval[0], intervals[i][0]);
        newInterval[1] = std::max(newInterval[1], intervals[i][1]);
        ++i;
    }
    out.push_back(newInterval);
    // Phase 3: the rest starts after the merged interval ends.
    while (i < n) out.push_back(intervals[i++]);
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>>;
    VV a = {{1, 3}, {6, 9}};
    std::vector<int> na = {2, 5};
    assert((insert(a, na) == VV{{1, 5}, {6, 9}}));

    VV b = {{1, 2}, {3, 5}, {6, 7}, {8, 10}, {12, 16}};
    std::vector<int> nb = {4, 8};
    assert((insert(b, nb) == VV{{1, 2}, {3, 10}, {12, 16}}));

    VV c = {}; // empty list: result is just the new interval
    std::vector<int> nc = {5, 7};
    assert((insert(c, nc) == VV{{5, 7}}));

    VV d = {{1, 5}}; // insert before everything
    std::vector<int> nd = {-3, 0};
    assert((insert(d, nd) == VV{{-3, 0}, {1, 5}}));

    VV e = {{1, 5}}; // insert after everything
    std::vector<int> ne = {6, 8};
    assert((insert(e, ne) == VV{{1, 5}, {6, 8}}));

    std::cout << "OK 03_insert_interval.cpp\n";
    return 0;
}
```

What is new:

- `out.reserve(intervals.size() + 1);` asks the vector for one heap shelf big enough for everything up front (m5), so `push_back` never has to move to a bigger shelf. Optional, but free.
- `out.push_back(intervals[i++]);` is `out.append(intervals[i]); i += 1` in one line. `i++` means "use the current value of `i`, then add one". It is fine here; do not use it in tricky expressions.
- `std::min` and `std::max` are Python's `min` and `max` for two values.
- `newInterval` is taken by non-const reference (`std::vector<int>&`) and changed in place, exactly like Python's `new[0] = ...` on the caller's list. That is what the given signature asks for; in your own code you would take a copy.

Walk through `[[1,3],[6,9]]` with new `[2,5]`. `i = 0`. Phase 1: does `[1,3]` end before 2? `3 < 2` is false, so phase 1 copies nothing. Phase 2: does `[1,3]` start at or before 5? `1 <= 5`, yes:

`new = [min(2,1), max(5,3)] = [1,5]`, `i = 1`. Does `[6,9]` start at or before 5? `6 <= 5`, no. Push `[1,5]`. Phase 3: copy `[6,9]`. Answer `[[1,5],[6,9]]`.

Complexity. One pass: $O(n)$ time and $O(n)$ memory for the output.

Robotics. A scheduler keeps a sorted list of "arm busy" windows and inserts a newly planned trajectory segment. A keep-out list along a path, indexed by distance along the path, gets a new obstacle zone inserted and merged with its neighbours. Anything sorted by time or arc length that must stay non-overlapping uses this exact insert.

Follow-ups you may get. - *Can you find the insertion point faster than scanning?* Use `std::lower_bound` (binary search) to find the first overlap in $O(\log n)$, but shifting the vector to insert is $O(n)$ anyway. A `std::map<int,int>` keyed by start gives $O(\log n + k)$ where k intervals are merged. - *Do it in place?* Erase the overlapped range and insert the merged interval: `intervals.erase(first, last); intervals.insert(first, merged);`. - *Why `<` in phase 1 but `<=` in phase 2?* Because touching counts as overlap: `[1,2]` and a new `[2,5]` must merge, so "ends strictly before" is the condition for leaving an interval alone.

4. 3Sum (LeetCode 15, medium)

The problem in plain words. Given a list of integers, find every *unique* group of three numbers (from three different positions) that add up to zero. Return the groups in any order; two groups with the same three values count as the same group and must appear only once.

```
nums = [-1, 0, 1, 2, -1, -4]

sorted:  -4  -1  -1   0   1   2
index:    0   1   2   3   4   5

-1 + -1 + 2 = 0    -> [-1, -1, 2]
-1 +  0 + 1 = 0    -> [-1,  0, 1]
(the second -1 gives the same groups again, so it must be skipped)

answer: [[-1, -1, 2], [-1, 0, 1]]
```

Edge cases: fewer than three numbers gives an empty answer; `[0, 0, 0, 0]` gives exactly one group `[0, 0, 0]`; no group at all (`[0, 1, 1]`) gives an empty list. Up to 3,000 numbers.

By hand. Sort the numbers first. Pick the smallest as the "anchor" and now you need two more numbers that add to minus the anchor. With the rest sorted, put one finger on the left end and one on the right end. If the three add to less than zero, the sum is too small, so move the left finger right (to a bigger number). If more than zero, move the right finger left. If exactly zero, write it down, then move both fingers past any repeats. When the fingers meet, move the anchor to the next *different* number and repeat.

The idea. 1. Sort. For each anchor `i`: if `nums[i] > 0`, stop (three positives cannot sum to zero). If `nums[i] == nums[i-1]`, skip (same anchor, same groups). 2. `lo = i + 1`, `hi = n - 1`. While `lo < hi`: sum too small, `++lo`; too big, `--hi`; zero, record it and step both fingers past duplicates.

The Python you would have written.

```
def threeSum(nums):
    nums.sort()
    n = len(nums)
    out = []
    for i in range(n - 2):
        if nums[i] > 0:
            break
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        lo, hi = i + 1, n - 1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0:
                lo += 1
            elif s > 0:
                hi -= 1
            else:
                out.append([nums[i], nums[lo], nums[hi]])
                while lo < hi and nums[lo] == nums[lo + 1]: lo += 1
                while lo < hi and nums[hi] == nums[hi - 1]: hi -= 1
                lo += 1; hi -= 1
    return out
```

In C++.

```
// 04_three_sum.cpp - LeetCode 15. Sort, fix one element, two pointers for the other two, skip duplicates.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>> threeSum(std::vector<int>& nums) {
    std::vector<std::vector<int>> out;
    std::sort(nums.begin(), nums.end());
    const int n = static_cast<int>(nums.size());
    for (int i = 0; i < n - 2; ++i) {
        if (nums[i] > 0) break;
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int lo = i + 1, hi = n - 1;
        while (lo < hi) {
            const int sum = nums[i] + nums[lo] + nums[hi];
            if (sum < 0) ++lo;
            else if (sum > 0) --hi;
            else {
                out.push_back({nums[i], nums[lo], nums[hi]});
                // Skip duplicates on both sides before moving on, otherwise the same triple repeats.
                while (lo < hi && nums[lo] == nums[lo + 1]) ++lo;
                while (lo < hi && nums[hi] == nums[hi - 1]) --hi;
                ++lo; --hi;
            }
        }
    }
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>;
    auto sorted = [](VV v) { std::sort(v.begin(), v.end()); return v; }; // answer order is unspecified

    std::vector<int> a = {-1, 0, 1, 2, -1, -4};
    assert(sorted(threeSum(a)) == sorted(VV{{-1, -1, 2}, {-1, 0, 1}}));

    std::vector<int> b = {0, 1, 1};
    assert(threeSum(b).empty());

    std::vector<int> c = {0, 0, 0, 0};
    assert((threeSum(c) == VV{{0, 0, 0}}));
```

```

std::vector<int> d = {-2, 0, 0, 2, 2};           // duplicate skipping on lo and hi
assert((threeSum(d) == VV{{-2, 0, 2}}));

std::vector<int> e = {1, 2};                   // fewer than three elements
assert(threeSum(e).empty());

std::cout << "OK 04_three_sum.cpp\n";
return 0;
}

```

What is new:

- `std::sort(nums.begin(), nums.end());` is `nums.sort()`. No comparator: ints sort ascending by themselves (s1).
- `for (int i = 0; i < n - 2; ++i)` is `for i in range(n - 2)`. With `n` an `int` (m6), a list of two numbers gives `n - 2 == 0` and the loop never runs, which is correct. With `size_t`, `n - 2` on a list of one would roll round to a huge number.
- `break` and `continue` mean the same as in Python.
- `int lo = i + 1, hi = n - 1;` declares two ints on one line, Python's `lo, hi = i + 1, n - 1`.
- `out.push_back({nums[i], nums[lo], nums[hi]});` is `out.append(...)`; the braces build the three-element vector on the spot.
- `++lo; --hi;` is `lo += 1; hi -= 1`.

Walk through the example. Sorted: `[-4, -1, -1, 0, 1, 2]`, `n = 6`.

- `i=0` (anchor -4): `lo=1, hi=5: -4-1+2 = -3 < 0, lo=2. -4-1+2 = -3, lo=3. -4+0+2 = -2, lo=4. -4+1+2 = -1, lo=5`. Fingers meet. Nothing found.
- `i=1` (anchor -1): `lo=2, hi=5: -1-1+2 = 0`. Record `[-1, -1, 2]`. Skip repeats: `nums[2]` vs `nums[3]` differ, `nums[5]` vs `nums[4]` differ. `lo=3, hi=4: -1+0+1 = 0`. Record `[-1, 0, 1]`. `lo=4, hi=3`, stop.
- `i=2` (anchor -1 again): `nums[2] == nums[1]`, skip. This is what stops the duplicate groups.
- `i=3` (anchor 0): `lo=4, hi=5: 0+1+2 = 3 > 0, hi=4`. Stop.
- `i=4` is not `< n-2 = 4`; loop ends. Answer `[[-1, -1, 2], [-1, 0, 1]]`.

Complexity. The sort is $O(n \log n)$; then for each of n anchors the two fingers together walk at most n steps: $O(n^2)$ time. Extra memory is $O(1)$ beyond the sort and the output.

Robotics. Two fingers walking inward over sorted data is how you match events by timestamp: pairing each lidar scan with the nearest IMU sample, or finding pairs of measurements whose time difference is within a tolerance. The discipline of skipping repeats is the same one you need when several detections fall into the same grid cell and must be counted once.

Follow-ups you may get. - Why compare with `nums[i-1]` and not `nums[i+1]` when skipping anchors?

Skipping ahead would throw away the *first* -1 and lose `[-1, -1, 2]`. You skip an anchor only if the previous anchor was the same value and has already done the work. - 3Sum Closest, or 4Sum? Same skeleton. For closest, track the best `|sum - target|` instead of testing for zero. For 4Sum, add one more outer loop: $O(n^3)$. - Is it OK that the sort changes the input? The signature takes a non-const reference, so the problem allows it. If the caller needs the original order, copy first.

5. Container With Most Water (LeetCode 11, medium)

The problem in plain words. You are given a list of heights. Each height is a vertical wall standing at position `i` on the ground. Pick two walls; together with the ground they form a container. It holds water up to the height of the *shorter* wall, across the distance between them. Return the largest amount of water any pair of walls can hold, that is the largest `min(height[i], height[j]) * (j - i)`.

```
height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
```

```
8 |      #                #
```

```

7 |   #           #   #
6 |   # #       #   #
5 |   # #   #   #   #
4 |   # #   # # #   #
3 |   # #   # # # # #
2 |   # # # # # # # #
1 | # # # # # # # # #
  +-----+
    0  1  2  3  4  5  6  7  8

```

best pair: wall 1 (height 8) and wall 8 (height 7)

water = $\min(8, 7) * (8 - 1) = 7 * 7 = 49$

Edge cases: two walls of height 1 hold 1; a single wall holds nothing (0); the outer two walls can be the answer ([4, 3, 2, 1, 4] gives 16). Up to 100,000 walls, so trying every pair is too slow.

By hand. Start with the two outermost walls: that is the widest container you can make. Now ask: which wall is worth moving? The shorter one. Moving the taller wall inward can never help, because the water level is set by the shorter wall and the width only shrinks. So move the shorter wall one step inward, measure again, and keep going until the two walls meet. Remember the biggest measurement.

The idea. 1. $lo = 0$, $hi = n - 1$, $best = 0$. 2. Measure $\min(\text{height}[lo], \text{height}[hi]) * (hi - lo)$; keep the max. 3. If $\text{height}[lo] < \text{height}[hi]$ move lo right, else move hi left. Stop when they meet.

The Python you would have written.

```

def maxArea(height):
    lo, hi = 0, len(height) - 1
    best = 0
    while lo < hi:
        best = max(best, min(height[lo], height[hi]) * (hi - lo))
        if height[lo] < height[hi]:
            lo += 1          # the shorter wall is on the left: move it
        else:
            hi -= 1
    return best

```

In C++.

```

// 05_container_water.cpp - LeetCode 11. Two pointers from the outside in; always move the shorter wall.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

int maxArea(std::vector<int>& height) {
    int lo = 0, hi = static_cast<int>(height.size()) - 1;
    int best = 0;
    while (lo < hi) {
        best = std::max(best, std::min(height[lo], height[hi]) * (hi - lo));
        // The shorter wall bounds the area. Moving the taller one can only shrink width without
        // raising the limit, so the shorter side is the only move that might improve.
        if (height[lo] < height[hi]) ++lo;
        else --hi;
    }
    return best;
}

int main() {
    std::vector<int> a = {1, 8, 6, 2, 5, 4, 8, 3, 7};
    assert(maxArea(a) == 49);

    std::vector<int> b = {1, 1};
    assert(maxArea(b) == 1);

    std::vector<int> c = {4, 3, 2, 1, 4};    // outer walls are the answer
    assert(maxArea(c) == 16);
}

```

```

std::vector<int> d = {1, 2, 1};
assert(maxArea(d) == 2);

std::vector<int> e = {7};           // single wall: no container
assert(maxArea(e) == 0);

std::cout << "OK 05_container_water.cpp\n";
return 0;
}

```

What is new:

- `int lo = 0, hi = static_cast<int>(height.size()) - 1;` is `lo, hi = 0, len(height) - 1`. The cast matters (m6): on an empty list `size() - 1` would roll round, but `static_cast<int>(0) - 1` is `-1`, and while `(lo < hi)` simply does not run.
- `std::max(best, std::min(height[lo], height[hi]) * (hi - lo))` is the same line as Python with `std::` in front. Both come from `<algorithm>`.
- `++lo` and `--hi` are `lo += 1` and `hi -= 1`.

Walk through the example. `n = 9, lo = 0, hi = 8, best = 0`.

lo	hi	heights	water	best	move
0	8	1, 7	$1 \times 8 = 8$	8	$1 < 7$: <code>lo</code> → 1
1	8	8, 7	$7 \times 7 = 49$	49	$8 < 7$ false: <code>hi</code> → 7
1	7	8, 3	$3 \times 6 = 18$	49	<code>hi</code> → 6
1	6	8, 8	$8 \times 5 = 40$	49	equal, else branch: <code>hi</code> → 5
1	5	8, 4	$4 \times 4 = 16$	49	<code>hi</code> → 4
1	4	8, 5	$5 \times 3 = 15$	49	<code>hi</code> → 3
1	3	8, 2	$2 \times 2 = 4$	49	<code>hi</code> → 2
1	2	8, 6	$6 \times 1 = 6$	49	<code>hi</code> → 1

`lo == hi`, stop. Answer 49.

Complexity. The two fingers together take at most n steps: $O(n)$ time, $O(1)$ memory.

Robotics. "Shrink from the outside and only ever move the limiting side" is the same reasoning you use when tightening a time window between two sensors' valid ranges, or when picking the widest clearance corridor between two obstacle profiles along a scan line: the more constraining side is the only one worth moving.

Follow-ups you may get. - *Prove that moving the taller wall never helps.* Say `height[lo] <= height[hi]`. Any container using `lo` and some `j` between them has water at most `height[lo] * (j - lo)`, which is less than `height[lo] * (hi - lo)`, the one we just measured. So `lo` can never do better; retire it. - *What if the two walls are equal?* Move either; the code moves `hi`. Both choices are safe by the argument above applied to each side. - *Trapping Rain Water (LeetCode 42)?* Also two fingers, but you sum water over every position using running left and right maxima. Chapter 05.

6. Rotate Image (LeetCode 48, medium)

The problem in plain words. You are given a square grid of numbers, n rows by n columns. Rotate it 90 degrees clockwise, *in place*: change the grid you were given rather than building a new one.

before	after (turned a quarter turn to the right)
1 2 3	7 4 1

```
4 5 6    -->    8 5 2
7 8 9          9 6 3
```

The top row 1 2 3 became the right column, read top to bottom. Edge cases: a 1x1 grid does not change; a 2x2 `[[1,2],[3,4]]` becomes `[[3,1],[4,2]]`; four rotations bring the original back. `n` is at most 20.

By hand. Rotating by hand means moving every number to a new place at once, which is hard to do in place. But there is a two-step trick that is easy. First *transpose*: flip the grid over its main diagonal (the line from top-left to bottom-right), so rows become columns. Then *reverse each row*. Do those two easy things and the grid has been rotated.

original		step 1: transpose (flip over the \ diagonal)		step 2: reverse each row
1 2 3		1 4 7		7 4 1
4 5 6	---->	2 5 8	---->	8 5 2
7 8 9		3 6 9		9 6 3

Look at step 1: the number at row `i`, column `j` moved to row `j`, column `i`. The diagonal 1 5 9 did not move. Look at step 2: each row was simply read backwards.

The idea. 1. For every pair of positions above the diagonal (`j > i`), swap `matrix[i][j]` with `matrix[j][i]`. Starting `j` at `i + 1` swaps each pair exactly once. 2. Reverse every row.

The Python you would have written.

```
def rotate(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j] # transpose
    for row in matrix:
        row.reverse() # flip each row
```

In C++.

```
// 06_rotate_image.cpp - LeetCode 48. Rotate an n x n matrix 90 degrees clockwise in place:
// transpose, then reverse each row.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <utility>
#include <vector>

void rotate(std::vector<std::vector<int>>& matrix) {
    const int n = static_cast<int>(matrix.size());
    // Transpose: swap across the main diagonal. j starts at i+1 so each pair is swapped once.
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            std::swap(matrix[i][j], matrix[j][i]);
    // Reverse each row: transpose + horizontal flip == 90 degrees clockwise.
    for (auto& row : matrix) std::reverse(row.begin(), row.end());
}

int main() {
    using VV = std::vector<std::vector<int>>;
    VV a = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    rotate(a);
    assert((a == VV{{7, 4, 1}, {8, 5, 2}, {9, 6, 3}}));

    VV b = {{5, 1, 9, 11}, {2, 4, 8, 10}, {13, 3, 6, 7}, {15, 14, 12, 16}};
    rotate(b);
    assert((b == VV{{15, 13, 2, 5}, {14, 3, 4, 1}, {12, 6, 8, 9}, {16, 7, 10, 11}}));

    VV c = {{1}}; // 1x1 is a fixed point
```

```

rotate(c);
assert((c == VV{{1}}));

VV d = {{1, 2}, {3, 4}};
rotate(d);
assert((d == VV{{3, 1}, {4, 2}}));
rotate(d); rotate(d); rotate(d);           // four rotations = identity
assert((d == VV{{1, 2}, {3, 4}}));

std::cout << "OK 06_rotate_image.cpp\n";
return 0;
}

```

What is new:

- `void rotate(std::vector<std::vector<int>>& matrix)` returns nothing and takes the grid by reference (m1), so the caller's grid is the one that changes. Without the `&` you would rotate a copy and the caller would see nothing.
- `std::swap(matrix[i][j], matrix[j][i]);` is Python's `a, b = b, a`. It comes from `<utility>`.
- `for (auto& row : matrix) std::reverse(row.begin(), row.end());` is for `row in matrix: row.reverse()`. The `&` on `auto& row` is essential: `auto row` would be a copy of the row, and reversing the copy leaves the grid untouched.
- `std::reverse(first, last)` reverses whatever lies between two bookmarks (s4); here, the whole row.

Walk through the example. `n = 3`.

Transpose: `i=0`: swap `(0,1)` with `(1,0)`: 2 and 4 trade places. Swap `(0,2)` with `(2,0)`: 3 and 7. `i=1`: swap `(1,2)` with `(2,1)`: 6 and 8. `i=2`: `j` would start at 3, nothing to do. Grid is now `[[1,4,7],[2,5,8],[3,6,9]]`.

Reverse rows: `[1,4,7] → [7,4,1]`; `[2,5,8] → [8,5,2]`; `[3,6,9] → [9,6,3]`. Done.

Complexity. Every cell is touched a constant number of times: $O(n^2)$ time (there are n^2 cells, so this is one pass). $O(1)$ extra memory.

Robotics. Rotating an occupancy grid by 90 degrees when the map convention changes (image (`row`, `col`) with `row` growing downward versus map (`x`, `y`) with `y` growing upward) is exactly this operation: the transpose is the axis swap and the row reversal is the axis flip. It also explains, in miniature, why the inverse of a rotation matrix is its transpose.

Follow-ups you may get. - *Counter-clockwise?* Transpose, then reverse each *column* (or reverse the rows first, then transpose). - *Not square?* An `m`-by-`n` grid cannot be rotated in place inside the same buffer. Build an `n`-by-`m` output with `out[j][n - 1 - i] = in[i][j]`. - *What goes wrong if `j` starts at 0 instead of `i + 1`?* Every pair is swapped twice and the grid ends up exactly as it started.

7. Top K Frequent Elements (LeetCode 347, medium)

The problem in plain words. Given a list of integers and a number `k`, return the `k` values that appear most often. The problem promises the answer is unique, and you may return the values in any order.

```

nums = [1, 1, 1, 2, 2, 3], k = 2

count:   1 appears 3 times
         2 appears 2 times
         3 appears 1 time

the 2 most frequent: 1 and 2      answer: [1, 2] (or [2, 1])

```

Edge cases: a single element with `k = 1` returns it; `k` may equal the number of distinct values (return them all); negative numbers are fine. Up to 100,000 numbers, and `k` is never larger than the number of distinct values.

By hand. First make a tally: one line per distinct value with how many times it appears. Then you need the k biggest tallies. Sorting the whole tally works, but there is a neater way that never keeps more than k things in hand: keep a small pile of "candidates so far" that always shows the *weakest* candidate on top. Add each value's tally to the pile; if the pile now has more than k things, throw away the weakest. At the end the pile holds exactly the k strongest.

The idea. 1. Count with an `unordered_map` (s4): `++freq[x]`. 2. For each `(count, value)` pair, push it onto a min-heap (s2, smallest count on top). If the heap size passes k, pop the top (the weakest). 3. Empty the heap into the answer.

The Python you would have written.

```
from collections import Counter
import heapq

def topKFrequent(nums, k):
    freq = Counter(nums)
    heap = []
    for val, cnt in freq.items():
        heapq.heappush(heap, (cnt, val))    # heapq is a min-heap: weakest on top
        if len(heap) > k:
            heapq.heappop(heap)            # throw away the weakest
    return [val for cnt, val in heap]
```

In C++.

```
// 07_top_k_frequent.cpp - LeetCode 347. Count with a hash map, keep the k largest counts in a min-heap.
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <unordered_map>
#include <utility>
#include <vector>

std::vector<int> topKFrequent(std::vector<int>& nums, int k) {
    std::unordered_map<int, int> freq;
    for (int x : nums) ++freq[x];

    // Min-heap on (count, value): the top is the weakest of the current k candidates.
    // When the heap exceeds k, pop the weakest. Heap size never exceeds k+1 -> O(n log k).
    using CountVal = std::pair<int, int>;
    std::priority_queue<CountVal, std::vector<CountVal>, std::greater<CountVal>> heap;
    for (const auto& [val, cnt] : freq) {
        heap.push({cnt, val});
        if (static_cast<int>(heap.size()) > k) heap.pop();
    }
    std::vector<int> out;
    out.reserve(static_cast<std::size_t>(k));
    while (!heap.empty()) {
        out.push_back(heap.top().second);
        heap.pop();
    }
    return out;    // any order is accepted by the problem
}

int main() {
    auto sorted = [](std::vector<int> v) { std::sort(v.begin(), v.end()); return v; };

    std::vector<int> a = {1, 1, 1, 2, 2, 3};
    assert(sorted(topKFrequent(a, 2)) == (std::vector<int>{1, 2}));

    std::vector<int> b = {1};
    assert(sorted(topKFrequent(b, 1)) == (std::vector<int>{1}));

    std::vector<int> c = {4, 4, 4, 5, 5, 6, 6, 7};    // k equal to the number of distinct values
    assert(sorted(topKFrequent(c, 4)) == (std::vector<int>{4, 5, 6, 7}));
}
```

```

std::vector<int> d = {-1, -1, 2, 2, 2, 3};           // negatives are fine as hash keys
assert(sorted(topKFrequent(d, 1)) == (std::vector<int>{2}));

std::cout << "OK 07_top_k_frequent.cpp\n";
return 0;
}

```

What is new:

- `std::unordered_map<int, int> freq;` for `(int x : nums) ++freq[x];` is `Counter(nums)`. `freq[x]` creates the entry at 0 the first time (s4), then `++` bumps it.
- `using CountVal = std::pair<int, int>;` names the pair type (count, value) so the next line is readable.
- `std::priority_queue<CountVal, std::vector<CountVal>, std::greater<CountVal>> heap;` is the min-heap of pairs from s2: the pair with the smallest first (the count) sits on top. This is why the pair is (count, value) and not (value, count).
- `for (const auto& [val, cnt] : freq)` is for `val, cnt` in `freq.items()`, with the pair unpacked in place.
- `if (static_cast<int>(heap.size()) > k) heap.pop();` is `if len(heap) > k`. The cast keeps an unsigned `size()` from being compared with a signed `k` (m6).
- `heap.top().second` reads the value half of the top pair; then `heap.pop()` removes it (s2: look, then pop).

Walk through the example. `freq = {1: 3, 2: 2, 3: 1}`, `k = 2`. (An `unordered_map` may hand these out in any order; the end result is the same. Take this order.)

<code>push (3, 1)</code>	<code>heap: top -> (3,1)</code>	<code>size 1</code>
<code>push (2, 2)</code>	<code>heap: top -> (2,2) (3,1)</code>	<code>size 2</code>
<code>push (1, 3)</code>	<code>heap: top -> (1,3) (2,2) (3,1)</code>	<code>size 3 > 2: pop the top, (1,3)</code>
	<code>heap: top -> (2,2) (3,1)</code>	<code>size 2</code>
<code>drain:</code>	<code>out = [2, 1]</code>	

Any order is accepted, so `[2, 1]` is a correct answer.

Complexity. Counting is one pass, $O(n)$. Each of the d distinct values costs a push and maybe a pop on a heap that never holds more than $k+1$ items: $O(d \log k)$. Total $O(n \log k)$ time, $O(n)$ memory for the tally.

Robotics. "The k object classes seen most often in the last minute", "the k map cells hit most by a lidar sweep", "the k loop-closure candidates with the most votes in a place recogniser": count into a hash map, keep a bounded heap. The same bounded min-heap is how a KD-tree search keeps the k nearest neighbours found so far.

Follow-ups you may get. - *Why a min-heap and not a max-heap?* With a max-heap you would pop k times from a heap of all d values: $O(d \log d)$. The size- k min-heap shows you the weakest candidate, so evicting is $O(\log k)$. - *Can you do it in $O(n)$?* Bucket sort by count: `buckets[count].push_back(value)` with $n + 1$ buckets, then walk from the highest bucket down until you have k values. - *What about ties?* The problem promises there are none. If there could be, put the tie-break in the pair, for example `(count, -value)`.

8. Min Stack (LeetCode 155, medium)

The problem in plain words. Design a stack (the pile of plates from s3) with the usual `push(x)`, `pop()`, and `top()`, plus one extra operation `getMin()` that returns the smallest value currently anywhere in the stack. Every one of the four operations must take constant time, no matter how many plates are on the pile. `pop`, `top` and `getMin` are only ever called on a non-empty stack.

<code>push(-2)</code>	<code>push(0)</code>	<code>push(-3)</code>	<code>getMin() -> -3</code>
<code>pop()</code>			<code>top() -> 0</code>
			<code>getMin() -> -2</code>

Edge cases: duplicates of the minimum (`push(1)`, `push(1)`, `push(2)`, `pop()`, `pop()` must still report `getMin() == 1`); a rising sequence where the minimum never changes. Up to 30,000 operations.

By hand. Keeping "the minimum so far" as a single number breaks the moment you pop it: what was the minimum *before* it? You would have to search. So keep a *second* pile that remembers the history of minimums. Whenever you push a value that is at least as small as the current minimum, also push it onto the second pile. Whenever you pop a value that equals the top of the second pile, pop that too. The top of the second pile is always the answer to `getMin`.

Here is the state of both piles after each operation in the example (tops on the right):

operation	data_	mins_	getMin()	why
push(-2)	[-2]	[-2]	-2	mins_ was empty, so push
push(0)	[-2, 0]	[-2]	-2	0 <= -2 is false: not pushed on mins_
push(-3)	[-2, 0, -3]	[-2, -3]	-3	-3 <= -2: pushed on both
pop()	[-2, 0]	[-2]	-2	popped -3 equals mins_ top, so pop it there too
top()	-> 0			

The idea. - `push(v)`: push on `data_`; if `mins_` is empty or `v <= mins_.top()`, push on `mins_` too. - `pop()`: if `data_.top() == mins_.top()`, pop `mins_`; then pop `data_`. - `top()`: `data_.top()`. `getMin()`: `mins_.top()`.

The `<=` (not `<`) is what makes duplicates work: pushing 1, 1 puts *two* 1s on `mins_`, so popping one of them leaves the other as the minimum.

The Python you would have written.

```
class MinStack:
    def __init__(self):
        self.data = []
        self.mins = []

    def push(self, val):
        self.data.append(val)
        if not self.mins or val <= self.mins[-1]:
            self.mins.append(val)

    def pop(self):
        if self.data[-1] == self.mins[-1]:
            self.mins.pop()
        self.data.pop()

    def top(self):
        return self.data[-1]

    def getMin(self):
        return self.mins[-1]
```

In C++.

```
// 08_min_stack.cpp - LeetCode 155. A stack with O(1) getMin: a second stack tracks the running minimum.
#include <cassert>
#include <iostream>
#include <stack>

class MinStack {
public:
    MinStack() = default;

    void push(int val) {
        data_.push(val);
        // mins_ holds the minimum of everything at or below this position.
        // Push on <= (not <) so duplicates of the minimum survive a pop of one copy.
        if (mins_.empty() || val <= mins_.top()) mins_.push(val);
    }

    void pop() {
        if (data_.top() == mins_.top()) mins_.pop(); // this element was the current minimum
        data_.pop();
    }

    int top() const { return data_.top(); }
```

```

int getMin() const { return mins_.top(); }

private:
    std::stack<int> data_;
    std::stack<int> mins_;
};

int main() {
    MinStack s;
    s.push(-2); s.push(0); s.push(-3);
    assert(s.getMin() == -3);
    s.pop();
    assert(s.top() == 0);
    assert(s.getMin() == -2);

    MinStack dup;                // duplicate minimums: popping one must not lose the other
    dup.push(1); dup.push(1); dup.push(2);
    assert(dup.getMin() == 1);
    dup.pop(); dup.pop();
    assert(dup.getMin() == 1);
    dup.pop();

    MinStack rising;             // increasing pushes: mins_ stays size 1
    rising.push(5); rising.push(6); rising.push(7);
    assert(rising.getMin() == 5 && rising.top() == 7);
    rising.pop(); rising.pop(); rising.pop();
    rising.push(3);
    assert(rising.getMin() == 3);

    std::cout << "OK 08_min_stack.cpp\n";
    return 0;
}

```

What is new:

- `class MinStack { public: ... private: ... };` is the box-with-buttons from m3. The two piles `data_` and `mins_` are private compartments (the trailing underscore is a common naming habit for members); the four operations are public buttons. Python's `self.data` becomes `data_`.
- `MinStack()` = default; says "the constructor does nothing special": the two `std::stack`s build themselves empty, like `self.data = []`. There is no destructor because both compartments clean up after themselves: rule of zero (m3).
- `std::stack<int> data_;` is the pile of plates from s3 (push, pop, top, empty).
- `int top() const` and `int getMin() const`: these only read, so they carry the `const` sticker (m2) and can be called on a `const MinStack&`.
- `void push(int val)` and `void pop()` change the compartments, so they are not `const`.

Walk through the example. That is exactly the table above: after `push(-2)`, `push(0)`, `push(-3)` we have `data_ = [-2, 0, -3]` and `mins_ = [-2, -3]`, so `getMin()` is -3. `pop()` sees `data_.top() == -3 == mins_.top()`, pops both: `data_ = [-2, 0]`, `mins_ = [-2]`. `top()` is 0 and `getMin()` is -2.

For the duplicate case: `push(1)`, `push(1)`, `push(2)` gives `data_ = [1, 1, 2]` and, thanks to `<=`, `mins_ = [1, 1]`. `pop()` removes the 2 (not equal to 1, so `mins_` untouched). `pop()` removes a 1 and one 1 from `mins_`. `mins_ = [1]`, so `getMin()` is still 1.

Complexity. Each operation touches the tops of two piles: $O(1)$ time. Memory is $O(n)$ in the worst case (a falling sequence puts every value on both piles).

Robotics. A pile of running aggregates is the shape of any scoped state with cheap undo: pushing a transform onto a chain while keeping the cumulative pose, or pushing a cost onto a planner's backtracking stack while tracking the best cost seen so far. The two-stack trick works for any "combine" operation that you can recompute from the history: min, max, sum, matrix product.

Follow-ups you may get. - *One stack instead of two?* Store pairs (value, minSoFar) on a single stack. Same time, about twice the memory per element. - *What breaks with < instead of <=?* Push 1, 1, pop once: the only 1 on mins_ is gone and getMin() reports the wrong thing (or reads an empty stack). - *O(1) extra memory?* There is a trick that stores encoded differences $2*v - min$ in a long long stack. Know that it exists; it is rarely expected.

Part C — Check yourself

Answer each in plain words, with a drawing where one helps. If you cannot, reread that section.

1. Draw `int a = 5; int* p = &a; int& r = a;` as boxes. Then show what `*p = 6` and `r = 7` do.
2. What is `x` after `void f(int a) { ++a; }` is called as `f(x)` with `x = 1`? Which two signatures would make the caller see the change?
3. Name three things a reference cannot do that a pointer can.
4. Read `const int* p` and `int* const p` out in words, and say which assignment each forbids.
5. Why do we pass a `std::string` as `const std::string&` instead of `std::string`?
6. A getter is written without `const`. What stops compiling, and what is the right fix?
7. Two objects `a` then `b` are created in one block. In what order do their destructors run, and who calls them?
8. What does RAII mean in the library-book picture, and name two standard classes that do it.
9. State the rule of zero in one sentence.
10. Draw the `shared_ptr` counter for: create `s1`, copy to `s2` inside a block, leave the block, then `s1.reset()`. When is the object deleted? What does a `weak_ptr` see at each step?
11. Why is `int& bad() { int x = 1; return x; }` wrong? Draw the stack trays before and after the return.
12. Why does `while (--n >= 0)` never stop when `n` is a `size_t`? Write two safe backwards loops.
13. What question does a `std::sort` comparator answer, and why must it use `<` rather than `<=`?
14. Write the type of a priority queue of (double, int) pairs with the smallest double on top.
15. What does `m["z"]` do on a missing key, what should you use for a read-only check, and what is wrong with `m.erase(it); ++it;?`

03. Grids and BFS/DFS

This is the chapter everything else in robotics path planning is built on. A robot's map is a grid. "Find the shortest way to the goal" is BFS. "Which cells can the robot reach" is DFS. Almost every grid question you will be asked is one of four patterns, and this chapter teaches each pattern from zero, with a hand trace, the Python you would have written, and then the C++.

Rough time: 6 days at about 2 hours a day. Day 1 is Part A only. Do not start the problems until you can explain BFS and DFS in your own words from Part A.

What you need before this chapter (all in chapter 01): `std::vector<std::vector<int>>`, `std::pair`, `std::queue`, `auto [a, b] = ...`, and how to compile with `g++`. If any of those look foreign, go back to chapter 01 first; this chapter will not make sense otherwise.

Part A — The four patterns, from zero

A1. What a grid is, and how to index it

A grid is a table of cells. Here is a 3-row by 4-column grid with a wall in it (# is a wall, . is free):

	c=0	c=1	c=2	c=3
r=0	.	.	.	.
r=1	.	#	#	.
r=2	.	.	.	.

Two facts you must have burned in:

1. **grid[r][c] means row r, then column c.** r grows *downwards*, c grows to the *right*. So `grid[1][2]` is row 1, column 2: the second #. This is the same as Python's `grid[r][c]` on a list of lists. It is *not* (x, y); if you think "x then y" you will swap them and get a bug that is very hard to see.
2. **The size.** `rows = grid.size()` and `cols = grid[0].size()`. In Python that was `len(grid)` and `len(grid[0])`.

In Python you would write the grid above as:

```
grid = [
    ['.','.','.','.'],
    ['.','#','#','.'],
    ['.','.','.','.'],
]
rows, cols = len(grid), len(grid[0])
```

In C++ it is the same thing with types written out:

```
std::vector<std::vector<char>> grid = {
    {'.','.','.','.'},
    {'.','#','#','.'},
    {'.','.','.','.'},
};
int rows = static_cast<int>(grid.size());
int cols = static_cast<int>(grid[0].size());
```

(You can skip this paragraph on a first read.) Why the `static_cast<int>?` `size()` gives back an "unsigned" number, a type that cannot be negative. Mixing it with ordinary `ints` makes the compiler warn. Converting it to `int` once at the top, and using `int` everywhere after, keeps things simple. Just copy those two lines in every grid problem.

Neighbours. A cell's 4 neighbours are up, right, down, left. For the cell at (r, c) :

	$(r-1, c)$	up	
$(r, c-1)$	(r, c)	$(r, c+1)$	left, me, right
	$(r+1, c)$	down	

Instead of writing four separate `ifs`, everyone stores the four *offsets* in two little arrays and loops over them. Python:

```
for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]: # up, right, down, left
    nr, nc = r + dr, c + dc
```

C++:

```
const int dr[4] = {-1, 0, 1, 0}; // change in row: up, right, down, left
const int dc[4] = {0, 1, 0, -1}; // change in column: up, right, down, left
for (int k = 0; k < 4; ++k) {
    int nr = r + dr[k];
    int nc = c + dc[k];
    // ... use (nr, nc)
}
```

Read it as: "for each of the 4 directions k , the neighbour is my row plus $dr[k]$, my column plus $dc[k]$ ". nr/nc mean "new row / new column". You will type these two arrays in every grid problem for the rest of your life.

Staying inside the grid. The cell at $(0, 0)$ has no "up" neighbour: $(-1, 0)$ is outside. In Python, `grid[-1][0]` silently gives you the *last* row, which is a bug you never notice. In C++, `grid[-1][0]` is undefined behaviour: it may crash, or may silently read garbage. So every grid algorithm has this check, and it must come *before* you touch `grid[nr][nc]`:

```
bool inBounds(int r, int c, int rows, int cols) {
    return r >= 0 && r < rows && c >= 0 && c < cols;
}
// ...
if (!inBounds(nr, nc, rows, cols)) continue; // skip neighbours outside the grid
if (grid[nr][nc] == '#') continue; // skip walls; only safe AFTER the bounds check
```

That is all the grid machinery there is: `grid[r][c]`, the `dr/dc` arrays, and `inBounds`. Everything below is built from those three pieces.

In your own words: "I index the grid as row then column, keep the four direction offsets in two arrays, and always check bounds before I read a neighbour."

A1b. Three tools you need: a queue, a stack, and recursion

Everything in this chapter is built from three simple tools. If you know them, skip ahead.

A queue is a line at a shop. People join at the back and are served from the front, so whoever arrived first is served first. In C++:

```
#include <queue>
std::queue<int> q;
q.push(5); // 5 joins the back of the line
q.push(7); // 7 joins behind it
int x = q.front(); // look at who is first: 5
q.pop(); // the first person leaves (this returns nothing; you looked with front() already)
bool e = q.empty(); // is the line empty? no, 7 is still there
```

Python's `collections.deque` with `append` and `popleft` is the same thing. The one habit to change: in C++, "look" (`front()`) and "remove" (`pop()`) are two separate steps.

A stack is a pile of plates. You put a plate on top, and you take a plate from the top, so the *last* one put on is the *first* one taken off. In C++:

```
#include <stack>
std::stack<int> st;
st.push(5);      // plate 5 on the pile
st.push(7);      // plate 7 on top of it
int x = st.top(); // look at the top plate: 7
st.pop();        // take it off; 5 is on top now
```

A Python list with `append` and `pop()` is a stack.

Recursion is a function that calls itself on a smaller version of the problem. Think of Russian dolls: to open the whole set, open one doll and then "open the whole set" that is inside, which is the same task but smaller, until you reach the tiny solid doll and stop. The "stop" rule is called the base case, and every recursive function must have one or it runs forever.

```
void countDown(int n) {
    if (n == 0) return; // base case: nothing left to do
    std::cout << n << "\n";
    countDown(n - 1);   // the same task, one smaller
}
```

Each call has its own copy of `n`. The computer keeps a pile of the unfinished calls (that pile is literally called the call stack), and when a call finishes, the one below it carries on. That is what lets DFS "back up" for free.

A2. BFS: the ripple

The picture. Drop a stone into a pond at the start cell. The ripple spreads outward one ring at a time: first all cells 1 step away, then all cells 2 steps away, and so on. When the ripple first touches the goal, the ring number *is* the shortest distance. That is Breadth-First Search, and that is why it finds shortest paths: it explores in order of distance.

The tool. A *queue*: you add cells to the back and take cells from the front, so cells are processed in the order they were discovered. Cells discovered at distance 1 are all processed before any cell discovered at distance 2.

The hand trace. Start at `S = (0, 0)` in the grid from A1. We keep a `dist` table, all cells set to `-1` meaning "not reached yet". Reaching a cell means writing its distance and putting it in the queue.

Step 0: `dist[0][0] = 0`, `queue = [(0,0)]`.

```
dist:  0  -1  -1  -1
      -1  #   #  -1
      -1  -1  -1  -1
```

Step 1: take `(0,0)` from the front. Its neighbours: up is out of bounds, right is `(0,1)`, down is `(1,0)`, left is out of bounds. Both unreached, so they get distance `0 + 1 = 1` and go in the queue. `queue = [(0,1), (1,0)]`.

```
dist:  0  1  -1  -1
      1  #   #  -1
      -1 -1  -1  -1
```

Step 2: take `(0,1)`. Neighbours: right `(0,2)` unreached $\rightarrow$ dist 2. Down `(1,1)` is a wall, skip. Left `(0,0)` already has a distance, skip. `queue = [(1,0), (0,2)]`.

Step 3: take (1,0). Down (2,0) unreachable → dist 2. queue = [(0,2), (2,0)].

```
dist:    0   1   2  -1
         1   #   #  -1
         2  -1  -1  -1
```

Step 4: take (0,2). Right (0,3) → dist 3. queue = [(2,0), (0,3)]. Step 5: take (2,0). Right (2,1) → dist 3. queue = [(0,3), (2,1)].

```
dist:    0   1   2   3
         1   #   #  -1
         2   3  -1  -1
```

Keep going and the table fills in:

```
dist:    0   1   2   3
         1   #   #   4
         2   3   4   5
```

Notice: every cell was reached exactly once, and its number is the true shortest distance from S. The queue processed all the 1s before the 2s, all the 2s before the 3s. That ordering is the whole reason BFS works.

The Python you would have written.

```
from collections import deque

def bfs(grid, sr, sc):
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    dist[sr][sc] = 0
    q = deque([(sr, sc)])
    while q:
        r, c = q.popleft()
        for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != '#' and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

The same thing in C++. Read the two side by side; every line has a twin.

```
#include <queue>
#include <utility>    // std::pair
#include <vector>

std::vector<std::vector<int>> bfs(const std::vector<std::vector<char>>& grid, int sr, int sc) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));    // rows x cols, all -1
    const int dr[4] = {-1, 0, 1, 0};
    const int dc[4] = {0, 1, 0, -1};

    std::queue<std::pair<int, int>> q;    // a queue of (row, col) pairs
    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front();    // look at the front...
        q.pop();                    // ...then remove it. Two calls, unlike popleft().
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k];
            const int nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;    // outside the grid
        }
    }
}
```

```

        if (grid[nr][nc] == '#') continue; // wall
        if (dist[nr][nc] != -1) continue; // already reached
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
return dist;
}

```

Python	C++	Note
<code>deque([(sr, sc)])</code>	<code>std::queue<std::pair<int,int>> q; q.push({sr, sc});</code>	You must say what the queue holds: pairs of ints.
<code>r, c = q.popleft()</code>	<code>auto [r, c] = q.front(); q.pop();</code>	C++ <code>pop()</code> returns nothing. Look with <code>front()</code> , then <code>pop()</code> .
<code>[[[-1]*cols for _ in range(rows)]]</code>	<code>std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));</code>	"rows copies of a row of cols minus-ones".
<code>0 <= nr < rows</code>	<code>nr >= 0 && nr < rows</code>	No chained comparisons in C++.
<code>q.append((nr, nc))</code>	<code>q.push({nr, nc});</code>	<code>{nr, nc}</code> builds the pair.
<code>while q:</code>	<code>while (!q.empty())</code>	A queue is not "truthy" in C++.

The one rule that people get wrong. Mark a cell as reached (write its `dist`) *when you push it*, not when you pop it. If you wait until popping, the same cell can be pushed by several neighbours before it is popped, the queue fills with duplicates, and the distances are still right but level counting (next paragraph) breaks. Marking on push means every cell enters the queue exactly once.

Variant 1: counting rings ("how many minutes / steps"). Sometimes you do not want a distance per cell, you want "how many rounds until everything is reached". Process the queue one ring at a time: look at how many cells are in the queue right now, pop exactly that many, and then count one round.

```

int rounds = 0;
while (!q.empty()) {
    const int ringSize = static_cast<int>(q.size()); // how many cells are in THIS ring
    for (int i = 0; i < ringSize; ++i) {
        auto [r, c] = q.front(); q.pop();
        // ... push unreached neighbours as before ...
    }
    ++rounds; // one whole ring done
}

```

The `ringSize` snapshot matters: the queue grows while you are inside the loop, so you must record its size before you start.

Variant 2: many starting points ("multi-source BFS"). If there are several starts, push *all* of them with distance 0 before the loop. Nothing else changes. Each cell then ends up with the distance to the *nearest* start. Rotting Oranges and 01 Matrix are exactly this.

In your own words: "BFS uses a queue so cells are processed in order of distance. I mark a cell when I push it so it enters the queue once. The first time I reach a cell is by a shortest path. For several starting points I push all of them first; for a round count I process the queue one ring at a time."

A3. DFS: walk until stuck, then back up

The picture. You are in a maze with a piece of chalk. Walk forward, marking every cell you step on. When you cannot go any further, back up to the last cell that still has an unmarked neighbour and try that. Eventually every cell connected to your start is marked. That is Depth-First Search. It does *not* find shortest paths (you may wander a long way round), but it answers "what can I reach from here?" and "how many separate regions are there?", which is what most DFS problems ask.

The tool. Recursion. "Visit this cell, then visit each unvisited neighbour" is a function that calls itself. The call stack does the "back up" for you: when the recursive call returns, you are back at the previous cell.

The hand trace. Same grid, start at (0,0), mark with x:

```
visit (0,0) -> mark. Try up: out. Try right (0,1):
  visit (0,1) -> mark. Try up: out. Try right (0,2):
    visit (0,2) -> mark. Try right (0,3):
      visit (0,3) -> mark. up out, right out, down (1,3):
        visit (1,3) -> mark. down (2,3):
          visit (2,3) -> mark. down out, left (2,2):
            visit (2,2) -> mark. ... this keeps going along the bottom row ...
            ... (2,1), (2,0), (1,0) all marked, then it is stuck: everything around is marked or a wall
            returns, returns, returns ... all the way back up to (0,0)
(0,0) tries down (1,0): already marked. Done.
```

Every free cell got marked exactly once. The path taken was long and wiggly, and that is fine for "can I reach it", not for "how far is it".

The Python you would have written.

```
def dfs(grid, r, c, visited):
    rows, cols = len(grid), len(grid[0])
    if not (0 <= r < rows and 0 <= c < cols):    # outside
        return
    if grid[r][c] == '#' or visited[r][c]:        # wall or already marked
        return
    visited[r][c] = True
    for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]:
        dfs(grid, r + dr, c + dc, visited)
```

Notice the shape: the checks that *stop* the recursion are at the top, then mark, then recurse on the four neighbours. That shape is every DFS.

The same thing in C++.

```
void dfs(const std::vector<std::vector<char>>& grid, int r, int c,
         std::vector<std::vector<bool>>& visited) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    if (r < 0 || r >= rows || c < 0 || c >= cols) return;    // outside
    if (grid[r][c] == '#' || visited[r][c]) return;          // wall or already marked
    visited[r][c] = true;
    dfs(grid, r - 1, c, visited);    // up
    dfs(grid, r, c + 1, visited);    // right
    dfs(grid, r + 1, c, visited);    // down
    dfs(grid, r, c - 1, visited);    // left
}
```

`visited` is passed by reference (&) so all the recursive calls share one table. Without the &, each call would get its own copy and nothing would ever count as visited. This is the single most common C++ mistake in this chapter.

A cheaper "visited". Very often you are allowed to change the grid itself. Then instead of a separate visited table you overwrite the cell: turn '1' into '0', turn the fresh orange into a rotten one, turn the old colour into the new colour. The overwrite *is* the mark. It is less code and less memory; just be aware you are modifying the input, and copy it first if the caller still needs the original.

When recursion is too deep. Each recursive call uses a little stack memory, and the stack is about 8 MB on Linux. On a 1000 by 1000 grid that is all free cells, DFS can recurse a million levels deep and crash. Python would raise `RecursionError`; C++ just dies with a segmentation fault. The fix is to run DFS with an explicit stack instead of recursion: it is the BFS loop from A2 with `std::stack` in place of `std::queue` and `top()` in place of `front()`. You will see it in Number of Islands. Use recursion while learning; switch to the explicit stack when the grid can be large.

BFS or DFS?

You want...	Use	Why
Shortest distance / fewest steps	BFS	Explores in order of distance
"Can I reach it" / "which cells are connected"	Either; DFS is shorter to write	Both visit everything reachable
Count separate regions (islands)	DFS (or BFS) from each unvisited cell	Each start that finds new cells is a new region
Try every path and undo (Word Search)	DFS with backtracking (A4)	Needs the "back up" behaviour
Nearest-of-many (distance to closest 0)	Multi-source BFS	Push all sources first

In your own words: "DFS visits a cell, marks it, and recurses into unvisited neighbours; the call stack backs up for me. It finds everything reachable but not shortest paths. On a large grid recursion can overflow the stack, so I would use an explicit stack instead."

A4. Backtracking: mark, try, unmark

Plain DFS marks a cell once and leaves it marked, because it only cares *whether* a cell can be reached. Some problems ask for a *specific path*: "does the word CAT appear as a path of adjacent cells, using each cell at most once?" Now a cell being used on one attempted path must become free again when that attempt fails, so a different path can use it.

The pattern is three lines around the recursion:

```
mark the cell as in use
try each neighbour (recurse)
unmark the cell          <-- this is the only difference from DFS
```

Tiny trace. Looking for "AB" in

```
A  B
B  A
```

Start at $(0,0) = A$, matches letter 0. Mark it. Look for B next to it: $(0,1)$ is B, matches letter 1, that was the last letter, return true. Done. Now suppose the word were "ABA": from $(0,1)$ we need an A: $(1,1)$ is A, found. But if $(1,1)$ had been B, the attempt fails, we *unmark* $(0,1)$, back at $(0,0)$ we try down $(1,0) = B$, and so on. The unmark is what lets $(0,1)$ be reused by a later attempt starting somewhere else.

The cost: DFS visits each cell once, so it is linear in the grid size. Backtracking can revisit cells on different paths, so its worst case is exponential in the path length. That is expected for "find a specific path" problems and there is no way round it.

In your own words: "Backtracking is DFS where the visited mark is temporary: mark, recurse, unmark, so each cell can be reused on a different path. It is exponential in the worst case, which is unavoidable for search-for-a-path problems."

A5. DP on a grid: fill in a table

Some grid problems are not about searching at all. "How many ways can a robot walk from the top-left to the bottom-right if it can only move right or down?" There is no wandering here; each cell's answer depends only on the cell above and the cell to its left.

The picture. Fill in a table. The number of ways to reach a cell is (ways to reach the cell above) + (ways to reach the cell to the left), because the last step came from one of those two. The top row and left column are all 1 (only one way: straight along the edge).

ways:	1	1	1	1
	1	2	3	4
	1	3	6	10

`ways[2][3] = ways[1][3] + ways[2][2] = 4 + 6 = 10.`

That is dynamic programming on a grid: define what each cell means (`ways[r][c]` = number of paths to `(r, c)`), write how it comes from its neighbours (the *transition*), fill in the edges (*base cases*), and loop. With obstacles, an obstacle cell gets 0. For "minimum cost path", replace the sum with `cost[r][c] + min(above, left)`.

In your own words: "Grid DP: I define what each cell stores, state the transition from the cell above and the cell to the left, set the base cases along the edges, then fill the table row by row. Time is rows times columns."

A6. Why this matters for robots

A mobile robot's map is an *occupancy grid*: a 2D array of cells, each free, occupied, or unknown. In ROS this is `nav_msgs/OccupancyGrid`, a flat array of bytes with a width and height, exactly the `r * cols + c` flat indexing you will see in some solutions.

- **BFS from the goal** across the free cells gives every cell its distance to the goal. That is the *wavefront planner*: the robot just walks downhill. Rotting Oranges is a wavefront.
- **Multi-source BFS from every obstacle** gives every cell its distance to the nearest obstacle. That is the *distance transform*, and it is how a costmap's *inflation layer* is built (cells near walls get a high cost so the planner keeps its distance). 01 Matrix is exactly this.
- **DFS / flood fill from the robot** gives the set of reachable cells, used to find frontiers for exploration and to reject unreachable goals.
- **A*** is BFS's grown-up sibling for weighted grids, and it is what the global planner in the ROS navigation stack actually runs. You build it at the end of this chapter.

So every grid problem below is really the inner loop of a planner. When you see one, name the robotics version in your head: "this is a wavefront", "this is the distance transform".

A7. A* in one paragraph (details in Part C)

BFS treats every step as cost 1 and explores rings blindly in all directions. A *does two things differently*. First, it allows steps to have different costs (a diagonal costs about 1.414, a muddy cell costs more), so instead of a plain queue it uses a priority queue that always pops the cheapest cell so far. That alone is called Dijkstra. Second, it adds a guess of the remaining distance to the goal (for example the straight-line distance) to each

cell's priority, so the search leans toward the goal instead of spreading in a circle. If the guess never over-estimates, the first time the goal is popped the path is optimal. You will write this from scratch, with tests, in Part C. Understand BFS thoroughly first; A is BFS with a smarter queue.

A8. The toolkit file

code/03-grids-bfs-dfs/00_grid_toolkit.cpp contains the BFS and DFS functions from A2 and A3 (single-source BFS, multi-source BFS, recursive DFS, stack-based DFS) on a small map, with asserts. Compile and run it, then change the map and predict the output before you run it again:

```
cd code
g++ -std=c++17 -Wall -Wextra -O2 03-grids-bfs-dfs/00_grid_toolkit.cpp -o toolkit && ./toolkit
```

You do not need to memorise that file. You need to be able to type A2's BFS and A3's DFS from a blank screen. Practise that now, before the problems.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution line by line, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Robot Return to Origin (LeetCode 657, easy)

The problem in plain words. A robot starts at the point (0, 0). You are given a string of moves, each character one of U (up), D (down), L (left), R (right). Each move is one step. Return true if, after all the moves, the robot is back at (0, 0), otherwise false.

```
moves = "UD"      -> up one, down one      -> back at origin -> true
moves = "LL"      -> left twice, now at (-2, 0) -> not at origin -> false
moves = "RRDD"    -> ends at (2, -2)        -> false
```

The string can be empty (then the robot never moved, answer true), and it can be up to 20,000 characters.

By hand. You do not need to draw the path. Count how far left versus right, and how far up versus down. The robot is back home only if the counts cancel: as many L as R, and as many U as D.

The idea. Keep two counters x and y. R adds 1 to x, L subtracts 1; U adds 1 to y, D subtracts 1. At the end check both are zero.

The Python you would have written.

```
def judgeCircle(moves):
    x = y = 0
    for m in moves:
        if m == 'U': y += 1
        elif m == 'D': y -= 1
        elif m == 'R': x += 1
        elif m == 'L': x -= 1
    return x == 0 and y == 0
```

In C++. This is the full file. main() at the bottom runs the tests.

```
// 01_robot_return_to_origin.cpp – LeetCode 657. Robot Return to Origin (easy)
#include <cassert>
#include <iostream>
#include <string>

// Net displacement is all that matters: sum the unit vectors and check for (0,0).
bool judgeCircle(std::string moves) {
    int x = 0, y = 0;
    for (const char m : moves) {
        switch (m) {
            case 'U': ++y; break;
            case 'D': --y; break;
            case 'R': ++x; break;
            case 'L': --x; break;
            default: break; // input guarantees only U/D/L/R
        }
    }
    return x == 0 && y == 0;
}

int main() {
    assert(judgeCircle("UD") == true);
    assert(judgeCircle("LL") == false);
    assert(judgeCircle("RRDD") == false);
    assert(judgeCircle("LDRRLRUULR") == false);
    assert(judgeCircle("") == true); // no moves: already at origin
    assert(judgeCircle("UDLR") == true);
    std::cout << "OK 01_robot_return_to_origin.cpp\n";
    return 0;
}
```

Line by line, what is new compared with Python:

- `bool judgeCircle(std::string moves)` says the function returns a `bool` and takes a string.
- `for (const char m : moves)` is Python's `for m in moves`. `const char` is the type of one character; `const` because we only read it.
- `switch (m) { case 'U': ++y; break; ... }` is the C++ way to write a chain of `if m == ...` on a single character. Every case needs `break` or execution falls through into the next case. `default:` is the "else". You could write `if / else if` instead; both are fine.
- `++y` is `y += 1`.

Walk through "RRDD". Start `x=0, y=0`. R: `x=1`. R: `x=2`. D: `y=-1`. D: `y=-2`. End: `x == 0` is false, so return false.

Complexity. One pass over the string: time proportional to its length, constant extra memory.

Robotics. This is dead reckoning at its simplest: integrate the commanded motions to estimate where you are. Real robots do this with wheel encoders and it drifts, which is why they also use sensors and a Kalman filter (chapter 06).

Follow-ups you may get. - *What if moves could be diagonal?* Add two more cases, or store direction vectors in a map from `char` to `(dx, dy)`. - *What if the robot turns instead of moving in absolute directions?* Keep a heading (0 to 3) and rotate it on L/R; that is Robot Bounded In Circle (LeetCode 1041).

2. Flood Fill (LeetCode 733, easy)

The problem in plain words. You are given an image as a grid of integers (each integer is a colour), a starting cell `(sr, sc)`, and a new colour. Recolour the starting cell *and every cell connected to it that has the same colour as the starting cell*. "Connected" means you can reach it by moving up, down, left or right through cells of that same colour. Return the modified image. This is the paint-bucket tool in an image editor.

```
image = 1 1 1      start (1, 1), new colour 2
        1 1 0
        1 0 1
```

The starting cell has colour 1. All the 1s connected to it (not the lone 1 in the bottom-right corner, which is cut off by 0s) become 2:

```
result = 2 2 2
         2 2 0
         2 0 1
```

By hand. Put your finger on the start. Colour it. Look at its four neighbours: any that has the *original* colour, move there and repeat. Stop when nothing next to you has the original colour. That is DFS from A3, where "already visited" is simply "no longer the original colour".

The idea. Write a recursive function `fill(r, c)`: if `(r, c)` is outside the image or is not the original colour, stop. Otherwise set it to the new colour and call `fill` on the four neighbours. Because a filled cell no longer has the original colour, it is never filled twice.

One trap: if the new colour equals the original colour, "not the original colour" is never true and the recursion never stops. Check for that up front and do nothing.

The Python you would have written.

```
def floodFill(image, sr, sc, color):
    rows, cols = len(image), len(image[0])
    original = image[sr][sc]
    if original == color:
        return image          # nothing to do; avoids infinite recursion
    def fill(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or image[r][c] != original:
            return
        image[r][c] = color
        fill(r + 1, c); fill(r - 1, c); fill(r, c + 1); fill(r, c - 1)
    fill(sr, sc)
    return image
```

In C++.

```
// 02_flood_fill.cpp - LeetCode 733. Flood Fill (easy)
#include <cassert>
#include <iostream>
#include <vector>

// Recursive DFS. The recolouring itself is the visited mark: once a cell is
// newColor it no longer equals the original colour and is skipped.
void fill(std::vector<std::vector<int>>& img, int r, int c, int from, int to) {
    const int rows = static_cast<int>(img.size());
    const int cols = static_cast<int>(img[0].size());
    if (r < 0 || r >= rows || c < 0 || c >= cols || img[r][c] != from) return;
    img[r][c] = to;
    fill(img, r + 1, c, from, to);
    fill(img, r - 1, c, from, to);
    fill(img, r, c + 1, from, to);
    fill(img, r, c - 1, from, to);
}

std::vector<std::vector<int>> floodFill(std::vector<std::vector<int>>& image, int sr, int sc, int color) {
    // If the start already has the target colour, filling would never terminate
    // without this guard (every neighbour "still" matches the original colour).
    if (image[sr][sc] != color) fill(image, sr, sc, image[sr][sc], color);
    return image;
}

int main() {
    {
        std::vector<std::vector<int>> img = {{1, 1, 1}, {1, 1, 0}, {1, 0, 1}};
    }
}
```

```

    const std::vector<std::vector<int>> want = {{2, 2, 2}, {2, 2, 0}, {2, 0, 1}};
    assert(floodFill(img, 1, 1, 2) == want);
}
{
    std::vector<std::vector<int>> img = {{0, 0, 0}, {0, 0, 0}};
    const std::vector<std::vector<int>> want = img;
    assert(floodFill(img, 0, 0, 0) == want); // same colour: unchanged, no infinite loop
}
{
    std::vector<std::vector<int>> img = {{5}};
    assert(floodFill(img, 0, 0, 7) == std::vector<std::vector<int>>{{7}});
}
{
    // Diagonal cells are not 4-connected: the two 1s stay separate.
    std::vector<std::vector<int>> img = {{1, 0}, {0, 1}};
    const std::vector<std::vector<int>> want = {{9, 0}, {0, 1}};
    assert(floodFill(img, 0, 0, 9) == want);
}
std::cout << "OK 02_flood_fill.cpp\n";
return 0;
}

```

What is new:

- C++ has no nested functions, so `fill` is a separate function above `floodFill`, and everything it needs (`img`, the original colour `from`, the new colour `to`) is passed as parameters.
- `std::vector<std::vector<int>>& img`: the `&` means "work on the caller's image, do not copy it". Without the `&` every recursive call would paint its own private copy and the caller would see no change. This is the mistake to watch for in every recursive grid function.
- The four neighbour checks and the colour check are combined into one `if (...) return;` at the top. Order matters: the bounds checks come first so `img[r][c]` is never read out of range.
- `floodFill` returns `image` by value at the end; that copies the grid once, which is what LeetCode's signature asks for.

Walk through the example. `from = 1, to = 2`. `fill(1,1)`: it is 1, set to 2. Recurse down `fill(2,1)`: it is 0, stop. Up `fill(0,1)`: `1 → 2`; from there `fill(-1,1)` is out, `fill(1,1)` is now 2 so stop, `fill(0,2)`: `1 → 2`, ..., `fill(0,0)`: `1 → 2`, `fill(1,0)`: `1 → 2`, `fill(2,0)`: `1 → 2`. Everything connected is now 2. The cell `(2,2)` was never reached because every route to it goes through a 0.

Complexity. Each cell is painted at most once, so time is proportional to the number of cells. Memory is the recursion depth, which in the worst case (a snake-shaped region) is also the number of cells.

Robotics. Flood fill from the robot's own cell over the free cells of an occupancy grid gives the region the robot can actually reach. Cells outside it are unreachable goals; cells on its edge next to unknown space are *frontiers*, which is how exploration picks where to go next.

Follow-ups you may get. - *Iterative version?* Replace recursion with a `std::stack<std::pair<int,int>>`: push the start, loop while not empty, pop, paint, push same-colour neighbours. - *8-connected (diagonals count)?* Use the 8-entry `dr/dc` arrays instead of four calls.

3. Number of Islands (LeetCode 200, medium)

The problem in plain words. A grid of characters where '1' is land and '0' is water. An island is a group of '1' cells connected up/down/left/right (diagonal does *not* count). The grid is surrounded by water. Count the islands.

```

1 1 0 0 0
1 1 0 0 0      three islands: the 2x2 block top-left,
0 0 1 0 0      the single cell in the middle,
0 0 0 1 1      and the pair bottom-right.

```

The grid can be up to 300 by 300.

By hand. Scan the grid left to right, top to bottom. The first time you see a '1' that you have not already coloured in, that is a new island: add one to the count, then colour in the *whole* island so you do not count it again. Colouring in the whole island is flood fill from problem 2.

The idea. For every cell: if it is '1', increment the count and run a DFS that turns every connected '1' into '0' ("sink" the island). Later scans then skip those cells. Sinking the island in place is the visited mark; no separate table needed.

The Python you would have written.

```
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    def sink(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        sink(r + 1, c); sink(r - 1, c); sink(r, c + 1); sink(r, c - 1)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                sink(r, c)
    return count
```

In C++. This version uses an explicit stack instead of recursion. Read A3 again for why: a 300 by 300 grid of all land would recurse 90,000 levels deep, which is risky. The stack version is the same DFS with the "back up" done by a `std::stack` instead of the call stack.

```
// 03_number_of_islands.cpp - LeetCode 200. Number of Islands (medium)
#include <cassert>
#include <iostream>
#include <stack>
#include <utility>
#include <vector>

// Iterative DFS that sinks each island by overwriting '1' with '0' in place.
// Mutating the input is the cheapest visited set. Copy the grid first if the
// caller still needs the original.
void sink(std::vector<std::vector<char>>& g, int sr, int sc) {
    const int rows = static_cast<int>(g.size());
    const int cols = static_cast<int>(g[0].size());
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    std::stack<std::pair<int, int>> st;
    st.push({sr, sc});
    g[sr][sc] = '0';
    while (!st.empty()) {
        auto [r, c] = st.top();
        st.pop();
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || g[nr][nc] != '1') continue;
            g[nr][nc] = '0'; // mark when pushing so a cell is never pushed twice
            st.push({nr, nc});
        }
    }
}

int numIslands(std::vector<std::vector<char>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
```

```

        if (grid[r][c] == '1') {
            ++islands;
            sink(grid, r, c);
        }
    }
    return islands;
}

int main() {
    {
        std::vector<std::vector<char>> g = {
            {'1', '1', '1', '1', '0'},
            {'1', '1', '0', '1', '0'},
            {'1', '1', '0', '0', '0'},
            {'0', '0', '0', '0', '0'}};
        assert(numIslands(g) == 1);
    }
    {
        std::vector<std::vector<char>> g = {
            {'1', '1', '0', '0', '0'},
            {'1', '1', '0', '0', '0'},
            {'0', '0', '1', '0', '0'},
            {'0', '0', '0', '1', '1'}};
        assert(numIslands(g) == 3); // diagonals do not connect
    }
    {
        std::vector<std::vector<char>> g;
        assert(numIslands(g) == 0);
    }
    {
        std::vector<std::vector<char>> g = {{'0'}};
        assert(numIslands(g) == 0);
    }
    {
        // Large all-land grid: recursion would be 250k deep; the stack version is fine.
        std::vector<std::vector<char>> g(500, std::vector<char>(500, '1'));
        assert(numIslands(g) == 1);
    }
    std::cout << "OK 03_number_of_islands.cpp\n";
    return 0;
}

```

What is new:

- `std::stack<std::pair<int,int>> st;` is a stack of (row, col). `st.push`, `st.top`, `st.pop` match queue's `push`, `front`, `pop`. The only difference from the BFS loop in A2 is `top()` instead of `front()`, which makes it last-in-first-out and therefore a DFS.
- `g[nr][nc] = '0'` happens *when pushing*. Same rule as BFS: mark on push so each cell is pushed once.
- `if (grid.empty() || grid[0].empty()) return 0;` guards against an empty grid before `grid[0]` is touched. `grid[0]` on an empty vector is undefined behaviour in C++, not an exception.
- The nested for loops without braces around a single `if` are normal C++ style; add braces if you find it clearer.

Walk through the example. Scan: (0,0) is '1' → count = 1, sink the 2x2 block (four cells become '0'). Continue scanning: (0,1), (1,0), (1,1) are now '0', skip. (2,2) is '1' → count = 2, sink it (just one cell). (3,3) is '1' → count = 3, sink (3,3) and (3,4). End of grid: 3.

Complexity. Every cell is scanned once and sunk at most once: time proportional to rows × cols. The stack holds at most one entry per cell.

Robotics. This is connected-component labelling on a map. Grouping obstacle cells into distinct obstacles, grouping frontier cells into frontier regions for exploration, and clustering laser hits into objects all use exactly this loop.

Follow-ups you may get. - *Do not modify the input.* Keep a `std::vector<std::vector<bool>>` visited (or a flat `std::vector<char>` of size rows × cols) and check that instead of overwriting. - *Largest island instead of count?* Have sink return how many cells it sank; keep the max. - *Islands appear one at a time and you must report the count after each?* That is Number of Islands II (LeetCode 305), solved with union-find (chapter 04).

4. Rotting Oranges (LeetCode 994, medium)

The problem in plain words. A grid where each cell is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, every fresh orange that is directly next to a rotten one (up/down/left/right) becomes rotten. Return the number of minutes until no fresh orange is left. If some fresh orange can never be reached by the rot, return -1.

minute 0	minute 1	minute 2	minute 3	minute 4	
2 1 1	2 2 1	2 2 2	2 2 2	2 2 2	
1 1 0	2 1 0	2 2 0	2 2 0	2 2 0	
0 1 1	0 1 1	0 1 1	0 2 1	0 2 2	-> answer 4

The orange at (2,2) is the last to rot, at minute 4. If the grid were 2 1 1 / 0 1 1 / 1 0 1, the orange at (2,0) is walled off by zeros and never rots: answer -1. If there are no fresh oranges at all, the answer is 0.

By hand. Look at the minute-by-minute pictures. The rot spreads outward from *all* rotten oranges at once, one ring per minute. That is exactly the BFS ripple from A2, with two twists from the variants: it starts from *many* sources at once (every rotten orange), and we want to *count rings* (minutes) rather than record a distance per cell.

The idea. 1. Scan the grid. Push every rotten orange into the queue. Count the fresh ones. 2. Loop one ring at a time: snapshot the queue size, pop that many cells, and for each fresh neighbour mark it rotten (write 2 into the grid), decrement the fresh count, push it. After the ring, add one minute. 3. Stop when the queue is empty or no fresh oranges remain. If fresh is zero, return the minutes; otherwise -1.

The check `fresh > 0` in the loop condition stops us counting an extra minute in which nothing rots.

The Python you would have written.

```
from collections import deque

def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: q.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    while q and fresh > 0:
        for _ in range(len(q)):                # one ring
            r, c = q.popleft()
            for dr, dc in ((-1,0),(0,1),(1,0),(0,-1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
            minutes += 1
    return minutes if fresh == 0 else -1
```

In C++.

```
// 04_rotting_oranges.cpp - LeetCode 994. Rotting Oranges (medium) - multi-source BFS
#include <cassert>
#include <iostream>
```

```

#include <queue>
#include <utility>
#include <vector>

// Every rotten orange is a BFS source at time 0. Process the queue level by
// level; each level is one minute. The answer is the number of levels that
// actually rotted something, and -1 if any fresh orange survives.
int orangesRotting(std::vector<std::vector<int>>& grid) {
    const int rows = static_cast<int>(grid.size());
    const int cols = rows ? static_cast<int>(grid[0].size()) : 0;
    std::queue<std::pair<int, int>> q;
    int fresh = 0;
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) {
            if (grid[r][c] == 2) q.push({r, c});
            else if (grid[r][c] == 1) ++fresh;
        }
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    int minutes = 0;
    while (!q.empty() && fresh > 0) {
        const int levelSize = static_cast<int>(q.size()); // snapshot: the queue grows inside
        for (int i = 0; i < levelSize; ++i) {
            auto [r, c] = q.front();
            q.pop();
            for (int k = 0; k < 4; ++k) {
                const int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || grid[nr][nc] != 1) continue;
                grid[nr][nc] = 2; // the grid itself is the visited set
                --fresh;
                q.push({nr, nc});
            }
        }
        ++minutes;
    }
    return fresh == 0 ? minutes : -1;
}

int main() {
    {
        std::vector<std::vector<int>> g = {{2, 1, 1}, {1, 1, 0}, {0, 1, 1}};
        assert(orangesRotting(g) == 4);
    }
    {
        std::vector<std::vector<int>> g = {{2, 1, 1}, {0, 1, 1}, {1, 0, 1}};
        assert(orangesRotting(g) == -1); // bottom-left orange is isolated
    }
    {
        std::vector<std::vector<int>> g = {{0, 2}};
        assert(orangesRotting(g) == 0); // nothing fresh: zero minutes
    }
    {
        std::vector<std::vector<int>> g = {{1}};
        assert(orangesRotting(g) == -1); // fresh but no source
    }
    {
        std::vector<std::vector<int>> g = {{2, 1, 1, 1, 2}};
        assert(orangesRotting(g) == 2); // two sources meet in the middle
    }
    std::cout << "OK 04_rotting_oranges.cpp\n";
    return 0;
}

```

What is new:

- `const int cols = rows ? static_cast<int>(grid[0].size()) : 0;` is a ternary: "if rows is non-zero, use `grid[0].size()`, else 0". It avoids touching `grid[0]` on an empty grid.

- `const int levelSize = static_cast<int>(q.size());` is Python's `for _ in range(len(q))`. It must be taken *before* the inner loop because the queue grows during it.
- `grid[nr][nc] = 2;` is both "rot it" and "mark it visited". No separate visited table.
- `return fresh == 0 ? minutes : -1;` is Python's `minutes if fresh == 0 else -1`.

Walk through the example. Queue starts with `[(0,0)]`, `fresh = 6`. Minute 1 ring: pop `(0,0)`, rot `(0,1)` and `(1,0)`, `fresh = 4`, `minutes = 1`. Minute 2 ring: pop `(0,1)`, rot `(0,2)`; pop `(1,0)`, rot `(1,1)`; `fresh = 2`, `minutes = 2`. Minute 3 ring: pop `(0,2)`, nothing new; pop `(1,1)`, rot `(2,1)`; `fresh = 1`, `minutes = 3`. Minute 4 ring: pop `(2,1)`, rot `(2,2)`; `fresh = 0`, `minutes = 4`. Loop ends because `fresh` is 0. Return 4.

Complexity. Every cell enters the queue at most once: time and memory proportional to `rows × cols`.

Robotics. This is the wavefront planner. Seed the queue with the goal cell (or several goals), BFS outward over free cells, and every cell ends up labelled with its distance to the nearest goal. The robot then simply steps to whichever neighbour has the smaller number. The "minutes" here are the wavefront's distance rings.

Follow-ups you may get. - *Why not run BFS from each rotten orange separately and take the minimum?* That is `(number of rotten) × (grid size)`; pushing all sources at once gives the same answer in one pass. - *What if you also need to know which orange rotted each cell?* Store a source id alongside the cell when you push it, or keep a parallel `source` grid. - *Why mark when pushing rather than when popping?* So a fresh orange next to two rotten ones is pushed once, not twice, and the ring sizes stay correct.

5. Shortest Path in Binary Matrix (LeetCode 1091, medium)

The problem in plain words. You are given a square grid of 0s and 1s, `n` rows by `n` columns. 0 is a free cell, 1 is blocked. You start at the top-left cell `(0, 0)` and want to reach the bottom-right cell `(n-1, n-1)`. From a cell you may move to any of its **eight** neighbours (up, down, left, right, and the four diagonals), as long as the neighbour is free. Return the length of the shortest such path, where length is the **number of cells on the path, including the start and the goal**. If there is no path, or the start or goal is blocked, return `-1`.

```
grid =  0 0 0      One shortest path: (0,0) -> (0,1) -> (1,2) -> (2,2).
        1 1 0      That is 3 moves but 4 cells, so the answer is 4.
        1 1 0
```

Note the second move, `(0,1) -> (1,2)`: that is a diagonal step, down and right at once. Without diagonals the answer would be 5 (`(0,0) (0,1) (0,2) (1,2) (2,2)`).

More examples and the edge cases:

```
0 1      one diagonal step from (0,0) to (1,1): 2 cells    -> 2
1 0

1 0 0
1 1 0      the start cell is blocked                        -> -1
1 1 0

0          a 1x1 grid: start is the goal, the path is 1 cell -> 1

0 0 0
0 1 0      go round the blocked centre: (0,0)(0,1)(1,2)(2,2) -> 4
0 0 0
```

`n` is at most 100, so the grid has at most 10,000 cells.

Cells, not steps. Most path problems count moves. This one counts cells, which is always moves + 1. So the start cell alone has "length 1", and a 1x1 grid returns 1, not 0. The whole difference in the code is one character: the start gets distance 1 instead of 0.

By hand. This is the ripple from A2 with two changes. First, the ripple spreads to eight neighbours instead of four, so it also grows diagonally. Second, we do not need the distance of every cell; we can stop the moment the ripple touches the goal. Drop the stone at (0,0) with ring number 1. Ring 2 is every free neighbour of the start. Ring 3 is every free unreached neighbour of ring 2, and so on. The ring number of the goal, when it first appears, is the answer.

The eight neighbours. For the cell at (r, c):

```
(r-1, c-1) (r-1, c) (r-1, c+1)
(r, c-1) (r, c) (r, c+1)
(r+1, c-1) (r+1, c) (r+1, c+1)
```

Every neighbour is "row plus one of {-1, 0, +1}, column plus one of {-1, 0, +1}", except the middle one where both are 0 (that is the cell itself). So instead of an 8-entry table you can write two nested loops over -1..1 and skip the (0, 0) case. Both ways are fine; the solution below uses the loops.

The idea. 1. If the start or the goal is blocked, return -1 immediately. 2. Make a `dist` table of zeros. Zero means "not reached yet" (a real distance is never zero here, because the start is already 1). 3. Set `dist[0][0] = 1` and push (0,0). 4. Loop: pop the front. If it is the goal, return its distance. Otherwise, for each of the eight neighbours that is inside the grid, free, and unreached, write `dist + 1` and push it. 5. If the queue runs dry, the goal was never reached: return -1.

Why returning at the first pop is safe. BFS pops cells in order of distance: all the 2s come out before any 3, all the 3s before any 4. So the first time the goal is popped, no cell with a smaller distance is still waiting, and its distance is the smallest possible. You could also return at the moment you *push* the goal, since in BFS the distance written on push is already final; that saves finishing one ring. Returning on pop is the habit worth keeping, because it is the version that still works for Dijkstra and A* in Part C, where a pushed distance can later be improved and only the pop is final.

The Python you would have written.

```
from collections import deque

def shortestPathBinaryMatrix(grid):
    n = len(grid)
    if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
        return -1
    dist = [[0] * n for _ in range(n)]          # 0 = not reached
    dist[0][0] = 1                             # path length counts cells
    q = deque([(0, 0)])
    while q:
        r, c = q.popleft()
        if r == n - 1 and c == n - 1:
            return dist[r][c]
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                if dr == 0 and dc == 0:
                    continue
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and dist[nr][nc] == 0:
                    dist[nr][nc] = dist[r][c] + 1
                    q.append((nr, nc))
    return -1
```

In C++.

```
// 05_shortest_path_binary_matrix.cpp - LeetCode 1091. Shortest Path in Binary Matrix (medium)
// 8-connected BFS; path length counts cells, so the start contributes 1.
#include <cassert>
```

```

#include <iostream>
#include <queue>
#include <utility>
#include <vector>

int shortestPathBinaryMatrix(std::vector<std::vector<int>>& grid) {
    const int n = static_cast<int>(grid.size());
    if (n == 0 || grid[0][0] != 0 || grid[n - 1][n - 1] != 0) return -1;
    std::vector<std::vector<int>> dist(n, std::vector<int>(n, 0)); // 0 = unvisited
    std::queue<std::pair<int, int>> q;
    dist[0][0] = 1;
    q.push({0, 0});
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        if (r == n - 1 && c == n - 1) return dist[r][c]; // first pop of the goal is optimal in BFS
        // All eight neighbours, generated by a double loop instead of a table.
        for (int dr = -1; dr <= 1; ++dr)
            for (int dc = -1; dc <= 1; ++dc) {
                if (dr == 0 && dc == 0) continue;
                const int nr = r + dr, nc = c + dc;
                if (nr < 0 || nr >= n || nc < 0 || nc >= n || grid[nr][nc] != 0 || dist[nr][nc] != 0) continue;
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
    }
    return -1;
}

int main() {
    {
        std::vector<std::vector<int>> g = {{0, 1}, {1, 0}};
        assert(shortestPathBinaryMatrix(g) == 2); // one diagonal step
    }
    {
        std::vector<std::vector<int>> g = {{0, 0, 0}, {1, 1, 0}, {1, 1, 0}};
        assert(shortestPathBinaryMatrix(g) == 4);
    }
    {
        std::vector<std::vector<int>> g = {{1, 0, 0}, {1, 1, 0}, {1, 1, 0}};
        assert(shortestPathBinaryMatrix(g) == -1); // start blocked
    }
    {
        std::vector<std::vector<int>> g = {{0}};
        assert(shortestPathBinaryMatrix(g) == 1); // start == goal
    }
    {
        std::vector<std::vector<int>> g = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
        assert(shortestPathBinaryMatrix(g) == 4); // around the centre: (0,0)(0,1)(1,2)(2,2) or mirror
    }
    std::cout << "OK 05_shortest_path_binary_matrix.cpp\n";
    return 0;
}

```

What is new:

- `if (n == 0 || grid[0][0] != 0 || grid[n - 1][n - 1] != 0) return -1;` checks the empty grid *first*. `||` stops at the first true condition, so when `n == 0` the `grid[0][0]` part is never evaluated. That order matters: reading `grid[0][0]` on an empty vector is undefined behaviour in C++, not an `IndexError`.
- `std::vector<std::vector<int>> dist(n, std::vector<int>(n, 0));` is `[[0] * n for _ in range(n)]`.
- `for (int dr = -1; dr <= 1; ++dr) for (int dc = -1; dc <= 1; ++dc) { ... }` is the pair of `for dr in (-1, 0, 1) loops`. The outer `for` has no braces because its whole body is the single inner `for`. If you would rather have a table, this is the same thing:

```
cpp const int dr[8] = {-1, -1, -1, 0, 0, 1, 1, 1}; const int dc[8] = {-1, 0, 1, -1, 1, -1, 0, 1}; for
(int k = 0; k < 8; ++k) { /* nr = r + dr[k], nc = c + dc[k] */ }
```

- `const int nr = r + dr, nc = c + dc;` declares two `const int`s in one statement, exactly like `nr, nc = r + dr, c + dc`.

- The one long `if (... ) continue;` bundles four checks: outside the grid, blocked, already reached. The bounds checks come first for the same reason as in A1: `grid[nr][nc]` must not be read until you know `nr` and `nc` are valid.
- `return dist[r][c];` inside the `while` leaves the function straight away, as in Python.

Walk through the example (0 0 0 / 1 1 0 / 1 1 0). `dist[0][0] = 1`, queue `[(0,0)]`.

```
pop (0,0)  not the goal. Of its 8 neighbours only (0,1) is inside and free:
           dist[0][1] = 2.                               queue [(0,1)]
pop (0,1)  free unreached neighbours: (0,2) and (1,2) diagonally below-right:
           both get 3.                                     queue [(0,2), (1,2)]
pop (0,2)  (1,1) blocked, (1,2) already 3, nothing new.   queue [(1,2)]
pop (1,2)  (2,2) below gets 4; (2,1) blocked.           queue [(2,2)]
pop (2,2)  this is the goal: return dist[2][2] = 4.
```

Final dist table (0 means never reached):

```
1 2 3
0 0 3
0 0 4
```

Complexity. Every cell is pushed at most once and looks at 8 neighbours, so time is proportional to the number of cells: $O(n^2)$. The `dist` table and the queue are also $O(n^2)$.

Robotics. An 8-connected search over a binary occupancy grid is the simplest global planner. Notice one thing this problem allows that a real planner must not: the 0 1 / 1 0 example squeezes diagonally between two blocked cells that touch only at a corner. A robot has width and would clip both. The A* project in Part C refuses that move on purpose.

Follow-ups you may get. - *Diagonal moves should cost $\sqrt{2}$, not 1.* Then steps no longer have equal cost and plain BFS is wrong; use Dijkstra (a priority queue keyed on distance) or A, Part C. - *Why BFS and not DFS? DFS finds a path, not the shortest one. Only BFS explores in order of distance.* - Return the path itself, not just its length.* Store a `parent` cell for each cell when you push it, then walk back from the goal to the start and reverse.

6. 01 Matrix (LeetCode 542, medium)

The problem in plain words. You are given a grid of 0s and 1s. For every cell, find the distance to the nearest 0, counting a move up, down, left or right as 1 (no diagonals). Return a grid of the same shape holding those distances. Cells that are 0 have distance 0 to themselves. There is always at least one 0 in the grid.

```
input:  0 0 0      output: 0 0 0
        0 1 0        0 1 0
        1 1 1        1 2 1
```

The middle 1 at (1,1) touches a 0 directly, so its distance is 1. The bottom-middle cell (2,1) is two moves from the nearest 0 (for example (2,1) -> (1,1) -> (0,1) or (2,1) -> (2,0) -> (1,0)), so it gets 2.

A second example, with only one zero:

```
input:  0 1 1      output: 0 1 2
        1 1 1        1 2 3
```

Here every cell is just its Manhattan distance to the corner, because there is only one source. A 1x1 grid `[[0]]` returns `[[0]]`. The grid has at most 10,000 cells.

By hand. If there were a single 0, you would run the ripple from A2 starting there and the `dist` table is the answer. With several zeros, imagine dropping a stone into the pond at every zero at the same instant. All the ripples spread at the same speed. A cell is first touched by whichever ripple started closest to it, and the ring number at that moment is the distance to the nearest zero. That is Variant 2 in A2, multi-source BFS: push every zero into the queue with distance 0 before the loop starts, and run the ordinary BFS.

Why "all at once" gives the nearest zero. In single-source BFS, the first time a cell is reached is by a shortest path from the one source. With all zeros in the queue at distance 0, the queue still pops cells in order of distance: all the 0s, then every cell at distance 1 from *some* zero, then every cell at distance 2 from *some* zero. A cell at true distance d from its nearest zero is reached in ring d and cannot be reached earlier, because that would mean a zero closer than d . If it helps, picture one imaginary super-source joined to every zero by a free edge: single-source BFS from it gives exactly the same rings.

The naive way, and why it is slow. The first instinct is "for each 1, run a BFS until I hit a 0". Each of those BFS runs can scan the whole grid, and there can be nearly $\text{rows} \times \text{cols}$ ones, so the total is $(\text{rows} \times \text{cols})^2$. On a 10,000-cell grid that is up to 10^8 cell visits. The multi-source version visits each cell once: 10^4 . Same answer, ten thousand times less work.

The idea. 1. Make `dist` all -1 ("not reached"). 2. Scan the grid. Every 0 gets `dist = 0` and goes in the queue. 3. Ordinary BFS: pop, and for each in-bounds neighbour with `dist == -1`, write `dist + 1` and push. 4. Return `dist`.

Notice there is no "is this neighbour a 1?" check. Every zero is already marked with distance 0 before the loop, so the `dist == -1` test alone skips zeros and reached cells, and only unreached ones remain.

The Python you would have written.

```
from collections import deque

def updateMatrix(mat):
    rows, cols = len(mat), len(mat[0])
    dist = [[-1] * cols for _ in range(rows)]
    q = deque()
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                dist[r][c] = 0
                q.append((r, c))          # every zero is a source
    while q:
        r, c = q.popleft()
        for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

In C++.

```
// 06_01_matrix.cpp - LeetCode 542. 01 Matrix (medium) - distance transform via multi-source BFS
#include <cassert>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

// Seed the queue with every 0 at distance 0, then expand. The first time BFS
// reaches a cell is from its nearest zero, so the distance is final on push.
std::vector<std::vector<int>> updateMatrix(std::vector<std::vector<int>>& mat) {
    const int rows = static_cast<int>(mat.size());
    const int cols = rows ? static_cast<int>(mat[0].size()) : 0;
    std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));
    std::queue<std::pair<int, int>> q;
    for (int r = 0; r < rows; ++r)
```

```

    for (int c = 0; c < cols; ++c)
        if (mat[r][c] == 0) {
            dist[r][c] = 0;
            q.push({r, c});
        }
constexpr int dr[4] = {-1, 0, 1, 0};
constexpr int dc[4] = {0, 1, 0, -1};
while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();
    for (int k = 0; k < 4; ++k) {
        const int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || dist[nr][nc] != -1) continue;
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
return dist;
}

int main() {
{
    std::vector<std::vector<int>>> m = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    const std::vector<std::vector<int>>> want = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    assert(updateMatrix(m) == want);
}
{
    std::vector<std::vector<int>>> m = {{0, 0, 0}, {0, 1, 0}, {1, 1, 1}};
    const std::vector<std::vector<int>>> want = {{0, 0, 0}, {0, 1, 0}, {1, 2, 1}};
    assert(updateMatrix(m) == want);
}
{
    std::vector<std::vector<int>>> m = {{0}};
    assert(updateMatrix(m) == std::vector<std::vector<int>>>{{0}});
}
{
    // A single zero in the corner: distances are Manhattan distances to it.
    std::vector<std::vector<int>>> m = {{0, 1, 1}, {1, 1, 1}};
    const std::vector<std::vector<int>>> want = {{0, 1, 2}, {1, 2, 3}};
    assert(updateMatrix(m) == want);
}
std::cout << "OK 06_01_matrix.cpp\n";
return 0;
}

```

What is new:

- `std::vector<std::vector<int>>> updateMatrix(...)` returns a whole grid. `return dist;` at the end hands the table back to the caller; C++ moves it rather than copying, so this is cheap.
- `const int cols = rows ? static_cast<int>(mat[0].size()) : 0;` is the same empty-grid guard you saw in Rotting Oranges: only touch `mat[0]` if there is a row 0.
- `constexpr int dr[4] = {-1, 0, 1, 0};` is the direction table from A1. `constexpr` is `const` with the extra promise that the value is known at compile time. For a table of literals it behaves exactly like `const`; either spelling is fine here.
- The two scanning `for` loops and the `if` have no braces around the single statement they contain, and then braces around the two-line body. That is normal C++ style; add braces everywhere if you prefer.
- `auto [r, c] = q.front(); q.pop();` is `r, c = q.popleft();`, as in A2.
- The `main()` tests compare two `std::vector<std::vector<int>>>` with `==`. That works out of the box: vectors compare element by element, like Python lists.

Walk through the first example (0 0 0 / 0 1 0 / 1 1 1). The scan pushes the five zeros in reading order: queue [(0,0), (0,1), (0,2), (1,0), (1,2)], all with dist 0.

```

pop (0,0)  neighbours (0,1) and (1,0) already 0.      nothing new
pop (0,1)  down (1,1) unreachable -> 1.               queue [.., (1,1)]
pop (0,2)  down (1,2) already 0.                     nothing new
pop (1,0)  down (2,0) unreachable -> 1. right (1,1) done. queue [.., (1,1), (2,0)]
pop (1,2)  down (2,2) unreachable -> 1.               queue [(1,1), (2,0), (2,2)]
pop (1,1)  down (2,1) unreachable -> 2.               queue [(2,0), (2,2), (2,1)]
pop (2,0)  right (2,1) already 2.                     nothing new
pop (2,2)  left (2,1) already 2.                     nothing new
pop (2,1)  everything around is reached.               queue empty, done

```

The table now reads 0 0 0 / 0 1 0 / 1 2 1. Note (2,1) was pushed once, by (1,1), even though (2,0) and (2,2) also touch it: they arrived later and found it already marked.

Complexity. Every cell enters the queue exactly once and looks at 4 neighbours, so time is proportional to rows × cols: $O(\text{rows} \times \text{cols})$. Memory is the dist table plus the queue, also $O(\text{rows} \times \text{cols})$.

Robotics. This is the *distance transform*, also called brushfire, and it is how a costmap's inflation layer is built: every occupied cell is a source, BFS gives every free cell its distance to the nearest obstacle, and that distance is turned into a cost so the planner keeps the robot away from walls. The cells where two wavefronts meet (equal distance to two different obstacles) form the Voronoi diagram, which some planners follow as the "safest corridor" through a map.

Follow-ups you may get. - *Can you do it without a queue?* Yes, with two DP sweeps: top-left to bottom-right taking $\min(\text{above}, \text{left}) + 1$, then bottom-right to top-left taking $\min(\text{below}, \text{right}) + 1$. Same $O(\text{rows} \times \text{cols})$, no queue, but it only works for Manhattan distance. - *Euclidean distance instead of Manhattan?* Steps no longer have equal cost, so use Dijkstra from all sources, or a proper Euclidean distance transform (Felzenszwalb's algorithm is linear time). - *What if there are no zeros at all?* LeetCode promises at least one. Without it the queue starts empty and every cell stays -1, which is a reasonable "unreachable" answer to return or to document.

7. Unique Paths II (LeetCode 63, medium)

The problem in plain words. A robot sits in the top-left cell of a grid with rows rows and cols columns and wants to reach the bottom-right cell. It can only move **right** or **down**, one cell at a time. Some cells contain an obstacle (1); the rest are free (0). The robot cannot enter an obstacle cell. Count how many different paths lead from the top-left to the bottom-right. Two paths are different if they differ in at least one cell.

```

grid =  0 0 0      . . .      Only two paths avoid the centre:
        0 1 0      . # .      right, right, down, down
        0 0 0      . . .      down, down, right, right
                                     -> 2

```

Without the obstacle there would be 6 paths (the ways table in A5 gives 6 for a 3x3 grid). Edge cases: if the start or the goal is an obstacle the answer is 0; a 1x1 free grid has one path (stand still), answer 1; a full row or full column of obstacles blocks everything, answer 0. The grid is at most 100 by 100, and the final answer is promised to fit in a 32-bit int.

By hand. This is the grid DP from A5, with one extra rule. Write in each cell the number of ways to reach it. The number for a cell is (ways to reach the cell above) + (ways to reach the cell to the left), because the last move into it came from one of those two. An obstacle cell gets 0: you cannot stand there, so no path passes through it. Fill the table row by row, left to right. The example fills in like this:

```

grid:      . . .      ways:    1  1  1
          . # .          1  0  1
          . . .          1  1  2

```

Row 0 is all 1s (only one way along the top edge). In row 1, (1,0) is 1 (from above), (1,1) is the obstacle so 0, and (1,2) = above 1 + left 0 = 1. In row 2, (2,0) = 1, (2,1) = above 0 + left 1 = 1, (2,2) = above 1 + left 1 = 2. The bottom-right cell holds the answer.

The idea. 1. If the start cell is an obstacle, return 0. 2. Define $ways[r][c]$ = number of right/down paths from (0,0) to (r,c). 3. $ways[0][0] = 1$. For every other cell: if it is an obstacle, 0; otherwise $ways[r-1][c] + ways[r][c-1]$, treating a missing neighbour (off the top or left edge) as 0. 4. Fill row by row, left to right, so both the cell above and the cell to the left are ready when you need them. Return $ways[rows-1][cols-1]$.

The Python you would have written. This is the two-dimensional table exactly as drawn.

```
def uniquePathsWithObstacles(obstacleGrid):
    rows, cols = len(obstacleGrid), len(obstacleGrid[0])
    if obstacleGrid[0][0] == 1:
        return 0
    ways = [[0] * cols for _ in range(rows)]
    ways[0][0] = 1
    for r in range(rows):
        for c in range(cols):
            if obstacleGrid[r][c] == 1:
                ways[r][c] = 0
                continue
            if r > 0:
                ways[r][c] += ways[r - 1][c]      # from above
            if c > 0:
                ways[r][c] += ways[r][c - 1]      # from the left
    return ways[rows - 1][cols - 1]
```

Squashing the table to one row. Look at what row 2 of the table needs: only row 1 (the cell above) and row 2 itself (the cell to the left). Row 0 is never looked at again. So you do not need to keep the whole table, just one row that you overwrite in place. Call it `dp`. When you start processing row `r`, `dp[c]` still holds row `r-1`'s value for column `c`, which is exactly "the cell above". Sweep `c` from left to right: by the time you reach column `c`, `dp[c-1]` has already been updated to row `r`, which is exactly "the cell to the left". So the update is one line:

```
dp[c] = dp[c] + dp[c-1]
      ^^^^^  ^^^^^^^
      above  left (already updated this row)
```

An obstacle sets `dp[c] = 0`, and column 0 has no left neighbour so it keeps its "from above" value unchanged. The C++ file uses this rolling row. Here is the trace of `dp` for the example, one line per row:

start	<code>dp = [1, 0, 0]</code>	(<code>dp[0] = 1</code> is the start cell)
after row 0	<code>dp = [1, 1, 1]</code>	1, then 0+1, then 0+1
after row 1	<code>dp = [1, 0, 1]</code>	1 from above; obstacle -> 0; 1 + 0
after row 2	<code>dp = [1, 1, 2]</code>	1 from above; 0 + 1; 1 + 1
answer = <code>dp[2]</code>	<code>= 2</code>	

Compare each line with a row of the 2-D table above: they are the same numbers.

In C++.

```
// 07_unique_paths_ii.cpp - LeetCode 63. Unique Paths II (medium) - DP with obstacles
#include <cassert>
#include <iostream>
#include <vector>

// dp[c] = number of ways to reach (current row, c) moving only right/down.
// Rolling a single row: dp[c] already holds "from above", add dp[c-1] "from left".
// An obstacle zeroes its cell so nothing flows through it.
int uniquePathsWithObstacles(std::vector<std::vector<int>>& obstacleGrid) {
    const int rows = static_cast<int>(obstacleGrid.size());
    const int cols = rows ? static_cast<int>(obstacleGrid[0].size()) : 0;
    if (rows == 0 || cols == 0 || obstacleGrid[0][0] == 1) return 0;
```

```

std::vector<long long> dp(cols, 0); // long long: counts can exceed int mid-grid even if the answer fits
dp[0] = 1;
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c) {
        if (obstacleGrid[r][c] == 1) dp[c] = 0;
        else if (c > 0) dp[c] += dp[c - 1];
    }
return static_cast<int>(dp[cols - 1]);
}

int main() {
    {
        std::vector<std::vector<int>> g = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
        assert(uniquePathsWithObstacles(g) == 2);
    }
    {
        std::vector<std::vector<int>> g = {{0, 1}, {0, 0}};
        assert(uniquePathsWithObstacles(g) == 1);
    }
    {
        std::vector<std::vector<int>> g = {{1}};
        assert(uniquePathsWithObstacles(g) == 0); // start blocked
    }
    {
        std::vector<std::vector<int>> g = {{0}};
        assert(uniquePathsWithObstacles(g) == 1); // 1x1 free: one empty path
    }
    {
        std::vector<std::vector<int>> g = {{0, 0}, {1, 1}, {0, 0}};
        assert(uniquePathsWithObstacles(g) == 0); // a full wall row blocks everything
    }
    {
        // 3x7 without obstacles is the classic Unique Paths answer, C(8,2) = 28.
        std::vector<std::vector<int>> g(3, std::vector<int>(7, 0));
        assert(uniquePathsWithObstacles(g) == 28);
    }
    std::cout << "OK 07_unique_paths_ii.cpp\n";
    return 0;
}

```

What is new:

- `std::vector<long long> dp(cols, 0);` is the single rolling row, `[0] * cols`. The element type is `long long`, a 64-bit integer, instead of `int`. See the next paragraph for why.
- `dp[0] = 1;` is the start cell. There is no separate handling of row 0: the loop treats row 0 like any other row, and because `dp` starts as `[1, 0, 0, ...]`, sweeping left to right turns it into `[1, 1, 1, ...]`, which is the correct top row.
- `if (obstacleGrid[r][c] == 1) dp[c] = 0; else if (c > 0) dp[c] += dp[c - 1];` is the whole transition. `dp[c] += dp[c - 1]` is "above plus left" in place. For `c == 0` nothing happens, so the value carries down from the row above. If column 0 has an obstacle, `dp[0]` becomes 0 and stays 0 for every row below, which is right: nothing can get past it.
- `return static_cast<int>(dp[cols - 1]);` converts back to `int` because that is the return type LeetCode asks for. The cast says "I know this narrows, and I have checked it fits".

Why long long and not int. In Python an integer grows as large as it needs to. In C++ an `int` holds at most 2,147,483,647, and if a sum goes past that the behaviour is undefined: typically it silently wraps to a negative number, and nothing tells you. LeetCode promises the *final* answer fits in an `int`, but it says nothing about the cells in the middle of the table. A large open region whose every exit to the goal is walled off can hold path counts far bigger than the answer. `long long` goes up to about 9.2×10^{18} , four thousand million times more headroom, which covers every grid you will realistically be tested on. Get in the habit: when a DP table holds counts or sums, make it `long long` unless you have a reason not to.

Walk through the example (0 0 0 / 0 1 0 / 0 0 0). rows = cols = 3, start free, `dp = [1, 0, 0]`.

r=0 c=0	free, c==0: nothing.	dp = [1, 0, 0]
r=0 c=1	free: dp[1] += dp[0] -> 1.	dp = [1, 1, 0]
r=0 c=2	free: dp[2] += dp[1] -> 1.	dp = [1, 1, 1]

```

r=1 c=0 free, c==0: nothing (1 from above) dp = [1, 1, 1]
r=1 c=1 obstacle: dp[1] = 0. dp = [1, 0, 1]
r=1 c=2 free: dp[2] += dp[1] -> 1 + 0 = 1. dp = [1, 0, 1]
r=2 c=0 free, c==0: nothing. dp = [1, 0, 1]
r=2 c=1 free: dp[1] += dp[0] -> 0 + 1 = 1. dp = [1, 1, 1]
r=2 c=2 free: dp[2] += dp[1] -> 1 + 1 = 2. dp = [1, 1, 2]
return dp[2] = 2

```

Complexity. One visit per cell with constant work: time proportional to rows $\times$ cols, $O(\text{rows} \times \text{cols})$. Memory is one row: $O(\text{cols})$, instead of $O(\text{rows} \times \text{cols})$ for the full table.

Robotics. Nobody counts robot paths for its own sake, but the *sweep* is everywhere: fill a table in an order where each cell's inputs are already done. Value iteration on a grid world does exactly this when moves are restricted to one direction, and dynamic time warping, used to align a recorded trajectory against a reference one, is the same table with `min` in place of `+`. The rolling-row trick is also how you keep such tables small on an embedded board.

Follow-ups you may get. - *No obstacles: is there a formula?* Yes. Every path is rows-1 downs and cols-1 rights in some order, so the count is the binomial coefficient $C(\text{rows} + \text{cols} - 2, \text{rows} - 1)$. The 3x7 test in `main()` checks $C(8, 2) = 28$. - *Return one of the paths, not the count.* Keep a full 2-D table, then walk back from the goal: at each cell step to whichever of above/left has a non-zero count, and reverse the list. - *Could long long overflow too?* In principle, on a 100x100 grid with a huge walled-off region, yes. Two robust fixes: use `unsigned long long`, whose overflow is defined to wrap around modulo 2^{64} so the final answer (which fits) still comes out exact, or do a first pass that marks cells which cannot reach the goal and treat them as obstacles. - *Minimum-cost path instead of counting?* Same sweep with `cost[r][c] + min(above, left)`; that is the next problem.

8. Minimum Path Sum (LeetCode 64, medium)

The problem in plain words. You are given a grid of non-negative integers. Each number is the cost of stepping on that cell. Start at the top-left cell, finish at the bottom-right cell, and you may only move **right** or **down**. The cost of a path is the sum of every cell on it, including the first and the last. Return the smallest possible cost.

```

grid =  1  3  1
        1  5  1
        4  2  1

```

Cheapest path: 1 -> 3 -> 1 -> 1 -> 1, going right, right, down, down.
Cost = 1 + 3 + 1 + 1 + 1 = 7.

The path down the left side would be 1 + 1 + 4 + 2 + 1 = 9.
Going through the 5 costs at least 1 + 1 + 5 + 1 + 1 = 9.

Edge cases: a single cell returns that cell's value (`[[5]]` gives 5). A single row or a single column has only one path, so the answer is the sum of the row or column (`[[1],[2],[3]]` gives 6). The grid is at most 200 by 200 and every value is between 0 and 200, so an `int` is plenty. Because you can only go right or down, there is always at least one path; there is no "no path" case.

By hand. You could list every path and add them up, but there are far too many on a big grid. Instead notice that the last step onto any cell came either from the cell **above** it or from the cell to its **left**. So if you already know the cheapest way to reach those two cells, the cheapest way to reach this one is the cheaper of the two, plus this cell's own cost. That is the grid DP from A5, with `min` in place of the sum. Fill in a cost table from the top-left corner outward. The top row can only be reached from the left, the left column only from above.

```

grid:   1  3  1      cost:  1  4  5
        1  5  1          2  7  6
        4  2  1          6  8  7  <- answer, bottom-right

```

Reading a few cells: $\text{cost}[0][1] = 1 + 3 = 4$ (only from the left). $\text{cost}[1][0] = 1 + 1 = 2$ (only from above). $\text{cost}[1][1] = \min(\text{above } 4, \text{left } 2) + 5 = 7$. $\text{cost}[2][2] = \min(\text{above } 6, \text{left } 8) + 1 = 7$.

The idea. 1. $\text{cost}[r][c]$ means "cheapest total to arrive at (r, c) ". 2. Base cases: $\text{cost}[0][0] = \text{grid}[0][0]$; along the top row $\text{cost}[0][c] = \text{cost}[0][c-1] + \text{grid}[0][c]$; down the left column $\text{cost}[r][0] = \text{cost}[r-1][0] + \text{grid}[r][0]$. 3. Everywhere else: $\text{cost}[r][c] = \min(\text{cost}[r-1][c], \text{cost}[r][c-1]) + \text{grid}[r][c]$. 4. Fill row by row, left to right, so both the cell above and the cell to the left are already done. Return the bottom-right entry.

The Python you would have written.

```
def minPathSum(grid):
    rows, cols = len(grid), len(grid[0])
    cost = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if r == 0 and c == 0:
                cost[r][c] = grid[0][0]
            elif r == 0:
                cost[r][c] = cost[r][c - 1] + grid[r][c]      # only from the left
            elif c == 0:
                cost[r][c] = cost[r - 1][c] + grid[r][c]      # only from above
            else:
                cost[r][c] = min(cost[r - 1][c], cost[r][c - 1]) + grid[r][c]
    return cost[rows - 1][cols - 1]
```

In C++. The C++ version keeps only **one row** of the table instead of the whole thing. This is the "rolling row" trick from problem 7. Read it slowly the first time.

```
// 08_minimum_path_sum.cpp - LeetCode 64. Minimum Path Sum (medium) - DP
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// dp[c] = cheapest cost to reach (current row, c). Rolling row again:
// before the update dp[c] is the cost from above, dp[c-1] the cost from the left.
int minPathSum(std::vector<std::vector<int>>& grid) {
    const int rows = static_cast<int>(grid.size());
    const int cols = rows ? static_cast<int>(grid[0].size()) : 0;
    if (rows == 0 || cols == 0) return 0;
    std::vector<int> dp(cols, 0);
    dp[0] = grid[0][0];
    for (int c = 1; c < cols; ++c) dp[c] = dp[c - 1] + grid[0][c]; // first row: only from the left
    for (int r = 1; r < rows; ++r) {
        dp[0] += grid[r][0]; // first column: only from above
        for (int c = 1; c < cols; ++c) dp[c] = std::min(dp[c], dp[c - 1]) + grid[r][c];
    }
    return dp[cols - 1];
}

int main() {
    {
        std::vector<std::vector<int>> g = {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}};
        assert(minPathSum(g) == 7); // 1->3->1->1->1
    }
    {
        std::vector<std::vector<int>> g = {{1, 2, 3}, {4, 5, 6}};
        assert(minPathSum(g) == 12);
    }
    {
        std::vector<std::vector<int>> g = {{5}};
        assert(minPathSum(g) == 5);
    }
    {
        std::vector<std::vector<int>> g = {{1}, {2}, {3}};
    }
}
```

```

    assert(minPathSum(g) == 6); // single column
}
{
    std::vector<std::vector<int>> g = {{9, 1, 1}, {9, 9, 1}, {9, 9, 1}};
    assert(minPathSum(g) == 13); // hug the top and right edges
}
std::cout << "OK 08_minimum_path_sum.cpp\n";
return 0;
}

```

What is new:

- `std::vector<int> dp(cols, 0);` is a single row of `cols` zeros. Row `r` of the table only ever looks at row `r-1` and at itself, so we do not need to keep older rows. `dp` holds "the row we are currently filling in". The full-table version would be `std::vector<std::vector<int>> cost(rows, std::vector<int>(cols, 0));` exactly like the Python; both are correct, the rolling row just uses less memory.
- How the rolling row reads "above" and "left". When we are on row `r` and about to update `dp[c]`, we have not yet overwritten it, so `dp[c]` still holds the value from row `r-1`: that is "above". `dp[c-1]` has just been overwritten for row `r`: that is "left". So `std::min(dp[c], dp[c - 1]) + grid[r][c]` is exactly `min(above, left) + here`.
- `for (int c = 1; c < cols; ++c) dp[c] = dp[c - 1] + grid[0][c];` fills in the top row: a running sum, because the top row can only be reached from the left.
- `dp[0] += grid[r][0];` at the start of every later row is the left column: the old `dp[0]` is the cell above, so add this cell's cost. It replaces the Python `elif c == 0` branch.
- `std::min(a, b)` is Python's `min(a, b)` for two values. It lives in `<algorithm>`, hence the include at the top.
- `const int cols = rows ? static_cast<int>(grid[0].size()) : 0;` is the same empty-grid guard you saw in problem 4: do not touch `grid[0]` if there are no rows.
- `std::vector<std::vector<int>>& grid` takes the grid by reference to avoid copying 40,000 ints. The function never writes to it; LeetCode's signature just happens to be non-const.

Walk through the example. `dp` starts as the top row of costs: after the first loop, `dp = [1, 4, 5]`.

```

row 1:  dp[0] += 1          -> dp = [2, 4, 5]      (4 and 5 are still "above")
        c=1: min(4, 2) + 5 = 7 -> dp = [2, 7, 5]
        c=2: min(5, 7) + 1 = 6 -> dp = [2, 7, 6]      this is row 1 of the cost table
row 2:  dp[0] += 4          -> dp = [6, 7, 6]
        c=1: min(7, 6) + 2 = 8 -> dp = [6, 8, 6]
        c=2: min(6, 8) + 1 = 7 -> dp = [6, 8, 7]      this is row 2 of the cost table
return dp[2] = 7

```

Compare with the full table drawn above: each `dp` after a row is exactly that row.

Complexity. Every cell is visited once with constant work, so time is proportional to `rows × cols`, $O(\text{rows} \cdot \text{cols})$. Memory is one row, $O(\text{cols})$; the Python version uses $O(\text{rows} \cdot \text{cols})$.

Robotics. This is a cost-to-go table over a traversability map: each cell stores the cheapest way to reach it, and a robot can then follow the table by stepping to the cheaper neighbour. The right-or-down restriction is what makes a plain table fill work. Allow all four directions and the table has cycles, so you need Dijkstra instead; that is what the NavFn global planner in ROS computes over the whole costmap before it extracts a path by walking downhill.

Follow-ups you may get. - What if you can move in all four directions? Then a cell can depend on cells not yet filled, so row-by-row DP is wrong. Use Dijkstra (A without the heuristic, Part C) with cell cost as the edge weight. - Negative cell values? The DP still works because right/down paths never revisit a cell. Dijkstra would not. - Return the path, not just the cost? Keep the full table, then walk back from the bottom-right: at each cell step to whichever of above/left has cost equal to `cost[r][c] - grid[r][c]`.

9. Word Search (LeetCode 79, medium)

The problem in plain words. You are given a board of letters and a word. Return `true` if the word can be spelled by walking on the board: start at any cell, and each next letter must be in a cell directly **next to** the previous one (up, down, left or right; not diagonal). You may **not** step on the same cell twice within one spelling. Otherwise return `false`.

```
board = A B C E
        S F C S
        A D E E

word = "ABCCED" -> true    A(0,0) B(0,1) C(0,2) C(1,2) E(2,2) D(2,1)
word = "SEE"    -> true    S(1,3) E(2,3) E(2,2)
word = "ABCB"   -> false    A(0,0) B(0,1) C(0,2) and then B again would have to
                        be the same B at (0,1), which is not allowed
```

Edge cases: a 1 by 1 board with 'a' and word "a" is true; the same board with "ab" is false. An empty board or empty word gives false. The board is at most 6 by 6 and the word at most 15 letters, which is a strong hint that trying every path is acceptable.

By hand. Put your finger on every cell that holds the first letter. From there, look at the four neighbours for the second letter, move your finger, and keep going. Two things happen. When no neighbour has the next letter you are stuck: lift your finger, go back one cell, and try a different neighbour there. And when you consider a neighbour, you must ignore cells your finger has already passed over on *this* attempt. That is backtracking from A4: mark the cell as in use, try the neighbours, unmark it when you come back.

The neat trick in this solution is that the board itself is the "in use" list. When you step on a cell you overwrite its letter with '#'. No letter in the word is ever '#', so a marked cell can never match, which is exactly what "may not reuse" means. When you step back off the cell, you write the original letter back.

The idea. 1. `dfs(r, c, i)` answers: "standing on (r, c), can I match `word[i:]` from here?" 2. If `i` equals the word length, every letter has been matched: return `true`. 3. If (r, c) is outside the board, or `board[r][c]` is not `word[i]`, return `false`. A '#' cell fails this test automatically. 4. Otherwise save the letter, write '#', and ask the four neighbours about `i + 1`. If any says `true`, the answer is `true`. 5. Put the saved letter back **before** returning, whatever the answer was. 6. `exist` calls `dfs(r, c, 0)` from every cell and returns `true` the first time it succeeds.

The Python you would have written. The board must be a list of lists (not a list of strings) so that cells can be overwritten.

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != word[i]:
            return False
        saved = board[r][c]
        board[r][c] = '#' # mark: '#' never matches a letter
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                 dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = saved # unmark
        return found
    return any(dfs(r, c, 0) for r in range(rows) for c in range(cols))
```

In C++.

```
// 09_word_search.cpp - LeetCode 79. Word Search (medium) - backtracking
#include <cassert>
#include <iostream>
#include <string>
#include <vector>
```

```

// Backtracking: mark the cell as used, recurse on the next letter, unmark on the
// way out so a different path can reuse it. Unmarking is what makes this
// backtracking rather than plain DFS.
bool dfs(std::vector<std::vector<char>>& board, const std::string& word, int r, int c, int i) {
    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());
    if (i == static_cast<int>(word.size())) return true;
    if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] != word[i]) return false;
    const char saved = board[r][c];
    board[r][c] = '#'; // in-place visited mark; '#' never matches a letter
    const bool found = dfs(board, word, r + 1, c, i + 1) || dfs(board, word, r - 1, c, i + 1) ||
        dfs(board, word, r, c + 1, i + 1) || dfs(board, word, r, c - 1, i + 1);
    board[r][c] = saved; // unmark: the board is restored for the next start cell
    return found;
}

bool exist(std::vector<std::vector<char>>& board, std::string word) {
    if (board.empty() || board[0].empty() || word.empty()) return false;
    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
            if (dfs(board, word, r, c, 0)) return true;
    return false;
}

int main() {
    std::vector<std::vector<char>> board = {
        {'A', 'B', 'C', 'E'},
        {'S', 'F', 'C', 'S'},
        {'A', 'D', 'E', 'E'};
    assert(exist(board, "ABCCED") == true);
    assert(exist(board, "SEE") == true);
    assert(exist(board, "ABCB") == false); // would need to reuse the B
    // Board is restored after every search.
    assert(board[0][0] == 'A' && board[1][1] == 'F');

    std::vector<std::vector<char>> one = {{'a'}};
    assert(exist(one, "a") == true);
    assert(exist(one, "ab") == false);
    std::vector<std::vector<char>> empty;
    assert(exist(empty, "a") == false);

    // Backtracking matters: seven A's ring the B. Starting at (0,0) the DFS
    // walks a short arc, fails to find C, unmarks, and a later start at (1,2)
    // snakes the long way round: (1,2)(0,2)(0,1)(0,0)(1,0)(2,0)(2,1) then C.
    std::vector<std::vector<char>> tricky = {
        {'A', 'A', 'A'},
        {'A', 'B', 'A'},
        {'A', 'A', 'C'};
    assert(exist(tricky, "AAAAAAC") == true); // 7 A's then C
    assert(exist(tricky, "AAAAAAAC") == false); // 8 A's: one too many
    assert(exist(tricky, "ABA") == true);
    assert(exist(tricky, "ABC") == false); // B and C are only diagonal neighbours
    std::cout << "OK 09_word_search.cpp\n";
    return 0;
}

```

What is new:

- `dfs` is a free function above `exist` (no nested functions in C++), so the board and the word are passed in. `std::vector<std::vector<char>>& board` is a reference: the marks written by one call must be visible to every other call, and the unmark must land on the same board. `const std::string& word` is a reference too, to avoid copying the word on every call; `const` because we only read it.
- `word[i]` is a `char` and `board[r][c]` is a `char`, so `board[r][c] != word[i]` compares two single characters, the same as Python's `!=` on one-letter strings.

- `if (i == static_cast<int>(word.size())) return true;` is `if i == len(word)`. The cast is the signed/unsigned issue from A1 again.
- `const char saved = board[r][c]; board[r][c] = '#';` is the mark. A char literal uses single quotes in C++; `"#"` with double quotes would be a string and would not compile here.
- The four recursive calls joined by `||` are Python's `or` chain. C++ `||` also short-circuits: as soon as one call returns `true`, the remaining calls are skipped.
- `const bool found = ...; board[r][c] = saved; return found;` is the important shape. The result is stored in a variable so the `unmark` **always** runs before the function returns. If you wrote `if (dfs(...)) return true;` inside the chain, the `'#'` would stay on the board after a success and the caller's board would be corrupted. The test `assert(board[0][0] == 'A' && board[1][1] == 'F');` in `main` checks exactly this.
- `bool exist(std::vector<std::vector<char>>& board, std::string word)` takes `word` by value, which is fine for a 15-letter string and matches LeetCode's signature.

Walk through the example. The code tries neighbours in the order down, up, right, left.

First, an attempt that fails: "ABCB" starting at (0,0). A matches, mark it. Need B: down is S, no; up is outside; right (0,1) is B, mark it. Need C: down F, no; up outside; right (0,2) is C, mark it. The board now looks like this:

```
# # # E
S F C S
A D E E
```

Need B from (0,2): down (1,2) is C, no; up outside; right (0,3) is E, no; left (0,1) is '#', no. The only B is under a mark, so this attempt is dead. `dfs` at (0,2) writes C back and returns `false`. Back at (0,1) the last direction, left, is (0,0) which is '#', no; write B back, return `false`. Back at (0,0), left is outside; write A back, return `false`. The board is exactly as it was:

```
A B C E
S F C S
A D E E
```

`exist` keeps scanning. The other A at (2,0) has neighbours S and D, neither is B, so it fails immediately. No start works: `false`.

Now the attempt that succeeds: "ABCCED" starting at (0,0). Same first three steps as before, so (0,0), (0,1), (0,2) get marked. Need C from (0,2): down (1,2) is C, mark it. Need E: down (2,2) is E, mark it. Need D from (2,2): down is outside; up (1,2) is '#', no; right (2,3) is E, no; left (2,1) is D, mark it. Now `i` is 6, which equals the word length, so the next call returns `true` straight away.

```
# # # E      every '#' is a letter of ABCCED, in use on this path
S F # S
A # # E
```

`true` travels back up the chain. Each level still runs its `unmark` line on the way out, so by the time `exist` returns `true` the board is fully restored.

Complexity. There are `rows × cols` starting cells. From each, the first step has up to 4 choices and every later step at most 3 (you never go back to the marked cell you came from), so the worst case is proportional to `rows × cols × 3L` where `L` is the word length: $O(M \cdot N \cdot 3^L)$. With a 6 by 6 board and 15 letters this is fine. The recursion is at most `L` deep, so memory is $O(L)$.

Robotics. Backtracking with state restoration is the skeleton of any small exhaustive search: enumerating coverage paths on a small grid, trying task orderings for a robot arm, or assigning robots to goals under constraints. In each case you commit to a choice, recurse, and undo the choice so the next candidate starts from the same state, exactly like writing `'#'` and then writing the letter back.

Follow-ups you may get. - *Many words on one board?* That is Word Search II (LeetCode 212): put all the words in a trie and run the same DFS once, stopping as soon as the current prefix is not in the trie. - *Can you prune before searching?* Count the letters on the board; if the word needs more of any letter than the board has, return `false` without a single DFS call. - *Why not a separate `visited` table?* You can, and you must if the board is `const`. The `'#'` trick is shorter and uses no extra memory; just remember to restore.

10. Pacific Atlantic Water Flow (LeetCode 417, medium)

The problem in plain words. This one is famous for being hard to read, so here it is slowly. You are given a grid of heights; think of it as a rectangular island. The **Pacific Ocean** touches the island along its **top edge and left edge**. The **Atlantic Ocean** touches it along the **bottom edge and right edge**.

		P a c i f i c							
		+-----+							
			1	2	2	3	5		
P			3	2	3	4	4		A
a			2	4	5	3	1		t
c			6	7	1	4	5		l
			5	1	1	2	4		a
		+-----+							n
		A t l a n t i c							t i c

Rain falls on a cell. Water can flow from a cell to any of its four neighbours (up, down, left, right) whose height is **equal or lower**. It can keep flowing like that, cell to cell, as long as it never goes uphill. Water that flows off an edge of the grid falls into the ocean on that edge. Since every cell in the top row and left column is already touching the Pacific, rain on those cells reaches the Pacific with no flowing at all; likewise the bottom row and right column for the Atlantic.

The question: return the list of cells `[r, c]` such that rain on that cell can reach **both** oceans (by two different routes if necessary). Any order is accepted by LeetCode; our code returns them row by row.

For the grid above the answer is 7 cells, marked *:

1	2	2	3	[5]*	(0,4): top edge (Pacific) and right edge (Atlantic) at once
3	2	3	[4]*[4]*		(1,3): 4 -> 3 -> 2 -> 2 -> 1 up to the Pacific, 4 -> 4 right to the Atlantic
2	4	[5]*	3	1	(2,2): 5 -> 4 -> 2 left to the Pacific, 5 -> 3 -> 1 right to the Atlantic
[6]*	[7]*	1	4	5	(3,0): left edge, and 6 -> 5 down to the bottom edge
[5]*	1	1	2	4	(4,0): left edge (Pacific) and bottom edge (Atlantic) at once

Edge cases: a 1 by 1 grid touches all four edges, so the answer is `[[0,0]]`. An empty grid gives an empty list. A completely flat grid returns every cell (water can flow anywhere on a flat surface). A cell that is lower than all four neighbours, like the 0 in the middle of a 3 by 3 ring of 1s, cannot flow anywhere, so it is in the answer only if it sits on an edge. The grid is at most 200 by 200 and heights are between 0 and 100,000.

By hand. The obvious approach is to pick each cell, follow the water downhill in every possible direction, and see which oceans you hit. On the 5 by 5 example that is 25 separate searches, and on 200 by 200 it is 40,000 searches over 40,000 cells each: far too slow.

The trick is to turn the question around. Instead of asking "can this cell's water get down to the Pacific?", ask "starting from the Pacific, which cells can I **climb up** to?" Water flows from higher to equal-or-lower, so if you walk in reverse you go from lower to equal-or-higher. Start with every cell on the Pacific edge (they can obviously reach the Pacific), and spread to any neighbour whose height is **at least** the current cell's height. Every cell you reach this way can send water down the same route in reverse. That is one multi-source BFS from A2, with the visited rule changed from "not a wall" to "not lower than me". Do the same from the Atlantic edge. The answer is the cells reached by both.

The idea. 1. Make two boolean tables `pac` and `atl`, all `false`. 2. Push every cell of row 0 and column 0 into a Pacific queue and mark it in `pac`. Push every cell of the last row and last column into an Atlantic queue and mark it in `atl`. 3. Run BFS on each queue. A neighbour is allowed if it is inside the grid, not yet marked in this table, and `heights[neighbour] >= heights[current]`. Mark it and push it. 4. Return every `(r, c)` with `pac[r][c]` and `atl[r][c]` both true.

The Python you would have written.

```
from collections import deque

def pacificAtlantic(heights):
    rows, cols = len(heights), len(heights[0])
    def climb(starts):
        reach = [[False] * cols for _ in range(rows)]
        q = deque()
        for r, c in starts:
            reach[r][c] = True
            q.append((r, c))
        while q:
            r, c = q.popleft()
            for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)):
                nr, nc = r + dr, c + dc
                if (0 <= nr < rows and 0 <= nc < cols and not reach[nr][nc]
                    and heights[nr][nc] >= heights[r][c]): # uphill or flat only
                    reach[nr][nc] = True
                    q.append((nr, nc))
        return reach
    pac_edge = {(r, 0) for r in range(rows)} | {(0, c) for c in range(cols)}
    atl_edge = {(r, cols - 1) for r in range(rows)} | {(rows - 1, c) for c in range(cols)}
    pac = climb(pac_edge)
    atl = climb(atl_edge)
    return [[r, c] for r in range(rows) for c in range(cols) if pac[r][c] and atl[r][c]]
```

The edges are sets so that the corner cell `(0, 0)`, which is in both row 0 and column 0, is not pushed twice.

In C++.

```
// 10_pacific_atlantic.cpp - LeetCode 417. Pacific Atlantic Water Flow (medium)
// Reverse BFS from each ocean: instead of asking "can this cell reach the
// ocean?" (n*m searches), ask "which cells can the ocean climb up to?" (2 searches).
#include <cassert>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

// Multi-source BFS that moves only to neighbours with height >= current.
void climb(const std::vector<std::vector<int>>& h, std::vector<std::vector<char>>& reach,
    std::queue<std::pair<int, int>>& q) {
    const int rows = static_cast<int>(h.size());
    const int cols = static_cast<int>(h[0].size());
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            if (reach[nr][nc] || h[nr][nc] < h[r][c]) continue; // water flows downhill, so we climb uphill
            reach[nr][nc] = 1;
            q.push({nr, nc});
        }
    }
}

std::vector<std::vector<int>> pacificAtlantic(std::vector<std::vector<int>>& heights) {
    std::vector<std::vector<int>> out;
    const int rows = static_cast<int>(heights.size());
    const int cols = rows ? static_cast<int>(heights[0].size()) : 0;
```

```

if (rows == 0 || cols == 0) return out;
// vector<char> instead of vector<bool>: bool is a bit-packed proxy and you cannot take references into it.
std::vector<std::vector<char>> pac(rows, std::vector<char>(cols, 0));
std::vector<std::vector<char>> atl(rows, std::vector<char>(cols, 0));
std::queue<std::pair<int, int>> qp, qa;
for (int r = 0; r < rows; ++r) {
    pac[r][0] = 1; qp.push({r, 0});
    atl[r][cols - 1] = 1; qa.push({r, cols - 1});
}
for (int c = 0; c < cols; ++c) {
    if (!pac[0][c]) { pac[0][c] = 1; qp.push({0, c}); }
    if (!atl[rows - 1][c]) { atl[rows - 1][c] = 1; qa.push({rows - 1, c}); }
}
climb(heights, pac, qp);
climb(heights, atl, qa);
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c)
        if (pac[r][c] && atl[r][c]) out.push_back({r, c});
return out; // row-major order, which is what the tests below expect
}

int main() {
{
    std::vector<std::vector<int>> h = {
        {1, 2, 2, 3, 5},
        {3, 2, 3, 4, 4},
        {2, 4, 5, 3, 1},
        {6, 7, 1, 4, 5},
        {5, 1, 1, 2, 4}};
    const std::vector<std::vector<int>> want = {{0, 4}, {1, 3}, {1, 4}, {2, 2}, {3, 0}, {3, 1}, {4, 0}};
    assert(pacificAtlantic(h) == want);
}
{
    std::vector<std::vector<int>> h = {{1}};
    const std::vector<std::vector<int>> want = {{0, 0}}; // touches both oceans
    assert(pacificAtlantic(h) == want);
}
{
    std::vector<std::vector<int>> h;
    assert(pacificAtlantic(h).empty());
}
{
    // Flat grid: every cell reaches both oceans.
    std::vector<std::vector<int>> h = {{1, 1}, {1, 1}};
    assert(pacificAtlantic(h).size() == 4);
}
{
    // A pit: the centre 0 cannot flow uphill to any neighbour, the 8 border cells can.
    std::vector<std::vector<int>> h = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
    const auto out = pacificAtlantic(h);
    assert(out.size() == 8);
    for (const auto& rc : out) assert(!(rc[0] == 1 && rc[1] == 1));
}
{
    // (0,0) is the lowest cell: it touches the Pacific but nothing lets it climb to the Atlantic side.
    std::vector<std::vector<int>> h = {{1, 2}, {3, 4}};
    const std::vector<std::vector<int>> want = {{0, 1}, {1, 0}, {1, 1}};
    assert(pacificAtlantic(h) == want);
}
std::cout << "OK 10_pacific_atlantic.cpp\n";
return 0;
}

```

What is new:

- `climb` is the BFS from A2 with one line changed: `if (reach[nr][nc] || h[nr][nc] < h[r][c]) continue;` skips neighbours that are already reached **or lower** than the current cell. Remember the direction is flipped because we search uphill.
- `climb` takes the queue **by reference** (`std::queue<std::pair<int,int>>& q`) and the reach table by reference. The caller fills the queue with the edge cells, then hands it over. This lets one function serve both oceans. `h` is a `const` reference because it is only read.

- `std::vector<std::vector<char>>` for the two reach tables instead of `std::vector<bool>`. In C++ `std::vector<bool>` is a special case that packs eight flags into a byte, and because of that you cannot take an ordinary reference to one of its elements. Using `char` with the values 0 and 1 behaves like a normal container and is the usual workaround. `if (pac[r][c] && atl[r][c])` works because a non-zero `char` counts as true.
- `constexpr int dr[4] = ...` is the same as `const int dr[4]` from A1; `constexpr` just promises the value is known at compile time. Either spelling is fine.
- `std::queue<std::pair<int, int>> qp, qa;` declares two queues in one statement.
- `if (!pac[0][c]) { pac[0][c] = 1; qp.push({0, c}); }` is the "do not push the corner twice" check, the same job the Python sets do.
- `out.push_back({r, c});` appends a two-element `std::vector<int>` built from the braces, so the result has the shape `[[r, c], ...]` that LeetCode expects.
- `std::vector<std::vector<int>> out;` is declared first and returned in the early-exit case, so an empty grid returns an empty list.

Walk through the example. The Pacific queue starts with the nine edge cells: (0,0) (1,0) (2,0) (3,0) (4,0) (0,1) (0,2) (0,3) (0,4), all marked P.

```
pop (0,0) h=1: right (0,1) and down (1,0) already marked.
pop (1,0) h=3: right (1,1) h=2 is lower, skip.
pop (2,0) h=2: right (2,1) h=4 >= 2, mark, push.
pop (3,0) h=6: right (3,1) h=7 >= 6, mark, push.
pop (4,0) h=5: right (4,1) h=1, skip.
pop (0,1) h=2: down (1,1) h=2 >= 2 (equal is allowed), mark, push.
pop (0,2) h=2: down (1,2) h=3, push.    pop (0,3) h=3: down (1,3) h=4, push.
pop (0,4) h=5: down (1,4) h=4, skip.
pop (2,1) h=4: right (2,2) h=5, push.    pop (3,1) h=7: neighbours 1 and 1, skip.
pop (1,1), (1,2): nothing new.          pop (1,3) h=4: right (1,4) h=4, push.
pop (2,2) h=5: right 3 and down 1, skip. pop (1,4) h=4: down (2,4) h=1, skip. Queue empty.
```

The Atlantic search starts from the last row and last column and climbs the same way. The two reach tables, and their overlap:

pac:	P P P P P	atl:	 A	both:	 *
	P P P P P		. . . A A		. . . * *
	P P P . .		. . A A A		. . * . .
	P P . . .		A A A A A		* * . . .
	P		A A A A A		*

Reading both row by row gives (0,4) (1,3) (1,4) (2,2) (3,0) (3,1) (4,0), the seven cells from the drawing at the top.

Complexity. Two BFS passes, each marking every cell at most once: time proportional to rows × cols, $O(\text{rows} \cdot \text{cols})$. Memory is the two reach tables and the queues, also $O(\text{rows} \cdot \text{cols})$. The naive "search from every cell" approach would be $O((\text{rows} \cdot \text{cols})^2)$.

Robotics. Searching backwards from the goal set is standard practice: a wavefront run from all docking stations at once tells every cell which dock is nearest, in one pass instead of one per cell. The "only climb to equal-or-higher" rule is a traversability constraint, like a slope limit on a terrain map, and intersecting two reachability sets is how you find cells a robot can reach from its current position *and* from which it can still reach a rendezvous point.

Follow-ups you may get. - *Why not DFS from every cell?* It is correct but $O((\text{rows} \cdot \text{cols})^2)$. Two searches from the oceans give the same answer in linear time. - *Only the count is needed?* Same code; count the cells where both tables are set instead of building the output vector. - *Water can only flow strictly downhill (not to equal heights)?* Change `<` to `<=` in the skip test in `climb`; the structure is unchanged.

Part C — Build: A* on a grid, from scratch, with tests

Everything in Part B was one function with a `main()` under it. This part is different: you build a small *project*. A planner library, a demo program that draws a path on a map, and a test suite that proves the planner is right. It is the kind of thing you can put in a public repository and talk about, and it is the algorithm that the global planner of a ROS navigation stack actually runs.

Rough time: two days. Day 1 is C1 to C3 (understand A* and write the 40-line version yourself). Day 2 is C4 and C5 (read the project, build it, break it).

You need A2 (the BFS ripple) cold before starting. A* is BFS with a smarter queue; if the queue version is not automatic yet, go back and type it out once more.

C1. What A* is, with a tiny example

Here is the map from A1 again, now with a start `S` and a goal `G`. `#` is a wall. Moves are up, down, left, right, and every move costs 1. The question: what is the cheapest way from `S` to `G`?

	c0	c1	c2	c3
r0	S	.	.	.
r1	.	#	#	.
r2	.	.	.	G

By eye: along the top row and down the right side is 5 moves. Down the left side and along the bottom row is also 5. There is nothing cheaper, because `G` is 2 rows down and 3 columns right, and each move changes the row or the column by one, so you need at least $2 + 3 = 5$ moves.

BFS from A2 answers this. It also fills the *whole* map with distances before it is done, including the bottom-left corner, which is nowhere near the goal. That is the waste A* removes.

Three numbers per cell. A* keeps, for every cell it has touched:

- `g`: the cost of the cheapest way we have found *so far* from `S` to this cell. Exact, because we added up real moves.
- `h`: a *guess* of the cost still to go from this cell to `G`. On a 4-direction grid the natural guess is "rows still to go plus columns still to go", the Manhattan distance. For `(0,1)` that is $|0 - 2| + |1 - 3| = 4$.
- `f = g + h`: the guessed total cost of a route through this cell.

The to-do list. Instead of BFS's queue (oldest first), A *keeps a to-do list of cells it has seen but not yet dealt with, and always takes the cell with the smallest f**: the one that looks like it lies on the cheapest total route. Taking a cell means: look at its neighbours, and for each one work out `g` (my `g` plus 1), `h`, `f`, and put it on the list. When the cell taken from the list is `G`, stop; its `g` is the answer.

Here is every step on the map above. "g/h/f" is written per cell.

step	to-do list (cell: g/h/f)	taken	added to the list
0	(0,0): 0/5/5	(0,0)	(0,1): 1/4/5 and (1,0): 1/4/5
1	(0,1): 1/4/5, (1,0): 1/4/5	(0,1). A tie: same f, same g. We take the one with the smaller row number; any fixed rule will do.	(0,2): 2/3/5. (1,1) is a wall.
2	(0,2): 2/3/5, (1,0): 1/4/5	(0,2). Same f again; among equal f we prefer the larger g (more on this below).	(0,3): 3/2/5. (1,2) is a wall.
3	(0,3): 3/2/5, (1,0): 1/4/5	(0,3)	(1,3): 4/1/5

step	to-do list (cell: g/h/f)	taken	added to the list
4	(1,3): 4/1/5, (1,0): 1/4/5	(1,3)	(2,3): 5/0/5
5	(2,3): 5/0/5, (1,0): 1/4/5	(2,3). This is the goal. Answer: g = 5.	stop

Mark the cells that were taken off the list with x:

```

      c0  c1  c2  c3
r0    S   x   x   x
r1    .   #   #   x
r2    .   .   .   G

```

Six cells taken. (1,0) sat on the list the whole time and was never taken, and the three bottom-left cells were never even added. BFS on the same map takes all 10 free cells before it reaches G. That is the whole point: the guess *h* pulls the search toward the goal.

One thing to notice in the table: every *f* was 5. That is because the guess happened to be exactly right along the top row (no detour was needed). When a detour *is* needed, cells on the detour get a larger *f*, and A* only takes them once every cell with a smaller *f* has been tried. *f* never has to go down, because *h* is only ever an under-estimate (rule 2 below).

A bigger picture: BFS versus A* on an open 5x5. Start top-left, goal bottom-right, no walls. x marks a cell that was taken off the list before the goal was reached.

BFS (25 cells taken)	A* (9 cells taken)
S x x x x	S x x x x
x x x x x	 x
x x x x x	 x
x x x x x	 x
x x x x G	 G

BFS grows its ripple in every direction, and the goal is the very last cell it reaches, so it looks at everything. A* walks straight there.

A detail hidden in that picture: on an open grid every cell has *f* = 8 (its *g* plus its *h* always adds up to the true distance, because nothing is in the way). So *which* *f* = 8 cell to take is decided purely by the tie-break. The rule used here, and in the project below, is: among equal *f*, take the cell with the *larger g*, the one that has already travelled furthest. With the opposite rule (smaller *g* first) A* on this open grid takes all 25 cells, exactly like BFS. The tie-break is not a detail you can skip.

The three rules.

- h = 0 turns A* into Dijkstra.** With no guess at all, *f* = *g*, and "take the smallest *f*" becomes "take the cheapest cell found so far". That is Dijkstra's algorithm. On a grid where every move costs 1 it visits cells in the same order as BFS, just with a slower queue. So *A is Dijkstra plus a guess, and Dijkstra is A* with a guess of zero. The project uses this: `dijkstra()` is literally `aStar()` with the zero heuristic.
- h must never over-estimate.** A guess that is too small only makes A *look at a few more cells*. A guess that is too big can make it return the wrong answer. A guess that never over-estimates is called *admissible**. Here is a bad one, on a map where the two routes have different lengths:

```
c0 c1 c2 c3 r0 S . . . r1 . # # . r2 . # # G r3 . . . .
```

The top route costs 5, the bottom route costs 7. Now suppose the guess for cell (0,3) is wrong and says 20 (the honest value is 2). Then (0,3) gets *f* = 3 + 20 = 23 and sinks to the bottom of the list. Every cell on the bottom route has *f* at most 7, so all of them are taken first, and the goal is taken with *g* = 7 from the

bottom. A returns 7. The real answer is 5. The over-estimate hid the best route. Manhattan on a 4-direction grid can never over-estimate (you cannot get anywhere in fewer than rows + cols moves), so it is safe. C3 shows a case where it is not* safe.

The heuristics used in this chapter also satisfy a slightly stronger condition called *consistent*: the guess never drops by more than the cost of one move. That is what allows us to mark a cell as finished the first time it is taken off the list and never look at it again. C4 points out the exact line that depends on it.

1. **The to-do list is a priority queue.** "Give me the entry with the smallest f" must be fast, and the list can hold thousands of entries. A plain list scanned every time would be proportional to its length per step. A binary heap does it in about $\log(n)$ steps. In Python that is `heapq`; in C++ it is `std::priority_queue`. Both push and pop in $\log(n)$.

In your own words: "A* is BFS with a to-do list ordered by $f = g + h$, where g is the exact cost so far and h is a guess of the cost to go. It always takes the smallest f . If h never over-estimates, the first time the goal is taken its g is the true shortest cost. With $h = 0$ it is Dijkstra."

C2. From BFS to A*: what changes in the code

Put the BFS loop from A2 next to the A* loop. Lines marked `<--` are the ones that change.

BFS (A2)	A*
-----	-----
<code>dist[start] = 0</code>	<code>g[start] = 0</code>
<code>queue = [start]</code>	<code>open = [(f = h(start), start)]</code>
<code>while queue not empty:</code>	<code>while open not empty:</code>
<code>cell = queue.pop_front()</code>	<code>cell = open.pop_smallest_f()</code> <-- cheapest-looking first, not oldest
	if cell is closed: continue <-- a stale duplicate; skip it
	mark cell closed
	if cell == goal: <-- stop at the goal
	rebuild path, return g[cell]
for each free neighbour n of cell:	for each free neighbour n of cell:
if dist[n] is unset:	<code>new_g = g[cell] + cost(cell, n)</code> <-- moves may cost different amounts
<code>dist[n] = dist[cell] + 1</code>	if g[n] is unset or <code>new_g < g[n]</code> : <-- "unset" became "cheaper than before"
	<code>g[n] = new_g</code>
	<code>parent[n] = cell</code> <-- remember how we got here
<code>queue.push_back(n)</code>	<code>open.push((new_g + h(n), n))</code>

Four changes: the queue is ordered by f ; a cell can be improved after it has been pushed, so we push again and skip stale copies later; we stop when the goal is taken, not when the queue runs dry; and we remember each cell's parent so the path can be rebuilt.

Now the same thing in real C++, on a 4-direction grid with every move costing 1. This is a complete file. It is saved as `code/03-grids-bfs-dfs/11_astar_minimal.cpp`; type it out yourself first, then compile with `g++ -std=c++17 -Wall -Wextra -O2 11_astar_minimal.cpp -o astar_min`, and run it. It prints the path through the tiny map and OK.

```
#include <algorithm> // std::reverse
#include <cassert>
#include <cstdlib> // std::abs
#include <functional> // std::greater
#include <iostream>
#include <queue> // std::priority_queue
#include <string>
#include <tuple>
#include <utility> // std::pair
#include <vector>

// A* on a 4-connected grid ('#' = wall). Every step costs 1.
// Returns the cost of the cheapest path from (sr,sc) to (gr,gc), or -1 if there
// is none, and fills `path` with the cells from start to goal inclusive.
int aStar4(const std::vector<std::string>& grid, int sr, int sc, int gr, int gc,
          std::vector<std::pair<int, int>>& path) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    const int dr[4] = {-1, 0, 1, 0};
```

```

const int dc[4] = {0, 1, 0, -1};
auto h = [&](int r, int c) { return std::abs(r - gr) + std::abs(c - gc); }; // rows+cols to goal

std::vector<int> g(rows * cols, -1); // best cost so far per cell; -1 = never reached
std::vector<int> parent(rows * cols, -1); // which cell we came from
std::vector<char> closed(rows * cols, 0); // 1 once a cell has been taken off the list for good
using Entry = std::tuple<int, int, int, int>; // (f, -g, r, c): smallest f first, then LARGEST g
std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open;

g[sr * cols + sc] = 0;
open.push({h(sr, sc), 0, sr, sc});
while (!open.empty()) {
    const auto [f, negG, r, c] = open.top(); // the entry with the smallest f
    open.pop();
    const int idx = r * cols + c;
    if (closed[idx]) continue; // stale duplicate of a cell already done: skip it
    closed[idx] = 1;
    if (r == gr && c == gc) { // goal taken off the list: its g is the answer
        for (int i = idx; i != -1; i = parent[i]) path.push_back({i / cols, i % cols});
        std::reverse(path.begin(), path.end()); // we walked goal -> start, flip it
        return -negG;
    }
    for (int k = 0; k < 4; ++k) {
        const int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue; // outside
        if (grid[nr][nc] == '#') continue; // wall
        const int ni = nr * cols + nc;
        const int ng = -negG + 1; // one more step
        if (g[ni] == -1 || ng < g[ni]) { // first time here, or a cheaper way than before
            g[ni] = ng;
            parent[ni] = idx;
            open.push({ng + h(nr, nc), -ng, nr, nc}); // push even if ni is already in the list
        }
    }
}
return -1; // list ran empty: the goal is unreachable
}

int main() {
    const std::vector<std::string> tiny = {
        "S...",
        ".##.",
        "...G"};
    std::vector<std::pair<int, int>> path;
    assert(aStar4(tiny, 0, 0, 2, 3, path) == 5);
    assert(path.size() == 6);
    assert(path.front() == std::make_pair(0, 0));
    assert(path.back() == std::make_pair(2, 3));
    for (const auto& [r, c] : path) std::cout << "(" << r << "," << c << ") ";
    std::cout << "\n";

    const std::vector<std::string> walled = {
        "S#.",
        ".#.",
        ".#G"};
    path.clear();
    assert(aStar4(walled, 0, 0, 2, 2, path) == -1);
    assert(path.empty());

    path.clear();
    assert(aStar4(tiny, 0, 0, 0, 0, path) == 0 && path.size() == 1); // start == goal
    std::cout << "OK 11_astar_minimal.cpp\n";
    return 0;
}

```

Output:

```

(0,0) (0,1) (0,2) (0,3) (1,3) (2,3)
OK 11_astar_minimal.cpp

```

That is the top route from the C1 trace, cost 5.

What is new, line by line, for a Python developer.

- **Flat indexing.** `g`, `parent` and `closed` are one-dimensional vectors of size `rows * cols`, and `cell (r, c)` lives at `r * cols + c`. Going back is `i / cols` for the row and `i % cols` for the column. One allocation instead of `rows` of them, and it is how real occupancy grids are stored (A6). You will see it again in the project.
- `auto h = [&](int r, int c) { ... };` is a *lambda*: a small local function, like a nested `def` in Python. The `[&]` means "you may use the variables around you", here `gr` and `gc`. Calling it is just `h(r, c)`.
- `std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open;` is the heap. Three things in the angle brackets: what it holds, which container it is built on (always `std::vector`), and how to compare two entries. Here is the trap: by default `std::priority_queue` puts the *largest* element on top. That is the opposite of Python's `heapq`, which always pops the smallest. To get the smallest `f` on top you must hand it the "backwards" comparison `std::greater<Entry>`. If you forget, the search takes the most-expensive-looking cell every time and behaves like a search for the *longest* route. The project's test suite catches that mistake instantly; your own eyes may not.
- **The entry is a tuple (f, -g, r, c).** C++ tuples compare element by element from the left, exactly like Python tuples, so a heap of tuples is ordered by `f` first. This is the same trick you would use with `heapq.heappush(pq, (f, -g, r, c))`. Storing `-g` rather than `g` is the tie-break from C1: among equal `f`, the *larger* `g` has the *smaller* `-g` and so comes out first. `top()` and `pop()` are two calls, the same as `front()` and `pop()` on the queue in A2, and `const auto [f, negG, r, c] = open.top();` unpacks the tuple the way `f, neg_g, r, c = heapq.heappop(pq)` does.
- **Push duplicates, skip stale ones.** Suppose (2,2) is already on the list with `g = 6` and we now find a way to reach it with `g = 4`. In a textbook you would "decrease the key" of the existing entry. Neither `std::priority_queue` nor `heapq` can reach into the heap and edit an entry. So we push a *second* entry for (2,2) with the better `f` and leave the old one where it is. The better entry comes out first (smaller `f`), we deal with the cell and mark it `closed`. Later the old entry surfaces; `if (closed[idx]) continue;` throws it away. This is called *lazy deletion*. The heap can hold a few entries per cell instead of one, at most one per incoming edge, which on a grid is 4 or 8. Cheap, and no hand-written heap needed.
- `std::vector<char> closed, not std::vector<bool>.` `vector<bool>` is a special case in C++ that packs eight flags into a byte and behaves strangely (you cannot take a reference to one element). A vector of `char` is a plain byte array. The project uses `std::uint8_t` for the same reason.
- **Rebuilding the path.** `parent[ni] = idx` records, for every cell, which cell we stepped in from. When the goal is taken we walk `parent` links from the goal back to the start (the start's parent is `-1`, which stops the loop), collecting cells goal-first, then `std::reverse` flips the vector in place. Python: `path.reverse()` or `path[::-1]`.
- **-1 as "unset" for g.** Every move costs 1 here, so `g` is an `int` and `-1` is a safe sentinel. In the project moves can cost `sqrt(2)`, so `g` is a `double` and "unset" is `+infinity`, which is cleaner: `new_g < infinity` is always true the first time, so the `g[ni] == -1 ||` part of the check disappears.

Python (heapq)	C++	Note
<code>pq = []; heapq.heappush(pq, (f, -g, r, c))</code>	<code>std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open; open.push({f, -g, r, c});</code>	Must say <code>greater</code> to get smallest-first.
<code>f, ng, r, c = heapq.heappop(pq)</code>	<code>const auto [f, negG, r, c] = open.top(); open.pop();</code>	Two calls.
<code>while pq:</code>	<code>while (!open.empty())</code>	Not truthy in C++.
<code>path[::-1]</code>	<code>std::reverse(path.begin(), path.end());</code>	In place.
nested <code>def h(r, c):</code>	<code>auto h = [&](int r, int c) { ... };</code>	<code>[&]</code> captures surrounding variables.

In your own words: "Compared with BFS: the queue becomes a min-heap on f, I push a fresh entry whenever a cell's g improves and skip stale entries via a closed set, I stop when the goal is popped, and I rebuild the path from a parent array and reverse it. `std::priority_queue` is a max-heap by default, so the comparator is `std::greater`."

C3. 8 directions, diagonal cost, and the right heuristic

Robots do not only move up, down, left and right. Allowing the four diagonal moves as well ("8-connected") gives shorter, more natural paths. Three things change.

A diagonal step costs $\sqrt{2}$. Moving one cell right and one cell down at the same time covers the diagonal of a square with side 1. Pythagoras: $\sqrt{1*1 + 1*1} = \sqrt{2} = 1.414\dots$. If you charged 1 for a diagonal, the planner would treat diagonals as free shortcuts and zig-zag diagonally everywhere. If you charged 2, it would never use one. 1.414 is the truth, and it is what the project charges.

Manhattan is now a bad guess. Say the goal is 2 rows down and 3 columns right, with nothing in the way:

	c0	c1	c2	c3	
r0	S	.	.	.	
r1	.	x	.	.	two diagonals, then one step right
r2	.	.	x	G	cost = 1.414 + 1.414 + 1 = 3.83

Manhattan says $2 + 3 = 5$. The true cost is 3.83. The guess is *bigger* than the truth: it over-estimates, so it is not admissible on an 8-connected grid, and rule 2 from C1 says A* can now return a longer path than necessary. (The project's `MatchesDijkstraOnRandomMaps` test would catch it.)

The right guess is the *octile* distance: you take $\min(dr, dc)$ diagonal steps, which cover one row and one column each, and then $\max(dr, dc) - \min(dr, dc)$ straight steps for what is left over:

```
octile = max(dr, dc) + (sqrt(2) - 1) * min(dr, dc)
        = 3 + 0.414 * 2 = 3.83           for dr = 2, dc = 3
```

That is exactly the true cost when nothing is in the way, so it never over-estimates, and it is as tight as a guess can be. The straight-line (Euclidean) distance $\sqrt{dr*dr + dc*dc}$ is $\sqrt{13} = 3.61$ here: also safe, but looser, so A* takes more cells before it is sure.

Guess	Formula	Safe on 4-connected	Safe on 8-connected
Zero	0	yes (it is Dijkstra)	yes
Manhattan	$dr + dc$	yes, and exact on an open map	no , over-estimates diagonals
Octile	$\max(dr, dc) + (\sqrt{2} - 1) * \min(dr, dc)$	yes, but loose	yes, and exact on an open map
Euclidean	$\sqrt{dr*dr + dc*dc}$	yes, loose	yes, loose

(dr and dc mean $|\text{row} - \text{goal row}|$ and $|\text{col} - \text{goal col}|$.) The rule: use the guess that is exact on an open map for your move set. Manhattan for 4 directions, octile for 8.

Do not cut corners. Look at this robot R wanting to step diagonally to (1,1):

	c0	c1
r0	R	#
r1	.	.

The diagonal move passes right through the corner where the wall at $(0,1)$ touches $(1,0)$. A point could squeeze through; a robot with any width would clip the wall. So the project refuses a diagonal step unless *both* cells it brushes past, $(r + dr, c)$ and $(r, c + dc)$, are free. Here $(0,1)$ is blocked, so the robot goes $(0,0) \rightarrow (1,0) \rightarrow (1,1)$ instead: cost 2, not 1.414. And in this map:

```

      c0  c1
r0    R   #
r1    #   .

```

both brushed cells are walls, and $(1,1)$ is simply unreachable. The NoCornerCutting test checks both cases.

In your own words: "With 8 directions a diagonal costs $\sqrt{2}$. Manhattan then over-estimates and is not admissible; octile is exact on an open grid and is the right heuristic. A diagonal step is only allowed when both orthogonal neighbours are free, otherwise the robot clips the corner."

C4. The project, file by file

The project lives in `code/03-grids-bfs-dfs/astar/`:

astar/	
CMakeLists.txt	how to build the library, the demo and the tests
include/astar/grid.hpp	Grid: the map (flat byte array, inBounds, isFree, fromAscii)
include/astar/astar.hpp	the public interface: Cell, Connectivity, Heuristic, Result, aStar(), dijkstra()
src/astar.cpp	the planner itself
src/demo.cpp	astar_demo: reads a map file, draws the path
tests/astar_test.cpp	13 GoogleTest cases
maps/sample.txt	an ASCII map with S and G
README.md	design notes (the same points as this section, shorter)

Two ideas that are new if you have only ever written single-file C++:

- **Headers and sources.** A `.hpp` file *declares* things (the shape of a struct, the signature of a function). A `.cpp` file *defines* them (the bodies). Anyone who wants to use the planner includes the header; the compiled body lives in a library they link against. Python has no split like this because it has no separate compile step.
- **A library, a demo and a test as three targets.** The planner is compiled once into `libastar.a`. The demo and the test executables each link it. Change the planner, rebuild, and both pick it up.

Read the files in this order.

CMakeLists.txt

CMake is the build description, the closest thing C++ has to a `pyproject.toml` plus a Makefile. It says what to build from which sources, with which flags, and what depends on what. `cmake -S . -B build` reads this file and writes a Makefile into `build/`; `cmake --build build` runs it.

```

cmake_minimum_required(VERSION 3.16)
project(astar_grid LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

# Library: the planner itself. PUBLIC include dir so consumers get it too.
add_library(astar src/astar.cpp)
target_include_directories(astar PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_compile_options(astar PRIVATE -Wall -Wextra -Wpedantic)

# Demo: astar_demo [maps/sample.txt] [4|8]

```

```

add_executable(aster_demo src/demo.cpp)
target_link_libraries(aster_demo PRIVATE aster)
target_compile_options(aster_demo PRIVATE -Wall -Wextra -Wpedantic)

# Tests. libgtest-dev on Ubuntu 22.04 ships GTestConfig.cmake, so find_package
# gives us the GTest::gtest_main imported target. Fallback: build from source.
enable_testing()
find_package(GTest QUIET)
if(NOT GTest_FOUND AND EXISTS /usr/src/googletest)
    add_subdirectory(/usr/src/googletest ${CMAKE_BINARY_DIR}/googletest EXCLUDE_FROM_ALL)
    add_library(GTest::gtest_main ALIAS gtest_main)
elseif(NOT GTest_FOUND)
    message(FATAL_ERROR "GoogleTest not found: apt install libgtest-dev")
endif()

add_executable(aster_test tests/aster_test.cpp)
target_link_libraries(aster_test PRIVATE aster GTest::gtest_main)
target_compile_options(aster_test PRIVATE -Wall -Wextra)
add_test(NAME aster_test COMMAND aster_test)

```

What to notice:

- `add_library(aster src/aster.cpp)` makes the planner a library target. `target_include_directories(... PUBLIC include)` says "the headers are in `include/`, and anyone who links `aster` gets that include path too". That is why `demo.cpp` can write `#include "aster/aster.hpp"` without its own setting.
- `add_executable(aster_demo src/demo.cpp)` plus `target_link_libraries(aster_demo PRIVATE aster)` is "build this program, and link the planner into it". Same again for `aster_test`.
- `set(CMAKE_CXX_STANDARD 17)` is the `-std=c++17` you have been typing by hand. `-Wall -Wextra -Wpedantic` are the warnings you have been enabling by hand. The `CMAKE_BUILD_TYPE Release` default turns on `-O3` optimisation.
- `find_package(GTest QUIET)` looks for an installed GoogleTest. On Ubuntu 22.04 `apt install libgtest-dev` provides it and the `GTest::gtest_main` target appears. The `elseif` fallback builds GoogleTest from `/usr/src/googletest` if only the sources are there.
- `enable_testing()` and `add_test(NAME aster_test COMMAND aster_test)` register the test executable with `ctest`, so `ctest --test-dir build` knows what to run.

`include/aster/grid.hpp`

The map. A Grid is a width, a height, and one flat array of bytes, 0 for free and 1 for blocked.

```

// grid.hpp – a binary occupancy grid: 0 free, 1 blocked. Row-major, index r*width+c.
#pragma once

#include <stdint>
#include <string>
#include <vector>

namespace aster {

struct Grid {
    int width = 0;
    int height = 0;
    std::vector<std::uint8_t> cells; // flat, row-major; uint8_t not bool so it is a real byte array

    Grid() = default;
    Grid(int w, int h) : width(w), height(h), cells(static_cast<std::size_t>(w) * h, 0) {}

    int index(int r, int c) const { return r * width + c; }
    bool inBounds(int r, int c) const { return r >= 0 && r < height && c >= 0 && c < width; }
    bool isFree(int r, int c) const { return inBounds(r, c) && cells[index(r, c)] == 0; }
    void setBlocked(int r, int c, bool blocked = true) { cells[index(r, c)] = blocked ? 1 : 0; }

    // '#' is blocked, every other character ('.', 'S', 'G', ' ') is free.
    // Throws std::invalid_argument if rows have different lengths.
    static Grid fromAscii(const std::vector<std::string>& rows);
};

```

```
};

} // namespace astar
```

What to notice:

- `#pragma once` means "if this header is included twice in one file, ignore the second time". Every header has it (or an equivalent `#ifndef` guard).
- `namespace astar { ... }` groups everything under a name, like a Python module. Users write `astar::Grid`. It stops a `Grid` from a different library colliding with ours.
- **A flat `std::vector<std::uint8_t>`**. One allocation of `width * height` bytes, cell `(r, c)` at `r * width + c`. This is the C2 layout and it is the layout of a ROS `OccupancyGrid` and of a `nav2` costmap: a byte array with a width and a height. It is also faster than a vector of vectors, because the whole map is one contiguous block in memory. `uint8_t` rather than `bool` for the `vector<bool>` reason from C2.
- `struct` with functions inside it. `index`, `inBounds`, `isFree`, `setBlocked` are *member functions*, the same as methods on a Python class; `this` is implicit like `self`. The `const` after `inBounds(int r, int c)` `const` promises the function does not change the grid, which is what allows calling it on a `const Grid&`.
- `isFree` does the bounds check *and* the wall check. So the planner can ask "is the neighbour free?" in one call and never index out of range. That is the A1 rule packaged up.
- `Grid(int w, int h) : width(w), height(h), cells(static_cast<std::size_t>(w) * h, 0) {}` is a *constructor* with a *member initialiser list*: the part after the colon sets each field directly. `Grid()` = default; also keeps the do-nothing constructor so an empty `Grid` can be declared and filled later.
- `static Grid fromAscii(...)` is a function attached to the type rather than to an object, like a `@classmethod` factory in Python. Its body is in `astar.cpp`. `'#'` is a wall, every other character is free, so `S` and `G` are free cells. Rows of different lengths throw `std::invalid_argument`, and a test checks that.

`include/astar/astar.hpp`

The public interface of the planner: the types you pass in and the type you get back.

```
// astar.hpp – A* on a Grid. Dijkstra is A* with the zero heuristic.
#pragma once

#include <cstdint>
#include <functional>
#include <vector>

#include "astar/grid.hpp"

namespace astar {

struct Cell {
    int r = 0;
    int c = 0;
};
inline bool operator==(Cell a, Cell b) { return a.r == b.r && a.c == b.c; }
inline bool operator!=(Cell a, Cell b) { return !(a == b); }

enum class Connectivity { Four, Eight };

// Built-in heuristics. All are admissible and consistent for the connectivity
// they are meant for: Manhattan for Four, Octile or Euclidean for Eight.
// Manhattan is NOT admissible on an 8-connected grid (it overestimates diagonals).
enum class Heuristic { Zero, Manhattan, Octile, Euclidean };

using HeuristicFn = std::function<double(Cell, Cell)>;

double zeroHeuristic(Cell a, Cell b);
double manhattan(Cell a, Cell b);
double octile(Cell a, Cell b);
double euclidean(Cell a, Cell b);
HeuristicFn heuristicFor(Heuristic h);
```

```

struct Result {
    bool found = false;
    double cost = 0.0;           // +infinity when !found
    std::vector<Cell> path;      // start..goal inclusive; empty when !found
    std::size_t expanded = 0;   // nodes popped from the open set and closed
};

// Orthogonal moves cost 1, diagonal moves cost sqrt(2). Diagonal moves are
// refused when either orthogonal neighbour is blocked (no corner cutting).
Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, const HeuristicFn& h);
Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, Heuristic h);
Result dijkstra(const Grid& grid, Cell start, Cell goal, Connectivity conn);

} // namespace astar

```

What to notice:

- `struct Cell { int r; int c; }` with `operator==` and `operator!=` defined next to it. That is what lets the tests write `EXPECT_EQ(res.path.front(), start)`: without `==` the compiler has no idea how to compare two `Cells`. In Python you would write `__eq__`.
- `enum class Connectivity { Four, Eight }` is a Python Enum. The `class` keyword makes it *scoped*: you must write `Connectivity::Four`, and it will not silently convert to an `int`. `Heuristic` is the same for the four built-in guesses from C3.
- `using HeuristicFn = std::function<double(Cell, Cell)>;` names a type meaning "anything I can call with two `Cells` that gives back a `double`": a plain function, a lambda, anything. It is the C++ spelling of a `Callable[[Cell, Cell], float]` type hint. It exists so you can pass your *own* guess in; the `CustomHeuristicFunction` test does exactly that with a lambda.
- **Result is what comes back**, and it is a struct rather than a bare number because a planner has four things to say: `found` (is there a path at all), `cost` (the total, in step units, `+infinity` when not found), `path` (every cell from start to goal inclusive, empty when not found), and `expanded` (how many cells were taken off the list). `expanded` is the number you looked at in the C1 pictures: 6 versus 10, 9 versus 25. It is how you *measure* whether a heuristic is helping, and it is what you would log on a robot to tune planning time.
- `aStar` appears twice with the same name and different last parameters. That is *overloading*, which Python does not have: the compiler picks the version by the argument type. One takes any callable, the other takes an enum and looks up the built-in function. `dijkstra` is a third name for `aStar` with the zero guess.
- The `const Grid&` and `const HeuristicFn&` parameters mean "look at the caller's grid, do not copy it". A 1000 by 1000 map is a megabyte; copying it per call would be silly.

src/astar.cpp

The planner. It is the C2 function grown up: double costs, 4 or 8 directions, corner-cutting check, a `Result` instead of an `int`.

```

// astar.cpp – A* with a lazy-deletion binary heap and a closed set.
#include "astar/astar.hpp"

#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <limits>
#include <queue>
#include <stdexcept>

namespace astar {

Grid Grid::fromAscii(const std::vector<std::string>& rows) {
    Grid g(rows.empty() ? 0 : static_cast<int>(rows[0].size()), static_cast<int>(rows.size()));
    for (int r = 0; r < g.height; ++r) {

```

```

        if (static_cast<int>(rows[r].size()) != g.width) throw std::invalid_argument("ragged ascii map");
        for (int c = 0; c < g.width; ++c) g.cells[g.index(r, c)] = rows[r][c] == '#' ? 1 : 0;
    }
    return g;
}

namespace {
constexpr double kSqrt2 = 1.4142135623730951;
// First four entries are orthogonal (cost 1), last four diagonal (cost sqrt2).
constexpr int kDr[8] = {-1, 0, 1, 0, -1, -1, 1, 1};
constexpr int kDc[8] = {0, 1, 0, -1, -1, 1, -1, 1};

struct Node {
    double f;
    double g;
    int idx;
};

// priority_queue is a max-heap, so "greater" puts the smallest f on top.
// Tie-break on larger g: among equal f, prefer the node closer to the goal,
// which avoids sweeping a whole plateau of equal-f cells on open maps.
struct NodeGreater {
    bool operator()(const Node& a, const Node& b) const {
        if (a.f != b.f) return a.f > b.f;
        return a.g < b.g;
    }
};

// namespace

double zeroHeuristic(Cell, Cell) { return 0.0; }
double manhattan(Cell a, Cell b) { return std::abs(a.r - b.r) + std::abs(a.c - b.c); }
double octile(Cell a, Cell b) {
    const int dr = std::abs(a.r - b.r), dc = std::abs(a.c - b.c);
    return (dr + dc) + (kSqrt2 - 2.0) * std::min(dr, dc); // = max + (sqrt2-1)*min
}
double euclidean(Cell a, Cell b) { return std::hypot(a.r - b.r, a.c - b.c); }

HeuristicFn heuristicFor(Heuristic h) {
    switch (h) {
        case Heuristic::Manhattan: return manhattan;
        case Heuristic::Octile: return octile;
        case Heuristic::Euclidean: return euclidean;
        case Heuristic::Zero: break;
    }
    return zeroHeuristic;
}

Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, const HeuristicFn& h) {
    Result res;
    res.cost = std::numeric_limits<double>::infinity();
    if (!grid.isFree(start.r, start.c) || !grid.isFree(goal.r, goal.c)) return res;

    const int n = grid.width * grid.height;
    const double inf = std::numeric_limits<double>::infinity();
    std::vector<double> gScore(n, inf);
    std::vector<int> parent(n, -1);
    std::vector<char> closed(n, 0);
    std::priority_queue<Node, std::vector<Node>, NodeGreater> open;

    const int s = grid.index(start.r, start.c);
    const int t = grid.index(goal.r, goal.c);
    gScore[s] = 0.0;
    open.push({h(start, goal), 0.0, s});
    const int nDirs = conn == Connectivity::Four ? 4 : 8;

    while (!open.empty()) {
        const Node cur = open.top();
        open.pop();
        // Lazy deletion: a node can sit in the heap several times with different
        // g. Only the first (best) pop counts; later copies are stale and skipped.
        if (closed[cur.idx]) continue;
        closed[cur.idx] = 1;
        ++res.expanded;
    }
}

```

```

    if (cur.idx == t) {
        res.found = true;
        res.cost = cur.g;
        for (int i = t; i != -1; i = parent[i]) res.path.push_back({i / grid.width, i % grid.width});
        std::reverse(res.path.begin(), res.path.end());
        return res;
    }

    const int r = cur.idx / grid.width, c = cur.idx % grid.width;
    for (int k = 0; k < nDirs; ++k) {
        const int nr = r + kDr[k], nc = c + kDc[k];
        if (!grid.isFree(nr, nc)) continue;
        // No corner cutting: a diagonal step needs both orthogonal cells free,
        // otherwise a real robot would clip the obstacle corner.
        if (k >= 4 && (!grid.isFree(r + kDr[k], c) || !grid.isFree(r, c + kDc[k]))) continue;
        const int ni = grid.index(nr, nc);
        // With a consistent heuristic a closed node already has its optimal g.
        if (closed[ni]) continue;
        const double ng = cur.g + (k < 4 ? 1.0 : kSqrt2);
        if (ng < gScore[ni]) {
            gScore[ni] = ng;
            parent[ni] = cur.idx;
            open.push({ng + h({nr, nc}, goal), ng, ni});
        }
    }
}

return res; // open set exhausted: no path, cost stays +inf, path stays empty
}

Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, Heuristic h) {
    return aStar(grid, start, goal, conn, heuristicFor(h));
}

Result dijkstra(const Grid& grid, Cell start, Cell goal, Connectivity conn) {
    return aStar(grid, start, goal, conn, zeroHeuristic);
}

} // namespace astar

```

What to notice:

- namespace { ... } with no name is an *anonymous namespace*: everything inside is private to this file, like a leading underscore in Python. `kSqrt2`, the direction arrays, `Node` and `NodeGreater` are implementation details nobody else needs.
- **The direction arrays are ordered on purpose.** Entries 0 to 3 are up, right, down, left; entries 4 to 7 are the diagonals. So `k < 4 ? 1.0 : kSqrt2` gives the step cost, and `nDirs` is 4 or 8 depending on `Connectivity`, and the same loop serves both.
- `struct Node { double f; double g; int idx; }` is the heap entry, a struct instead of the tuple from C2. `NodeGreater` is the comparison, written out by hand: `a.f > b.f` for smallest-*f*-first (the "backwards" comparator again), and on equal *f*, `a.g < b.g` so the larger *g* wins. That is the C1 tie-break; the README calls it out because without it the `HeuristicReducesExpansions` test fails on an open map.
- `gScore` is double with `+infinity` as "unset", as promised in C2. `parent` is int with `-1`, `closed` is a byte per cell.
- The early return: if the start or the goal is a wall or outside the map, return a `Result` with `found = false` and `cost = infinity` before doing anything. `expanded` is 0.
- The loop is the C2 loop. `if (closed[cur.idx]) continue;` is the lazy deletion. `++res.expanded` counts every cell that is really taken. `if (cur.idx == t)` rebuilds the path from `parent` and reverses it.
- **The corner-cutting check** is the line `if (k >= 4 && (!grid.isFree(r + kDr[k], c) || !grid.isFree(r, c + kDc[k]))) continue;`. For a diagonal move (`k >= 4`) it looks at the two cells the robot would brush past and refuses the move if either is blocked. C3 drew this.
- **if (closed[ni]) continue; when looking at neighbours.** This skips a neighbour that is already finished, and it is only correct because the heuristics are *consistent* (C1, rule 2): with a consistent guess, the first time a cell is taken its *g* is already the best possible, so a finished cell can never be improved.

Hand in an inconsistent guess through `HeuristicFn` (for example `2 * manhattan`, "weighted A*") and the planner still terminates, but the path may be a little longer than optimal. That trade is sometimes made on purpose to plan faster.

- The octile body is written as $(dr + dc) + (\sqrt{2} - 2) * \min(dr, dc)$, which is the same as $\max + (\sqrt{2} - 1) * \min$ from C3 after a little algebra ($dr + dc = \max + \min$).
- `heuristicFor` is a switch on the enum that returns the matching function. The case `Heuristic::Zero`: `break;` followed by `return zeroHeuristic;` outside the switch is a style that keeps the compiler happy about every path returning something.
- Cost: every cell is taken at most once and pushes at most 8 entries, each heap operation is log of the heap size, so the time is proportional to $(\text{cells} \times \text{directions} \times \log \text{cells})$, written $O(E \log V)$. Memory is three vectors of one entry per cell plus the heap.

tests/astar_test.cpp

The proof. If you have never used GoogleTest, here is all you need:

1. `#include <gtest/gtest.h>`, and link with `GTest::gtest_main`, which supplies `main()` for you.
2. `TEST(SuiteName, TestName) { ... }` defines one test; every `TEST` runs automatically.
3. `EXPECT_EQ(a, b)`, `EXPECT_TRUE(x)`, `EXPECT_FALSE(x)`, `EXPECT_LT(a, b)` check a fact and *carry on* if it fails, so one test can report several failures. `EXPECT_NEAR(a, b, 1e-9)` is the `pytest.approx` for doubles; `EXPECT_DOUBLE_EQ` is the same with a rounding-error tolerance; `EXPECT_THROW(expr, ExceptionType)` is `pytest.raises`.
4. `ASSERT_TRUE(x)` and friends check and *stop this test* on failure. Use them before code that would crash if the fact were false, like `path.front()` on a path that might be empty.
5. Run `./build/astar_test` to see every test, or `./build/astar_test --gtest_filter='AStar.NoPath'` for one.

```
// astar_test.cpp - GoogleTest suite for the grid A*.
#include <cmath>
#include <cstdlib>
#include <limits>
#include <random>
#include <string>
#include <vector>

#include <gtest/gtest.h>

#include "astar/astar.hpp"

using astar::Cell;
using astar::Connectivity;
using astar::Grid;
using astar::Heuristic;
using astar::Result;

namespace {
constexpr double kSqrt2 = 1.4142135623730951;

Grid openGrid(int w, int h) { return Grid(w, h); }

// Every consecutive pair of cells is one king move apart, every cell is free,
// and the summed step costs equal the reported cost.
void expectContiguous(const Grid& g, const Result& res, Cell start, Cell goal) {
    ASSERT_TRUE(res.found);
    ASSERT_FALSE(res.path.empty());
    EXPECT_EQ(res.path.front(), start);
    EXPECT_EQ(res.path.back(), goal);
    double sum = 0.0;
    for (std::size_t i = 0; i < res.path.size(); ++i) {
        EXPECT_TRUE(g.isFree(res.path[i].r, res.path[i].c));
        if (i == 0) continue;
        const int dr = std::abs(res.path[i].r - res.path[i - 1].r);
        const int dc = std::abs(res.path[i].c - res.path[i - 1].c);
        EXPECT_LE(dr, 1);
```

```

    EXPECT_LE(dc, 1);
    EXPECT_FALSE(dr == 0 && dc == 0);
    sum += (dr + dc == 2) ? kSqrt2 : 1.0;
}
EXPECT_NEAR(sum, res.cost, 1e-9);
}
} // namespace

TEST(Grid, FromAsciiAndBounds) {
    const Grid g = Grid::fromAscii({"..#", "S.G"});
    EXPECT_EQ(g.width, 3);
    EXPECT_EQ(g.height, 2);
    EXPECT_TRUE(g.isFree(0, 0));
    EXPECT_FALSE(g.isFree(0, 2));
    EXPECT_TRUE(g.isFree(1, 0)); // 'S' and 'G' are free cells
    EXPECT_FALSE(g.inBounds(-1, 0));
    EXPECT_FALSE(g.inBounds(0, 3));
    EXPECT_FALSE(g.isFree(2, 0)); // out of bounds counts as not free
    EXPECT_THROW(Grid::fromAscii({"...", ".."}), std::invalid_argument);
}

TEST(AStar, StraightLineFourConnected) {
    const Grid g = openGrid(5, 1);
    const Result res = astar::aStar(g, {0, 0}, {0, 4}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 4.0);
    EXPECT_EQ(res.path.size(), 5u);
    expectContiguous(g, res, {0, 0}, {0, 4});
}

TEST(AStar, AroundObstacle) {
    // Wall down column 2 with a gap at the bottom row.
    const Grid g = Grid::fromAscii({
        "..#..",
        "..#..",
        "..#..",
        "....."});
    const Result res = astar::aStar(g, {0, 0}, {0, 4}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 10.0); // down 3, right 4, up 3
    expectContiguous(g, res, {0, 0}, {0, 4});
    for (const auto& cell : res.path) EXPECT_NE(cell.c == 2 && cell.r < 3, true);
}

TEST(AStar, NoPath) {
    const Grid g = Grid::fromAscii({
        ".#.",
        ".#.",
        ".#."});
    const Result res = astar::aStar(g, {0, 0}, {0, 2}, Connectivity::Eight, Heuristic::Octile);
    EXPECT_FALSE(res.found);
    EXPECT_TRUE(res.path.empty());
    EXPECT_TRUE(std::isinf(res.cost));
    EXPECT_EQ(res.expanded, 3u); // the whole left column was explored before giving up
}

TEST(AStar, BlockedStartOrGoal) {
    const Grid g = Grid::fromAscii({"#.", ".."});
    EXPECT_FALSE(astar::aStar(g, {0, 0}, {1, 1}, Connectivity::Four, Heuristic::Manhattan).found);
    EXPECT_FALSE(astar::aStar(g, {1, 1}, {0, 0}, Connectivity::Four, Heuristic::Manhattan).found);
    EXPECT_FALSE(astar::aStar(g, {1, 1}, {5, 5}, Connectivity::Four, Heuristic::Manhattan).found);
}

TEST(AStar, StartEqualsGoal) {
    const Grid g = openGrid(3, 3);
    const Result res = astar::aStar(g, {1, 1}, {1, 1}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 0.0);
    ASSERT_EQ(res.path.size(), 1u);
    EXPECT_EQ(res.path[0], (Cell{1, 1}));
    EXPECT_EQ(res.expanded, 1u);
}

```

```

TEST(AStar, EightConnectedDiagonalCostsSqrt2) {
    const Grid g = openGrid(3, 3);
    const Result res = astar::aStar(g, {0, 0}, {2, 2}, Connectivity::Eight, Heuristic::Octile);
    ASSERT_TRUE(res.found);
    EXPECT_NEAR(res.cost, 2.0 * kSqrt2, 1e-12);
    EXPECT_EQ(res.path.size(), 3u);
    // Four-connected on the same grid must walk the long way.
    const Result four = astar::aStar(g, {0, 0}, {2, 2}, Connectivity::Four, Heuristic::Manhattan);
    EXPECT_DOUBLE_EQ(four.cost, 4.0);
}

TEST(AStar, NoCornerCutting) {
    // One orthogonal neighbour blocked: the diagonal is refused, go around (cost 2, not sqrt2).
    const Grid oneBlocked = Grid::fromAscii({".#", ".."});
    const Result r1 = astar::aStar(oneBlocked, {0, 0}, {1, 1}, Connectivity::Eight, Heuristic::Octile);
    ASSERT_TRUE(r1.found);
    EXPECT_DOUBLE_EQ(r1.cost, 2.0);
    EXPECT_EQ(r1.path.size(), 3u);
    // Both orthogonal neighbours blocked: squeezing between them is impossible.
    const Grid pinched = Grid::fromAscii({".#", "#."});
    const Result r2 = astar::aStar(pinched, {0, 0}, {1, 1}, Connectivity::Eight, Heuristic::Octile);
    EXPECT_FALSE(r2.found);
}

TEST(AStar, PathIsContiguousOnMaze) {
    const Grid g = Grid::fromAscii({
        "S..#.....",
        ".#.#.####.",
        ".#...#....",
        ".####.#.#.",
        ".....#..G"});
    for (Connectivity conn : {Connectivity::Four, Connectivity::Eight}) {
        const Heuristic h = conn == Connectivity::Four ? Heuristic::Manhattan : Heuristic::Octile;
        const Result res = astar::aStar(g, {0, 0}, {4, 9}, conn, h);
        expectContiguous(g, res, {0, 0}, {4, 9});
    }
}

TEST(AStar, MatchesDijkstraOnRandomMaps) {
    std::mt19937 rng(42); // fixed seed: the test is deterministic
    std::bernoulli_distribution blocked(0.3);
    std::uniform_int_distribution<int> coord(0, 19);
    int comparedFound = 0;
    for (int trial = 0; trial < 60; ++trial) {
        Grid g(20, 20);
        for (auto& cell : g.cells) cell = blocked(rng) ? 1 : 0;
        const Cell s{coord(rng), coord(rng)}, t{coord(rng), coord(rng)};
        const Connectivity conn = trial % 2 ? Connectivity::Eight : Connectivity::Four;
        for (Heuristic h : {conn == Connectivity::Four ? Heuristic::Manhattan : Heuristic::Octile,
                           Heuristic::Euclidean}) {
            const Result a = astar::aStar(g, s, t, conn, h);
            const Result d = astar::dijkstra(g, s, t, conn);
            EXPECT_EQ(a.found, d.found);
            if (!a.found) continue;
            ++comparedFound;
            EXPECT_NEAR(a.cost, d.cost, 1e-9);
            expectContiguous(g, a, s, t);
            expectContiguous(g, d, s, t);
        }
    }
    EXPECT_GT(comparedFound, 20); // sanity: the random maps were not all disconnected
}

TEST(AStar, DijkstraIsAStarWithZeroHeuristic) {
    const Grid g = openGrid(15, 15);
    const Result d = astar::dijkstra(g, {7, 7}, {14, 0}, Connectivity::Four);
    const Result z = astar::aStar(g, {7, 7}, {14, 0}, Connectivity::Four, Heuristic::Zero);
    EXPECT_DOUBLE_EQ(d.cost, z.cost);
    EXPECT_EQ(d.expanded, z.expanded);
}

```

```

TEST(AStar, HeuristicReducesExpansions) {
    // Start in the middle so Dijkstra has to grow a full diamond before it
    // touches the corner goal; A* with a good heuristic heads straight there.
    const Grid g = openGrid(41, 41);
    const Cell s{20, 20}, t{40, 40};
    const Result d4 = astar::dijkstra(g, s, t, Connectivity::Four);
    const Result a4 = astar::aStar(g, s, t, Connectivity::Four, Heuristic::Manhattan);
    EXPECT_DOUBLE_EQ(a4.cost, d4.cost);
    EXPECT_LT(a4.expanded * 4, d4.expanded);

    const Result d8 = astar::dijkstra(g, s, t, Connectivity::Eight);
    const Result a8 = astar::aStar(g, s, t, Connectivity::Eight, Heuristic::Octile);
    const Result e8 = astar::aStar(g, s, t, Connectivity::Eight, Heuristic::Euclidean);
    EXPECT_NEAR(a8.cost, d8.cost, 1e-9);
    EXPECT_NEAR(e8.cost, d8.cost, 1e-9);
    EXPECT_LT(a8.expanded * 4, d8.expanded);
    EXPECT_LE(a8.expanded, e8.expanded); // octile is the tighter bound, so it expands no more
}

TEST(AStar, CustomHeuristicFunction) {
    const Grid g = openGrid(6, 6);
    // Weighted A* (w = 2): still finds a path, cost may be suboptimal in general,
    // but on an open grid with Manhattan it stays optimal.
    const astar::HeuristicFn weighted = [](Cell a, Cell b) { return 2.0 * astar::manhattan(a, b); };
    const Result w = astar::aStar(g, {0, 0}, {5, 5}, Connectivity::Four, weighted);
    ASSERT_TRUE(w.found);
    EXPECT_DOUBLE_EQ(w.cost, 10.0);
}

```

What to notice:

- `expectContiguous` is a helper used by several tests. It checks the whole *shape* of an answer, not just the cost: the path starts at `start`, ends at `goal`, every cell is free, every consecutive pair is one move apart (row and column each change by at most 1, and not both 0), and the summed step costs (1 or `sqrt2` each) equal the reported `cost`. Any bug in the parent array or the path reversal fails here.
- `Grid::fromAscii({"S...", ".##.", "...G"})` style maps are used everywhere. The tests are readable as pictures.

What each test proves:

Test	Map	Proves
<code>Grid.FromAsciiAndBounds</code>	<code>..# / S.G</code>	The parser: width, height, # blocked, S/G free, out-of-bounds is not free, ragged rows throw.
<code>AStar.StraightLineFourConnected</code>	5 by 1 open	The simplest possible path: cost 4, 5 cells.
<code>AStar.AroundObstacle</code>	wall with a gap at the bottom	The detour is found and costs 10; no path cell is inside the wall.
<code>AStar.NoPath</code>	full-height wall	<code>found == false</code> , empty path, cost is infinite, and <code>expanded == 3</code> (the whole reachable side was searched before giving up).
<code>AStar.BlockedStartOrGoal</code>	2 by 2 with a wall	A start or goal on a wall, or outside the map, gives <code>found == false</code> without crashing.
<code>AStar.StartEqualsGoal</code>	3 by 3 open	Cost 0, path of one cell, one expansion.
<code>AStar.EightConnectedDiagonalCostsSqrt2</code>	3 by 3 open	Corner to corner is $2 * \text{sqrt2}$ with 8 directions and 4 with 4 directions.
<code>AStar.NoCornerCutting</code>	the two C3 pictures	One blocked side forces a cost-2 detour; two blocked sides make the goal unreachable.
<code>AStar.PathIsContiguousOnMaze</code>	a 5 by 10 maze	The path shape is valid for both connectivities.

Test	Map	Proves
AStar.MatchesDijkstraOnRandomMaps	60 random 20 by 20 maps, 30% walls, fixed seed	A with each admissible guess gives the same cost* as Dijkstra, both connectivities. This is the test that catches a non-admissible heuristic, and the strongest one in the file.
AStar.DijkstraIsAStarWithZeroHeuristic	15 by 15 open	dijkstra() and aStar(..., Zero) agree on cost and on expanded.
AStar.HeuristicReducesExpansions	41 by 41 open, start in the middle	A* takes fewer than a quarter of the cells Dijkstra takes, and octile takes no more than Euclidean.
AStar.CustomHeuristicFunction	6 by 6 open	A lambda can be passed as the heuristic (weighted A*, 2 * manhattan).

Read `MatchesDijkstraOnRandomMaps` twice. `std::mt19937 rng(42)` is a random generator with a fixed seed, so the "random" maps are the same every run and a failure can be reproduced. `std::bernoulli_distribution blocked(0.3)` gives `true` 30% of the time. Cross-checking a fast algorithm against a slow one you trust, on many random inputs, is the single most useful testing idea you can take from this chapter.

src/demo.cpp

A small command-line program: read a map file, find S and G, run A* and Dijkstra, print the map with the path drawn as *, and print the cost and both expansion counts.

```
// demo.cpp – astar_demo [map.txt] [4|8]. Reads an ASCII map with 'S' and 'G',
// runs A*, prints the map with the path drawn as '*'.
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

#include "astar/astar.hpp"

int main(int argc, char** argv) {
    const std::string file = argc > 1 ? argv[1] : "maps/sample.txt";
    const bool eight = argc > 2 && std::string(argv[2]) == "8";

    std::ifstream in(file);
    if (!in) {
        std::cerr << "cannot open " << file << "\n";
        return 1;
    }
    std::vector<std::string> rows;
    for (std::string line; std::getline(in, line);)
        if (!line.empty()) rows.push_back(line);

    astar::Cell start{-1, -1}, goal{-1, -1};
    for (int r = 0; r < static_cast<int>(rows.size()); ++r)
        for (int c = 0; c < static_cast<int>(rows[r].size()); ++c) {
            if (rows[r][c] == 'S') start = {r, c};
            if (rows[r][c] == 'G') goal = {r, c};
        }
    if (start.r < 0 || goal.r < 0) {
        std::cerr << "map needs an S and a G\n";
        return 1;
    }

    const astar::Grid grid = astar::Grid::fromAscii(rows);
    const auto conn = eight ? astar::Connectivity::Eight : astar::Connectivity::Four;
    const auto h = eight ? astar::Heuristic::Octile : astar::Heuristic::Manhattan;
    const astar::Result res = astar::aStar(grid, start, goal, conn, h);
    const astar::Result dij = astar::dijkstra(grid, start, goal, conn);
```

```

for (const auto& cell : res.path)
    if (cell != start && cell != goal) rows[cell.r][cell.c] = '*';
for (const auto& row : rows) std::cout << row << "\n";
if (!res.found) {
    std::cout << "no path\n";
    return 0;
}
std::cout << "cost " << res.cost << ", steps " << res.path.size() - 1
    << ", A* expanded " << res.expanded << ", Dijkstra expanded " << dij.expanded << "\n";
return 0;
}

```

What to notice:

- `int main(int argc, char** argv)` is how a C++ program receives command-line arguments: `argc` is how many, `argv[i]` is the *i*-th as a C string, `argv[0]` is the program name. Python's `sys.argv`. `argc > 1 ? argv[1] : "maps/sample.txt"` is "use the first argument if given, else a default".
- `std::ifstream in(file);` opens a file for reading; `if (!in)` is the "could not open" check. `for (std::string line; std::getline(in, line);)` reads it line by line, the same as `for line in open(file)`. Empty lines are skipped.
- The map is read as a `std::vector<std::string>` first so we can find S and G and later overwrite cells with `*` for printing; `Grid::fromAscii` turns the same rows into the byte grid the planner wants.
- It runs both `aStar` and `dijkstra` so the last line shows the two expansion counts side by side. That is exercise 2 below, already built in.

maps/sample.txt

```

S.....#.....
.####.#.#####.
.#...#.#...#..
.#.####.#.#.#..
.#.....#.#.#..
.#####.#.##..
.....#....G

```

A 7 by 16 map. The top-left has a spiral that leads nowhere useful; the way out is down the left edge and along the bottom.

C5. Build it, run it, break it

Build and test. From the repository root:

```

cd "code/03-grids-bfs-dfs/astar"
cmake -S . -B build           # configure: read CMakeLists.txt, write build/Makefile
cmake --build build           # compile the library, the demo and the tests
ctest --test-dir build        # run the registered tests

```

Expected from `ctest`:

```

1/1 Test #1: astar_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 1

```

For the per-test view run the test binary directly:

```
./build/astar_test
```

```

[ RUN      ] Grid.FromAsciiAndBounds
[         OK ] Grid.FromAsciiAndBounds (0 ms)

```

```
[ RUN      ] AStar.StraightLineFourConnected
[         OK ] AStar.StraightLineFourConnected (0 ms)
...
[ RUN      ] AStar.CustomHeuristicFunction
[         OK ] AStar.CustomHeuristicFunction (0 ms)
[ PASSED   ] 13 tests.
```

Run the demo. The second argument is 4 or 8 for the connectivity.

```
./build/astar_demo maps/sample.txt 4
```

```
S.....#.....
*#####.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#####*#####.
*****#*****G
cost 31, steps 31, A* expanded 50, Dijkstra expanded 66
```

A* took 50 cells off its list, Dijkstra (same code, zero guess) took 66. The map has 71 free cells. The heuristic helps, but on a twisty map with the goal behind walls it helps less than on an open one, because the guess "straight-line distance" is optimistic about every wall.

```
./build/astar_demo maps/sample.txt 8
```

```
S.....#.....
*#####.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#####*#####.
*****#*****G
cost 31, steps 31, A* expanded 52, Dijkstra expanded 67
```

Same path with 8 directions. Every diagonal that would shorten the route on this map clips a corner, so the corner cutting rule refuses it and the 8-connected answer is identical. The expansion counts differ by a couple of cells because the octile guess gives slightly different f values, so ties are broken in a different order.

Exercise 1: a map with no path. Create `maps/nopath.txt`:

```
S..#...
...#...
...#...
...#...G
```

```
./build/astar_demo maps/nopath.txt 4
```

```
S..#...
...#...
...#...
...#...G
no path
```

No crash, no -1 in a double, no exception: `found` is false, `cost` is `+infinity`, `path` is empty. Inside `Result`, `expanded` is 12, the number of cells on the start's side of the wall, because the search had to exhaust them all before it could be sure. Callers test `found` first, always.

Rebuild and run just that test:

```
cmake --build build
./build/astar_test --gtest_filter='*Tiny*
```

```
[ RUN      ] AStar.TinyExampleFromTheTutorial
[ OK       ] AStar.TinyExampleFromTheTutorial (0 ms)
[ PASSED   ] 1 test.
```

The `6u` is an unsigned literal, because `expanded` and `path.size()` are unsigned types and the compiler warns when you compare signed with unsigned (the `A1 static_cast<int>` issue from the other side). Now try breaking things and watch which test catches it: change `a.f > b.f` to `a.f < b.f` in `NodeGreater`, or delete the corner-cutting line, or swap `0ctile` for `Manhattan` in the 8-connected tests. Rebuild, run `ctest`, read the failure message. Put it back.

Where this runs on a real robot. The `nav2` global planner (`NavFn`) is Dijkstra or A over the costmap, with cells near obstacles made expensive by the inflation layer (the 01 Matrix distance transform from A6). The *Smac planners* are A over a lattice of motion primitives instead of grid cells. Both have exactly this structure: a heap of `f`, a closed set, a parent array, a heuristic that must not over-estimate. On a 1000 by 1000 costmap that is a few million heap operations, a few tens of milliseconds, which is why a global plan at 1 Hz is easy.

What to write in your CV about this.

Implemented a grid path planner (A and Dijkstra, 4- and 8-connected, admissible heuristics, no corner cutting) in C++17 as a CMake library with a demo tool and a 13-case GoogleTest suite, including randomised cross-checks of A against Dijkstra.

In your own words: "The planner is a library with a `Grid` of flat bytes and an `aStar` function returning found, cost, path and expansion count. The open set is a `priority_queue` with lazy deletion, the closed set a byte per cell, ties go to larger `g`, diagonals cost $\sqrt{2}$ and never cut corners. The tests cross-check A* against Dijkstra on seeded random maps, and the demo prints both expansion counts so I can see the heuristic working."

Part D — Check yourself

Answer these in your own words before marking the phase done in the tracker. Every answer is in this chapter.

1. In `grid[r][c]`, which index goes down and which goes right? What is `grid.size()` and `grid[0].size()`?
2. Write the `dr/dc` arrays for 4 directions from memory. Now for 8.
3. Why must the bounds check come before `grid[nr][nc]`? What happens in Python versus C++ if you get that wrong?
4. Describe BFS as a picture. Why does it find the shortest path?
5. Why do you mark a cell visited when you push it rather than when you pop it?
6. How do you count "rings" (minutes, steps) in BFS? Why must you snapshot the queue size?
7. What changes in BFS when there are several starting points? What does each cell's distance then mean?
8. Describe DFS as a picture. Name two questions DFS answers and one it does not.
9. What does the `&` in `std::vector<std::vector<bool>>& visited` do, and what breaks without it?
10. When can recursive DFS crash, and what do you replace it with?
11. What is the one-line difference between DFS and backtracking? Why is backtracking slower?
12. For "number of ways to reach each cell moving only right or down", what is the transition and what are the base cases?

13. Which problem in this chapter is the wavefront planner? Which is the distance transform?
14. In A*, what are g, h and f? What happens when h is zero? What happens when h guesses too high?
15. Why does `std::priority_queue` need a "backwards" comparator to pop the smallest f, and why do we push duplicates instead of updating entries?

04. Graphs and Dijkstra

Chapter 03 was about grids: a map made of square cells. This chapter is about graphs: a map made of *places* joined by *roads*, where the roads can have any length. A robot's road network, a warehouse of shelves and aisles, the list of tasks that must be done before other tasks, the frames of a robot arm: all of these are graphs. The star of the chapter is Dijkstra's algorithm, which finds the shortest route on a map with real road lengths. It sits underneath almost every path planner in robotics, so by the end you should be able to type it from a blank file without looking anything up.

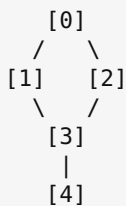
Rough time: 5 days at about 2 hours a day. Day 1 and most of day 2 are Part A. Do the Dijkstra hand trace in A5 on paper before you type anything.

What you need before this chapter. Chapter 01 for `std::vector`, `std::pair`, `std::queue` and `auto [a, b] = ...`. Chapter 03 for BFS (the ripple) and DFS (the chalk), which this chapter reuses on graphs. If "BFS uses a queue and marks on push" does not sound familiar, reread chapter 03 A2 and A3 first.

Part A — Graphs, four tools, and Dijkstra

A1. What a graph is, and how to store it

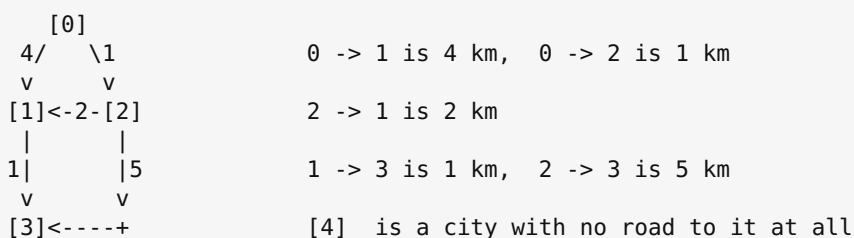
The picture. Five cities, joined by roads:



Each box is a *node* (a city). Each line is an *edge* (a road). That is all a graph is: a set of things, and a set of connections between pairs of them. Numbers on the nodes are just names, 0 to 4, so we can use them as indexes into arrays.

Three words you need:

- **Undirected vs directed.** The roads above are two-way: you can drive 0 to 1 and 1 to 0. That is an *undirected* graph. If some roads are one-way streets, we draw arrows and call the graph *directed*. "Take course B before course A" is a one-way arrow, so that will be a directed graph.
- **Weighted.** If every road has a length (or a cost, or a travel time), the graph is *weighted*. We write the number on the road:



- **Neighbours.** The neighbours of a node are the nodes one road away. In the first picture, the neighbours of 3 are 1, 2 and 4.

How to store it: every city keeps a list of its neighbours. Write a table with one row per node:

node	neighbours
0	1, 2
1	0, 3
2	0, 3
3	1, 2, 4
4	3

This table is called an *adjacency list*. Notice road 0-1 appears twice: in row 0 (as 1) and in row 1 (as 0). That is what "two-way" means in storage. For a directed graph each arrow appears once, in the row of the node it starts from.

In Python you would write it as a list of lists:

```
n = 5
edges = [(0, 1), (0, 2), (1, 3), (2, 3), (3, 4)]
adj = [[] for _ in range(n)]      # n empty lists
for u, v in edges:
    adj[u].append(v)
    adj[v].append(u)              # delete this line for one-way roads
# adj == [[1, 2], [0, 3], [0, 3], [1, 2, 4], [3]]
```

In C++, the table is a vector of vectors. Problems usually hand you the edges as `std::vector<std::vector<int>>`, where each inner vector is {from, to}:

```
using Graph = std::vector<std::vector<int>>;           // unweighted

Graph buildA1(int n, const std::vector<std::vector<int>>& edges) {
    Graph adj(n);                                     // n empty neighbour lists
    for (const auto& e : edges) {                     // e is one edge, e[0] and e[1] are its two ends
        adj[e[0]].push_back(e[1]);                   // e[1] is a neighbour of e[0]
        adj[e[1]].push_back(e[0]);                   // and e[0] is a neighbour of e[1]; delete for one-way roads
    }
    return adj;
}
```

Line by line:

- `using Graph = std::vector<std::vector<int>>;` gives the long type a short name. From now on `Graph` means "a vector of int-vectors". It is like a Python type alias.
- `Graph adj(n);` makes `n` empty inner vectors. This is `[[] for _ in range(n)]`. If you forget the `(n)` you get zero rows and `adj[0]` crashes.
- `for (const auto& e : edges)` loops over each edge without copying it. `e` is one {`u`, `v`} vector, so `e[0]` is `u` and `e[1]` is `v`. (It is `e[0]`, not `e.first`; that is a pair thing and this is a vector.)
- `adj[e[0]].push_back(e[1]);` is `adj[u].append(v)`.

Weighted roads. Each neighbour now comes with a length, so each row stores *pairs* (neighbour, length):

node	(neighbour, length)
0	(1, 4) (2, 1)
1	(3, 1)
2	(1, 2) (3, 5)
3	
4	

```
using WGraph = std::vector<std::vector<std::pair<int, int>>>; // weighted: (neighbour, length)

WGraph buildW(int n, const std::vector<std::vector<int>>& roads) {
    WGraph adj(n);
    for (const auto& r : roads) {                     // r[0] from, r[1] to, r[2] length
        adj[r[0]].push_back({r[1], r[2]});
    }
}
```

```

    }
    return adj;
}

```

- `std::pair<int, int>` is a two-slot box. `{r[1], r[2]}` builds one on the spot, the way a Python tuple `(v, w)` would.
- This version is directed (only one `push_back`). Add `adj[r[1]].push_back({r[0], r[2]});` for two-way roads.

To read the pairs back out, use the C++17 "unpack" syntax you met in chapter 03:

```

for (const auto& [v, w] : adj[u]) {    // v is the neighbour, w is the road length
    // ...
}

```

That is Python's `for v, w in adj[u]:`.

Compare with the grid from chapter 03. A grid is also a graph: every cell is a node and every cell is joined to its 4 neighbours. The difference is that on a grid we never *stored* the neighbours. We computed them with the `dr/dc` arrays: "my neighbour is my row plus `dr[k]`, my column plus `dc[k]`". A general graph has no such rule (city 3 is joined to 1, 2 and 4 for no geometric reason), so the neighbours have to be written down. That is the whole difference. Every algorithm in this chapter is the chapter 03 algorithm with `for (int v : adj[u])` in place of the `dr/dc` loop.

(You can skip this on a first read.) Two other storage options exist. If node names are not `0..n-1` (strings, or pointers as in Clone Graph), replace the outer vector with `std::unordered_map<Key, std::vector<Key>>`, which is Python's `defaultdict(list)`. If the graph is tiny and you often ask "is there a road from `u` to `v`?", an `n`-by-`n` table of 0/1 (an *adjacency matrix*) answers in one lookup, at the cost of `n`-squared memory. The adjacency list is the default for everything in this chapter.

Try it. In `00_graph_toolkit.cpp`, the first graph in `main` is built with `buildUndirected`. Change it to `buildDirected` and predict which of the asserts on the next three lines will fail before you run it.

In your own words: "A graph is nodes joined by edges. I store it as an adjacency list: a vector with one inner vector per node holding that node's neighbours, or pairs of (neighbour, length) when roads have lengths. For a two-way road I push both directions."

A2. BFS and DFS on a graph

Chapter 03 A2 taught BFS as a ripple spreading from a stone dropped in a pond, and A3 taught DFS as walking a maze with chalk. Both work on a graph without any new idea. The only change is where the neighbours come from.

BFS: the ripple, on the 5-node graph. Drop the stone at node 0. Keep a `dist` array, `-1` meaning "not reached yet", and a queue.

```

Start:   dist = [0, -1, -1, -1, -1]    queue = [0]

Pop 0.   Neighbours 1 and 2, both unreached: dist 1.
         dist = [0, 1, 1, -1, -1]      queue = [1, 2]

Pop 1.   Neighbours 0 (reached, skip) and 3 (unreached): dist 2.
         dist = [0, 1, 1, 2, -1]       queue = [2, 3]

Pop 2.   Neighbours 0 and 3, both already reached. Nothing to do.
         dist = [0, 1, 1, 2, -1]       queue = [3]

Pop 3.   Neighbours 1, 2 (reached) and 4 (unreached): dist 3.
         dist = [0, 1, 1, 2, 3]        queue = [4]

```

Pop 4. Neighbour 3, reached. Queue is empty. Done.

`dist[v]` is now the fewest roads from 0 to `v`. Every node went into the queue exactly once, and they came out in order of distance: the 1s before the 2, the 2 before the 3. That is the same reason BFS worked on the grid.

Notice the step "Pop 2: neighbour 3 already reached". On a grid this happened too, but on a graph it is more important: graphs have loops (0-1-3-2-0 is a loop), so without the `dist[v] != -1` check you would go round the loop forever. On a graph the visited check is never optional.

```
from collections import deque

def bfs(adj, src):
    dist = [-1] * len(adj)
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

```
std::vector<int> bfsDistances(const Graph& adj, int src) {
    const int n = static_cast<int>(adj.size());
    std::vector<int> dist(n, -1); // -1 means "not reached yet"
    std::queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) { // every neighbour of u
            if (dist[v] != -1) continue; // already reached
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

Compare it with the chapter 03 BFS: the queue holds a single `int` instead of a `(row, col)` pair, and for `(int v : adj[u])` replaces the four-direction loop and the bounds check. There is no `inBounds` because the adjacency list only ever contains real nodes.

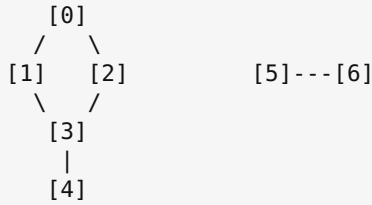
DFS: the chalk, on a graph. Visit a node, mark it, and walk into each unmarked neighbour. The call stack backs you up when you get stuck.

```
def dfs(adj, u, visited):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs(adj, v, visited)
```

```
void dfs(const Graph& adj, int u, std::vector<bool>& visited) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v]) dfs(adj, v, visited);
}
```

`visited` is passed by reference (&) so every recursive call marks the same array. Same rule as chapter 03.

Counting connected components. Suppose the map has two more cities, 5 and 6, joined to each other but to nothing else:



How many separate groups of cities are there? Two. To count them: walk through every node in order; whenever you meet one that is not yet marked, that is a new group, so add one to the count and run DFS from it to mark the whole group. The DFS from 0 marks 0, 1, 2, 3, 4. Nodes 1 to 4 are then skipped. Node 5 is unmarked: second group, DFS marks 5 and 6. Node 6 is skipped. Answer 2.

```

def count_components(adj):
    visited = [False] * len(adj)
    count = 0
    for s in range(len(adj)):
        if not visited[s]:
            count += 1
            dfs(adj, s, visited)
    return count
  
```

```

int countComponents(const Graph& adj) {
    const int n = static_cast<int>(adj.size());
    std::vector<bool> visited(n, false);
    int count = 0;
    for (int s = 0; s < n; ++s) {
        if (visited[s]) continue;           // already part of a group we counted
        ++count;                           // a new group starts here
        dfs(adj, s, visited);              // mark the whole group
    }
    return count;
}
  
```

This is Number of Islands from chapter 03 with cities instead of cells.

(You can skip this on a first read.) Recursive DFS is fine for graphs up to a few tens of thousands of nodes deep. For a very long chain of nodes it can overflow the call stack, and the fix is the same as in chapter 03: an explicit `std::vector<int>` used as a stack, pop the back, skip if already visited, mark, push the unvisited neighbours. `00_graph_toolkit.cpp` has this version as `dfsOrder`. The `if (visited[u]) continue;` after the pop is needed there because the same node can be pushed twice before it is popped once.

Try it. Add the edge `{2, 4}` to the undirected graph in `00_graph_toolkit.cpp` and predict the new `bfsDistances` result before running.

In your own words: "BFS and DFS on a graph are the chapter 03 versions with `for v in adj[u]` instead of the four directions. The visited check is essential because graphs have loops. Counting components is: for every unvisited node, count one and DFS from it."

A3. Topological sort

The picture. Getting dressed. Socks go on before shoes. Underwear before trousers, trousers before shoes. Shirt before jacket. Draw each rule as an arrow "must happen before":

```

socks(0) -----> shoes(3)
                ^
underwear(1) --> trousers(2) ---+
  
```

```
shirt(4) --> jacket(5)
```

A *topological order* is any list of the items such that every arrow points forward in the list. socks, underwear, shirt, trousers, jacket, shoes works. So does underwear, trousers, socks, shoes, shirt, jacket. There are usually many valid orders; we only need one.

This only makes sense if the arrows have no loop. "Shoes before socks" together with "socks before shoes" is impossible to obey. A directed graph with no loop is called a *DAG* (directed acyclic graph). Topological sort either gives you an order or tells you there is a loop.

Kahn's algorithm. The rule a person would use: "put on anything that has nothing left to wait for; when it is on, the things that were waiting for it have one fewer thing to wait for."

The number of arrows pointing *into* a node is its *in-degree*: how many things it is still waiting for. Write the in-degree beside each node:

```
socks(0) [0] -----> shoes(3) [2]
                        ^
underwear(1) [0] --> trousers(2) [1]
shirt(4) [0] --> jacket(5) [1]
```

Everything with in-degree 0 goes into a queue: queue = [0, 1, 4], order so far [].

Pop 0 (socks).	Its arrow goes to 3. indeg[3] becomes 1. Not zero yet.		
	in-degrees: 0:0 1:0 2:1 3:1 4:0 5:1	order = [0]	queue = [1, 4]
Pop 1 (underwear).	Arrow to 2. indeg[2] becomes 0. Push 2.		
	in-degrees: 0:0 1:0 2:0 3:1 4:0 5:1	order = [0, 1]	queue = [4, 2]
Pop 4 (shirt).	Arrow to 5. indeg[5] becomes 0. Push 5.		
	in-degrees: 0:0 1:0 2:0 3:1 4:0 5:0	order = [0, 1, 4]	queue = [2, 5]
Pop 2 (trousers).	Arrow to 3. indeg[3] becomes 0. Push 3.		
	in-degrees: all 0	order = [0, 1, 4, 2]	queue = [5, 3]
Pop 5 (jacket).	No arrows out.	order = [0, 1, 4, 2, 5]	queue = [3]
Pop 3 (shoes).	No arrows out. Queue empty.	order = [0, 1, 4, 2, 5, 3]	

Six items, six in the order: it worked. Read the order back: socks, underwear, shirt, trousers, jacket, shoes. Every arrow points forward.

How a loop shows up. Add a bad rule: 0 -> 1, 1 -> 2, 2 -> 0 (a triangle), plus a lone node 3.

```
[0] --> [1]
  ^      |
  |      v
[2] <----+      [3]

in-degrees: 0:1 1:1 2:1 3:0
```

Only node 3 has in-degree 0. Pop it; it has no arrows out. Queue is empty. order = [3], which has 1 item, but there are 4 nodes. The three on the triangle each wait for another one on the triangle, so none of them ever reaches in-degree 0. The test is simply: **if the order is shorter than n, there is a cycle.**

```
from collections import deque

def kahn(adj):
    n = len(adj)
    indeg = [0] * n
    for u in range(n):
```

```

    for v in adj[u]:
        indeg[v] += 1
q = deque(u for u in range(n) if indeg[u] == 0)
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
return order # shorter than n means there is a cycle

```

```

std::vector<int> kahnTopo(const Graph& adj) {
    const int n = static_cast<int>(adj.size());
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v]; // one more arrow points at v
    std::queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u); // nothing has to come before u
    std::vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v); // u is done, so v has one fewer thing to wait for
    }
    return order; // shorter than n means there is a cycle
}

```

What is new:

- `++indeg[v]` adds one to the in-degree of `v`. This is `indeg[v] += 1`.
- `if (--indeg[v] == 0)` does two things in one line: subtract one *first*, then compare with zero. `--x` (minus signs in front) means "decrease, then give me the new value". If you wrote `indeg[v]-- == 0` (signs after), it would compare the *old* value, which is off by one and wrong. Write the signs in front.
- The shape is the BFS loop from A2. Kahn's algorithm is BFS where "reached" means "all my prerequisites are done".

(You can skip this on a first read.) There is a second way to do topological sort, with DFS: when a node's DFS call finishes, append it to a list; reverse the list at the end. Detecting a cycle there needs three colours (unseen, in progress, finished) and a node found "in progress" is on a loop. `00_graph_toolkit.cpp` has this as `hasDirectedCycle`. Learn Kahn's first; it is the one Course Schedule wants.

Try it. In the dag graph in `00_graph_toolkit.cpp`, add the edge `{3, 0}` and predict the size of `kahnTopo(dag)` before running.

In your own words: "Topological sort orders the nodes of a directed graph so every arrow points forward. Kahn's algorithm counts in-degrees, queues the nodes with in-degree 0, and each time it pops one it lowers its neighbours' in-degrees and queues those that hit 0. If fewer than `n` nodes come out, there is a cycle."

A4. Union-find

The picture: friend groups. Six people, numbered 0 to 5. At the start nobody knows anyone, so there are six groups of one. We learn facts one at a time: "0 and 1 are friends", "2 and 3 are friends", "1 and 3 are friends". After those three facts, 0, 1, 2 and 3 are all one group (friends of friends count), and 4 and 5 are still alone. The questions we want to answer fast are: "are `x` and `y` in the same group?" and "merge the groups of `x` and `y`".

How it is stored. Each person points to one other person in their group (or to themselves). Follow the arrows and you always arrive at one special person, the *representative* of the group. Two people are in the same group exactly when the arrows lead them to the same representative. The arrows live in one array, `parent`, where `parent[x]` is who `x` points to and `parent[x] == x` means "`x` is a representative".

Start: everyone points to themselves.

```
parent = [0, 1, 2, 3, 4, 5]      (0) (1) (2) (3) (4) (5)      six groups
```

Fact "0 and 1 are friends": `union(0, 1)`. Find the representatives (0 and 1, themselves), and make one point at the other.

```
parent = [0, 0, 2, 3, 4, 5]      (0)      (2) (3) (4) (5)
                                ^
                                (1)
```

Fact "2 and 3": `union(2, 3)`.

```
parent = [0, 0, 2, 2, 4, 5]      (0)      (2)      (4) (5)
                                ^      ^
                                (1)      (3)
```

Fact "1 and 3": `union(1, 3)`. `find(1)` follows `1 -> 0`, representative 0. `find(3)` follows `3 -> 2`, representative 2. They differ, so point 2 at 0.

```
parent = [0, 0, 0, 2, 4, 5]      (0)      (4) (5)
                                ^      ^
                                (1)      (2)
                                    ^
                                    (3)
```

Now `find(3)` walks `3 -> 2 -> 0` and `find(1)` walks `1 -> 0`: same representative, same group. A fourth fact "0 and 3 are friends" would find the same representative for both and do nothing; we say the union *fails*, and that failure is useful: it means the two were already connected, which in a road map means the new road closes a loop. That is the whole trick behind Redundant Connection and Graph Valid Tree.

Path compression. Long chains of arrows make `find` slow. So while walking up, we re-point each node we pass closer to the representative. The version in the code is called *path halving*: at each step, point the current node at its grandparent. Before and after `find(5)` on a chain:

```
before:   (5) -> (4) -> (3) -> (2) -> (0)      parent[5]=4, parent[4]=3, parent[3]=2, parent[2]=0
after:    (5) -----> (3) -----> (0)      parent[5]=3  (5 now skips 4)
          (4) -----> (3)                  parent[3]=0  (3 now skips 2)
          (2) -----> (0)
```

The chain of length 4 became chains of length 2, and the next `find(5)` will flatten it further. Together with the rule "attach the *smaller* group under the larger one" (union by size), the arrows never get long. The cost of one `find` or `union` is then *almost constant*: formally $\alpha(n)$, the inverse Ackermann function, which is at most 4 for any `n` you will ever meet. Say "nearly constant time, formally $\alpha(n)$ " and move on.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]  # point at grandparent
            x = self.parent[x]
        return x
    def union(self, a, b):
```

```

a, b = self.find(a), self.find(b)
if a == b:
    return False # already in the same group
if self.size[a] < self.size[b]:
    a, b = b, a
self.parent[b] = a
self.size[a] += self.size[b]
return True

```

```

#include <numeric> // std::iota

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]]; // path halving: point at my grandparent
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) { // false if a and b were already in one group
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b); // attach the smaller group under the larger
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

```

What is new:

- `struct DSU { ... };` is a class: some data (`parent`, `sz`) and some functions that work on it. It is Python's `class DSU:` with `self`. written for you.
- `explicit DSU(int n) : parent(n), sz(n, 1) { ... }` is the constructor (`__init__`). The part after the colon sets `parent` to `n` zeros and `sz` to `n` ones before the body runs. `explicit` just stops C++ from silently turning an `int` into a `DSU`; always write it on one-argument constructors.
- `std::iota(parent.begin(), parent.end(), 0)` fills the vector with `0, 1, 2, ...`. It is `list(range(n))`. It lives in `<numeric>`.
- `unite`, not `union`: `union` is a reserved word in C++.
- `std::swap(a, b)` is `a, b = b, a`.
- `DSU dsu(n)` on the caller's side builds one for `n` people. If the nodes are numbered `1..n`, build `DSU dsu(n + 1)` and ignore index `0`.

Number of groups at any time = `n` minus the number of unions that returned `true`.

Try it. In `00_graph_toolkit.cpp`, change `dsu.unite(1, 2)` to `dsu.unite(2, 3)` and predict which assert fails.

In your own words: "Union-find keeps a `parent` array where following the arrows leads to a group's representative. `find` walks up and flattens the path as it goes; `union` points one representative at the other, smaller group under larger. Each operation is nearly constant time, formally $\alpha(n)$. A union that fails means the two were already connected, which is how it detects a loop."

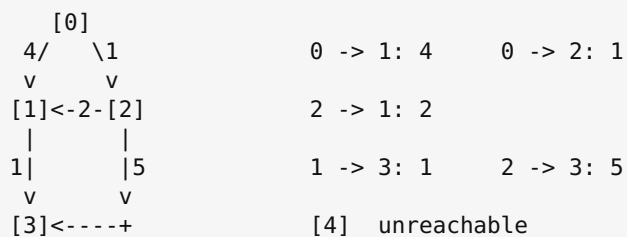
A5. Dijkstra

The story. BFS finds the route with the fewest roads. It assumes every road is the same length, 1 km. Real roads are not. On the weighted map from A1, the route 0 -> 1 uses one road but it is 4 km, while 0 -> 2 -> 1 uses two roads and is only 1 + 2 = 3 km. BFS would pick the 4 km route. Dijkstra fixes this with one change of habit: instead of expanding cities in the order they were *discovered*, always expand the city with the **smallest known distance so far**.

Think of it as a search party. Every city we have heard of has a "best distance so far" written on it. At each step we go to the closest city we have not yet visited, and from there we look down every road: if going through this city gives a neighbour a better distance than it had, we update the neighbour's number. Because we always take the closest unvisited city, the number on a city is *final* the moment we visit it: no other route could be shorter, since every other route would have to pass through a city that is already farther away. (This argument needs all roads to be zero or longer. Hold that thought until the negative-road example.)

The tool. A *priority queue*: a bag where you can throw things in, and taking something out always gives you the smallest. In chapter 03 the queue gave us "first in, first out". Here we need "smallest distance out first". C++ has `std::priority_queue`, Python has `heapq`.

The hand trace. The directed weighted graph from A1, starting at 0:



We keep `dist`, the best distance known so far (INF = "no route known yet"), and the priority queue `pq`, holding (distance, node) pairs. The table shows both after every step.

step	action	dist[0..4]	pq (smallest first)
0	start: dist[0] = 0, push (0,0)	0 INF INF INF INF	(0,0)
1	pop (0,0). Roads: 0->1 = 4, 0->2 = 1. dist[1] INF -> 4, push (4,1) dist[2] INF -> 1, push (1,2)	0 4 1 INF INF	(1,2) (4,1)
2	pop (1,2). Roads: 2->1 = 2, 2->3 = 5. 1+2 = 3 < 4: dist[1] -> 3, push (3,1) 1+5 = 6 < INF: dist[3] -> 6, push (6,3)	0 3 1 6 INF	(3,1) (4,1) (6,3)
3	pop (3,1). 3 == dist[1], so it is fresh. Road 1->3 = 1: 3+1 = 4 < 6, dist[3] -> 4, push (4,3)	0 3 1 4 INF	(4,1) (4,3) (6,3)
4	pop (4,1). 4 > dist[1] = 3: STALE, skip.	0 3 1 4 INF	(4,3) (6,3)
5	pop (4,3). Fresh. Node 3 has no roads out.	0 3 1 4 INF	(6,3)
6	pop (6,3). 6 > dist[3] = 4: STALE, skip.	0 3 1 4 INF	(empty)

Final: `dist` = [0, 3, 1, 4, INF]. Node 1 is 3 km away (via 2), node 3 is 4 km away (0 -> 2 -> 1 -> 3), node 4 was never reached.

Look at step 2. Node 1 already had the number 4 and an entry (4,1) in the queue. We found a better route, wrote 3, and pushed a *second* entry (3,1). We did not remove the old one; a priority queue cannot find and delete an entry in the middle. So the queue briefly holds both:

```

pq:  (3,1)  (4,1)  (6,3)
      ^      ^
      the new the old, stale entry for node 1
  
```

The new one comes out first (step 3) because $3 < 4$. When the old one comes out (step 4), we compare its distance 4 with `dist[1] = 3`, see that it is worse than what we already know, and skip it. That comparison is the line `if (d > dist[u]) continue;` and it is the only thing that stops us doing wasted work on old entries.

The Python you would have written.

```
import heapq

def dijkstra(adj, src):
    INF = float('inf')
    dist = [INF] * len(adj)
    parent = [-1] * len(adj)
    dist[src] = 0
    pq = [(0, src)]           # heapq keeps the smallest tuple at pq[0]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue         # stale entry
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                parent[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, parent
```

The C++ template. This is the one to memorise.

```
#include <limits>
#include <queue>
#include <utility>
#include <vector>

using WGraph = std::vector<std::vector<std::pair<int, int>>>>;

constexpr long long INF = std::numeric_limits<long long>::max() / 4;

std::vector<long long> dijkstra(const WGraph& adj, int src, std::vector<int>* parent = nullptr) {
    int n = static_cast<int>(adj.size());
    std::vector<long long> dist(n, INF);
    if (parent) parent->assign(n, -1);
    using State = std::pair<long long, int>;           // (distance, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq; // smallest on top
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;                    // stale entry: we already found a better path to u
        for (const auto& [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {                        // found a shorter way to v
                dist[v] = nd;
                if (parent) (*parent)[v] = u;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}
```

Line by line:

- `constexpr long long INF = std::numeric_limits<long long>::max() / 4;` `INF` is "no route known". `long long` is a 64-bit integer, big enough for any sum of distances. `max()` is the largest value it can hold. **Why divide by 4?** The code computes `d + w`. If `d` were the largest possible number and you added anything, it would wrap round to a huge negative number, and a negative "distance" would beat every real one.

`max / 4` is still enormous but leaves room to add to it safely. `constexpr` means "a constant, known at compile time".

- `std::vector<int>* parent = nullptr` is an optional extra argument: a *pointer* to a vector where the caller wants the parent of each node written, or `nullptr` (Python `None`) if they do not care. `if (parent)` is "if the caller gave me one". `parent->assign(n, -1)` fills it with `n` copies of `-1`, and `(*parent)[v] = u` writes into it. If pointers are still new, just read those three lines as "if there is a parent array, keep it up to date".
- `using State = std::pair<long long, int>;` a queue entry is a pair. **Why (distance, node) and not (node, distance)?** Pairs compare by their *first* slot, and only look at the second on a tie. The queue must sort by distance, so distance has to be first. Put the node first and the queue would hand you the lowest-numbered node, not the closest one, and every answer would be wrong.
- `std::priority_queue<State, std::vector<State>, std::greater<State>> pq;` **Why three things in the angle brackets, and why `std::greater`?** By default `std::priority_queue` gives you the *largest* element (a pile where the biggest is on top). Dijkstra needs the smallest. The third argument is the comparison to use; `std::greater<State>` flips the pile so the smallest is on top. The middle argument, `std::vector<State>`, is just the container it is built on and is always written the same way. You cannot skip to the third argument without writing the second. Writing `std::priority_queue<State> pq;` on autopilot is the single most common Dijkstra bug: it compiles and gives wrong answers.
- `auto [d, u] = pq.top(); pq.pop();` look at the smallest, then remove it. Two calls, as with `std::queue`. Python's `heappop` did both.
- `if (d > dist[u]) continue;` the stale-entry skip from the drawing above. Without it the code is still correct but can do a great deal of repeated work.
- `for (const auto& [v, w] : adj[u])` for each road out of `u`, `v` is the far end and `w` is its length.
- `long long nd = d + w;` the new candidate distance to `v` through `u`.
- `if (nd < dist[v]) { ... }` if that beats the best known, record it, remember we came from `u`, and push the new entry. This is called *relaxing* the road.

Rebuilding the path. `parent[v]` is the node we were standing on when we last improved `dist[v]`. In the trace, `parent = [-1, 2, 0, 1, -1]`: we reached 1 from 2, 2 from 0, 3 from 1. To get the route to 3, walk backwards until you hit `-1`, then reverse:

```
3 -> parent[3] = 1 -> parent[1] = 2 -> parent[2] = 0 -> parent[0] = -1, stop
collected: [3, 1, 2, 0]          reversed: [0, 2, 1, 3]
```

```
#include <algorithm>    // std::reverse

std::vector<int> reconstructPath(const std::vector<int>& parent, int target) {
    std::vector<int> path;
    for (int v = target; v != -1; v = parent[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

`for (int v = target; v != -1; v = parent[v])` is a for-loop that does not count; each round it moves `v` one step towards the start. `std::reverse` flips the vector in place, like `path[::-1]` but without making a copy.

Stopping early. If you only need the distance to one target, add `if (u == target) break;` right after the stale-entry check. The first time a node is *popped* fresh, its distance is final. (Not the first time it is pushed. Node 1 was pushed with 4 and only later got 3.)

Why it fails with a negative road. Three cities: 0 -> 1 costs 2, 0 -> 2 costs 5, and the road 2 -> 1 costs -4 (someone pays you to drive it).

```
      2
[0] ----> [1]
  |         ^
 5 |         | -4
```

```

v      |
[2] -----+

```

The true cheapest route to 1 is $0 \rightarrow 2 \rightarrow 1 = 5 + (-4) = 1$. Now run Dijkstra from 0 with the early stop for target 1. Step 1: pop 0, set $\text{dist}[1] = 2$, $\text{dist}[2] = 5$. Step 2: the smallest entry is (2, 1). We pop node 1, declare it final, and return 2. Wrong. The promise "once I pop you, nothing can improve you" was built on "every road makes the trip longer or leaves it the same", and the -4 road broke that promise. (The version without the early stop happens to repair this tiny case by pushing 1 again later, but it can be made to do an enormous amount of rework, and with a negative loop it never stops.) Rule: any road with a negative length means you use Bellman-Ford from A6, not Dijkstra.

Complexity. Every road is looked at once, and every push or pop on the queue costs about $\log$ of the queue size. Total: proportional to (nodes + roads) times $\log(\text{nodes})$. Written $O((V + E) \log V)$. Memory is the adjacency list plus the `dist` array: $O(V + E)$.

(You can skip this on a first read.) Some people mark a node "done" in a `visited` array when it is popped instead of comparing `d` with `dist[u]`. That is equally correct. What is *not* correct is marking it done when it is *pushed*: that turns Dijkstra back into BFS and is wrong the moment two roads have different lengths. Also, `int` distances are fine on LeetCode where the limits are small; the template uses `long long` so you never have to think about it.

Practice drill: type it in 15 minutes. Set a timer and, from a blank file with no reference, type: the includes, using `WGraph`, `INF`, the `dijkstra` function, `reconstructPath`, and a `main` with a 4-node graph where the direct road loses to a two-road route, plus one unreachable node, with asserts. Compile with `-Wall -Wextra`. Do this three days running. When it comes out clean under 15 minutes, you know it.

In your own words: "Dijkstra keeps a priority queue of (distance, node) and always expands the closest unvisited node. Each pop with a fresh distance is final because all roads are non-negative; stale entries are skipped by comparing with `dist`. Relaxing a road means updating a neighbour when the route through me is shorter. Time is $O((V + E) \log V)$. Negative roads break it."

A6. Variants in plain words

Dijkstra on a grid. A grid is a graph whose neighbours are computed, so Dijkstra on a grid is the A5 template with the chapter 03 pieces plugged in: the queue holds `(cost, row, col)`, `dist` is a 2-D vector, and the neighbours come from `dr/dc` with the bounds check. Three numbers do not fit in a pair, so use a `std::tuple<int, int, int>`; tuples compare slot by slot from the left, exactly like pairs, so `cost` still goes first. Unpack with `auto [e, r, c] = pq.top();`. You will see this in `Path with Minimum Effort` and `Swim in Rising Water`.

Minimise the worst road (minimax). Sometimes the cost of a route is not the *sum* of its roads but the *largest single road* on it: a truck that must fit under every bridge, a robot whose worst step must not tip it over. Picture:

```

      3      3
[0] ----- [1] ----- [3]
|           |           |
1 |         |           | 4
|           |           |
[2] -----+

```

From 0 to 3 there are two routes. The top route 0-1-3 has roads 3 and 3: total 6, worst road 3. The bottom route 0-2-3 has roads 1 and 4: total 5, worst road 4. By total length the bottom route wins ($5 < 6$). By worst road the top route wins ($3 < 4$). They are different questions with different answers. Dijkstra answers the worst-road question with a one-word change: replace `d + w` with `std::max(d, w)`. Everything else, including the stale skip and the "first pop is final" promise, still works, because "the worst road so far" can only stay the same or get worse as you extend a route, just as a sum can only grow. On the picture, minimax Dijkstra gives `[0, 3, 1, 3]` and plain Dijkstra gives `[0, 3, 1, 5]`.

At most k stops. "Cheapest flight from A to B using at most k stops." Now Dijkstra's greedy choice breaks. Here is why, with numbers:

```

[0] --5--> [1] --1--> [3]
 |         ^
 |         |
 1         1
  v         |
[2] --1--> [4]

```

Dijkstra finds the cheapest way to node 1: 0 -> 2 -> 4 -> 1, cost 3, and declares node 1 done at cost 3. From there 3 costs 4. But that route to 3 has three stops (2, 4, 1). If only one stop is allowed, the only legal route is 0 -> 1 -> 3, cost 6, and Dijkstra never considers it: it settled node 1 at the cheap price and threw away the expensive-but-short way of getting there. The problem is that "cheapest so far" is no longer the only thing that matters; how many roads you used matters too.

The clean fix is **Bellman-Ford**, which you can say in one sentence: *relax every road, k+1 times*. Round 1 finds the best cost using at most 1 road, round 2 using at most 2 roads, and so on. "At most k stops" means at most k+1 roads, so k+1 rounds.

```

def cheapest(n, roads, src, k):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):
        nxt = dist[:]
        for u, v, w in roads:
            if dist[u] != INF and dist[u] + w < nxt[v]:
                nxt[v] = dist[u] + w
        dist = nxt
    return dist

```

```

struct Edge { int u, v, w; };

std::vector<long long> bellmanFordK(int n, const std::vector<Edge>& edges, int src, int k) {
    std::vector<long long> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round < k + 1; ++round) {
        std::vector<long long> next = dist;
        for (const auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < next[e.v]) next[e.v] = dist[e.u] + e.w;
        dist = next;
    }
    return dist;
}

```

- `struct Edge { int u, v, w; };` a three-field record. `{0, 1, 5}` builds one.
- `std::vector<long long> next = dist;` copies the whole array. **The copy is the point.** If you updated `dist` in place, one round could chain several roads together (update `dist[2]`, then immediately use it to update `dist[4]`, and so on) and the stop limit would silently vanish. Reading from last round's `dist` and writing into `next` keeps each round to exactly one more road.
- `dist[e.u] != INF &&` do not add to INF; the sum would be nonsense (and with `int` it would overflow).

On the picture with `k = 1` (two rounds): round 1 gives `dist = [0, 5, 1, INF, INF]`; round 2 gives `dist = [0, 5, 1, 6, 2]`. Node 3 costs 6. With `k = 3`, four rounds, node 3 gets down to 4.

Full Bellman-Ford is the same loop run `n - 1` times with no copy needed. It is slower than Dijkstra, $O(V \text{ times } E)$, but it works with negative roads, which is the other time you reach for it.

A*: Dijkstra plus a guess. Dijkstra spreads out in every direction, like the BFS ripple. If you know where the goal is, add a *guess* of the remaining distance (straight-line distance, or Manhattan distance on a grid) to each queue entry's priority. The search then leans towards the goal. As long as the guess never over-

estimates, the first time the goal is popped its distance is exact. With the guess set to zero, A* is Dijkstra. Chapter 03 Part C builds A* on a grid from scratch; reread it after this chapter and you will see it is this template with `+ h(v)` added to the priority only.

(You can skip this on a first read.) Two more variants exist that are worth recognising by name. If every road is 0 or 1, a plain `std::deque` does Dijkstra's job in $O(V + E)$: push to the front on a 0-road, to the back on a 1-road (`zeroOneBfs` in the toolkit). And "shortest path between every pair of nodes" on a small dense graph is Floyd-Warshall, three nested loops over an n -by- n table, $O(n^3)$.

Which one do I use?

The roads are...	Use
all the same length	BFS (chapter 03)
different non-negative lengths	Dijkstra
the cost is the worst road, not the sum	Dijkstra with <code>max</code>
at most k roads allowed, or some lengths negative	Bellman-Ford
non-negative, and I know where the goal is	A*

In your own words: "On a grid, Dijkstra uses a tuple state and the dr/dc neighbours. For 'minimise the worst road' I replace the sum with `max`. A stop limit breaks Dijkstra's greedy settling, so I use Bellman-Ford: relax every road $k+1$ times, reading from a copy of last round. A* is Dijkstra with a never-over-estimating guess added to the priority."

A7. The toolkit file

```
// Graph toolkit demo: adjacency lists, BFS, DFS (iterative), Kahn's topological sort,
// union-find, and the Dijkstra template with path reconstruction.
#include <algorithm>
#include <cassert>
#include <deque>
#include <functional>
#include <iostream>
#include <limits>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

using Graph = std::vector<std::vector<int>>>; // unweighted adjacency list
using WGraph = std::vector<std::vector<std::pair<int, int>>>>; // weighted: (neighbor, weight)

// ----- building from edge lists -----
Graph buildUndirected(int n, const std::vector<std::pair<int, int>>& edges) {
    Graph adj(n);
    for (const auto& [u, v] : edges) {
        adj[u].push_back(v);
        adj[v].push_back(u); // the only difference from a directed graph
    }
    return adj;
}

Graph buildDirected(int n, const std::vector<std::pair<int, int>>& edges) {
    Graph adj(n);
    for (const auto& [u, v] : edges) adj[u].push_back(v);
    return adj;
}

// ----- BFS: shortest hop count from src on an unweighted graph -----
std::vector<int> bfsDistances(const Graph& adj, int src) {
    std::vector<int> dist(adj.size(), -1); // -1 == unreachable, doubles as visited
    std::queue<int> q;
    dist[src] = 0;
```

```

q.push(src);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : adj[u]) {
        if (dist[v] != -1) continue;
        dist[v] = dist[u] + 1;
        q.push(v);
    }
}
return dist;
}

// ----- iterative DFS: preorder visit list -----
std::vector<int> dfsOrder(const Graph& adj, int src) {
    std::vector<bool> visited(adj.size(), false);
    std::vector<int> order;
    std::vector<int> st = {src};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        if (visited[u]) continue;
        visited[u] = true;
        order.push_back(u);
        for (auto it = adj[u].rbegin(); it != adj[u].rend(); ++it) // reverse so smallest is popped first
            if (!visited[*it]) st.push_back(*it);
    }
    return order;
}

// ----- connected components on an undirected graph -----
int countComponents(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<bool> visited(n, false);
    int count = 0;
    for (int s = 0; s < n; ++s) {
        if (visited[s]) continue;
        ++count;
        std::vector<int> st = {s};
        visited[s] = true;
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : adj[u])
                if (!visited[v]) { visited[v] = true; st.push_back(v); }
        }
    }
    return count;
}

// ----- directed cycle detection with white/gray/black colouring -----
bool hasDirectedCycle(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> color(n, 0); // 0 white, 1 gray (on stack), 2 black (done)
    std::function<bool(int)> dfs = [&](int u) {
        color[u] = 1;
        for (int v : adj[u]) {
            if (color[v] == 1) return true; // back edge to an ancestor: cycle
            if (color[v] == 0 && dfs(v)) return true;
        }
        color[u] = 2;
        return false;
    };
    for (int s = 0; s < n; ++s)
        if (color[s] == 0 && dfs(s)) return true;
    return false;
}

// ----- Kahn's topological sort; result shorter than n means a cycle -----
std::vector<int> kahnTopo(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v];
}

```

```

std::queue<int> q;
for (int u = 0; u < n; ++u)
    if (indeg[u] == 0) q.push(u);
std::vector<int> order;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : adj[u])
        if (--indeg[v] == 0) q.push(v);    // "removing" u frees v
}
return order;
}

// ----- union-find with path compression and union by size -----
struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];    // path halving: every step skips a level
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {                // false if a and b were already connected
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b);    // attach the smaller tree under the larger
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

// ----- the Dijkstra template (memorise this one) -----
constexpr long long INF = std::numeric_limits<long long>::max() / 4;    // /4 so INF + w never overflows

std::vector<long long> dijkstra(const WGraph& adj, int src, std::vector<int>* parent = nullptr) {
    int n = static_cast<int>(adj.size());
    std::vector<long long> dist(n, INF);
    if (parent) parent->assign(n, -1);
    using State = std::pair<long long, int>;    // (dist, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;    // min-heap!
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;    // stale entry: we already found a better path to u
        for (const auto& [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {        // relax
                dist[v] = nd;
                if (parent) (*parent)[v] = u;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}

std::vector<int> reconstructPath(const std::vector<int>& parent, int target) {
    std::vector<int> path;
    for (int v = target; v != -1; v = parent[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}

// ----- 0-1 BFS: weights are only 0 or 1, deque instead of heap -----
std::vector<int> zeroOneBfs(const WGraph& adj, int src) {
    std::vector<int> dist(adj.size(), std::numeric_limits<int>::max());
    std::deque<int> dq;

```

```

dist[src] = 0;
dq.push_back(src);
while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();
    for (const auto& [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v); else dq.push_back(v); // 0-edges keep the front
        }
    }
}
return dist;
}

// ----- Bellman-Ford: V-1 rounds of relaxing every edge; handles negative weights -----
struct Edge { int u, v, w; };
std::vector<long long> bellmanFord(int n, const std::vector<Edge>& edges, int src) {
    std::vector<long long> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round < n - 1; ++round)
        for (const auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) dist[e.v] = dist[e.u] + e.w;
    return dist;
}

int main() {
    // Undirected graph: 0-1, 0-2, 1-3, 2-3, 4-5 (two components)
    Graph g = buildUndirected(6, {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {4, 5}});
    assert(bfsDistances(g, 0) == std::vector<int>({0, 1, 1, 2, -1, -1}));
    assert(dfsOrder(g, 0) == std::vector<int>({0, 1, 3, 2}));
    assert(countComponents(g) == 2);

    // Directed graphs: a DAG and one with a cycle
    Graph dag = buildDirected(4, {{0, 1}, {0, 2}, {1, 3}, {2, 3}});
    assert(!hasDirectedCycle(dag));
    std::vector<int> topo = kahnTopo(dag);
    assert(topo.size() == 4 && topo[0] == 0 && topo[3] == 3);
    Graph cyc = buildDirected(3, {{0, 1}, {1, 2}, {2, 0}});
    assert(hasDirectedCycle(cyc));
    assert(kahnTopo(cyc).empty()); // nothing has in-degree 0 -> nothing emitted

    // Union-find: adding edges, the third closes a cycle
    DSU dsu(4);
    assert(dsu.unite(0, 1));
    assert(dsu.unite(1, 2));
    assert(!dsu.unite(0, 2)); // already connected
    assert(dsu.find(0) == dsu.find(2) && dsu.find(3) != dsu.find(0));

    // Dijkstra with path reconstruction. Directed weighted graph:
    // 0->1 (4), 0->2 (1), 2->1 (2), 1->3 (1), 2->3 (5)
    WGraph wg(5);
    auto addEdge = [&](int u, int v, int w) { wg[u].push_back({v, w}); };
    addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(2, 1, 2); addEdge(1, 3, 1); addEdge(2, 3, 5);
    std::vector<int> parent;
    std::vector<long long> dist = dijkstra(wg, 0, &parent);
    assert((dist == std::vector<long long>({0, 3, 1, 4, INF}))); // node 4 unreachable
    assert(reconstructPath(parent, 3) == std::vector<int>({0, 2, 1, 3}));

    // Bellman-Ford agrees on the same graph
    std::vector<Edge> edges = {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 5}};
    assert(bellmanFord(5, edges, 0) == dist);

    // 0-1 BFS: 0->1 (1), 0->2 (0), 2->1 (0)
    WGraph zg(3);
    zg[0].push_back({1, 1}); zg[0].push_back({2, 0}); zg[2].push_back({1, 0});
    assert(zeroOneBfs(zg, 0) == std::vector<int>({0, 0, 0}));

    // Edge case: single node graph
    WGraph one(1);
    assert(dijkstra(one, 0) == std::vector<long long>({0}));

    std::cout << "OK 00_graph_toolkit.cpp\n";
}

```

```
    return 0;
}
```

Compile and run it:

```
cd code
g++ -std=c++17 -Wall -Wextra -O2 04-graphs-dijkstra/00_graph_toolkit.cpp -o toolkit && ./toolkit
```

What to look at:

- `buildUndirected` and `buildDirected` are A1. The only difference is one line.
- `bfsDistances` is A2's BFS. `dfsOrder` is the explicit-stack DFS mentioned at the end of A2; note the `if (visited[u]) continue;` after the `pop`. `countComponents` is A2's counting loop.
- `kahnTopo` is A3. `hasDirectedCycle` is the DFS alternative with three colours.
- `DSU` is A4, character for character.
- `dijkstra` and `reconstructPath` are A5. `main` runs them on the exact graph from the A5 hand trace and asserts `{0, 3, 1, 4, INF}` and the path `{0, 2, 1, 3}`. Check those against your table.
- `zeroOneBfs` and `bellmanFord` are the A6 extras. `bellmanFord` is asserted to agree with `dijkstra` on the same graph.

You do not need to memorise the file. You need to be able to type A5's `dijkstra` from a blank screen, and to write A2, A3 and A4 with the file closed.

Part B — The problems

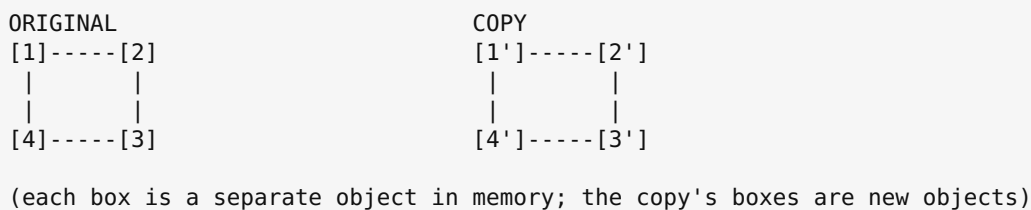
Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution line by line, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Clone Graph (LeetCode 133, medium)

The problem in plain words. You are handed a pointer to one node of an undirected graph. Each node is an object with a number `val` and a list `neighbors` of pointers to other nodes. The graph is connected, so from that one node you can reach every other node. Return a *deep copy* of the graph: a brand new set of node objects, joined to each other in exactly the same way, sharing nothing with the originals. Return the pointer to the copy of the node you were given.

What "deep copy" means, in a picture. The original is the square on the left. A deep copy is a second, separate square on the right. No arrow crosses from one side to the other:



A *shallow* copy would be a new node `1'` whose neighbour list still pointed at the *original* `2` and `4`. That is wrong: change the copy and the original changes too.

The example is given as an adjacency list, 1-indexed: `[[2,4],[1,3],[2,4],[1,3]]`, meaning node 1 is joined to 2 and 4, node 2 to 1 and 3, and so on. The answer is a graph with the same list. Edge cases: an empty graph (null pointer in, null pointer out); a single node with no neighbours; a node that lists itself as a neighbour (a self-loop, which must point at the copy of itself). Up to 100 nodes.

By hand. Start at node 1. Make a fresh box `1'` and write in a notebook "1 becomes 1'". Look at 1's neighbours. Node 2 is not in the notebook: make `2'`, write "2 becomes 2'", and remember to visit 2 later. Join `1'` to `2'`. Same for 4. Now visit 2: its neighbours are 1 (in the notebook: join `2'` to the existing `1'`, do not make another) and 3 (new). Keep going until there is nobody left to visit. The notebook is what stops you making a second copy of a node you have already copied, and it is what turns the loop 1-2-3-4-1 into a loop in the copy.

The idea. DFS over the original nodes with a dictionary `copies` from original pointer to copy pointer. The dictionary is the visited set *and* the lookup table for wiring. For each neighbour `v` of the node `u` being visited: if `v` has no copy yet, make one and put `v` on the stack; either way, append `copies[v]` to `copies[u].neighbors`.

The Python you would have written.

```
def cloneGraph(node):
    if node is None:
        return None
    copies = {node: Node(node.val)}          # original -> copy; also the visited set
    stack = [node]
    while stack:
        u = stack.pop()
        for v in u.neighbors:
            if v not in copies:
                copies[v] = Node(v.val)
                stack.append(v)
            copies[u].neighbors.append(copies[v])
    return copies[node]
```

In C++. The file includes LeetCode's `Node` class, the solution, and test helpers that build a graph, check the copy has the same shape with no shared pointers, and free everything.

```
// LeetCode 133. Clone Graph
#include <algorithm>
#include <cassert>
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <vector>

// LeetCode's definition
class Node {
public:
    int val;
    std::vector<Node*> neighbors;
    Node() : val(0), neighbors() {}
    explicit Node(int _val) : val(_val), neighbors() {}
    Node(int _val, std::vector<Node*> _neighbors) : val(_val), neighbors(std::move(_neighbors)) {}
};

// DFS with a map original -> copy. The map is both the "visited" set and the way
// to wire copied edges to already-created copies (which is what handles cycles).
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    std::unordered_map<Node*, Node*> copies;
    std::vector<Node*> st = {node};
    copies[node] = new Node(node->val);
    while (!st.empty()) {
        Node* u = st.back();
        st.pop_back();
        for (Node* v : u->neighbors) {
            if (!copies.count(v)) {                // first time we see v: create its copy, explore later
                copies[v] = new Node(v->val);
            }
            copies[u]->neighbors.push_back(copies[v]);
        }
    }
    return copies[node];
}
```

```

        st.push_back(v);
    }
    copies[u]->neighbors.push_back(copies[v]);
}
}
return copies[node];
}

// ----- test helpers -----
// Build from LeetCode's 1-indexed adjacency list; returns all nodes (node 1 first).
std::vector<Node*> buildGraph(const std::vector<std::vector<int>>& adj) {
    std::vector<Node*> nodes;
    for (int i = 0; i < static_cast<int>(adj.size()); ++i) nodes.push_back(new Node(i + 1));
    for (int i = 0; i < static_cast<int>(adj.size()); ++i)
        for (int j : adj[i]) nodes[i]->neighbors.push_back(nodes[j - 1]);
    return nodes;
}

// Collect every node reachable from start (so we can free a clone we only hold a pointer into).
std::vector<Node*> collect(Node* start) {
    std::vector<Node*> out;
    if (!start) return out;
    std::unordered_set<Node*> seen = {start};
    std::vector<Node*> st = {start};
    while (!st.empty()) {
        Node* u = st.back();
        st.pop_back();
        out.push_back(u);
        for (Node* v : u->neighbors)
            if (seen.insert(v).second) st.push_back(v);
    }
    return out;
}

void freeAll(const std::vector<Node*>& nodes) { for (Node* n : nodes) delete n; }

// Structural equality: same vals, same neighbor vals in the same order, and NO shared pointers.
bool sameStructure(Node* a, Node* b) {
    if (!a || !b) return a == b;
    std::unordered_map<Node*, Node*> pairing;
    std::vector<std::pair<Node*, Node*>> st = {{a, b}};
    pairing[a] = b;
    while (!st.empty()) {
        auto [x, y] = st.back();
        st.pop_back();
        if (x == y) return false; // clone must not alias the original
        if (x->val != y->val || x->neighbors.size() != y->neighbors.size()) return false;
        for (int i = 0; i < static_cast<int>(x->neighbors.size()); ++i) {
            Node* nx = x->neighbors[i];
            Node* ny = y->neighbors[i];
            auto it = pairing.find(nx);
            if (it == pairing.end()) { pairing[nx] = ny; st.push_back({nx, ny}); }
            else if (it->second != ny) return false;
        }
    }
    return true;
}

int main() {
    // Example 1: 4-cycle with chords, adjList = [[2,4],[1,3],[2,4],[1,3]]
    std::vector<Node*> g1 = buildGraph({{2, 4}, {1, 3}, {2, 4}, {1, 3}});
    Node* c1 = cloneGraph(g1[0]);
    assert(sameStructure(g1[0], c1));
    assert(collect(c1).size() == 4);
    freeAll(collect(c1));
    freeAll(g1);

    // Example 2: single node, no neighbors
    std::vector<Node*> g2 = buildGraph({{}});
    Node* c2 = cloneGraph(g2[0]);
    assert(c2 != g2[0] && c2->val == 1 && c2->neighbors.empty());
    delete c2;
    freeAll(g2);
}

```

```
// Example 3: empty graph
assert(cloneGraph(nullptr) == nullptr);

// Self-loop plus a chain: 1-1, 1-2, 2-3
std::vector<Node*> g3 = buildGraph({{1, 2}, {1, 3}, {2}});
Node* c3 = cloneGraph(g3[0]);
assert(sameStructure(g3[0], c3));
assert(c3->neighbors[0] == c3);           // self-loop preserved and points at the copy
freeAll(collect(c3));
freeAll(g3);

std::cout << "OK 01_clone_graph.cpp\n";
return 0;
}
```

What is new:

- `Node* node` is a pointer: the memory address of a `Node`. `node->val` reads the field the pointer points at; it is Python's `node.val`. `nullptr` is `None`. `if (!node)` is `if node is None`.
- `new Node(node->val)` creates a node on the heap and gives you its pointer. It is Python's `Node(node.val)`, except that in C++ nothing frees it for you; the test helpers `collect` and `freeAll` do that with `delete`. On LeetCode you just leak it.
- `std::unordered_map<Node*, Node*> copies;` is the notebook: a dictionary from pointer to pointer. `copies[node] = ...` inserts; `copies.count(v)` asks "is `v` a key?" and is Python's `v in copies`. Use `count` for the test, not `copies[v]`, because `copies[v]` on a missing key *inserts* a null entry, which would make every node look already copied.
- `std::vector<Node*> st = {node};` a vector used as a stack: `st.back()` is the top, `st.pop_back()` removes it, `st.push_back(v)` pushes. This is the explicit-stack DFS from the end of A2.
- `copies[u]->neighbors.push_back(copies[v]);` is `copies[u].neighbors.append(copies[v])`. The `->` is there because `copies[u]` is a pointer.

Walk through the example. `copies = {1: 1'}`, stack `[1]`. Pop 1. Neighbour 2: not in `copies`, make 2', push 2; join 1' -> 2'. Neighbour 4: make 4', push 4; join 1' -> 4'. Stack `[2, 4]`. Pop 4. Neighbour 1: already copied, join 4' -> 1'. Neighbour 3: make 3', push 3; join 4' -> 3'. Stack `[2, 3]`. Pop 3. Neighbours 2 and 4 both copied: join 3' -> 2', 3' -> 4'. Pop 2. Neighbours 1 and 3 copied: join 2' -> 1', 2' -> 3'. Stack empty. Return 1'. Four new nodes, eight neighbour entries, and no arrow points at an original.

Complexity. Each node is copied once and each neighbour entry is walked once: time proportional to nodes plus edges, $O(V + E)$. Memory is the map and the stack, $O(V)$.

Robotics. A pose graph in SLAM is nodes (robot poses) with pointers to each other through constraints. Before running an optimisation you may want to roll back, you deep-copy the graph exactly like this. A behaviour tree is snapshotted the same way. Any structure where objects hold raw pointers to each other needs this map-from-old-to-new trick, otherwise the "copy" still points into the original.

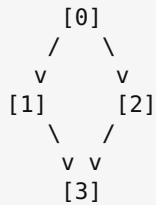
Follow-ups you may get. - *Recursive version?* `Node* clone(Node* u)` that returns `copies[u]` if it exists, else makes the copy, then for each neighbour appends `clone(v)`. Same map. - *Without pointers?* If nodes are numbered, the graph is an adjacency list and the deep copy is `Graph copy = adj;`, one line, because vectors copy by value. - *Who frees the memory?* In real code, own the nodes in a `std::vector<std::unique_ptr<Node>>` or store indexes instead of pointers.

2. Course Schedule (topological sort) (LeetCode 207, medium)

The problem in plain words. There are `numCourses` courses, numbered 0 to `numCourses - 1`. You are given a list of pairs `[a, b]`, each meaning "you must take course `b` before course `a`". Return `true` if it is possible to take every course in some order, and `false` if the rules contradict each other.

```
numCourses = 4, prerequisites = [[1,0], [2,0], [3,1], [3,2]]
```

"0 before 1", "0 before 2", "1 before 3", "2 before 3"



one possible order: 0, 1, 2, 3
answer: true

numCourses = 2, prerequisites = [[1,0], [0,1]]

[0] <--> [1] "0 before 1" and "1 before 0": impossible
answer: false

Edge cases: no prerequisites at all (always true); a cycle hanging off a chain (false). Up to 2000 courses and 5000 pairs.

By hand. Take any course that has no unfinished prerequisites. Cross it out. Now some other courses have one fewer prerequisite to wait for; if any reaches zero, it can be taken next. Keep going. If you cross out every course, true. If you get stuck with courses left over, each one waiting for another one in the pile, false. That is exactly Kahn's algorithm from A3.

The idea. 1. Build the graph: for each [a, b], draw an arrow $b \rightarrow a$ and add one to `indeg[a]`. 2. Put every course with `indeg == 0` in a queue. 3. Pop a course, count it as taken, and for each course it points at, lower its in-degree; if that hits zero, push it. 4. Return `taken == numCourses`.

The Python you would have written.

```

from collections import deque

def canFinish(numCourses, prerequisites):
    adj = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        adj[b].append(a)           # b before a: arrow b -> a
        indeg[a] += 1
    q = deque(c for c in range(numCourses) if indeg[c] == 0)
    taken = 0
    while q:
        u = q.popleft()
        taken += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return taken == numCourses
  
```

In C++.

```

// LeetCode 207. Course Schedule
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

// Kahn's algorithm: repeatedly take a course with no remaining prerequisites.
// If we can take all n, the prerequisite graph is a DAG.
bool canFinish(int numCourses, std::vector<std::vector<int>>& prerequisites) {
    std::vector<std::vector<int>> adj(numCourses); // prereq -> course
    std::vector<int> indeg(numCourses, 0);
    for (const auto& p : prerequisites) {
        adj[p[1]].push_back(p[0]);                // [a, b] means "b before a"
        ++indeg[p[0]];
    }
    std::queue<int> q;
    for (int c = 0; c < numCourses; ++c)
  
```

```

    if (indeg[c] == 0) q.push(c);
    int taken = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ++taken;
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    return taken == numCourses;           // nodes left with indeg > 0 sit on a cycle
}

int main() {
    std::vector<std::vector<int>> p1 = {{1, 0}};
    assert(canFinish(2, p1));

    std::vector<std::vector<int>> p2 = {{1, 0}, {0, 1}};    // 2-cycle
    assert(!canFinish(2, p2));

    std::vector<std::vector<int>> p3;                      // no prerequisites at all
    assert(canFinish(1, p3));
    assert(canFinish(5, p3));

    std::vector<std::vector<int>> p4 = {{1, 0}, {2, 1}, {3, 2}, {1, 3}};    // cycle 1->2->3->1 off a chain
    assert(!canFinish(4, p4));

    std::vector<std::vector<int>> p5 = {{1, 0}, {2, 0}, {3, 1}, {3, 2}};    // diamond DAG
    assert(canFinish(4, p5));

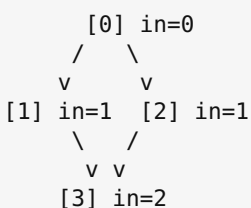
    std::cout << "OK 02_course_schedule.cpp\n";
    return 0;
}

```

What is new:

- `for (const auto& p : prerequisites)` with `p[0]` and `p[1]`: each pair arrives as a two-element vector, so it is `p[0]`, `p[1]`, not `.first`. The arrow goes from `p[1]` (the prerequisite) to `p[0]`.
- `++indeg[p[0]]`; is `indeg[a] += 1`.
- `if (--indeg[v] == 0) q.push(v)`; decrease first, then compare, as explained in A3.
- `return taken == numCourses`; returns a `bool` directly from the comparison.

Walk through the first example. Arrows: `0 -> 1`, `0 -> 2`, `1 -> 3`, `2 -> 3`. In-degrees drawn on the nodes:



Queue starts `[0]`. Pop 0, `taken = 1`; `indeg[1]` `1 -> 0`, push 1; `indeg[2]` `1 -> 0`, push 2. Pop 1, `taken = 2`; `indeg[3]` `2 -> 1`. Pop 2, `taken = 3`; `indeg[3]` `1 -> 0`, push 3. Pop 3, `taken = 4`. Queue empty. `4 == 4`, return `true`.

Second example: arrows `0 -> 1` and `1 -> 0`, in-degrees `[1, 1]`. Nothing starts at zero, the queue is empty from the beginning, `taken = 0`, return `false`.

Complexity. Building the graph is one pass over the pairs; the loop pops each course once and looks at each arrow once. $O(V + E)$ time and memory.

Robotics. Starting a robot's software is a dependency problem: the driver node before the sensor filter, the filter before the localiser, the localiser before the planner. A launch system or a build tool (colcon ordering packages) runs Kahn's algorithm on "must start before" arrows, and a cycle in those arrows is a configuration error caught by exactly this check. A mission with tasks that must finish before other tasks is scheduled the same way.

Follow-ups you may get. - *Return the order itself?* Push each popped course into a vector instead of counting; that is Course Schedule II (LeetCode 210). - *DFS version?* Three-colour DFS: a neighbour that is "in progress" means a cycle. - *Courses have durations; minimum time to finish all?* Process in topological order and keep $\text{finish}[v] = \max \text{ over prerequisites } u \text{ of } \text{finish}[u] + \text{duration}[v]$.

3. Redundant Connection (union-find) (LeetCode 684, medium)

The problem in plain words. A tree is a connected graph with no loops. Someone took a tree with n nodes, numbered 1 to n , and added one extra road, so now there are n roads and exactly one loop. You are given the n roads as pairs $[u, v]$. Return a road that can be removed so the graph is a tree again. If several roads would work, return the one that appears *last* in the input.

```
edges = [[1,2], [1,3], [2,3]]

    [1]
   / \
[2]--[3]    the three roads form a triangle; removing any one gives a tree.
            [2,3] is the last one in the input, so answer: [2, 3]

edges = [[1,2], [2,3], [3,4], [1,4], [1,5]]

[1]--[2]    the loop is 1-2-3-4-1. [1,5] is not on the loop, so it must stay.
 |    |      of the loop roads, [1,4] appears last.          answer: [1, 4]
[4]--[3]
 |
[5] (attached to 1)
```

Edge cases: n is at least 3; the loop may be closed early with more roads after it (those later roads are not on the loop and must not be returned). Up to 1000 nodes.

By hand. Add the roads one at a time and keep track of which nodes are already joined together. Road $[1,2]$: 1 and 2 were separate, now joined. Road $[1,3]$: 3 joins them. Road $[2,3]$: 2 and 3 are *already* joined, so this road closes a loop. That is the answer. Because there is only one loop, the first road that closes one is on it, and every other road of that loop came earlier in the input, so it is also the last loop road in input order, which is what the problem asks for.

The idea. Union-find from A4. For each road in input order, $\text{unite}(u, v)$. The first time unite returns `false` (the two ends already had the same representative), return that road.

The Python you would have written.

```
def findRedundantConnection(edges):
    dsu = DSU(len(edges) + 1)          # nodes are 1..n, so one extra slot
    for u, v in edges:
        if not dsu.union(u, v):
            return [u, v]
    return []
```

(DSU is the class from A4.)

In C++.

```
// LeetCode 684. Redundant Connection
#include <cassert>
#include <iostream>
#include <numeric>
#include <utility>
#include <vector>

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
}
```

```

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) std::swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    return true;
}

// n nodes, n edges => exactly one cycle. Add edges in input order; the first edge whose
// endpoints are already connected is the one that closes that cycle, and it is by
// construction the last edge of the cycle in input order, which is what LeetCode asks for.
std::vector<int> findRedundantConnection(std::vector<std::vector<int>>& edges) {
    DSU dsu(static_cast<int>(edges.size()) + 1); // nodes are 1..n, n == edges.size()
    for (const auto& e : edges)
        if (!dsu.unite(e[0], e[1])) return e;
    return {};
}

int main() {
    std::vector<std::vector<int>> e1 = {{1, 2}, {1, 3}, {2, 3}};
    assert(findRedundantConnection(e1) == std::vector<int>({2, 3}));

    std::vector<std::vector<int>> e2 = {{1, 2}, {2, 3}, {3, 4}, {1, 4}, {1, 5}};
    assert(findRedundantConnection(e2) == std::vector<int>({1, 4}));

    std::vector<std::vector<int>> e3 = {{1, 2}, {2, 3}, {1, 3}, {3, 4}, {4, 5}}; // cycle closed early
    assert(findRedundantConnection(e3) == std::vector<int>({1, 3}));

    std::vector<std::vector<int>> e4 = {{3, 4}, {1, 2}, {2, 4}, {3, 5}, {2, 5}}; // components merge before the cycle closes
    assert(findRedundantConnection(e4) == std::vector<int>({2, 5}));

    std::cout << "OK 03_redundant_connection.cpp\n";
    return 0;
}

```

What is new:

- The DSU struct is A4, pasted in. You will paste it into every union-find problem.
- `DSU dsu(static_cast<int>(edges.size()) + 1);` because the nodes are `1..n`, the array needs `n + 1` slots so that index `n` exists. `edges.size()` is `n` here (a tree with `n` nodes has `n - 1` roads, plus the extra one).
- `if (!dsu.unite(e[0], e[1])) return e;` `unite` returns `false` when the road closes a loop, and `return e` hands back the road as the `std::vector<int>` it arrived as.
- `return {};` an empty vector, for the case that never happens with valid input. The compiler wants every path to return something.

Walk through the first example. `parent = [0, 1, 2, 3]` (index 0 unused). Road `[1,2]`: `find(1) = 1`, `find(2) = 2`, different; sizes equal so `parent[2] = 1`. `parent = [0, 1, 1, 3]`. Road `[1,3]`: `find(1) = 1`, `find(3) = 3`; `parent[3] = 1`. `parent = [0, 1, 1, 1]`. Road `[2,3]`: `find(2)` walks `2 -> 1`, `find(3)` walks `3 -> 1`. Same representative, `unite` returns `false`, return `[2, 3]`.

after [1,2]:	(1) ^ (2) (3)	after [1,3]:	(1) ^ (2) (3)	[2,3]: both already lead to 1
--------------	------------------------	--------------	------------------------	-------------------------------

Complexity. One `unite` per road, each nearly constant time ($\alpha(n)$): $O(n \alpha(n))$, which you can call $O(n)$. Memory $O(n)$ for the two arrays.

Robotics. In pose-graph SLAM, a new constraint between two poses that are *already* connected through the graph is a loop closure, and loop closures are what correct drift. A union-find over pose ids answers "does this candidate close a loop?" before you spend time on the expensive matcher. Deduplicating edges in a topological map of a building uses the same check.

Follow-ups you may get. - *Why is the first failed union the right road?* There is exactly one loop; the road that closes it is by definition the last of its roads to be added. - *What if the roads were one-way (LeetCode 685)?* Union-find alone is not enough; you must also handle a node with two incoming arrows. Different problem. - *Count how many separate groups remain after all roads?* `n` minus the number of successful unions.

4. Graph Valid Tree (LeetCode 261, medium)

The problem in plain words. You are given n nodes numbered 0 to $n - 1$ and a list of two-way roads $[u, v]$, with no duplicate roads and no road from a node to itself. Return `true` if the roads form a *tree*: every node can reach every other node (connected), and there is no loop. Otherwise `false`.

$n = 5$, edges = $[[0,1], [0,2], [0,3], [1,4]]$

```

      [0]
     / | \
    [1][2][3]
     |
    [4]
  
```

connected, no loop answer: true

$n = 5$, edges = $[[0,1], [1,2], [2,0], [3,4]]$

```

[0]---[1]      4 roads, but 0-1-2 is a loop and 3-4 is cut off
  \   /
  [2] [3]---[4]
  
```

answer: false

$n = 4$, edges = $[[0,1], [2,3]]$

```

[0]---[1]   [2]---[3]   too few roads to be connected   answer: false
  
```

Edge cases: $n = 1$ with no roads is a tree (`true`). Up to 2000 nodes.

By hand. Two facts about trees. A tree with n nodes has exactly $n - 1$ roads. And with exactly $n - 1$ roads, "no loop" and "connected" become the same thing: every road that does not close a loop joins two separate groups, so $n - 1$ such roads take n groups down to 1. So: count the roads; if not $n - 1$, `false`. Then add the roads one by one with union-find; if any road closes a loop, `false`. Otherwise `true`.

The idea. 1. if `edges.size() != n - 1`: return `false`. 2. DSU `dsu(n)`; for each road, if `unite` fails, return `false`. 3. return `true`.

The Python you would have written.

```

def validTree(n, edges):
    if len(edges) != n - 1:
        return False
    dsu = DSU(n)
    for u, v in edges:
        if not dsu.union(u, v):
            return False           # this road closes a loop
    return True
  
```

In C++. The file has the union-find version and a BFS version (`validTreeBfs`) that does the same count check and then verifies every node is reachable from node 0.

```

// LeetCode 261. Graph Valid Tree (premium)
#include <cassert>
#include <iostream>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
  
```

```

// A tree on n nodes has exactly n-1 edges and no cycle (which then implies connected).
bool validTree(int n, std::vector<std::vector<int>>& edges) {
    if (static_cast<int>(edges.size()) != n - 1) return false;
    DSU dsu(n);
    for (const auto& e : edges)
        if (!dsu.unite(e[0], e[1])) return false;    // edge inside an existing component = cycle
    return true;                                     // n-1 successful unions => one component
}

// BFS alternative: connected AND n-1 edges. Same answer, shows the other way of thinking.
bool validTreeBfs(int n, std::vector<std::vector<int>>& edges) {
    if (static_cast<int>(edges.size()) != n - 1) return false;
    std::vector<std::vector<int>> adj(n);
    for (const auto& e : edges) { adj[e[0]].push_back(e[1]); adj[e[1]].push_back(e[0]); }
    std::vector<bool> visited(n, false);
    std::queue<int> q;
    q.push(0);
    visited[0] = true;
    int seen = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (!visited[v]) { visited[v] = true; ++seen; q.push(v); }
    }
    return seen == n;
}

int main() {
    std::vector<std::vector<int>> e1 = {{0, 1}, {0, 2}, {0, 3}, {1, 4}};
    assert(validTree(5, e1) && validTreeBfs(5, e1));

    std::vector<std::vector<int>> e2 = {{0, 1}, {1, 2}, {2, 3}, {1, 3}, {1, 4}};    // cycle 1-2-3
    assert(!validTree(5, e2) && !validTreeBfs(5, e2));

    std::vector<std::vector<int>> e3 = {{0, 1}, {2, 3}};    // too few edges: cannot be connected
    assert(!validTree(4, e3) && !validTreeBfs(4, e3));

    std::vector<std::vector<int>> e4 = {{0, 1}, {2, 3}, {1, 2}, {0, 3}};    // edge count is n-1 but there is a cycle (and node 4 is isolated)
    assert(!validTree(5, e4) && !validTreeBfs(5, e4));

    std::vector<std::vector<int>> e5;    // single node, no edges: a tree
    assert(validTree(1, e5) && validTreeBfs(1, e5));

    std::cout << "OK 04_graph_valid_tree.cpp\n";
    return 0;
}

```

What is new:

- `if (static_cast<int>(edges.size()) != n - 1) return false;` `edges.size()` is an unsigned number and `n - 1` is an `int`; comparing them directly makes the compiler warn, so the size is converted to `int` first. You saw the same cast in chapter 03.
- `DSU dsu(n);` nodes are `0..n-1` this time, so exactly `n` slots.
- In `validTreeBfs`, `int seen = 1;` counts node 0 as already seen, and each newly visited node adds one. `return seen == n;` is "did BFS reach everyone".

Walk through the first example. 4 roads, `n - 1 = 4`, fine. `parent = [0,1,2,3,4]`. `[0,1]: join`, `parent = [0,0,2,3,4]`. `[0,2]: join`, `parent = [0,0,0,3,4]`. `[0,3]: join`, `parent = [0,0,0,0,4]`. `[1,4]: find(1) = 0`, `find(4) = 4`, join, `parent = [0,0,0,0,0]`. All four succeeded, return `true`.

Second example: 4 roads, count is fine. `[0,1] join`, `[1,2] join`, `[2,0]: find(2)` and `find(0)` are both 0, `unite` fails, return `false`. We never even looked at `[3,4]`.

Third example: 2 roads but `n - 1 = 3`, return `false` at once.

Complexity. One nearly-constant `unite` per road: $O(n \alpha(n))$, effectively $O(n)$ time, $O(n)$ memory. The BFS version is $O(n + E)$, which is also $O(n)$ here because `E` is `n - 1`.

Robotics. A robot's TF tree (the graph of coordinate frames: map, odom, base_link, camera) must be a real tree: every frame has one parent and everything hangs off one root. `tf2` rejects a transform that would create a loop or a second parent. A URDF's links and joints must also form a tree; a closed chain such as a four-bar linkage fails this test and needs special handling. This check is what "is my frame graph valid" boils down to.

Follow-ups you may get. - *Why is the count check enough to guarantee connected once there is no loop?* A loop-free graph with n nodes and c groups has exactly $n - c$ roads; $n - 1$ roads forces $c = 1$. - *Count the groups instead (LeetCode 323)?* n minus the number of successful unions. - *Could you do it with DFS only?* Yes: count check, then DFS from 0 and see if every node was marked. With $n - 1$ roads, connected implies no loop.

5. Network Delay Time (Dijkstra) (LeetCode 743, medium)

The problem in plain words. There are n computers numbered 1 to n , joined by one-way cables. Each cable is given as $[u, v, w]$: a signal sent from u arrives at v after w milliseconds. A signal starts at computer k and spreads along every cable. Return the time at which the *last* computer receives it. If some computer can never receive it, return -1 .

```
times = [[2,1,1], [2,3,1], [3,4,1]], n = 4, k = 2

[1] <--1-- [2] --1--> [3] --1--> [4]

1 hears it at time 1, 3 at time 1, 4 at time 2. The last is 4, at time 2. answer: 2
```

Edge cases: a single computer with no cables (answer 0, it already has the signal); a cable pointing the wrong way so some computer is unreachable (-1); a direct cable that is slower than a two-cable route. Up to 100 computers and 6000 cables, times up to 100.

By hand. This is "shortest travel time from k to every computer", then take the biggest of those times. Shortest time on a map with different cable lengths is Dijkstra from A5. Once you have the time for every computer, the answer is the largest one, or -1 if any is still INF.

The idea. 1. Build the weighted adjacency list, size $n + 1$ because computers are $1..n$. 2. Run the Dijkstra template from k . 3. Scan $\text{dist}[1..n]$: any INF means -1 ; otherwise return the maximum.

The Python you would have written.

```
import heapq

def networkDelayTime(times, n, k):
    adj = [[] for _ in range(n + 1)]
    for u, v, w in times:
        adj[u].append((v, w))
    INF = float('inf')
    dist = [INF] * (n + 1)
    dist[k] = 0
    pq = [(0, k)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    ans = max(dist[1:]) # skip the unused slot 0
    return -1 if ans == INF else ans
```

In C++.

```
// LeetCode 743. Network Delay Time
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <utility>
```

```

#include <vector>

// Plain Dijkstra from k; answer is the largest finite distance (the last node to hear the signal).
int networkDelayTime(std::vector<std::vector<int>>& times, int n, int k) {
    std::vector<std::vector<std::pair<int, int>>> adj(n + 1); // 1-indexed nodes
    for (const auto& t : times) adj[t[0]].push_back({t[1], t[2]});

    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(n + 1, INF);
    using State = std::pair<int, int>; // (dist, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    dist[k] = 0;
    pq.push({0, k});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue; // stale heap entry
        for (const auto& [v, w] : adj[u]) {
            if (d + w < dist[v]) { // d is finite here, so no overflow
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    int ans = 0;
    for (int v = 1; v <= n; ++v) {
        if (dist[v] == INF) return -1; // some node never reached
        ans = std::max(ans, dist[v]);
    }
    return ans;
}

int main() {
    std::vector<std::vector<int>> t1 = {{2, 1, 1}, {2, 3, 1}, {3, 4, 1}};
    assert(networkDelayTime(t1, 4, 2) == 2);

    std::vector<std::vector<int>> t2 = {{1, 2, 1}};
    assert(networkDelayTime(t2, 2, 1) == 1);
    assert(networkDelayTime(t2, 2, 2) == -1); // edge is directed: 2 cannot reach 1

    std::vector<std::vector<int>> t3 = {{1, 2, 1}, {2, 3, 2}, {1, 3, 4}}; // direct edge loses to the 2-hop path
    assert(networkDelayTime(t3, 3, 1) == 3);

    std::vector<std::vector<int>> t4; // single node, no edges
    assert(networkDelayTime(t4, 1, 1) == 0);

    std::cout << "OK 05_network_delay_time.cpp\n";
    return 0;
}

```

What is new (compared with the A5 template):

- `std::vector<std::vector<std::pair<int, int>>> adj(n + 1);` the `WGraph` type written out, with `n + 1` rows for 1-indexed computers. Row 0 stays empty.
- `const int INF = std::numeric_limits<int>::max();` plain `int` and the full maximum this time. That is safe *only* because the code adds `w` to `d`, which was just popped from the queue and is therefore a real finite distance, never `INF`. If you ever write `dist[u] + w` without that guarantee, go back to the `long long` and `/ 4` of the template.
- `for (const auto& t : times) adj[t[0]].push_back({t[1], t[2]});` each cable is a 3-element vector: from, to, time.
- The final loop runs `v = 1` to `n` inclusive, ignoring slot 0, and returns `-1` the moment it sees `INF`. `std::max(ans, dist[v])` keeps the largest.

Walk through the example. Cables: 2 → 1 (1), 2 → 3 (1), 3 → 4 (1). Start at 2. The table shows `dist[1..4]` and the queue after each pop:

step	pop	what happens	dist[1]	dist[2]	dist[3]	dist[4]	pq
0	start	dist[2] = 0, push (0,2)	INF	0	INF	INF	(0,2)
1	(0,2)	2->1: 0+1 < INF, dist[1] = 1, push (1,1) 2->3: 0+1 < INF, dist[3] = 1, push (1,3)	1	0	1	INF	(1,1) (1,3)
2	(1,1)	computer 1 has no cables out	1	0	1	INF	(1,3)

3	(1,3)	3->4: 1+1 < INF, dist[4] = 2, push (2,4)	1	0	1	2	(2,4)
4	(2,4)	no cables out; queue empty	1	0	1	2	(empty)

Scan `dist[1..4] = [1, 0, 1, 2]`: no INF, maximum is 2. Return 2.

A second, smaller example where the direct cable loses: `times = [[1,2,1],[2,3,2],[1,3,4]]`, `n = 3`, `k = 1`. Pop (0,1): `dist[2] = 1`, `dist[3] = 4`. Pop (1,2): `1 + 2 = 3 < 4`, so `dist[3] = 3` and (3,3) is pushed while the old (4,3) stays in the queue. Pop (3,3): fresh, nothing out. Pop (4,3): `4 > 3`, stale, skip. Answer 3, not 4.

Complexity. Dijkstra with a binary heap: $O((V + E) \log V)$ time, $O(V + E)$ memory. With $V = 100$ and $E = 6000$ this is instant.

Robotics. The literal reading is "how long until a command reaches every node in a multi-hop robot network". The more common use of the same code is the *cost-to-go* layer of a global planner: run Dijkstra once from the *goal* over the costmap and every cell learns its cost to reach the goal. That field is what a local planner follows downhill, and it is the perfect heuristic for A* if you have the time to compute it.

Follow-ups you may get. - *Cables are two-way now?* Push both directions when building `adj`; nothing else changes. - *Which route did the signal take to computer 4?* Keep a parent array updated whenever `dist[v]` improves, then walk it back as in A5. - *No heap allowed?* Dijkstra with a plain array and a linear scan for the smallest unvisited node: $O(V^2)$, fine for $V = 100$ and actually better on very dense graphs.

6. Path with Minimum Effort (LeetCode 1631, medium)

The problem in plain words. A grid of heights. You start at the top-left cell and want to reach the bottom-right, moving up, down, left or right. The *effort* of a route is the largest single height difference between two consecutive cells on it (not the total climb). Return the smallest possible effort of any route.

```
heights = 1 2 2
          3 8 2
          5 3 5
```

Route A along the top then down the right: 1 -> 2 -> 2 -> 2 -> 5
 steps: `|1-2|=1`, `|2-2|=0`, `|2-2|=0`, `|2-5|=3` worst step 3

Route B down the left then along the bottom: 1 -> 3 -> 5 -> 3 -> 5
 steps: `|1-3|=2`, `|3-5|=2`, `|5-3|=2`, `|3-5|=2` worst step 2

No route does better than 2.

answer: 2

Edge cases: a single cell (effort 0); a single row (there is only one route, answer is its worst step). Up to 100 by 100 cells, heights up to a million.

By hand. This is "minimise the worst road" from A6, on a grid. Every step between two neighbouring cells is a road whose length is the height difference. We want the route whose biggest road is smallest. Dijkstra, but combining with `max` instead of `+`.

The idea. The A5 template with the grid pieces from A6: the queue holds `(effort, row, col)`, effort is a 2-D table, neighbours come from `dr/dc`. When relaxing, the new effort to a neighbour is `max(effort so far, |height difference|)`. Stop the first time the bottom-right cell is *popped*: that effort is final.

The Python you would have written.

```
import heapq

def minimumEffortPath(heights):
    rows, cols = len(heights), len(heights[0])
    INF = float('inf')
    effort = [[INF] * cols for _ in range(rows)]
    effort[0][0] = 0
    pq = [(0, 0, 0)]
    while pq:
        # (effort, r, c)
```

```

e, r, c = heapq.heappop(pq)
if (r, c) == (rows - 1, cols - 1):
    return e                                # first pop of the target is the answer
if e > effort[r][c]:
    continue
for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        ne = max(e, abs(heights[nr][nc] - heights[r][c]))
        if ne < effort[nr][nc]:
            effort[nr][nc] = ne
            heapq.heappush(pq, (ne, nr, nc))
return effort[rows - 1][cols - 1]

```

In C++. The file also contains a second solution, `minimumEffortPathBinarySearch`, that guesses an effort limit and uses BFS to ask "can I cross using only steps up to this limit?", then binary-searches the smallest limit that works. Read the Dijkstra one first.

```

// LeetCode 1631. Path With Minimum Effort
#include <algorithm>
#include <cassert>
#include <cstdlib>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <tuple>
#include <vector>

// Minimax Dijkstra on the grid: the "distance" of a path is the largest step along it,
// so relaxing uses max(d, step) instead of d + step. Everything else is the template.
int minimumEffortPath(std::vector<std::vector<int>>& heights) {
    int rows = static_cast<int>(heights.size());
    int cols = static_cast<int>(heights[0].size());
    const int INF = std::numeric_limits<int>::max();
    std::vector<std::vector<int>> effort(rows, std::vector<int>(cols, INF));
    using State = std::tuple<int, int, int>; // (effort, r, c)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    effort[0][0] = 0;
    pq.push({0, 0, 0});
    const int dr[4] = {1, -1, 0, 0};
    const int dc[4] = {0, 0, 1, -1};
    while (!pq.empty()) {
        auto [e, r, c] = pq.top();
        pq.pop();
        if (r == rows - 1 && c == cols - 1) return e; // early exit: first pop of target is optimal
        if (e > effort[r][c]) continue;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            int ne = std::max(e, std::abs(heights[nr][nc] - heights[r][c]));
            if (ne < effort[nr][nc]) {
                effort[nr][nc] = ne;
                pq.push({ne, nr, nc});
            }
        }
    }
    return effort[rows - 1][cols - 1]; // unreachable for a grid, keeps -Wall quiet
}

// Alternative: binary search the answer, BFS to check "can I cross using only steps <= limit?".
bool reachable(const std::vector<std::vector<int>>& h, int limit) {
    int rows = static_cast<int>(h.size()), cols = static_cast<int>(h[0].size());
    std::vector<std::vector<bool>> seen(rows, std::vector<bool>(cols, false));
    std::queue<std::pair<int, int>> q;
    q.push({0, 0});
    seen[0][0] = true;
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        if (r == rows - 1 && c == cols - 1) return true;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];

```

```

        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || seen[nr][nc]) continue;
        if (std::abs(h[nr][nc] - h[r][c]) > limit) continue;
        seen[nr][nc] = true;
        q.push({nr, nc});
    }
}
return false;
}

int minimumEffortPathBinarySearch(std::vector<std::vector<int>>& heights) {
    int lo = 0, hi = 1000000; // heights <= 1e6 per the constraints
    while (lo < hi) { // smallest limit that is reachable
        int mid = lo + (hi - lo) / 2;
        if (reachable(heights, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

int main() {
    std::vector<std::vector<int>> h1 = {{1, 2, 2}, {3, 8, 2}, {5, 3, 5}};
    assert(minimumEffortPath(h1) == 2 && minimumEffortPathBinarySearch(h1) == 2);

    std::vector<std::vector<int>> h2 = {{1, 2, 3}, {3, 8, 4}, {5, 3, 5}};
    assert(minimumEffortPath(h2) == 1 && minimumEffortPathBinarySearch(h2) == 1);

    std::vector<std::vector<int>> h3 = {{1, 2, 1, 1, 1}, {1, 2, 1, 2, 1}, {1, 2, 1, 2, 1},
                                       {1, 2, 1, 2, 1}, {1, 1, 1, 2, 1}};
    assert(minimumEffortPath(h3) == 0 && minimumEffortPathBinarySearch(h3) == 0);

    std::vector<std::vector<int>> h4 = {{7}}; // single cell: no steps at all
    assert(minimumEffortPath(h4) == 0 && minimumEffortPathBinarySearch(h4) == 0);

    std::vector<std::vector<int>> h5 = {{1, 10, 6, 7, 9, 10, 4, 9}}; // one row: forced path, answer is max step
    assert(minimumEffortPath(h5) == 9 && minimumEffortPathBinarySearch(h5) == 9);

    std::cout << "OK 06_path_with_minimum_effort.cpp\n";
    return 0;
}

```

What is new:

- using `State = std::tuple<int, int, int>`; three values, so a pair will not do. A tuple compares slot by slot from the left, so with effort in slot 0 the queue still pops the smallest effort. `#include <tuple>`.
- `pq.push({0, 0, 0})`; builds the tuple from braces, and `auto [e, r, c] = pq.top()`; unpacks it, exactly as with pairs.
- `std::abs(heights[nr][nc] - heights[r][c])` is Python's `abs(...)`. It lives in `<cstdlib>`.
- `int ne = std::max(e, ...)` is the one-word change from the plain template: `max` instead of `+`.
- `if (r == rows - 1 && c == cols - 1) return e`; sits *before* the stale check, right after the pop. Returning on pop is correct; returning when pushing would not be.
- The final `return effort[rows - 1][cols - 1]`; is never reached on a real grid (the target is always popped eventually); it is there so every path returns a value and the compiler stays quiet.

Walk through the example. `effort[0][0] = 0`, queue `{{(0,0,0)}`. Each line is one pop, with the pushes it causes.

```

pop (0, 0,0):  down (1,0): max(0,|3-1|)=2 push.    right (0,1): max(0,|2-1|)=1 push.
               queue: (1,0,1) (2,1,0)
pop (1, 0,1):  down (1,1): max(1,|8-2|)=6 push.    right (0,2): max(1,0)=1 push.
               queue: (1,0,2) (2,1,0) (6,1,1)
pop (1, 0,2):  down (1,2): max(1,0)=1 push.
               queue: (1,1,2) (2,1,0) (6,1,1)
pop (1, 1,2):  down (2,2): max(1,|5-2|)=3 push.    left (1,1): 6, no better.
               queue: (2,1,0) (3,2,2) (6,1,1)        <- target is in the queue at 3, but not popped
pop (2, 1,0):  down (2,0): max(2,|5-3|)=2 push.    right (1,1): max(2,5)=5 < 6, push.
               queue: (2,2,0) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,0):  right (2,1): max(2,|3-5|)=2 push.
               queue: (2,2,1) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,1):  right (2,2): max(2,|5-3|)=2 < 3, push (2,2,2).

```

```

queue: (2,2,2) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,2): this is the target. Return 2.

```

Notice the target first entered the queue with effort 3 (route A) and was later improved to 2 (route B) before it was popped. This is why you must return on *pop*, not on *push*.

Complexity. Every cell is a node and each has at most 4 roads: $O(RC \log(RC))$ time for R rows and C columns, $O(RC)$ memory for the table and the queue.

Robotics. Terrain traversability for a legged or wheeled robot on an elevation map: what tips the robot over is the *steepest single step* on the route, not the total climb, so the planner minimises the worst step. Same code with slope in place of height difference. A similar question arises for a manipulator: minimise the largest joint jump between consecutive waypoints.

Follow-ups you may get. - *Why does Dijkstra still work with `max`?* Because "the worst step so far" never decreases as a route gets longer, which is the only property the "first pop is final" argument needs. - *Another way?* Binary search on the answer with a BFS check per guess (in the file), or union-find over all steps sorted by size, stopping when start and target join. - *Total climb instead of worst step?* Then it is plain Dijkstra with `+`.

7. Swim in Rising Water (LeetCode 778, hard)

The problem in plain words. An n by n grid where `grid[r][c]` is the height of the ground at that cell. The numbers are all different and are exactly 0 to $n*n - 1$. Water is rising: at time t the water is at height t , and you can stand on any cell whose height is at most t , and swim between two neighbouring cells (up, down, left, right) if both are at most t . You start on the top-left cell. Return the earliest time t at which you can reach the bottom-right cell.

```

grid =  0  2          At t = 3 the whole grid is under water and you can cross.
       1  3          Before that the cell 3 (bottom-right) is dry land.    answer: 3

grid =  0  6  7
       2  5  8          Straight across the top needs t = 8 (the 8 is on the way).
       1  3  4          Down the left and along the bottom: 0,2,1,3,4 needs only t = 4.
                                   answer: 4

```

Edge cases: a single cell (answer is its own height); the start cell itself may be the tallest thing on the route, so the answer is at least `grid[0][0]`. n up to 50.

By hand. The time you need for a route is the *highest cell* on it (including the start and end). We want the route whose highest cell is lowest. That is the "minimise the worst road" question again, with the cost sitting on the cells instead of on the steps between them. Same Dijkstra with `max`.

The idea. The problem 6 code with two small changes: the start's cost is `grid[0][0]`, not 0, and the cost of stepping into a neighbour is `max(cost so far, grid[nr][nc])`.

The Python you would have written.

```

import heapq

def swimInWater(grid):
    n = len(grid)
    INF = float('inf')
    best = [[INF] * n for _ in range(n)]
    best[0][0] = grid[0][0]
    pq = [(grid[0][0], 0, 0)] # (time, r, c)
    while pq:
        t, r, c = heapq.heappop(pq)
        if (r, c) == (n - 1, n - 1):
            return t
        if t > best[r][c]:

```

```

        continue
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < n:
            nt = max(t, grid[nr][nc])
            if nt < best[nr][nc]:
                best[nr][nc] = nt
                heapq.heappush(pq, (nt, nr, nc))
    return best[n - 1][n - 1]

```

In C++. As in problem 6, the file has the Dijkstra solution and a binary-search alternative (swimInWaterBinarySearch) that guesses t and checks reachability with a DFS over cells of height at most t .

```

// LeetCode 778. Swim in Rising Water
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <tuple>
#include <vector>

// Same minimax idea as 1631, but the cost of a path is the max CELL value on it (not max edge):
// the water must have risen to at least grid[r][c] to stand on (r, c). Dijkstra with max().
int swimInWater(std::vector<std::vector<int>>& grid) {
    int n = static_cast<int>(grid.size());
    const int INF = std::numeric_limits<int>::max();
    std::vector<std::vector<int>> best(n, std::vector<int>(n, INF));
    using State = std::tuple<int, int, int>; // (time, r, c)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    best[0][0] = grid[0][0];
    pq.push({grid[0][0], 0, 0});
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!pq.empty()) {
        auto [t, r, c] = pq.top();
        pq.pop();
        if (r == n - 1 && c == n - 1) return t;
        if (t > best[r][c]) continue;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
            int nt = std::max(t, grid[nr][nc]);
            if (nt < best[nr][nc]) {
                best[nr][nc] = nt;
                pq.push({nt, nr, nc});
            }
        }
    }
    return best[n - 1][n - 1];
}

// Alternative: binary search on t, DFS over cells with value <= t.
bool canReach(const std::vector<std::vector<int>>& g, int t) {
    int n = static_cast<int>(g.size());
    if (g[0][0] > t) return false;
    std::vector<std::vector<bool>> seen(n, std::vector<bool>(n, false));
    std::vector<std::pair<int, int>> st = {{0, 0}};
    seen[0][0] = true;
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!st.empty()) {
        auto [r, c] = st.back();
        st.pop_back();
        if (r == n - 1 && c == n - 1) return true;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= n || seen[nr][nc] || g[nr][nc] > t) continue;
            seen[nr][nc] = true;
            st.push_back({nr, nc});
        }
    }
}

```

```

    }
    return false;
}

int swimInWaterBinarySearch(std::vector<std::vector<int>>& grid) {
    int n = static_cast<int>(grid.size());
    int lo = 0, hi = n * n - 1;           // grid is a permutation of 0..n*n-1
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canReach(grid, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

int main() {
    std::vector<std::vector<int>> g1 = {{0, 2}, {1, 3}};
    assert(swimInWater(g1) == 3 && swimInWaterBinarySearch(g1) == 3);

    std::vector<std::vector<int>> g2 = {{0, 1, 2, 3, 4}, {24, 23, 22, 21, 5}, {12, 13, 14, 15, 16},
                                         {11, 17, 18, 19, 20}, {10, 9, 8, 7, 6}};
    assert(swimInWater(g2) == 16 && swimInWaterBinarySearch(g2) == 16);

    std::vector<std::vector<int>> g3 = {{0}};           // single cell
    assert(swimInWater(g3) == 0 && swimInWaterBinarySearch(g3) == 0);

    std::vector<std::vector<int>> g4 = {{3, 2}, {0, 1}}; // start cell itself is the bottleneck
    assert(swimInWater(g4) == 3 && swimInWaterBinarySearch(g4) == 3);

    std::cout << "OK 07_swim_in_rising_water.cpp\n";
    return 0;
}

```

What is new:

- Almost nothing, and that is the point. Compare the file with problem 6 side by side: the state tuple is (time, r, c), the start is seeded with grid[0][0], and the relax line is `std::max(t, grid[nr][nc])`. Two solutions to two different-looking problems differ by two lines.
- `int lo = 0, hi = n * n - 1;` in the binary-search version uses the fact that the heights are exactly `0..n*n-1`; the Dijkstra version does not need that fact at all.

Walk through the 3 by 3 example. `best[0][0] = 0`, queue `{{(0,0,0)}}`.

```

pop (0, 0,0):  down (1,0): max(0,2)=2 push.   right (0,1): max(0,6)=6 push.
               queue: (2,1,0) (6,0,1)
pop (2, 1,0):  down (2,0): max(2,1)=2 push.   right (1,1): max(2,5)=5 push.
               queue: (2,2,0) (5,1,1) (6,0,1)
pop (2, 2,0):  right (2,1): max(2,3)=3 push.
               queue: (3,2,1) (5,1,1) (6,0,1)
pop (3, 2,1):  right (2,2): max(3,4)=4 push.   up (1,1): max(3,5)=5, no better.
               queue: (4,2,2) (5,1,1) (6,0,1)
pop (4, 2,2):  target. Return 4.

```

The cells 6, 7 and 8 were never popped; the search went round them along the low ground.

Complexity. $O(n^2 \log n)$ time for the Dijkstra version, $O(n^2)$ memory. The binary-search version is $O(n^2 \log(n^2))$, about the same.

Robotics. "Earliest time the goal becomes reachable" through a field that opens up over time: a flooding or tide map, an occupancy grid whose cells another robot clears on a schedule. Flip the sign and it is the *widest-corridor* path: on a clearance map, find the route whose narrowest point is as wide as possible, which is what you want for a large robot in a cluttered room.

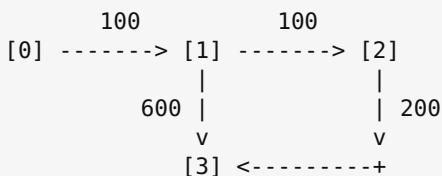
Follow-ups you may get. - How does this relate to problem 6? Both minimise the maximum along a route; there the cost is on the steps, here it is on the cells. A cell cost becomes a step cost by charging the cell you step into. - Union-find version? Sort the cells by height, add each one and join it to its lower neighbours, stop

when $(0,0)$ and $(n-1,n-1)$ share a representative; the height of the cell you just added is the answer. - *Why seed with $grid[0][0]$ and not 0?* The start cell can be the bottleneck. Test $[[3,2],[0,1]]$: the answer is 3, which a 0 seed would miss.

8. Cheapest Flights Within K Stops (LeetCode 787, medium)

The problem in plain words. There are n cities, numbered 0 to $n - 1$, and a list of one-way flights $[\text{from}, \text{to}, \text{price}]$. You want to fly from src to dst using at most k stops (intermediate cities), which means at most $k + 1$ flights. Return the cheapest total price, or -1 if it cannot be done.

```
n = 4, flights = [[0,1,100], [1,2,100], [2,0,100], [1,3,600], [2,3,200]], src = 0, dst = 3
```



(there is also a road $2 \rightarrow 0$ costing 100; it only leads back to the start)

```

k = 1:  0 -> 1 -> 3 costs 700 with 1 stop.      0 -> 1 -> 2 -> 3 costs 400 but has 2 stops.
        answer: 700
k = 0:  only direct flights, and there is no 0 -> 3.      answer: -1
k = 2:  0 -> 1 -> 2 -> 3 is allowed.              answer: 400
  
```

Edge cases: $\text{src} == \text{dst}$ (price 0, no flights needed); dst unreachable at all (-1); reachable but only with too many stops (-1). Up to 100 cities, k less than n .

By hand. The greedy answer is wrong, and you should see why before writing anything. Plain Dijkstra from 0 finds $\text{dist}[1] = 100$, $\text{dist}[2] = 200$, $\text{dist}[3] = 400$, and it reports 400 for city 3. But that route is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, two stops, not allowed when $k = 1$. Dijkstra only knows about cost; it has no idea how many flights it used. The correct answer is 700.

So think in rounds instead. "With at most 1 flight, what is the cheapest way to each city?" Only city 1, for 100. "With at most 2 flights?" Extend each of *last round's* answers by one flight: city 2 for 200, city 3 for 700. Two flights means one stop, so stop here: city 3 costs 700. That is Bellman-Ford from A6, limited to $k + 1$ rounds.

The idea. 1. $\text{dist} = [\text{INF}] * n$, $\text{dist}[\text{src}] = 0$. 2. Repeat $k + 1$ times: make a copy next of dist ; for every flight $u \rightarrow v$ with price w , if $\text{dist}[u]$ is not INF and $\text{dist}[u] + w < \text{next}[v]$, set $\text{next}[v]$; then $\text{dist} = \text{next}$. 3. Return $\text{dist}[\text{dst}]$, or -1 if it is still INF.

The copy is what enforces the limit: each round may only extend routes from the *previous* round by one flight.

The Python you would have written.

```

def findCheapestPrice(n, flights, src, dst, k):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):
        nxt = dist[:]
        for u, v, w in flights:
            if dist[u] != INF and dist[u] + w < nxt[v]:
                nxt[v] = dist[u] + w
        dist = nxt
    return -1 if dist[dst] == INF else dist[dst]
  
```

In C++.

```
// LeetCode 787. Cheapest Flights Within K Stops
#include <algorithm>
#include <cassert>
#include <iostream>
#include <limits>
#include <utility>
#include <vector>

// Bellman-Ford limited to k+1 rounds. Round i computes the cheapest price using at most i edges.
// Relaxing from a COPY of the previous round's dist is what stops a single round from
// chaining several edges together (which would break the stop limit).
int findCheapestPrice(int n, std::vector<std::vector<int>>& flights, int src, int dst, int k) {
    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round <= k; ++round) { // k stops == k+1 flights
        std::vector<int> next = dist;
        for (const auto& f : flights) {
            int u = f[0], v = f[1], w = f[2];
            if (dist[u] != INF && dist[u] + w < next[v]) next[v] = dist[u] + w; // INF check avoids overflow
        }
        dist = std::move(next);
    }
    return dist[dst] == INF ? -1 : dist[dst];
}

int main() {
    std::vector<std::vector<int>> f1 = {{0, 1, 100}, {1, 2, 100}, {2, 0, 100}, {1, 3, 600}, {2, 3, 200}};
    assert(findCheapestPrice(4, f1, 0, 3, 1) == 700); // 0->1->3; 0->1->2->3 needs 2 stops

    std::vector<std::vector<int>> f2 = {{0, 1, 100}, {1, 2, 100}, {0, 2, 500}};
    assert(findCheapestPrice(3, f2, 0, 2, 1) == 200);
    assert(findCheapestPrice(3, f2, 0, 2, 0) == 500); // no stops allowed: direct flight only

    std::vector<std::vector<int>> f3 = {{0, 1, 1}, {1, 2, 1}}; // dst not reachable within limit
    assert(findCheapestPrice(3, f3, 0, 2, 0) == -1);
    assert(findCheapestPrice(3, f3, 2, 0, 5) == -1); // not reachable at all (directed)

    // Where plain Dijkstra fails: cheapest path to 1 uses 2 stops, but the only
    // k-feasible route to 3 goes through 1 via the pricier direct edge.
    std::vector<std::vector<int>> f4 = {{0, 1, 5}, {0, 2, 1}, {2, 4, 1}, {4, 1, 1}, {1, 3, 1}};
    assert(findCheapestPrice(5, f4, 0, 3, 1) == 6); // 0->1->3 (Dijkstra would settle 1 at cost 3 and find nothing)
    assert(findCheapestPrice(5, f4, 0, 3, 3) == 4); // with enough stops the cheap route wins

    std::vector<std::vector<int>> none; // src == dst, no flights needed
    assert(findCheapestPrice(1, none, 0, 0, 0) == 0);

    std::cout << "OK 08_cheapest_flights_k_stops.cpp\n";
    return 0;
}
```

What is new:

- `for (int round = 0; round <= k; ++round)` runs $k + 1$ times: rounds 0 to k inclusive.
- `std::vector<int> next = dist;` copies the vector (vectors copy by value in C++). This is Python's `dist[:]`.
- `int u = f[0], v = f[1], w = f[2];` unpacks a flight given as a 3-element vector.
- `dist = std::move(next);` hands `next`'s contents to `dist` without copying them again. `dist = next;` would also be correct, just one copy slower. `std::move` is in `<utility>`.
- `return dist[dst] == INF ? -1 : dist[dst];` is `-1` if `dist[dst] == INF` else `dist[dst]`.
- The `dist[u] != INF &&` guard matters more here than in Python: `INT_MAX + w` overflows to a negative number, which would then look like a wonderful price.

Walk through the example with $k = 1$. Two rounds. Flights in input order: 0->1 (100), 1->2 (100), 2->0 (100), 1->3 (600), 2->3 (200).

```
start:    dist = [0, INF, INF, INF]

round 1:  next = copy of dist
          0->1: dist[0]=0, 0+100 < INF, next[1] = 100
          1->2: dist[1] is INF, skip           (reads the OLD dist, not next)
          2->0: dist[2] is INF, skip
          1->3: dist[1] is INF, skip
          2->3: dist[2] is INF, skip
          dist = [0, 100, INF, INF]           "cheapest with at most 1 flight"
```

```

round 2:  next = copy of dist
         0->1: 0+100, not better than 100
         1->2: 100+100 < INF, next[2] = 200
         2->0: dist[2] is INF (old copy), skip
         1->3: 100+600 < INF, next[3] = 700
         2->3: dist[2] is INF (old copy), skip
         dist = [0, 100, 200, 700]           "cheapest with at most 2 flights"

dist[3] = 700.  Return 700.

```

If you had updated `dist` in place instead of `next`, then in round 2 the line `2->3` would have seen the fresh `dist[2] = 200` and set `dist[3] = 400`, sneaking a third flight into a two-flight round. A third round ($k = 2$) is where 400 legitimately appears.

Complexity. $k + 1$ rounds, each one pass over all flights: $O(k \text{ times } E)$ time. Memory is two arrays of size n : $O(n)$.

Robotics. Any plan with a *hop budget*: a delivery drone that may recharge at most k waypoints, a message that may be relayed through at most k robots, a multi-robot handoff chain with at most k handoffs. The number of hops is a second dimension of the state, and Bellman-Ford's rounds walk through that dimension for free.

Follow-ups you may get. - *Can Dijkstra be repaired by carrying a stop count in each queue entry?* Only if you keep a separate best-cost table for every (city, stops used) pair, which is Dijkstra on a bigger graph, $O(kE \log(kV))$. Pruning alone is wrong: on $[[0,1,5],[0,2,1],[2,4,1],[4,1,1],[1,3,1]]$ with $k = 1$, it settles city 1 at cost 3 via 2 stops and never finds $0 \rightarrow 1 \rightarrow 3 = 6$. - *Negative prices?* Bellman-Ford handles them; Dijkstra does not. - *BFS version?* Level-by-level BFS for $k + 1$ levels, keeping the best cost per city, is the same algorithm written with a queue.

Part C — Check yourself

Answer these in plain words, on paper, without opening the files. If one stalls you, reread the section it points at.

1. Draw a 4-node undirected graph and write its adjacency list as a table. Then write the same graph as one-way roads in one direction only. Which rows changed? (A1)
2. Why did the grid in chapter 03 not need an adjacency list, while the city map does? (A1)
3. On the 5-node graph from A2, BFS from node 4 instead of node 0. Write the queue after each pop and the final `dist` array. (A2)
4. Why is the `dist[v] != -1` check optional-looking but actually essential on a graph, more than on a grid? (A2)
5. Write the in-degrees of every node for the "getting dressed" graph, then add the rule "shoes before socks". Run Kahn's algorithm by hand. How many nodes come out, and what does that tell you? (A3)
6. What is the difference between `--indeg[v] == 0` and `indeg[v]-- == 0`? Which one does Kahn's algorithm need? (A3)
7. Six people, unions (0,1), (2,3), (4,5), (1,3). Draw the parent arrows after each union. What does `union(0, 2)` return afterwards, and why is that return value useful? (A4)
8. Explain path compression with a before-and-after drawing of a chain of four arrows. Roughly how long does a `find` take after enough compressions? (A4)
9. On the A5 graph, change the road `0 -> 1` from 4 to 2. Redo the Dijkstra table: after each step, `dist` for every node and the contents of the queue. Which entries become stale? (A5)
10. Why must the queue entry be (distance, node) and not (node, distance)? What goes wrong with `std::priority_queue<State> pq;` written without the `std::greater` part? (A5)
11. Why is `INF` defined as `max / 4` rather than `max`? Give the concrete thing that would go wrong. (A5)

12. Give a three-city example with one negative road where Dijkstra with early stopping returns the wrong distance. Which algorithm would you use instead? (A5, A6)
13. For "minimise the worst road", which single expression in the Dijkstra template changes, and why does the "first pop is final" argument still hold? (A6, problems 6 and 7)
14. On the Cheapest Flights example with $k = 1$, what does plain Dijkstra return and why is it wrong? What does the `next = dist` copy in Bellman-Ford prevent? (A6, problem 8)
15. Match each to a tool without looking: fewest roads on an unweighted map; shortest travel time with different road lengths; is this graph a tree; can these tasks be ordered; shortest with at most k roads. (whole chapter)

05. Heaps, trees, design

This chapter is about three things a robot program needs all the time: a way to always grab the biggest (or smallest) item quickly (a heap), a way to walk a tree of pointers (a binary tree), and a way to keep only the most recently used things in memory (a cache). A planner picks the cheapest cell next; a sensor filter needs the biggest reading in the last k samples; a map viewer keeps only the last few tiles in RAM. Every one of those is a problem below.

Rough time: 5 days at about 2 hours a day. Day 1 and 2 are Part A only. Do the problems only when you can draw a heap push and a level-order loop from memory.

What you need before this chapter. From chapter 01: `std::vector`, `std::pair`, `auto [a, b]` and `struct`. From chapter 03: `std::queue`, `std::stack`, recursion, and the ring-by-ring BFS loop. If the BFS ring loop is fuzzy, reread chapter 03 A2 first; the tree part of this chapter reuses it.

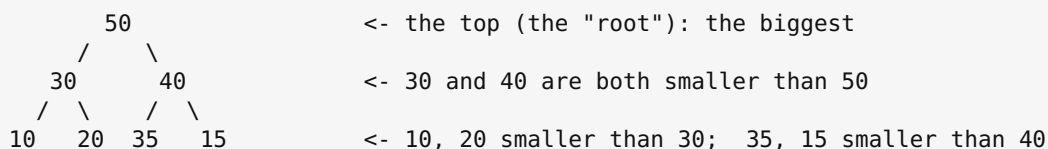
Part A — Heaps, trees and caches, from zero

A1. What a heap is

The picture. A heap is a pile where the biggest is always on top, and you only ever look at the top. Think of a stack of boxes arranged so that every box is bigger than the two boxes resting on it. You never search the pile. You ask "what is the biggest?" and the answer is sitting on top. You take it off, the pile shuffles a little, and the next biggest is on top.

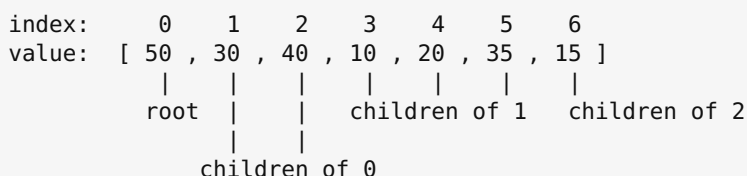
That is the only promise a heap makes: **every parent is bigger than (or equal to) its two children**. It does not promise that the left child is bigger than the right, or that anything is sorted. Only "parent beats children", all the way down.

The drawing. Here is a heap of 7 numbers drawn as a tree:



Check the rule: 50 beats 30 and 40. 30 beats 10 and 20. 40 beats 35 and 15. It holds everywhere. Notice 35 is bigger than 30 even though 35 is lower down. That is allowed: the rule only compares a node with its own children.

The trick: the tree lives in an ordinary array. There are no pointers. We number the nodes top to bottom, left to right, starting at 0, and store them in a plain array in that order:



The index arithmetic is the whole secret:

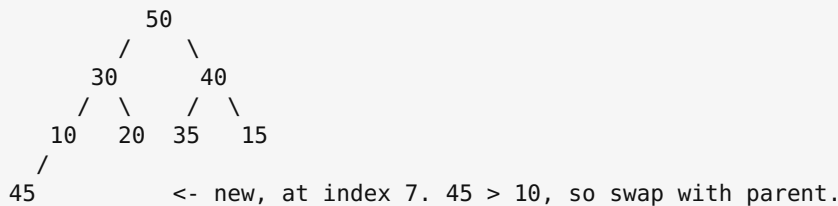
- the children of index i are at $2*i + 1$ and $2*i + 2$,
- the parent of index i is at $(i - 1) / 2$ (integer division, so the remainder is dropped).

Check it on the picture. Children of index 1 (value 30): $2*1+1 = 3$ and $2*1+2 = 4$, which hold 10 and 20. Correct. Children of index 2 (value 40): indices 5 and 6, values 35 and 15. Correct. Parent of index 5 (value 35): $(5-1)/2 = 2$, value 40. Correct. Parent of index 4: $(4-1)/2 = 3/2 = 1$ (integer division), value 30. Correct.

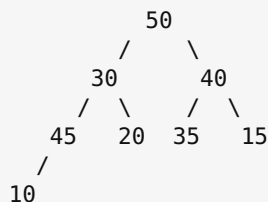
So "go to my parent" is one subtraction and one division, and "go to my left child" is one multiply and one add. That is why a heap is so fast: it is an array with a little arithmetic on top.

Push: put a new number in (sift up). Add the new number at the very end of the array (the next free spot in the bottom row), then let it climb: while it is bigger than its parent, swap with the parent. This is called *sift up*.

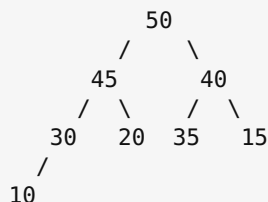
Push 45 into the heap above. Step 1: append at index 7. Its parent is $(7-1)/2 = 3$ (value 10).



Step 2: after the swap, 45 is at index 3. Its parent is $(3-1)/2 = 1$ (value 30). $45 > 30$, swap again.



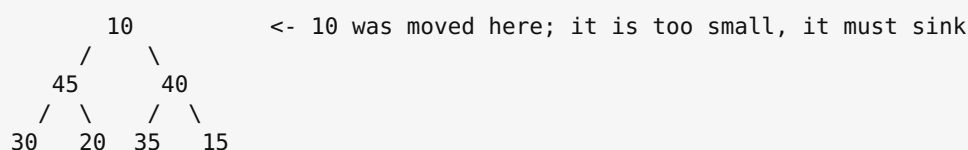
Step 3: 45 is at index 1. Its parent is index 0 (value 50). $45 < 50$, so stop.



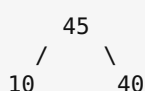
The array is now `[50, 45, 40, 30, 20, 35, 15, 10]`. Check the rule at every node: it holds.

Pop: take the biggest off (sift down). You want to remove the root. But removing index 0 would leave a hole at the top of the array. So: copy the *last* element into the root's slot, shrink the array by one, and let that element sink: while it is smaller than the bigger of its two children, swap with that bigger child. This is *sift down*.

Pop from the heap we just built. Step 1: the last element is 10 (index 7). Move it to index 0, drop the last slot. 50 is the answer we return.



Step 2: children of index 0 are 45 and 40. The bigger is 45. $10 < 45$, swap.



```

      /  \    /  \
    30   20 35   15

```

Step 3: 10 is at index 1. Children are 30 and 20. Bigger is 30. $10 < 30$, swap.

```

      45
     /  \
    30   40
   /  \  /  \
  10  20 35  15

```

Step 4: 10 is at index 3. Its children would be at 7 and 8, which do not exist. Stop. The array is [45, 30, 40, 10, 20, 35, 15] and 45, the second biggest, is on top. Exactly what we want.

Why push and pop cost $\log n$. Look at how the levels grow: level 0 has 1 node, level 1 has 2, level 2 has 4, level 3 has 8. Each level doubles. So a heap with n nodes has about $\log_2(n)$ levels (7 nodes: 3 levels; 8 nodes: 4 levels; 1000 nodes: 10 levels; a million nodes: 20 levels). Sift up moves at most one level per swap and sift down moves at most one level per swap, so each is at most $\log_2(n)$ swaps. A million elements, 20 swaps. That is what " $O(\log n)$ " means here: the height of the tree.

The Python you would have written. Python's `heapq` is a heap on a list, but it keeps the *smallest* on top (a min-heap). For "biggest on top" people push negatives.

```

import heapq
h = []
for x in [50, 30, 40, 10, 20, 35, 15]:
    heapq.heappush(h, x)      # smallest on top
print(h[0])                  # 10
print(heapq.heappop(h))      # 10
print(h[0])                  # 15

```

The same thing in C++. `std::priority_queue` is the heap. The biggest is on top by default.

```

#include <queue>           // std::priority_queue lives here
#include <functional>      // std::greater

std::priority_queue<int> pq;           // biggest on top (a max-heap)
pq.push(30);
pq.push(50);
pq.push(40);
int biggest = pq.top();                // 50: look at the top
pq.pop();                             // remove it; now 40 is on top
bool e = pq.empty();                  // false

```

- `std::priority_queue<int> pq;` says "a heap of ints, biggest on top". The name is "priority queue" because the item with the highest priority always comes out first.
- `push(x)` is `heapq.heappush`. `top()` is `h[0]`. `pop()` removes the top and, as with every C++ container, returns nothing; look with `top()` first, then `pop()`.
- `empty()` and `size()` are the same as on `std::queue`.

For **smallest on top** you add two more things inside the angle brackets:

```

std::priority_queue<int, std::vector<int>, std::greater<int>> minq; // smallest on top
minq.push(30); minq.push(50); minq.push(10);
int smallest = minq.top();           // 10

```

- The second thing, `std::vector<int>`, is the array the heap lives in. You almost always write exactly this.
- The third thing, `std::greater<int>`, is the comparison the heap uses. The default is `std::less<int>` and gives "biggest on top". `std::greater<int>` flips it to "smallest on top". Memorise this line as one unit; everyone types it the same way.

A heap of pairs. Very often you want to sort by one number but carry another along, for example (distance, id). Put the number you want to sort by *first* in the pair. Pairs compare by the first element, and only look at the second on a tie.

```
using P = std::pair<double, int>; // (distance, id)
std::priority_queue<P, std::vector<P>, std::greater<P>> pq; // smallest distance on top
pq.push({2.5, 7});
pq.push({0.5, 3});
pq.push({1.0, 9});
int nearestId = pq.top().second; // 3
```

- `using P = std::pair<double, int>;` gives the long type a short name so the next line is readable. It is like a Python alias `P = tuple`.
- `.first` and `.second` are the two halves of a pair. `{0.5, 3}` builds one.

(You can skip this on a first read.) `top()` on an empty priority queue is undefined behaviour, not an exception: check `empty()` first. `top()` also hands you a `const` reference, so you cannot change the top element in place; pop it, change it, push it back. If you want your own ordering on a struct, you write a small function (a lambda) that answers "should a come out *after* b?" and pass it as the third thing; `00_heap_tree_toolkit.cpp` shows the exact spelling with `decltype(cmp)` because it is fiddly to remember.

Try it. In `00_heap_tree_toolkit.cpp`, the `MaxHeap` struct is the sift-up and sift-down above in about 30 lines. Change `data` in `main` to `{50, 30, 40, 10, 20, 35, 15, 45}` and write down what `h.a` looks like after all the pushes before you run it. (Hint: the trace above.)

In your own words: "A heap is a tree stored in an array where every parent beats its children, so the biggest is always at index 0. Children of i are at $2i+1$ and $2i+2$. Push appends and sifts up, pop moves the last element to the root and sifts down; both cost the height, $\log n$. In C++ it is `std::priority_queue`, biggest on top by default, `std::greater` for smallest."

A2. The "top k" patterns: a heap of size k, and nth_element

Many questions ask for "the k largest", "the k closest", "the kth biggest". Two tools.

Tool 1: a small heap of size k. Picture a VIP list with k seats. Newcomers only get in if they beat the *worst* VIP currently seated, and when they get in, the worst VIP leaves. At the end the k seats hold the k best people, and the person in the worst seat is exactly the "kth best".

The heap trick: to find the k *largest*, keep a *min*-heap (smallest on top) of size k. That sounds backwards, but the top of a min-heap is the *worst* of the VIPs, which is exactly the one you need to compare newcomers against and throw out.

Find the 3 largest of `[7, 2, 9, 4, 8]`, so $k = 3$. The heap is drawn as a set with its top underlined.

see 7:	heap {7}	fewer than 3 seats used, just add
see 2:	heap {2, 7}	top is 2
see 9:	heap {2, 7, 9}	3 seats used, top is 2
see 4:	push 4 -> {2, 4, 7, 9}	4 people, one too many
	pop top -> {4, 7, 9}	2 was the worst, 2 leaves
see 8:	push 8 -> {4, 7, 8, 9}	one too many again
	pop top -> {7, 8, 9}	4 leaves

The heap holds `{7, 8, 9}`: the 3 largest. Its top, 7, is the 3rd largest. Each push and pop costs $\log k$, not $\log n$, because the heap never has more than $k+1$ things in it. For n numbers that is $n \log k$ work, and you only ever hold k numbers in memory. This is the tool when numbers arrive one at a time from a sensor and you cannot store them all.

The rule to remember: **k largest needs a min-heap, k smallest needs a max-heap**. The top is always "the worst of the best" so it can be thrown out.

In Python:

```
import heapq
def k_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return sorted(h)
# throw out the smallest of the k+1
# [7, 8, 9] for [7, 2, 9, 4, 8], k = 3
```

Tool 2: `std::nth_element`, "partially sort just enough to know who is at position k". If you sorted the whole array, position $k-1$ (counting from 0) would hold the k th smallest. But sorting everything is wasted work; you only care about one position. `std::nth_element` reorders the array *just enough* so that the element at position $k-1$ is the one that would be there after a full sort, everything before it is smaller or equal, and everything after it is bigger or equal. Nothing else is sorted.

```
before: [9, 2, 7, 4, 5, 1, 8, 3, 6]      want the 3rd smallest, position 2
after:  [1, 2, 3, ?, ?, ?, ?, ?, ?]    position 2 holds 3 (the true 3rd smallest)
      ^^^ these are <= 3      ^^^^^^^^^^^^^ these are >= 3, in no particular order
```

```
#include <algorithm> // std::nth_element
std::vector<int> v = {9, 2, 7, 4, 5, 1, 8, 3, 6};
int k = 3;
std::nth_element(v.begin(), v.begin() + (k - 1), v.end());
int third = v[k - 1]; // 3
```

- `std::nth_element(first, nth, last)` takes three positions: where the range starts, the one position you care about, and one past the end. `v.begin() + (k - 1)` is "position $k-1$ ".
- It changes the order of `v`. If you need the original order later, copy first.
- On average it does a small constant times n steps (it is a "quickselect": it partitions like quicksort but only recurses into the half that contains position k). Python has no built-in for this; `sorted(v)[k-1]` is the usual substitute and costs $n \log n$.

When to use which.

Situation	Use	Why
Numbers arrive one at a time (a stream), you cannot hold them all	heap of size k	Only k things in memory, $\log k$ per number
Everything is in a vector and you may reorder it	<code>std::nth_element</code>	Average n steps, no extra memory
k is close to n	just <code>std::sort</code>	The heap would be nearly $n \log n$ anyway
" k most frequent"	count into a hash map first, then either tool on the counts	The counting is the real work

In your own words: "For the k largest I keep a min-heap of size k ; its top is the worst of the best, so a newcomer only gets in if it beats that. That is $n \log k$ time and k memory, and it works on a stream. If all the data is in a vector I can reorder, `std::nth_element` puts the k th element in its sorted position in linear time on average."

A3. Monotonic deque and monotonic stack

Both of these are lists where you throw away things that can never be the answer again. The word "monotonic" just means the values in the list only go one way (always decreasing from front to back, say). That property is what makes them fast.

The problem the deque solves: sliding window maximum. You have a list of numbers and a window of width k that slides along it one step at a time. For every position of the window you want the biggest number inside it.

```
nums = [2, 5, 1, 4, 3], k = 3

[2, 5, 1] 4 3    -> max 5
2 [5, 1, 4] 3    -> max 5
2 5 [1, 4, 3]    -> max 4      answer: [5, 5, 4]
```

The slow way is to look at all k numbers every time: n times k work. The fast way keeps a short list of *candidates* and does a constant amount of work per number.

The picture. Imagine people standing in a line and the window is a viewing box that slides over them. You want the tallest person in the box. Here is the key thought: *a shorter person standing behind a taller one can never be the tallest in the window while the taller one is there*. The taller one entered later, so it leaves the window later. Until it leaves, the shorter person in front of it is useless. So the moment a new tall person arrives, we throw out every shorter person who arrived before them.

That leaves a list of people that is tallest at the front and shorter towards the back: a *monotonic decreasing* list. The front is always the answer. And when the front person walks out of the window (their index is too old), we drop them from the front.

We store *indices* (positions), not values, because we need the index to know when someone has left the window. We can always look up the value with `nums[index]`.

The trace. Deque shown as `[index(value), ...]`, front on the left. The window covers indices $i-k+1 \dots i$.

<code>i=0, value 2:</code>	deque empty, push 0.	deque [0(2)]	window not full yet
<code>i=1, value 5:</code>	back is 0(2), $2 \leq 5$, throw it out. push 1.	deque [1(5)]	window not full yet
<code>i=2, value 1:</code>	back is 1(5), $5 > 1$, keep it. push 2.	deque [1(5), 2(1)]	window is 0..2, answer <code>nums[1] = 5</code>
<code>i=3, value 4:</code>	front 1 still in window ($1 > 3-3=0$). back is 2(1), $1 \leq 4$, throw out. back is 1(5), $5 > 4$, keep. push 3.	deque [1(5), 3(4)]	window 1..3, answer 5
<code>i=4, value 3:</code>	front is 1, and $1 \leq 4-3 = 1$: too old, drop from front. back is 3(4), $4 > 3$, keep. push 4.	deque [3(4)] deque [3(4), 4(3)]	window 2..4, answer 4

Answers: `[5, 5, 4]`. Every index was pushed once and thrown out at most once, so the total work is about $2n$, no matter how the inner "throw out" loop looks.

Why a deque and not a stack or queue. We drop from the front (too old) and from the back (too short), and push at the back. A container that is fast at both ends is a *double-ended queue*, "deque" for short. Python has `collections.deque`; C++ has `std::deque` with `push_back`, `pop_back`, `front`, `pop_front`, `back`.

The Python and the C++ for this are Problem 4 below. Read the trace again; then the code will look obvious.

The stack version: next greater element. A close cousin. For each number, what is the first number to its right that is bigger? If there is none, say -1.

```
nums = [2, 1, 4, 3]
answer = [4, 4, -1, -1]      (2 -> 4, 1 -> 4, 4 -> nothing, 3 -> nothing)
```

Same thought, but now nothing ever "leaves a window", so only the back end matters, which makes it a stack. Walk left to right. Keep a stack of indices whose values are still *waiting* for their bigger number, tallest at the bottom. When a new number arrives, every waiting number smaller than it has just found its answer: pop them and write the answer. Then the new number starts waiting.

i=0, value 2:	stack empty. push 0.	stack [0(2)]
i=1, value 1:	top 2 < 1? no. push 1.	stack [0(2), 1(1)]
i=2, value 4:	top 1 < 4: answer[1] = 4, pop.	
	top 2 < 4: answer[0] = 4, pop.	
	push 2.	stack [2(4)]
i=3, value 3:	top 4 < 3? no. push 3.	stack [2(4), 3(3)]
end:	2 and 3 are still waiting: answer stays -1.	

The Python:

```
def next_greater(nums):
    ans = [-1] * len(nums)
    stack = [] # indices, values decreasing bottom to top
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            ans[stack.pop()] = x
        stack.append(i)
    return ans # [4, 4, -1, -1] for [2, 1, 4, 3]
```

The C++:

```
#include <vector>
std::vector<int> nextGreater(const std::vector<int>& a) {
    std::vector<int> ans(a.size(), -1); // same size as a, all -1
    std::vector<int> st; // the stack: indices, values decreasing bottom to top
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            ans[st.back()] = a[i]; // the waiting index just found its bigger number
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}
```

- `std::vector<int> ans(a.size(), -1)` is Python's `[-1] * len(nums)`: "a vector of that many copies of -1".
- We use a `std::vector` as the stack: `push_back` is `append`, `back()` is `stack[-1]`, `pop_back()` is `pop()`. `std::stack` would also work; `vector` is just as easy and lets you look inside while debugging.
- `while (!st.empty() && ...)`: the `!st.empty()` check must come first. `st.back()` on an empty vector is undefined behaviour in C++, where Python would raise `IndexError`.

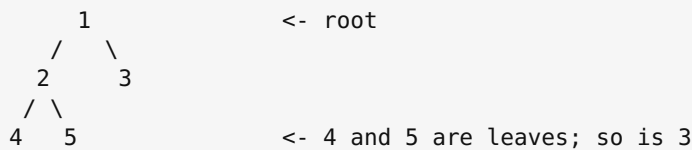
Trapping Rain Water (Problem 8) has a monotonic stack version, and so do "largest rectangle in a histogram" and "stock span". If a problem says "for each element, the nearest bigger/smaller one to the left/right", it is this.

Try it. Change `<` to `<=` in the `while` of `nextGreater` and run it on `[3, 3, 5]`. Predict the answer for the first 3 before you run it.

In your own words: "For a sliding window maximum I keep a deque of indices with values decreasing from front to back. A new value throws out every smaller value behind it, because those are older and smaller and can never win. The front is the window maximum, and I drop it when its index falls out of the window. Each index is pushed and popped once, so it is linear. Next-greater-element is the same idea with a stack."

A4. Binary trees in C++

The picture. A binary tree is a family tree where every node has at most two children, called left and right. The one at the top is the *root*. A node with no children is a *leaf*.



The node. In Python you would have had a small class with `val`, `left` and `right`. In C++ it is a `struct` with the same three fields, and it is the exact struct LeetCode gives you:

```

struct TreeNode {
    int val;           // the number stored in this node
    TreeNode* left;    // an arrow to the left child, or nullptr if there is none
    TreeNode* right;   // an arrow to the right child, or nullptr if there is none
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};
  
```

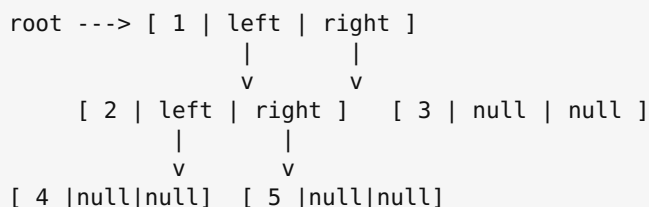
- `TreeNode* left;` is a *pointer*: the address of another node, like a house address written on a sticky note. An arrow in the drawing is a pointer in the code. `nullptr` means "no address, there is nothing there"; it is Python's `None`.
- The three lines starting `TreeNode(...)` are *constructors*: recipes for making a node. `: val(x), left(nullptr), right(nullptr)` fills the three fields. `new TreeNode(4)` makes a node holding 4 with no children and gives you its address.
- To follow an arrow you write `node->left` (Python `node.left`). The `->` means "go to the address stored in this pointer, then take that field".
- `if (node == nullptr)` or simply `if (!node)` is Python's `if node is None`.

Building the 5-node tree by hand:

```

TreeNode* root = new TreeNode(1);
root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);
  
```

In memory that is five separate boxes, joined by arrows:



The three traversals: when do you say the node's name? Walking a tree means visiting every node. The recursive shape is always "visit left subtree, visit right subtree", and the only choice is *when* you say the node's own name: before the children, between them, or after.

```

pre-order:  say ME, then left, then right  -> 1 2 4 5 3
in-order:   left, then say ME, then right  -> 4 2 5 1 3
post-order: left, then right, then say ME  -> 4 5 2 3 1
  
```

Trace in-order on the 5-node tree. Start at 1: first do the left subtree (rooted at 2). At 2: first do its left subtree (4). At 4: no left child, say **4**, no right child, done. Back at 2: say **2**. Now 2's right subtree (5): say **5**. Back at 1: say **1**. Then 1's right subtree (3): say **3**. Result 4 2 5 1 3.

Now pre-order: at 1 say **1** first, then go left: at 2 say **2**, go left: say **4**, back, go right: say **5**, back to 1, go right: say **3**. Result 1 2 4 5 3.

Post-order: at 1, do left first: at 2, do left first: at 4, nothing below, say **4**. Back at 2, do right: say **5**. Now 2 has finished both children, say **2**. Back at 1, do right: say **3**. Now 1 has finished both, say **1**. Result 4 5 2 3 1.

The Python:

```
def inorder(node, out):
    if node is None:
        return
    inorder(node.left, out)
    out.append(node.val)           # "say my name" in the middle
    inorder(node.right, out)
```

The C++:

```
void inorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    inorder(root->left, out);
    out.push_back(root->val);    // move this line to the top for pre-order, to the bottom for post-order
    inorder(root->right, out);
}
```

- `std::vector<int>& out`: the `&` means every recursive call appends to the *same* vector. Without it each call would get its own copy and the result would be empty. Same rule as the `visited` table in chapter 03.

Which one when? **Post-order** is the one you will use most: "finish the children, then use their results" is how you compute a height, delete a tree, or find a lowest common ancestor (A5). **Pre-order** is for copying or printing a tree. **In-order** on a *binary search tree* (left smaller, right bigger) gives the values in sorted order.

Level order: the queue and the ring loop. The fourth way to walk a tree is by depth: the root, then both its children, then all the grandchildren. That is BFS from chapter 03 with a queue, and it uses the same "snapshot the queue size, process exactly that many" loop that gave you the rings in Rotting Oranges.

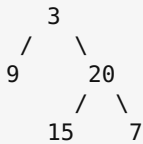
```
level 0: [1]
level 1: [2, 3]
level 2: [4, 5]
```

```
#include <queue>
std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> out;
    if (root == nullptr) return out;
    std::queue<TreeNode*> q;           // a queue of pointers to nodes
    q.push(root);
    while (!q.empty()) {
        int levelSize = static_cast<int>(q.size()); // how many nodes are on THIS level
        std::vector<int> level;
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* cur = q.front();
            q.pop();
            level.push_back(cur->val);
            if (cur->left) q.push(cur->left);    // children go to the back: next level
            if (cur->right) q.push(cur->right);
        }
        out.push_back(level);
    }
}
```

```
    return out;
}
```

- `std::queue<TreeNode*>`: the queue holds addresses of nodes, not copies of nodes.
- `levelSize` is taken *before* the inner loop, because pushing children makes the queue grow while we are inside it. Read chapter 03 A2, Variant 1, if this is not clear.
- `if (cur->left)` is short for `if (cur->left != nullptr)`.

Building a tree from a list, for tests. LeetCode writes a tree as a list in level order, with `null` for a missing child: `[3, 9, 20, null, null, 15, 7]` means



Read it like this: 3 is the root. The next two entries, 9 and 20, are its children. The next two, `null, null`, are 9's children (none). The next two, 15 and 7, are 20's children. The builder is a small BFS: keep a queue of parents that still need children, and hand out two list entries to each parent in turn. In the test files `-1` stands for `null` and the function is called `buildTree`; you will see it in every tree problem's file. You do not need to memorise it, only to know how to read the list.

Freeing nodes. Every `new` should be matched by a `delete`, or the program leaks memory. Deleting a tree is a post-order walk: delete both children first, then yourself. If you deleted yourself first you would lose the arrows to the children and could never reach them.

```
void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;           // children are gone; now this node
}
```

LeetCode does not check for leaks, but the test files in this repo free every tree they build. Get into the habit.

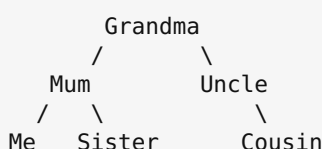
(You can skip this on a first read.) Recursion depth equals tree height. For a balanced tree of a million nodes that is 20 calls, no problem. For a tree that is one long chain (each node has only a right child) it is a million calls and the program crashes with a stack overflow. The fix is an explicit stack: `inorderIterative` in the toolkit file shows the in-order version (push the whole left spine, pop one, visit it, step right, repeat). Learn it after the recursive version is solid.

Try it. In `00_heap_tree_toolkit.cpp`, change the `preorder` function so that the `out.push_back` line is *after* the two recursive calls. Predict the vector it produces for the tree `{3, 9, 20, NONE, NONE, 15, 7}` before running (the assert will fail; that is expected).

In your own words: "A `TreeNode` is a value plus two pointers, left and right, `nullptr` when missing. Pre, in and post-order differ only in when I record the node: before, between, or after its children. Level order is BFS with a queue and the level-size snapshot loop. Deleting is post-order: children first, then delete root."

A5. Lowest common ancestor

The picture. A family tree:



The *lowest common ancestor* (LCA) of two people is the nearest person who is above both of them. For Me and Sister it is Mum. For Me and Cousin it is Grandma. For Mum and Me it is Mum herself: a person counts as their own ancestor in this problem.

The idea: ask both children, report what you found. Give every node one job. Someone hands it two names, p and q, and asks: "look in your subtree; what did you find?" A node answers with one of three things:

- "nothing": neither p nor q is anywhere below me, and I am not one of them;
- "p" (or "q"): I found exactly one of them, and here it is;
- "me": I found *both*, one on each side, so I am the meeting point, the LCA.

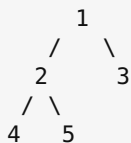
And there is one shortcut: if the node *is* p or q, it answers with itself straight away without looking further down. (If the other one is below it, this node is the answer anyway.)

A node computes its answer by asking its left child and its right child, then combining:

left says	right says	I say	
nothing	nothing	nothing	
p	nothing	p	(pass up whatever the one side found)
nothing	q	q	
p	q	ME	(both found on different sides: I am the LCA)
some node X	nothing	X	(a lower node already decided; pass it up unchanged)

Because a node answers only *after* both children have answered, this is a post-order walk from A4.

The trace. Use the 5-node tree from A4 and look for the LCA of 4 and 5.



```

ask 1: not 4 or 5. ask my children.
  ask 2: not 4 or 5. ask my children.
    ask 4: I AM 4. answer "4".
    ask 5: I AM 5. answer "5".
    2 hears "4" from the left and "5" from the right: both sides. answer "2" (I am the LCA).
  ask 3: not 4 or 5. ask children: both nullptr, both answer "nothing". answer "nothing".
1 hears "2" from the left and "nothing" from the right. pass up "2".
answer: 2
  
```

Now the LCA of 4 and 3:

```

ask 1: not 4 or 3. ask children.
  ask 2: not them. ask children.
    ask 4: I AM 4. answer "4".
    ask 5: not them; no children; answer "nothing".
    2 hears "4" and "nothing". pass up "4".
  ask 3: I AM 3. answer "3".
1 hears "4" from the left and "3" from the right: both sides. answer "1".
answer: 1
  
```

The Python:

```

def lca(root, p, q):
    if root is None or root is p or root is q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        # "nothing", or "me" if I am one of them
  
```

```

return root                                # found on both sides: I am the LCA
return left if left else right             # pass up whatever one side found

```

Note the `is`, not `==`: we compare *which node*, not what value it holds, because values can repeat. In C++ the same comparison is `root == p` on the pointers (two addresses are equal only if they are the same node). The C++ is Problem 6.

Every node is asked once, so this is one pass over the tree. The recursion goes as deep as the tree is tall.

In your own words: "Each node asks its two children what they found. Nothing from both means nothing. One side found something means pass it up. Both sides found something means I am the lowest common ancestor. A node that is `p` or `q` answers with itself immediately. It is a post-order walk, one pass, linear time."

A6. Design problems and the LRU cache

What a cache is. A cache is a small, fast storage that holds copies of the few things you used most recently, so you do not have to fetch them from the slow place (disk, network, a big computation) again. It has a fixed size, so when it is full and something new arrives, something old must go.

What "least recently used" means: the bookshelf story. You have a shelf with room for 3 books. Every time you use a book, you put it back at the *front* of the shelf. When you need a book that is not on the shelf, you get it from the library and put it at the front; if the shelf was full, the book at the *back* (the one you have not touched for the longest time) goes back to the library. That book is the "least recently used", and this rule is an LRU cache.

start (empty):	[		]			
use A:	[	A		]		
use B:	[	B	A	]		
use C:	[	C	B	A	]	full
use A again:	[	A	C	B	]	A moves to the front, nothing removed
use D (not on shelf):	[	D	A	C	]	shelf was full: B, at the back, is removed
use B (not on shelf):	[	B	D	A	]	C is removed

The cache in Problem 7 has two operations: `get(key)` returns the value stored under that key (or -1 if it is not there) and counts as "using" it, and `put(key, value)` stores a value (counts as using it too), removing the least recently used entry if the cache was full. Both must be fast no matter how big the cache is: a constant number of steps, not "walk along the shelf".

Why we need a hash map AND a linked list. Two questions must be answered quickly:

1. "Is key 7 on the shelf, and where?" Walking along the shelf is slow. A *hash map* (Python `dict`, C++ `std::unordered_map`) answers "where is key 7" in one step.
2. "Move this book to the front, and remove the one at the back." In an array (`std::vector`), moving a middle item to the front means shifting everything in between: slow. In a *doubly linked list*, every book has an arrow to the one before and the one after; moving a book means rewiring four arrows, no shifting. One step.

So we keep both, and they point at each other:

hash map (key -> where on the shelf)	the shelf: a doubly linked list
-----	front
key 1 ----->	[k=1,v=10] <--> [k=2,v=20] <--> [k=3,v=30]
key 2 ----->	^
key 3 ----->	^
	back

Each map entry stores a *bookmark* to that key's node in the list. Given a key, the map takes you straight to the node; from the node, the list lets you unhook it and rehook it at the front. And each list node stores its *key* as well as its value, so that when the back node is removed we know which map entry to delete too.

What `std::list::splice` does. In C++ the doubly linked list is `std::list`, and the bookmark is a `std::list` iterator (a pointer-like object that names one node). The move to the front is one call:

```
#include <list>
#include <unordered_map>

std::list<int> shelf = {1, 2, 3, 4}; // front is 1
std::unordered_map<int, std::list<int>::iterator> where; // key -> bookmark into the list
for (auto it = shelf.begin(); it != shelf.end(); ++it) where[*it] = it;

shelf.splice(shelf.begin(), shelf, where[3]); // move the node holding 3 to the front
// shelf is now {3, 1, 2, 4}; where[3] still points at the node holding 3
```

- `std::list<int>::iterator` is the type of a bookmark into a `std::list<int>`. `*it` is the value at the bookmark, `shelf.begin()` is the bookmark of the front node.
- `shelf.splice(pos, shelf, it)` means "unhook the node at `it` and rehook it just before `pos`". Nothing is copied and no memory is allocated; only the arrows change. That is why it is constant time.
- The stored bookmark `where[3]` is still valid after the splice, because the *node* did not move in memory, only its neighbours changed. This is the property that makes the whole design work. A `std::vector` would not give you this: adding to a vector can move every element to a new place in memory and every stored position becomes garbage.
- `shelf.erase(it)` removes one node, `shelf.pop_back()` removes the back, `shelf.back()` looks at the back, `shelf.emplace_front(key, value)` builds a new node at the front.

Python hides all of this in `OrderedDict.move_to_end`; C++ makes you build it, which is why the problem is popular.

How to attack a design problem. Write down the operations and what each returns, including the odd cases (put on a key that already exists, capacity 1). Then write down how fast each operation must be. Only then pick the containers; the speed requirement is what chooses them.

In your own words: "An LRU cache keeps the most recently used items and drops the least recently used when full. The hash map finds a key in one step; the doubly linked list moves a node to the front and drops the back in one step. The map stores list iterators as bookmarks, which stay valid because `std::list::splice` only rewires arrows. Each node also stores its key so eviction can erase the map entry."

A7. Where this shows up in robots

- **The heap is the planner's to-do list.** Dijkstra and A* (chapter 03, Part C) keep the cells still to be explored in a `std::priority_queue` of `(cost, cell)` pairs and always pop the cheapest. The pair-comparison rule from A1 is what makes ties break sensibly.
- **k nearest points.** Finding the k closest points in a laser scan or a point cloud is a bounded heap of size k (A2) exactly like Problem 2. Libraries like PCL use a tree to decide which points to *offer*, but the "keep the k best" part is this heap.
- **Sliding window filters.** "The biggest range reading in the last k samples" for spike rejection, computed at 1 kHz, is the monotonic deque of A3: constant work per sample.
- **Tile and submap caches.** A map viewer or a localiser keeps only the last N map tiles in RAM; that is an LRU cache (A6).
- **Trees.** Collision-checking hierarchies, octrees for 3D occupancy, and the tf frame tree in ROS are all trees; level order and post-order walks are how you print, prune, or compute bounds on them, and `LookupTransform` between two frames is an LCA query (A5).

A8. The toolkit file

code/05-heaps-trees-design/00_heap_tree_toolkit.cpp has everything from A1 to A6 in one file with asserts: a hand-written max-heap, the standard heap functions on a vector, `std::priority_queue` in its three spellings, `nth_element`, `TreeNode` with `buildTree` and the traversals, and `std::list::splice` with a map of bookmarks.

```
// Chapter 05 toolkit demo: hand-rolled binary heap, std heap algorithms,
// nth_element, TreeNode build-from-vector + traversals, std::list::splice.
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <list>
#include <queue>
#include <string>
#include <unordered_map>
#include <vector>

// ---- 1. A max-heap by hand on a std::vector -----
// Layout: node i has children 2i+1 and 2i+2, parent (i-1)/2. No pointers, no
// allocation per node, cache-friendly – that is why heaps live in arrays.
struct MaxHeap {
    std::vector<int> a;

    void push(int x) {
        a.push_back(x);
        siftUp(static_cast<int>(a.size()) - 1);
    }
    int top() const { return a.front(); }
    void pop() {
        std::swap(a.front(), a.back()); // move last leaf to the root ...
        a.pop_back();
        if (!a.empty()) siftDown(0);    // ... and let it sink to its place
    }
    bool empty() const { return a.empty(); }

    void siftUp(int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (a[p] >= a[i]) break;    // heap property restored
            std::swap(a[p], a[i]);
            i = p;
        }
    }
    void siftDown(int i) {
        int n = static_cast<int>(a.size());
        while (true) {
            int l = 2 * i + 1, r = 2 * i + 2, best = i;
            if (l < n && a[l] > a[best]) best = l;
            if (r < n && a[r] > a[best]) best = r;
            if (best == i) break;
            std::swap(a[i], a[best]);
            i = best;
        }
    }
};

// ---- 2. TreeNode as LeetCode defines it + level-order builder -----
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};
```

```

// Builds from LeetCode's level-order encoding; NONE marks a missing child.
// Children are only queued for non-null nodes, exactly like LeetCode's format.
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);
    int i = 1;
    int n = static_cast<int>(vals.size());
    while (!q.empty() && i < n) {
        TreeNode* cur = q.front();
        q.pop();
        if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
        ++i;
        if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
        ++i;
    }
    return root;
}

void deleteTree(TreeNode* root) {           // post-order: children first, then self
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

void inorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    inorder(root->left, out);
    out.push_back(root->val);
    inorder(root->right, out);
}

void preorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    out.push_back(root->val);
    preorder(root->left, out);
    preorder(root->right, out);
}

// Inorder without recursion: the explicit stack replaces the call stack.
std::vector<int> inorderIterative(TreeNode* root) {
    std::vector<int> out;
    std::vector<TreeNode*> stack;
    TreeNode* cur = root;
    while (cur != nullptr || !stack.empty()) {
        while (cur != nullptr) { stack.push_back(cur); cur = cur->left; }
        cur = stack.back();
        stack.pop_back();
        out.push_back(cur->val);
        cur = cur->right;
    }
    return out;
}

int main() {
    // ---- hand-rolled heap vs std::sort ----
    {
        std::vector<int> data = {5, 1, 9, 3, 7, 3, 8, 0, -2, 6};
        MaxHeap h;
        for (int x : data) h.push(x);
        std::vector<int> popped;
        while (!h.empty()) { popped.push_back(h.top()); h.pop(); }
        std::vector<int> sorted = data;
        std::sort(sorted.begin(), sorted.end(), std::greater<int>());
        assert(popped == sorted);
    }

    // ---- std::make_heap / push_heap / pop_heap on a vector ----
    {
        std::vector<int> v = {4, 10, 3, 5, 1};
        std::make_heap(v.begin(), v.end());           // O(n) build, max-heap by default
    }
}

```

```

assert(v.front() == 10);
v.push_back(42);
std::push_heap(v.begin(), v.end());           // sift the new back() up
assert(v.front() == 42);
std::pop_heap(v.begin(), v.end());             // moves max to back(), re-heaps [begin, end-1)
assert(v.back() == 42);
v.pop_back();
assert(v.front() == 10);
// min-heap: pass std::greater<> to every call
std::make_heap(v.begin(), v.end(), std::greater<>());
assert(v.front() == 1);
}

// ---- std::priority_queue: max by default, min via std::greater, custom via lambda ----
{
    std::priority_queue<int> maxq;
    for (int x : {3, 1, 4, 1, 5}) maxq.push(x);
    assert(maxq.top() == 5);

    std::priority_queue<int, std::vector<int>, std::greater<int>> minq;
    for (int x : {3, 1, 4, 1, 5}) minq.push(x);
    assert(minq.top() == 1);

    // pairs compare lexicographically: (dist, id) - smallest dist on top
    using P = std::pair<double, int>;
    std::priority_queue<P, std::vector<P>, std::greater<P>> pq;
    pq.push({2.5, 7}); pq.push({0.5, 3}); pq.push({1.0, 9});
    assert(pq.top().second == 3);

    // custom comparator with a lambda: "a comes AFTER b" == a has lower priority
    struct Task { int prio; std::string name; };
    auto cmp = [](const Task& a, const Task& b) { return a.prio < b.prio; }; // max-heap on prio
    std::priority_queue<Task, std::vector<Task>, decltype(cmp)> tasks(cmp);
    tasks.push({2, "plan"}); tasks.push({9, "estop"}); tasks.push({5, "localise"});
    assert(tasks.top().name == "estop");
}

// ---- nth_element: O(n) average selection ----
{
    std::vector<int> v = {9, 2, 7, 4, 5, 1, 8, 3, 6};
    int k = 3; // 3rd smallest (0-based index 2)
    std::nth_element(v.begin(), v.begin() + (k - 1), v.end());
    assert(v[k - 1] == 3);
    // everything left of the pivot is <= it, right is >= it, but NOT sorted
    for (int i = 0; i < k - 1; ++i) assert(v[i] <= v[k - 1]);
    for (int i = k; i < static_cast<int>(v.size()); ++i) assert(v[i] >= v[k - 1]);
}

// ---- TreeNode build + traversals ----
{
    // tree: 3 -> (9, 20), 20 -> (15, 7)
    TreeNode* root = buildTree({3, 9, 20, NONE, NONE, 15, 7});
    std::vector<int> in, pre;
    inorder(root, in);
    preorder(root, pre);
    assert(in == std::vector<int>({9, 3, 15, 20, 7}));
    assert(pre == std::vector<int>({3, 9, 20, 15, 7}));
    assert(inorderIterative(root) == in);
    deleteTree(root);
    assert(buildTree({}) == nullptr);
}

// ---- std::list::splice + iterator stability (the LRU building block) ----
{
    std::list<int> l = {1, 2, 3, 4};
    std::unordered_map<int, std::list<int>::iterator> where;
    for (auto it = l.begin(); it != l.end(); ++it) where[*it] = it;
    // move node "3" to the front in O(1): no allocation, no copy, iterators stay valid
    l.splice(l.begin(), l, where[3]);
    assert(l == std::list<int>({3, 1, 2, 4}));
}

```

```

    assert(*where[3] == 3 && where[3] == l.begin()); // the stored iterator still points at 3
    l.erase(where[2]);
    where.erase(2);
    assert(l == std::list<int>({3, 1, 4}));
}

std::cout << "OK 00_heap_tree_toolkit.cpp\n";
return 0;
}

```

What to look at:

- `MaxHeap::siftUp` and `MaxHeap::siftDown`: the two loops from A1. Match each line to the trace of push 45 and pop.
- `std::make_heap`, `std::push_heap`, `std::pop_heap`: the same heap as `priority_queue` but on a vector you can still look inside. `pop_heap` moves the biggest to the *back* of the vector and leaves it to you to `pop_back()`.
- The three `priority_queue` declarations in `main`: biggest on top, smallest on top, and a custom comparison with a lambda.
- `nth_element` and the two loops after it that check "everything before is smaller or equal, everything after is bigger or equal".
- `buildTree` for `{3, 9, 20, NONE, NONE, 15, 7}` and the asserted in-order and pre-order results. Draw the tree and check them.
- The `splice` block: after the move, `where[3]` still points at the right node.

Compile and run:

```

cd code
g++ -std=c++17 -Wall -Wextra -O2 05-heaps-trees-design/00_heap_tree_toolkit.cpp -o toolkit && ./toolkit

```

Part B — The problems

Same shape as chapter 03: the full problem with a drawing, how to do it by hand, the Python, the C++, a walk-through with real values, then the follow-ups. Read the statement, close the file, and try it in C++ for 25 minutes before reading the rest.

1. Kth Largest Element in an Array (LeetCode 215, medium)

The problem in plain words. You are given a list of whole numbers (not sorted, possibly with repeats) and a number k . Return the k th largest number. "kth largest" counts repeats: in `[5, 5, 4]` the 2nd largest is 5, not 4.

```

nums = [3, 2, 1, 5, 6, 4], k = 2

sorted biggest first:  6  5  4  3  2  1
                     ^  ^
                   1st 2nd          -> answer 5

```

k is always between 1 and the length of the list. The list has up to 100,000 numbers, and $k = 1$ means "the maximum", $k = \text{length}$ means "the minimum".

By hand. For a short list you sort it and count k from the big end. For a long list you would rather keep a short list of the k biggest seen so far, and for each new number ask "is it bigger than the smallest one on my short list?" If yes, it replaces that one. At the end the smallest on the short list is the answer. That is the VIP list from A2.

The idea. Two ways, both in the file.

1. A min-heap of size k (A2). Push every number; whenever the heap has more than k numbers, pop the smallest. At the end the top is the k th largest.
2. `std::nth_element` (A2). The k th largest is the element at position $n - k$ in ascending order. Ask `nth_element` to put the right element at that position.

The Python you would have written.

```
import heapq

def findKthLargest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

In C++.

```
// LeetCode 215. Kth Largest Element in an Array
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <vector>

// Approach 1: min-heap of size k. The heap holds the k largest seen so far;
// its top is the smallest of those, i.e. the answer. O(n log k) time, O(k) space.
int findKthLargest(std::vector<int>& nums, int k) {
    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
    for (int x : nums) {
        minHeap.push(x);
        if (static_cast<int>(minHeap.size()) > k) minHeap.pop(); // evict the smallest
    }
    return minHeap.top();
}

// Approach 2: quickselect via std::nth_element. O(n) average, O(n^2) worst
// (libstdc++ uses introselect so in practice it never degrades). Reorders nums.
int findKthLargestSelect(std::vector<int>& nums, int k) {
    // kth largest == element at index n-k once sorted ascending
    auto target = nums.begin() + (static_cast<int>(nums.size()) - k);
    std::nth_element(nums.begin(), target, nums.end());
    return *target;
}

int main() {
    auto check = [](std::vector<int> nums, int k, int expected) {
        std::vector<int> copy = nums; // heap version does not modify, select does
        assert(findKthLargest(nums, k) == expected);
        assert(findKthLargestSelect(copy, k) == expected);
    };
    check({3, 2, 1, 5, 6, 4}, 2, 5);
    check({3, 2, 3, 1, 2, 4, 5, 5, 6}, 4, 4);
    check({1}, 1, 1); // single element
    check({2, 1}, 2, 1); // k == n -> the minimum
    check({-1, -1, -1}, 2, -1); // duplicates and negatives
    std::cout << "OK 01_kth_largest.cpp\n";
    return 0;
}
```

What is new:

- `std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;` is the "smallest on top" heap from A1; it replaces `h = []` plus the `heapq` calls.
- `static_cast<int>(minHeap.size()) > k` is `len(h) > k`. `size()` gives an unsigned number; converting it to `int` before comparing with the `int` `k` keeps the compiler quiet.
- `auto target = nums.begin() + (n - k);` is a position (an iterator) inside the vector, "the slot at index `n - k`". `*target` reads the value at that position, like `nums[n - k]`.
- `std::nth_element(nums.begin(), target, nums.end());` reorders `nums` so that `*target` is the element a full sort would put there. This replaces `sorted(nums)[n - k]` but is linear on average.
- In `main`, check is a *lambda*: a small function written inline, like a Python `def` inside `main`. It copies `nums` so the `nth_element` version can reorder it without affecting the heap version.

Walk through the example. `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`. Heap shown with its top first.

```
3: push -> {3}
2: push -> {2, 3}
1: push -> {1, 2, 3}    size 3 > 2, pop 1    -> {2, 3}
5: push -> {2, 3, 5}    pop 2              -> {3, 5}
6: push -> {3, 5, 6}    pop 3              -> {5, 6}
4: push -> {4, 5, 6}    pop 4              -> {5, 6}
top = 5
```

With `nth_element`: `n = 6`, `k = 2`, so `target` is index 4. Ascending order is `[1, 2, 3, 4, 5, 6]` and index 4 holds 5. After the call `nums[4] == 5`, the four numbers before it are all `<= 5` and the one after is `>= 5`, in some order.

Complexity. Heap: one pass, each step costs `log k`: `O(n log k)` time, `O(k)` extra memory. `nth_element`: linear on average, `O(n)`, no extra memory, but it reorders the input.

Robotics. "The 95th-percentile range in this lidar scan" or "the `k`th smallest error in a batch of matches" is exactly this. If the batch is already in a vector, `nth_element` is the tool; if readings stream in from the sensor, the size-`k` heap is.

Follow-ups you may get. - *The numbers arrive one at a time and you cannot store them all?* The heap of size `k`; it only ever holds `k` numbers. - *Worst case of quickselect?* Quadratic with unlucky pivots; the standard library's `nth_element` mixes in a fallback so in practice it stays linear. - *Values are small whole numbers in a known range?* Count how many of each (counting sort), then walk down from the top until you have passed `k` of them.

2. K Closest Points to Origin (LeetCode 973, medium)

The problem in plain words. You are given a list of points on a flat plane, each an `[x, y]` pair of whole numbers, and a number `k`. Return the `k` points that are closest to the origin `(0, 0)`, measured by straight-line distance. The answer may be in any order.

```
points = [[3, 3], [5, -1], [-2, 4]], k = 2
```

```

y
4 |      *(-2,4)
3 |      *(3,3)
  |
0 +-----> x
-1 |      *(5,-1)
```

```
distance squared from (0,0):  (3,3) -> 9+9 = 18    (5,-1) -> 25+1 = 26    (-2,4) -> 4+16 = 20
two closest: (3,3) and (-2,4)                                -> answer [[3,3], [-2,4]]
```

Up to 10,000 points, coordinates between -10,000 and 10,000, `k` between 1 and the number of points.

By hand. Work out each point's distance from the origin, then pick the k smallest. One saving: you never need the actual distance, only which is smaller. The distance is $\sqrt{x^2 + y^2}$, and a bigger $x^2 + y^2$ always means a bigger square root, so compare $x^2 + y^2$ and skip the square root.

The idea. The VIP list from A2 again, but now we want the k *smallest* distances, so the heap keeps the *biggest* on top: the worst VIP is the farthest point, and a newcomer gets in only if it is closer than that. Push (distance squared, index of the point) pairs; when the heap exceeds size k , pop the top (the farthest). At the end, the heap holds the k closest.

The Python you would have written.

```
import heapq

def kClosest(points, k):
    h = []                                # max-heap via negative distances
    for i, (x, y) in enumerate(points):
        heapq.heappush(h, (-(x * x + y * y), i))
        if len(h) > k:
            heapq.heappop(h)             # drops the farthest
    return [points[i] for _, i in h]
```

In C++.

```
// LeetCode 973. K Closest Points to Origin
#include <algorithm>
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

// Max-heap of size k keyed on squared distance: the top is the FARTHEST of the
// k best, so a new point only enters if it beats that. sqrt is monotonic, so
// comparing squared distances gives the same order without the float call.
// O(n log k) time, O(k) space.
std::vector<std::vector<int>>> kClosest(std::vector<std::vector<int>>& points, int k) {
    using Entry = std::pair<long long, int>; // (squared distance, index into points)
    std::priority_queue<Entry> maxHeap;      // default std::less -> largest distance on top
    for (int i = 0; i < static_cast<int>(points.size()); ++i) {
        long long x = points[i][0], y = points[i][1];
        maxHeap.push({x * x + y * y, i});
        if (static_cast<int>(maxHeap.size()) > k) maxHeap.pop();
    }
    std::vector<std::vector<int>>> out;
    out.reserve(k);
    while (!maxHeap.empty()) {
        out.push_back(points[maxHeap.top().second]);
        maxHeap.pop();
    }
    return out;
}

// Alternative: nth_element with a distance comparator. O(n) average, reorders points.
std::vector<std::vector<int>>> kClosestSelect(std::vector<std::vector<int>>& points, int k) {
    auto dist2 = [](const std::vector<int>& p) {
        return static_cast<long long>(p[0]) * p[0] + static_cast<long long>(p[1]) * p[1];
    };
    std::nth_element(points.begin(), points.begin() + k, points.end(),
        [&](const std::vector<int>& a, const std::vector<int>& b) { return dist2(a) < dist2(b); });
    return std::vector<std::vector<int>>>(points.begin(), points.begin() + k);
}

// Order of the answer is unspecified, so compare as sorted sets.
static std::vector<std::vector<int>>> sorted(std::vector<std::vector<int>>> v) {
    std::sort(v.begin(), v.end());
    return v;
}

int main() {
    {
        std::vector<std::vector<int>>> pts = {{1, 3}, {-2, 2}};
        auto copy = pts;
        assert(sorted(kClosest(pts, 1)) == sorted({{-2, 2}}));
    }
}
```

```

    assert(sorted(kClosestSelect(copy, 1)) == sorted({{-2, 2}}));
}
{
    std::vector<std::vector<int>> pts = {{3, 3}, {5, -1}, {-2, 4}};
    auto copy = pts;
    assert(sorted(kClosest(pts, 2)) == sorted({{3, 3}, {-2, 4}}));
    assert(sorted(kClosestSelect(copy, 2)) == sorted({{3, 3}, {-2, 4}}));
}
{
    std::vector<std::vector<int>> pts = {{0, 1}, {1, 0}};    // k == n, returns everything
    assert(sorted(kClosest(pts, 2)) == sorted({{0, 1}, {1, 0}}));
}
{
    std::vector<std::vector<int>> pts = {{10000, 10000}};    // squares need 64-bit headroom
    assert(kClosest(pts, 1).size() == 1);
}
std::cout << "OK 02_k_closest_points.cpp\n";
return 0;
}

```

What is new:

- `using Entry = std::pair<long long, int>;` names the pair type (distance squared, index). `long long` is a 64-bit whole number; `10000 * 10000 + 10000 * 10000` fits in a plain `int`, but widening is the habit that saves you when the limits change.
- `std::priority_queue<Entry> maxHeap;` with nothing else in the brackets is "biggest on top", and pairs compare by their first element, the distance. There is no need for the Python minus-sign trick.
- `long long x = points[i][0], y = points[i][1];` reads the two coordinates into 64-bit numbers *before* multiplying, so the multiplication itself is done in 64 bits.
- `out.reserve(k);` tells the vector to make room for `k` items up front. Optional; it avoids a few reallocations.
- `while (!maxHeap.empty()) { out.push_back(points[maxHeap.top().second]); maxHeap.pop(); }` is the Python list comprehension: `.second` is the stored index, and `points[...]` is the point itself.
- The second function, `kClosestSelect`, does the same with `std::nth_element` and a comparison lambda `[&](a, b) { return dist2(a) < dist2(b); }` that says "a comes before b if it is closer". The `[&]` lets the lambda use `dist2` from the enclosing function.

Walk through the example. `points = [[3,3], [5,-1], [-2,4]]`, `k = 2`. Heap shown top first.

```

i=0, (3,3):    dist2 = 18. push (18,0)          -> {(18,0)}
i=1, (5,-1):   dist2 = 26. push (26,1)          -> {(26,1), (18,0)}      size 2, ok
i=2, (-2,4):   dist2 = 20. push (20,2)          -> {(26,1), (20,2), (18,0)}
                size 3 > 2: pop top (26,1)      -> {(20,2), (18,0)}
drain: pop (20,2) -> points[2] = (-2,4);  pop (18,0) -> points[0] = (3,3)
out = [[-2,4], [3,3]]

```

The test compares sorted copies because any order is accepted.

Complexity. One pass, `log k` per point: $O(n \log k)$ time, $O(k)$ memory. The `nth_element` version is $O(n)$ average and reorders the input.

Robotics. This is *k*-nearest-neighbour search on a point cloud with no index at all. Real libraries add a KD-tree so that most points are never even looked at, but the "keep a bounded max-heap of the *k* best while searching" part inside them is exactly this loop, and they compare squared distances for the same reason.

Follow-ups you may get. - *k* is close to *n*? The heap approaches $n \log n$; just sort by distance or use `nth_element`. - *Points keep arriving and you must always know the current k closest?* Keep the same heap alive between calls; it is already a streaming structure. - *3D and many queries from different centres?* Build a KD-tree once, then each query is about $\log n$.

3. Kth Smallest in a Sorted Matrix (LeetCode 378, medium)

The problem in plain words. You are given an n by n grid of whole numbers where every row is sorted left to right and every column is sorted top to bottom. Return the k th smallest number in the whole grid, counting repeats.

```
matrix =   1   5   9
          10  11  13
          12  13  15           k = 8

all values in order: 1, 5, 9, 10, 11, 12, 13, 13, 15
                    1 2 3  4  5  6  7  8  9      -> the 8th is 13
```

n is at most 300, values between -1,000,000,000 and 1,000,000,000, and k between 1 and $n*n$.

By hand. Because every row is sorted, the smallest value overall must be at the *front* of some row. Take it, cross it off, and now the next smallest is again at the front of some row (possibly the same row, one step along). Repeat k times. You only ever compare the n row fronts, never the whole grid.

The idea. The "merge n sorted lists" pattern. Put the first cell of every row into a min-heap as `(value, row, col)`. Pop the smallest; that is the next value in order. Then push the next cell in that same row, if there is one. The k th pop is the answer.

The file also has a second method that uses no heap: guess a value, count how many cells are $\leq$ the guess by walking a staircase from the bottom-left corner, and binary-search the guess. Read it after the heap version is solid.

The Python you would have written.

```
import heapq

def kthSmallest(matrix, k):
    n = len(matrix)
    h = [(matrix[r][0], r, 0) for r in range(min(n, k))]
    heapq.heapify(h)
    while True:
        val, r, c = heapq.heappop(h)
        k -= 1
        if k == 0:
            return val
        if c + 1 < n:
            heapq.heappush(h, (matrix[r][c + 1], r, c + 1))
```

In C++.

```
// LeetCode 378. Kth Smallest Element in a Sorted Matrix
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>

// Approach 1: n-way merge with a min-heap. Seed the heap with the head of each
// row (rows are sorted), pop k-1 times, each pop pushes that row's next element.
// O(k log n) time, O(n) space where n = number of rows (capped at k).
int kthSmallest(std::vector<std::vector<int>>& matrix, int k) {
    int n = static_cast<int>(matrix.size());
    using Cell = std::tuple<int, int, int>; // (value, row, col) - tuple compares lexicographically
    std::priority_queue<Cell, std::vector<Cell>, std::greater<Cell>> minHeap;
    for (int r = 0; r < std::min(n, k); ++r) minHeap.push({matrix[r][0], r, 0});
    while (true) {
        auto [val, r, c] = minHeap.top();
        minHeap.pop();
        if (--k == 0) return val;
        if (c + 1 < n) minHeap.push({matrix[r][c + 1], r, c + 1});
    }
}
```

```

}

// Approach 2: binary search on the VALUE range, not on indices. For a candidate
// mid, count how many cells are <= mid in O(n) by walking from the bottom-left
// corner (a "staircase" walk). Smallest mid with count >= k is the answer.
// O(n log(max-min)) time, O(1) space. No heap at all.
int kthSmallestBinarySearch(std::vector<std::vector<int>>& matrix, int k) {
    int n = static_cast<int>(matrix.size());
    auto countLessEqual = [&](int target) {
        int count = 0;
        int r = n - 1, c = 0;
        while (r >= 0 && c < n) {
            if (matrix[r][c] <= target) { count += r + 1; ++c; } // whole column above r is <= target
            else --r;
        }
        return count;
    };
    int lo = matrix[0][0], hi = matrix[n - 1][n - 1];
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (countLessEqual(mid) >= k) hi = mid; // answer is <= mid
        else lo = mid + 1;
    }
    return lo; // lo is always an actual matrix value: it's the smallest value with count >= k
}

int main() {
    {
        std::vector<std::vector<int>> m = {{1, 5, 9}, {10, 11, 13}, {12, 13, 15}};
        assert(kthSmallest(m, 8) == 13);
        assert(kthSmallestBinarySearch(m, 8) == 13);
        assert(kthSmallest(m, 1) == 1);
        assert(kthSmallestBinarySearch(m, 1) == 1);
        assert(kthSmallest(m, 9) == 15);
        assert(kthSmallestBinarySearch(m, 9) == 15);
    }
    {
        std::vector<std::vector<int>> m = {{-5}}; // 1x1
        assert(kthSmallest(m, 1) == -5);
        assert(kthSmallestBinarySearch(m, 1) == -5);
    }
    {
        std::vector<std::vector<int>> m = {{1, 2}, {1, 3}}; // duplicates across rows
        assert(kthSmallest(m, 2) == 1);
        assert(kthSmallestBinarySearch(m, 2) == 1);
        assert(kthSmallest(m, 3) == 2);
        assert(kthSmallestBinarySearch(m, 3) == 2);
    }
    std::cout << "OK 03_kth_smallest_sorted_matrix.cpp\n";
    return 0;
}

```

What is new:

- using `Cell = std::tuple<int, int, int>`; is a three-item pair, (value, row, col). Tuples compare item by item from the left, so the heap orders by value first. Put the thing you sort by first, as with pairs.
- `std::priority_queue<Cell, std::vector<Cell>, std::greater<Cell>> minHeap`; is the smallest-on-top heap of tuples.
- `for (int r = 0; r < std::min(n, k); ++r)` seeds the heap. If `k` is smaller than `n`, rows beyond the `k`th can never contribute, so we skip them.
- `auto [val, r, c] = minHeap.top(); minHeap.pop();` unpacks the tuple like Python's `val, r, c = heappop(h)`, in two calls (look, then remove).
- `if (--k == 0) return val;` decreases `k` first, then tests it. Same as `k -= 1`; if `k == 0`.
- In the second function, `auto countLessEqual = [&](int target) { ... };` is a lambda that can see `matrix` and `n` from the outer function (`[&]`). `int mid = lo + (hi - lo) / 2;` is the overflow-safe way to write `(lo + hi) / 2` when values can be near a billion.

Walk through the example. `k = 8`. Seed the heap with the row fronts: `{(1,0,0), (10,1,0), (12,2,0)}`. Heap shown with its top first.

```

pop (1, r0,c0)    k=7    push (5, r0,c1)    heap {5, 10, 12}
pop (5, r0,c1)    k=6    push (9, r0,c2)    heap {9, 10, 12}
pop (9, r0,c2)    k=5    row 0 is finished  heap {10, 12}
pop (10, r1,c0)   k=4    push (11, r1,c1)   heap {11, 12}
pop (11, r1,c1)   k=3    push (13, r1,c2)   heap {12, 13}
pop (12, r2,c0)   k=2    push (13, r2,c1)   heap {13(r1), 13(r2)}
pop (13, r1,c2)   k=1    row 1 is finished  heap {13(r2)}
pop (13, r2,c1)   k=0    return 13

```

The binary-search version on the same input: $lo = 1$, $hi = 15$. Guess 8: only 2 cells are ≤ 8 , fewer than 8, so $lo = 9$. Guess 12: 6 cells, $lo = 13$. Guess 14: 8 cells, $hi = 14$. Guess 13: 8 cells, $hi = 13$. Now $lo == hi == 13$. Answer 13.

Complexity. Heap: k pops, each $\log n$ with at most n things in the heap: $O(k \log n)$ time, $O(n)$ memory. Binary search on the value: each count is one staircase walk of at most $2n$ steps, and there are about $\log(\max - \min)$ guesses: $O(n \log(\text{range}))$ time, no extra memory.

Robotics. Merging several time-sorted message streams (one per sensor) into one ordered stream is exactly the n -way merge: the heap holds the next message from each stream. The binary-search-on-value trick is how you find a distance threshold that keeps exactly a chosen fraction of the closest laser returns.

Follow-ups you may get. - *Why not copy everything into one vector and use `nth_element`?* It works but costs nn time and memory and throws away the fact that the rows are sorted. - What if only the rows are sorted, not the columns? *The heap merge still works (it only needs sorted rows). The staircase count does not.* - Why does the binary search land on a value that is actually in the grid? The count only changes at values that exist in the grid, so the smallest guess with count $\geq k$ is one of them.

4. Sliding Window Maximum (LeetCode 239, hard)

The problem in plain words. You are given a list of numbers and a window width k . Slide the window from the left end to the right end one step at a time, and for every position report the largest number inside the window.

```

nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

[1 3 -1] -3 5 3 6 7    -> 3
1 [3 -1 -3] 5 3 6 7    -> 3
1 3 [-1 -3 5] 3 6 7    -> 5
1 3 -1 [-3 5 3] 6 7    -> 5
1 3 -1 -3 [5 3 6] 7    -> 6
1 3 -1 -3 5 [3 6 7]    -> 7          answer: [3, 3, 5, 5, 6, 7]

```

The list has up to 100,000 numbers and k is between 1 and the list length. With $k = 1$ the answer is the list itself; with $k = \text{length}$ it is one number, the maximum.

By hand. Slide the window and look at the three numbers each time. With 8 numbers that is fine; with 100,000 numbers and $k = 50,000$ it is far too slow. The fast way is the monotonic deque from A3: keep only the numbers that could still become a maximum.

The idea. For each index i : 1. If the index at the front of the deque is $\leq i - k$, it has slid out of the window: pop it from the front. 2. While the value at the back of the deque is $\leq$ the new value, pop the back (it is older and no bigger, so it can never win). 3. Push i at the back. 4. Once $i \geq k - 1$ (the first full window), the front of the deque is the maximum: record it.

The Python you would have written.

```

from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    out = []
    # indices; values decreasing from front to back

```

```

for i, x in enumerate(nums):
    if dq and dq[0] <= i - k:
        dq.popleft()
    while dq and nums[dq[-1]] <= x:
        dq.pop()
    dq.append(i)
    if i >= k - 1:
        out.append(nums[dq[0]])
return out

```

In C++.

```

// LeetCode 239. Sliding Window Maximum
#include <cassert>
#include <deque>
#include <iostream>
#include <vector>

// Monotonic deque of INDICES, values decreasing from front to back.
// Front is always the max of the current window. Each index is pushed and
// popped at most once -> O(n) total, O(k) space.
std::vector<int> maxSlidingWindow(std::vector<int>& nums, int k) {
    std::deque<int> dq;
    std::vector<int> out;
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i) {
        // 1. drop indices that fell out of the window [i-k+1, i]
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // 2. a new, larger value makes every smaller one behind it useless:
        // they are older AND smaller, so they can never be a window max again
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        // 3. once the first full window is in, record its max
        if (i >= k - 1) out.push_back(nums[dq.front()]);
    }
    return out;
}

int main() {
    {
        std::vector<int> nums = {1, 3, -1, -3, 5, 3, 6, 7};
        assert(maxSlidingWindow(nums, 3) == std::vector<int>({3, 3, 5, 5, 6, 7}));
    }
    {
        std::vector<int> nums = {1};
        assert(maxSlidingWindow(nums, 1) == std::vector<int>({1}));
    }
    {
        std::vector<int> nums = {9, 8, 7, 6}; // strictly decreasing: front expires each step
        assert(maxSlidingWindow(nums, 2) == std::vector<int>({9, 8, 7}));
    }
    {
        std::vector<int> nums = {1, 2, 3, 4}; // strictly increasing: deque holds one index
        assert(maxSlidingWindow(nums, 2) == std::vector<int>({2, 3, 4}));
    }
    {
        std::vector<int> nums = {4, 4, 4}; // ties: <= pops the older duplicate, still correct
        assert(maxSlidingWindow(nums, 3) == std::vector<int>({4}));
    }
    std::cout << "OK 04_sliding_window_maximum.cpp\n";
    return 0;
}

```

What is new:

- `#include <deque>` and `std::deque<int> dq;` is `collections.deque`. `push_back`, `pop_back`, `back()`, `front()`, `pop_front()` are `append`, `pop`, `dq[-1]`, `dq[0]`, `popleft`. As always in C++, "look" and "remove" are separate calls.
- `if (!dq.empty() && dq.front() <= i - k)` is `if dq and dq[0] <= i - k`. The `!dq.empty()` must come first; `front()` on an empty deque is undefined behaviour.
- `while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();` is the "throw out the shorter people behind" loop from A3.

- `if (i >= k - 1) out.push_back(nums[dq.front()]);` records the answer only once the window is full.

Walk through the example. Deque as `[index(value), ...]`, front on the left.

<code>i=0 (1):</code>	<code>push 0</code>	<code>[0(1)]</code>	
<code>i=1 (3):</code>	<code>back 1 <= 3, pop. push 1</code>	<code>[1(3)]</code>	
<code>i=2 (-1):</code>	<code>back 3 > -1, keep. push 2</code>	<code>[1(3), 2(-1)]</code>	<code>out: 3</code>
<code>i=3 (-3):</code>	<code>front 1 <= 0? no. back -1 > -3. push 3</code>	<code>[1(3), 2(-1), 3(-3)]</code>	<code>out: 3</code>
<code>i=4 (5):</code>	<code>front 1 <= 1? yes, pop front</code>	<code>[2(-1), 3(-3)]</code>	
	<code>back -3 <= 5 pop; back -1 <= 5 pop. push 4</code>	<code>[4(5)]</code>	<code>out: 5</code>
<code>i=5 (3):</code>	<code>front 4 <= 2? no. back 5 > 3. push 5</code>	<code>[4(5), 5(3)]</code>	<code>out: 5</code>
<code>i=6 (6):</code>	<code>front 4 <= 3? no. back 3 <= 6 pop,</code>		
	<code>back 5 <= 6 pop. push 6</code>	<code>[6(6)]</code>	<code>out: 6</code>
<code>i=7 (7):</code>	<code>front 6 <= 4? no. back 6 <= 7 pop. push 7</code>	<code>[7(7)]</code>	<code>out: 7</code>
<code>out = [3, 3, 5, 5, 6, 7]</code>			

Notice at `i=4`: the deque still held `-1` and `-3`, but `5` threw both out at once. They had no chance of ever being a maximum again.

Complexity. Each index is pushed once and popped at most once, so the whole thing is one pass: $O(n)$ time. The deque never holds more than `k` indices: $O(k)$ memory.

Robotics. "Maximum (or minimum) of the last `k` sensor readings" is a windowed filter: peak detection on a force sensor, the biggest range in the last `k` laser scans for an adaptive obstacle threshold, an envelope on a microphone channel. The deque makes it constant work per sample, which is what lets it run inside a 1 kHz control loop.

Follow-ups you may get. - *Sliding window minimum?* The same code with the comparison flipped (`>=` instead of `<=`). - *Could you use a heap?* Yes: a max-heap of `(value, index)`, and when the top's index is too old, pop it and look again. That is $O(n \log k)$; the deque is better. - *Sliding window median?* Two heaps (a max-heap for the lower half, a min-heap for the upper half) or a `std::multiset` with a pointer to the middle, $\log k$ per step.

5. Binary Tree Level Order Traversal (LeetCode 102, medium)

The problem in plain words. You are given the root of a binary tree. Return its values level by level: first a list with the root, then a list with the root's children left to right, then the grandchildren left to right, and so on. The answer is a list of lists.

tree given as `[3, 9, 20, null, null, 15, 7]`:



level 0: `[3]`

level 1: `[9, 20]`

level 2: `[15, 7]`

answer: `[[3], [9, 20], [15, 7]]`

The tree can be empty (answer: an empty list) and can have up to 2,000 nodes. A tree that is one long chain gives one single-item list per level.

By hand. Read the drawing row by row. Each row of the drawing is one level. To do it without a drawing: write the root on a piece of paper; then for each node on the paper, write its children on the next piece of paper; when the first paper is used up, that was one level, and you move to the next paper.

The idea. BFS with a queue and the level-size snapshot from A4 (the ring loop from chapter 03). Push the root. While the queue is not empty: note how many nodes are in the queue right now (that is this level), pop exactly that many, record their values, and push their children (they will be the next level). Append the level's list to the answer.

The Python you would have written.

```

from collections import deque

def levelOrder(root):
    out = []
    if root is None:
        return out
    q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):          # exactly this level
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        out.append(level)
    return out

```

In C++.

```

// LeetCode 102. Binary Tree Level Order Traversal
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};

// BFS with a queue. The trick that separates levels: snapshot q.size() at the
// start of each round – that many nodes are exactly the current level, and
// everything pushed during the round belongs to the next one.
// O(n) time, O(w) space where w = widest level.
std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> out;
    if (root == nullptr) return out;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = static_cast<int>(q.size());    // freeze it: q grows inside the loop
        std::vector<int> level;
        level.reserve(levelSize);
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* cur = q.front();
            q.pop();
            level.push_back(cur->val);
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        out.push_back(std::move(level));
    }
    return out;
}

// ---- test helpers -----
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);

```

```

int i = 1, n = static_cast<int>(vals.size());
while (!q.empty() && i < n) {
    TreeNode* cur = q.front();
    q.pop();
    if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
    ++i;
    if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
    ++i;
}
return root;
}

void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    {
        TreeNode* root = buildTree({3, 9, 20, NONE, NONE, 15, 7});
        assert(levelOrder(root) == std::vector<std::vector<int>>({{3}, {9, 20}, {15, 7}}));
        deleteTree(root);
    }
    {
        TreeNode* root = buildTree({1});
        assert(levelOrder(root) == std::vector<std::vector<int>>({{1}}));
        deleteTree(root);
    }
    {
        TreeNode* root = buildTree({1, NONE, 2, NONE, 3}); // right-leaning chain
        assert(levelOrder(root) == std::vector<std::vector<int>>({{1}, {2}, {3}}));
        deleteTree(root);
    }
    assert(levelOrder(nullptr).empty()); // empty tree
    std::cout << "OK 05_level_order_traversal.cpp\n";
    return 0;
}

```

What is new:

- `struct TreeNode { ... }` at the top is the node from A4; LeetCode supplies it, and the standalone file has to repeat it.
- `std::queue<TreeNode*> q;` holds node *addresses*. `q.push(root)` puts in the root's address, not a copy of the node.
- `int levelSize = static_cast<int>(q.size());` is the snapshot, Python's `range(len(q))`. It has to be taken before the loop because the queue grows inside it.
- `level.reserve(levelSize);` pre-sizes the level's vector. Optional.
- `if (cur->left) q.push(cur->left);` is `if node.left: q.append(node.left)`; a non-null pointer counts as true.
- `out.push_back(std::move(level));` hands the level's vector over to `out` without copying it. Plain `out.push_back(level)` would work too, one copy slower.
- `buildTree` and `deleteTree` at the bottom are the test helpers from A4. `NONE (-1)` stands for `null`.

Walk through the example. Queue shown front first.

start:	q = [3]		
level, size 1:	pop 3, record; push 9, 20	q = [9, 20]	out = [[3]]
level, size 2:	pop 9, record; no children	q = [20]	
	pop 20, record; push 15, 7	q = [15, 7]	out = [[3], [9, 20]]
level, size 2:	pop 15, record; no children	q = [7]	
	pop 7, record; no children	q = []	out = [[3], [9, 20], [15, 7]]
queue empty:	done		

Complexity. Every node is pushed and popped once: $O(n)$ time. The queue holds at most one level at a time, so memory is the widest level, $O(n)$ in the worst case.

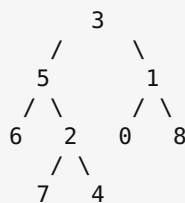
Robotics. Walking a tree level by level is how you print or check the tree hierarchy depth by depth. On an octree it is coarse-to-fine: visit every cell at resolution level d before any at level $d+1$. The frozen-level-size loop is the same one that expands one ring of a wavefront on a grid.

Follow-ups you may get. - *Zigzag order (LeetCode 103)*? Same loop; reverse every other level's list before appending. - *Right side view (LeetCode 199)*? The last value recorded in each level. - *Recursive version*? Pass a depth down: `if depth == out.size(): out.push_back({})`, then `out[depth].push_back(val)`; pre-order keeps each level left to right.

6. Lowest Common Ancestor (LeetCode 236, medium)

The problem in plain words. You are given the root of a binary tree and two nodes p and q that are both in the tree. Return the lowest common ancestor: the deepest node that has both p and q below it, where a node counts as being below itself.

tree [3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]:



$p = 5, q = 1 \rightarrow 3$ (they are on different sides of 3)
 $p = 5, q = 4 \rightarrow 5$ (4 is below 5, and 5 counts as its own ancestor)
 $p = 7, q = 4 \rightarrow 2$
 $p = 6, q = 8 \rightarrow 3$

Between 2 and 100,000 nodes; every value is different; p and q are different nodes that both exist in the tree.

By hand. Put a finger on p and a finger on q . Walk both fingers upwards to their parents, one step each, marking the nodes you pass. The first node both fingers have touched is the answer. That needs parent pointers, which `TreeNode` does not have, so the code uses the "ask both children" method from A5 instead.

The idea. Recursion that returns what it found (A5): 1. If the node is null, or is p , or is q , return the node. 2. Ask the left child and the right child. 3. If both returned something, this node is the LCA: return it. 4. Otherwise return whichever child returned something (or null if neither did).

The Python you would have written.

```
def lowestCommonAncestor(root, p, q):
    if root is None or root is p or root is q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

In C++.

```
// LeetCode 236. Lowest Common Ancestor of a Binary Tree
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>
```

```

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};

// Post-order "return what you found":
// - a subtree returns nullptr if it contains neither p nor q,
// - returns p or q if it contains exactly one of them (or that node itself),
// - the first node where BOTH sides return non-null is the LCA, and it
//   propagates unchanged up to the root.
// If p is an ancestor of q we return p as soon as we hit it, which is correct
// because the problem guarantees both nodes exist in the tree.
// O(n) time, O(h) recursion depth.
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (root == nullptr || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;           // p and q are on different sides: this is the split
    return left ? left : right;              // pass up whichever side found something
}

// BST variant (LeetCode 235): use the ordering, no recursion needed. O(h).
TreeNode* lowestCommonAncestorBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;                    // they split here (or one equals root)
    }
    return nullptr;
}

// ---- test helpers -----
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);
    int i = 1, n = static_cast<int>(vals.size());
    while (!q.empty() && i < n) {
        TreeNode* cur = q.front();
        q.pop();
        if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
        ++i;
        if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
        ++i;
    }
    return root;
}

TreeNode* find(TreeNode* root, int val) { // values are unique in the LeetCode tests
    if (root == nullptr || root->val == val) return root;
    TreeNode* l = find(root->left, val);
    return l ? l : find(root->right, val);
}

void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    {
        TreeNode* root = buildTree({3, 5, 1, 6, 2, 0, 8, NONE, NONE, 7, 4});
        assert(lowestCommonAncestor(root, find(root, 5), find(root, 1))->val == 3);
        assert(lowestCommonAncestor(root, find(root, 5), find(root, 4))->val == 5); // p is ancestor of q
        assert(lowestCommonAncestor(root, find(root, 7), find(root, 4))->val == 2);
        assert(lowestCommonAncestor(root, find(root, 6), find(root, 8))->val == 3);
        deleteTree(root);
    }
}

```

```

TreeNode* root = buildTree({1, 2}); // two nodes
assert(lowestCommonAncestor(root, find(root, 1), find(root, 2))->val == 1);
deleteTree(root);
}
{
    TreeNode* root = buildTree({6, 2, 8, 0, 4, 7, 9, NONE, NONE, 3, 5}); // a BST (LeetCode 235)
    assert(lowestCommonAncestorBST(root, find(root, 2), find(root, 8))->val == 6);
    assert(lowestCommonAncestorBST(root, find(root, 2), find(root, 4))->val == 2);
    assert(lowestCommonAncestor(root, find(root, 3), find(root, 5))->val == 4); // general version agrees
    deleteTree(root);
}
std::cout << "OK 06_lowest_common_ancestor.cpp\n";
return 0;
}

```

What is new:

- `if (root == nullptr || root == p || root == q) return root;` compares *pointers*: two pointers are equal only when they name the same node. That is Python's `is`. Comparing `root->val == p->val` would be wrong if values could repeat.
- `TreeNode* left = lowestCommonAncestor(root->left, p, q);` stores the child's answer, which is a pointer (possibly `nullptr`).
- `if (left && right) return root;` treats a non-null pointer as true.
- `return left ? left : right;` is Python's `left if left else right`.
- `lowestCommonAncestorBST` is a bonus for the case where the tree is a binary search tree (LeetCode 235): if both values are smaller than the current node go left, if both are bigger go right, otherwise this is the split. No recursion needed.
- `find(root, val)` in the test helpers turns a value into the node pointer, so the tests can say `find(root, 5)` for `p`.

Walk through the example. `p = 7, q = 4`, expected 2. Each line is one call; indentation is depth.

```

call(3): not p or q.
  call(5): not p or q.
    call(6): not p or q. call(null) -> null, call(null) -> null. return null.
    call(2): not p or q.
      call(7): I am p. return 7.
      call(4): I am q. return 4.
    both non-null: return 2.
  left = null, right = 2. return 2.
call(1): not p or q.
  call(0): ... both children null. return null.
  call(8): ... return null.
return null.
left = 2, right = null. return 2.

```

And `p = 5, q = 4`: `call(3)` asks `call(5)`, which returns 5 at once because it *is* `p`, without ever looking at 4. `call(1)` returns null. So 3 returns 5. Correct, because the problem promises `q` is in the tree, and it is not on 1's side, so it must be under 5.

Complexity. Every node is visited once: $O(n)$ time. The recursion is as deep as the tree is tall: $O(\text{height})$ memory.

Robotics. ROS `tf2` finds the transform between two frames by locating their lowest common ancestor in the frame tree and composing the two chains up to it; `lookupTransform` is an LCA query with parent pointers. The same appears in a robot's link tree (URDF) when you need the pose of one link relative to another.

Follow-ups you may get. - *`p` or `q` might not be in the tree (LeetCode 1644)?* Count how many of the two you actually met during the walk; return the candidate only if the count is 2. - *Nodes have parent pointers (LeetCode 1650)?* Walk up from both, first equalising depths, or collect `p`'s ancestors in a set and walk up from `q` until you hit one. - *Many LCA queries on one fixed tree?* Precompute " 2^j -th ancestor" tables (binary lifting): $n \log n$ setup, $\log n$ per query.

7. LRU Cache (LeetCode 146, medium)

The problem in plain words. Build a class `LRUCache` with a fixed capacity and two operations:

- `get(key)`: if the key is in the cache, return its value and mark it as just used; otherwise return -1.
- `put(key, value)`: store the value under the key and mark it as just used. If the key was already there, just update its value. If it was new and the cache was already full, first throw out the entry that was used longest ago.

Both operations must take a constant number of steps, however big the cache is.

capacity 2	shelf, front = most recently used	
<code>put(1, 1)</code>	<code>[(1,1)]</code>	
<code>put(2, 2)</code>	<code>[(2,2) (1,1)]</code>	
<code>get(1) -> 1</code>	<code>[(1,1) (2,2)]</code>	1 moves to the front
<code>put(3, 3)</code>	<code>[(3,3) (1,1)]</code>	full: (2,2) at the back is thrown out
<code>get(2) -> -1</code>		2 is gone
<code>put(4, 4)</code>	<code>[(4,4) (3,3)]</code>	(1,1) thrown out
<code>get(1) -> -1</code>		
<code>get(3) -> 3</code>	<code>[(3,3) (4,4)]</code>	
<code>get(4) -> 4</code>	<code>[(4,4) (3,3)]</code>	

Capacity is at least 1, keys and values are non-negative whole numbers, and there can be up to 200,000 operations.

By hand. Play the bookshelf game from A6 on paper, as in the table above. Every `get` or `put` moves that entry to the front; a `put` of a new key on a full shelf first removes the back entry.

The idea. A `std::list` of `(key, value)` pairs as the shelf, front = most recently used, and a `std::unordered_map` from key to a bookmark (iterator) into the list.

- `get`: look the key up in the map. Missing: return -1. Found: splice its node to the front and return the value.
- `put`: look the key up. Found: change the value in the node, splice to the front. New: if the list is at capacity, read the key of the back node, erase that key from the map, pop the back node. Then push the new pair at the front and store its bookmark in the map.

The Python you would have written.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()           # front = least recent, end = most recent

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        elif len(self.d) == self.cap:
            self.d.popitem(last=False)    # drop the least recent
        self.d[key] = value
```

(Python's `OrderedDict` is a hash map plus a doubly linked list underneath; in C++ you wire the two together yourself.)

In C++.

```

// LeetCode 146. LRU Cache
#include <cassert>
#include <iostream>
#include <list>
#include <unordered_map>
#include <utility>

// Recency order lives in a doubly linked list (front = most recently used).
// The hash map gives O(1) key -> list node. std::list iterators stay valid
// through splice/erase of OTHER nodes, which is exactly what makes this O(1);
// a std::vector would invalidate them on every move.
class LRUCache {
public:
    explicit LRUCache(int capacity) : capacity_(capacity) {}

    int get(int key) {
        auto it = map_.find(key);
        if (it == map_.end()) return -1;
        touch(it->second); // move to front, O(1)
        return it->second->second;
    }

    void put(int key, int value) {
        auto it = map_.find(key);
        if (it != map_.end()) {
            it->second->second = value; // update in place, then mark as recent
            touch(it->second);
            return;
        }
        if (static_cast<int>(items_.size()) == capacity_) {
            map_.erase(items_.back().first); // evict least recently used (back)
            items_.pop_back();
        }
        items_.emplace_front(key, value);
        map_[key] = items_.begin();
    }

private:
    using Node = std::pair<int, int>; // (key, value): key needed to erase from map on eviction
    using Iter = std::list<Node>::iterator;

    void touch(Iter it) {
        // relink the node at the front without copying or reallocating; `it` stays valid
        items_.splice(items_.begin(), items_, it);
    }

    int capacity_;
    std::list<Node> items_;
    std::unordered_map<int, Iter> map_;
};

int main() {
    {
        LRUCache c(2); // LeetCode example
        c.put(1, 1);
        c.put(2, 2);
        assert(c.get(1) == 1);
        c.put(3, 3); // evicts key 2 (1 was just touched)
        assert(c.get(2) == -1);
        c.put(4, 4); // evicts key 1
        assert(c.get(1) == -1);
        assert(c.get(3) == 3);
        assert(c.get(4) == 4);
    }
    {
        LRUCache c(1); // capacity 1: every put of a new key evicts
        c.put(5, 50);
        assert(c.get(5) == 50);
        c.put(6, 60);
        assert(c.get(5) == -1);
        assert(c.get(6) == 60);
        c.put(6, 61); // overwrite existing key, no eviction
        assert(c.get(6) == 61);
    }
}

```

```

}
{
    LRUCache c(2);
    c.put(2, 1);
    c.put(2, 2);           // update keeps size at 1
    assert(c.get(2) == 2);
    c.put(1, 1);
    c.put(4, 1);           // evicts 2 (1 is more recent)
    assert(c.get(2) == -1);
    assert(c.get(1) == 1);
}
{
    LRUCache c(3);
    assert(c.get(42) == -1); // get on empty cache
}
std::cout << "OK 07_lru_cache.cpp\n";
return 0;
}

```

What is new:

- `class LRUCache { public: ... private: ... };` is a Python class with the parts labelled. `public:` are the methods anyone can call; `private:` are the helpers and the data. The trailing `_` on `capacity_`, `items_`, `map_` is just a naming habit for member variables.
- `explicit LRUCache(int capacity) : capacity_(capacity) {}` is `__init__`. `explicit` stops C++ from silently turning a plain `int` into an `LRUCache`; always write it on one-argument constructors.
- `using Node = std::pair<int, int>;` and `using Iter = std::list<Node>::iterator;` name the two types so the rest reads easily. `Iter` is the bookmark type from A6.
- `std::list<Node> items_;` is the shelf; `std::unordered_map<int, Iter> map_;` is the key-to-bookmark map.
- `auto it = map_.find(key); if (it == map_.end()) return -1;` is if key not in self.d. `find` returns a map bookmark, and `end()` means "not found". `it->second` is the value stored under the key, which here is the list bookmark; so `it->second->second` is "the list node's value" (the pair's second half) and `it->second->first` would be its key.
- `items_.splice(items_.begin(), items_, it);` in touch is `move_to_end`, but to the front.
- `map_.erase(items_.back().first); items_.pop_back();` is `popitem(last=False)`: read the back node's key, erase it from the map, then drop the node.
- `items_.emplace_front(key, value); map_[key] = items_.begin();` builds the new node at the front, then stores the bookmark of the new front. The order matters: `begin()` must be read *after* the insert.

Walk through the example. Capacity 2. The list is drawn front first, and the map lists the keys that have bookmarks.

<code>put(1,1):</code>	not in map. size 0 < 2. <code>emplace_front (1,1)</code> . <code>map {1}</code> .	<code>list [(1,1)]</code>
<code>put(2,2):</code>	not in map. size 1 < 2. <code>emplace_front (2,2)</code> . <code>map {1,2}</code> .	<code>list [(2,2) (1,1)]</code>
<code>get(1):</code>	in map. <code>splice</code> its node to the front. return 1.	<code>list [(1,1) (2,2)]</code>
<code>put(3,3):</code>	not in map. size 2 == 2: back is (2,2): <code>map.erase(2)</code> , <code>pop_back</code> . <code>emplace_front (3,3)</code> . <code>map {1,3}</code> .	<code>list [(3,3) (1,1)]</code>
<code>get(2):</code>	not in map. return -1.	
<code>put(4,4):</code>	not in map. full: back is (1,1): <code>erase 1</code> , <code>pop_back</code> . <code>emplace_front (4,4)</code> . <code>map {3,4}</code> .	<code>list [(4,4) (3,3)]</code>
<code>get(1):</code>	not in map. return -1.	
<code>get(3):</code>	in map. <code>splice</code> to front. return 3.	<code>list [(3,3) (4,4)]</code>
<code>get(4):</code>	in map. <code>splice</code> to front. return 4.	<code>list [(4,4) (3,3)]</code>

Complexity. Every operation is a map lookup plus a few pointer rewirings: $O(1)$ per operation. Memory is $O(\text{capacity})$.

Robotics. A localiser keeps the last N lidar submaps resident and evicts by recency; a map renderer does the same with costmap tiles and textures. Both are this class with a bigger value type. The same `list + unordered_map` pair is what you would write for an object pool that recycles the least recently used slot.

Follow-ups you may get. - *Why not a `std::vector` for the shelf?* Moving a middle item to the front shifts everything (linear), and growing a vector can move every element in memory, so stored bookmarks become garbage. `std::list` has neither problem. - *Thread safety?* Wrap both operations in a `std::mutex`. A reader-writer lock does not help, because even `get` changes the recency order. - *LFU instead (evict the least frequently used, LeetCode 460)?* One list per frequency count plus a `minFreq` counter; each access splices the node from list `f` to list `f+1`. Still constant time, about three times the bookkeeping.

8. Trapping Rain Water (LeetCode 42, hard)

The problem in plain words. You are given a list of bar heights, each bar one unit wide, standing side by side. It rains. Water collects in the dips between taller bars and runs off the two ends. Return how many unit squares of water are trapped.

```
height = [4, 2, 0, 3, 2, 5]          # is a bar, ~ is trapped water

5 |                                     #
4 | # ~ ~ ~ ~ #
3 | # ~ ~ # ~ #
2 | # # ~ # # #
1 | # # ~ # # #
+-----+
  0 1 2 3 4 5
  4 2 0 3 2 5    <- height

count the ~ squares: 4 + 3 + 1 + 1 = 9    -> answer 9
```

A second example, the classic one:

```
height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

3 |                                     #
2 |           # ~ ~ ~ # # ~ #
1 |   # ~ # # ~ # # # # # #
+-----+
  0 1 2 3 4 5 6 7 8 9 10 11
  0 1 0 2 1 0 1 3 2 1 2 1    <- height

answer 6
```

The list can be empty or have one bar (answer 0), and can have up to 20,000 bars of height up to 100,000. If the bars only go up, or only go down, nothing is trapped.

By hand. Look at one column at a time. Water sits on top of bar `i` up to the level of the *lower* of two walls: the tallest bar anywhere to its left and the tallest bar anywhere to its right. So the water above bar `i` is $\min(\text{tallest on the left}, \text{tallest on the right}) - \text{height}[i]$, and never less than 0. In the first drawing, above bar 2 (height 0): tallest left is 4, tallest right is 5, min is 4, water is $4 - 0 = 4$. Above bar 3 (height 3): $\min(4, 5) - 3 = 1$. Add them all up.

The idea. Three versions, all in the file.

1. *Two arrays* (the obvious one): `leftMax[i]` = tallest bar from 0 to `i`, computed left to right; `rightMax[i]` = tallest from `i` to the end, computed right to left. Then sum $\min(\text{leftMax}[i], \text{rightMax}[i]) - \text{height}[i]$. Linear time, linear extra memory.
2. *Two pointers* (the one the file calls `trap`): the same formula without the arrays. Keep `l` at the left end and `r` at the right end, with running `leftMax` and `rightMax`. Look at the two bars under the pointers. If `height[l] < height[r]`, the tallest bar on the right is at least `height[r]`, which is taller than `height[l]`, so the right wall is not the limit: bar `l`'s water level is decided by `leftMax` alone. Settle it, `++l`. Otherwise settle bar `r` with `rightMax`, `--r`. Stop when they meet. Linear time, no extra memory.

3. *Monotonic stack (A3)*: keep a stack of bar indices with decreasing heights. When a taller bar arrives, each bar popped is the floor of a horizontal strip of water bounded by the new bar and the next bar left on the stack. This fills the water layer by layer instead of column by column.

The Python you would have written. The two-pointer version.

```
def trap(height):
    l, r = 0, len(height) - 1
    left_max = right_max = water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

In C++.

```
// LeetCode 42. Trapping Rain Water
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Water above bar i = min(maxLeft(i), maxRight(i)) - height[i].
// Two pointers: whichever side has the lower running max is the side whose
// water level is already decided (the other side has something at least as
// tall to hold it), so we can settle that bar and move inwards.
// O(n) time, O(1) space.
int trap(std::vector<int>& height) {
    int l = 0, r = static_cast<int>(height.size()) - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            leftMax = std::max(leftMax, height[l]);
            water += leftMax - height[l]; // never negative: leftMax >= height[l]
            ++l;
        } else {
            rightMax = std::max(rightMax, height[r]);
            water += rightMax - height[r];
            --r;
        }
    }
    return water;
}

// Alternative 1: prefix/suffix max arrays. Same formula, O(n) extra space,
// the most obvious version – say it first, then improve to two pointers.
int trapPrefixMax(std::vector<int>& height) {
    int n = static_cast<int>(height.size());
    if (n == 0) return 0;
    std::vector<int> leftMax(n), rightMax(n);
    leftMax[0] = height[0];
    for (int i = 1; i < n; ++i) leftMax[i] = std::max(leftMax[i - 1], height[i]);
    rightMax[n - 1] = height[n - 1];
    for (int i = n - 2; i >= 0; --i) rightMax[i] = std::max(rightMax[i + 1], height[i]);
    int water = 0;
    for (int i = 0; i < n; ++i) water += std::min(leftMax[i], rightMax[i]) - height[i];
    return water;
}

// Alternative 2: monotonic (decreasing) stack of indices. When a taller bar
// arrives, each popped bar is the floor of a horizontal strip bounded by the
```

```
// new bar and the next bar left on the stack. Fills water "layer by layer".
int trapMonotonicStack(std::vector<int>& height) {
    std::vector<int> stack; // indices, heights decreasing bottom->top
    int water = 0;
    for (int i = 0; i < static_cast<int>(height.size()); ++i) {
        while (!stack.empty() && height[stack.back()] < height[i]) {
            int floor = stack.back();
            stack.pop_back();
            if (stack.empty()) break; // no left wall
            int left = stack.back();
            int width = i - left - 1;
            int depth = std::min(height[left], height[i]) - height[floor];
            water += width * depth;
        }
        stack.push_back(i);
    }
    return water;
}

int main() {
    auto check = [](std::vector<int> h, int expected) {
        assert(trap(h) == expected);
        assert(trapPrefixMax(h) == expected);
        assert(trapMonotonicStack(h) == expected);
    };
    check({0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}, 6);
    check({4, 2, 0, 3, 2, 5}, 9);
    check({}, 0); // empty
    check({5}, 0); // single bar
    check({1, 2, 3, 4}, 0); // monotonic: nothing trapped
    check({4, 3, 2, 1}, 0);
    check({3, 0, 0, 3}, 6); // flat basin
    std::cout << "OK 08_trapping_rain_water.cpp\n";
    return 0;
}
```

What is new:

- `int l = 0, r = static_cast<int>(height.size()) - 1;` declares two ints on one line. For an empty vector `r` becomes -1, `l < r` is false at once, and the answer is 0 with no special case.
- `leftMax = std::max(leftMax, height[l]);` is `left_max = max(left_max, height[l])`. Needs `#include <algorithm>`.
- `water += leftMax - height[l];` is never negative because `leftMax` was just updated to include `height[l]`.
- In `trapPrefixMax`, `std::vector<int> leftMax(n), rightMax(n);` makes two vectors of `n` zeros, and the loop `for (int i = n - 2; i >= 0; --i)` walks right to left.
- In `trapMonotonicStack`, `if (stack.empty()) break;` after a pop guards against there being no left wall; without it `stack.back()` would read an empty vector, which is undefined behaviour in C++.
- `check` in `main` is a lambda that runs all three versions on the same input and asserts they agree.

Walk through the example. `height = [4, 2, 0, 3, 2, 5]`, two pointers.

<code>l=0 r=5</code>	<code>height[0]=4 < height[5]=5</code>	<code>leftMax=4</code>	<code>water += 4-4 = 0</code>	<code>-> 0</code>	<code>l=1</code>
<code>l=1 r=5</code>	<code>height[1]=2 < 5</code>	<code>leftMax=4</code>	<code>water += 4-2 = 2</code>	<code>-> 2</code>	<code>l=2</code>
<code>l=2 r=5</code>	<code>height[2]=0 < 5</code>	<code>leftMax=4</code>	<code>water += 4-0 = 4</code>	<code>-> 6</code>	<code>l=3</code>
<code>l=3 r=5</code>	<code>height[3]=3 < 5</code>	<code>leftMax=4</code>	<code>water += 4-3 = 1</code>	<code>-> 7</code>	<code>l=4</code>
<code>l=4 r=5</code>	<code>height[4]=2 < 5</code>	<code>leftMax=4</code>	<code>water += 4-2 = 2</code>	<code>-> 9</code>	<code>l=5</code>
<code>l=5 r=5</code>	<code>l < r</code> is false: stop. <code>water = 9</code>				

Every step moved the left pointer because the bar under the right pointer (5) was taller than every left bar. That is the whole justification: with a 5 waiting on the right, each left bar's water is capped by the left side only. Compare with the drawing: column 1 has 2 squares of water, column 2 has 4, column 3 has 1, column 4 has 2.

Complexity. Two pointers: one pass, $O(n)$ time, $O(1)$ extra memory. Two arrays: $O(n)$ time and $O(n)$ memory. Monotonic stack: $O(n)$ time (each index pushed and popped once), $O(n)$ memory in the worst case.

Robotics. The leftMax/rightMax formulation is a one-dimensional "flood fill to the rim": the fill depth of a terrain profile for traversability, or the puddle volume in a height map from a stereo camera. The 2D version (LeetCode 407) is a priority-queue BFS from the border cells, which is a wavefront over a cost map.

Follow-ups you may get. - *Why is moving the pointer on the shorter side safe?* If `height[l] < height[r]`, then `rightMax >= height[r] > height[l]`, so the level above bar `l` is limited by the left side only, and `leftMax` already knows it. - *2D version (LeetCode 407)?* Min-heap seeded with all border cells; pop the lowest, any lower neighbour traps `level - h`, push neighbours with `max(level, h)`. - *Container With Most Water (LeetCode 11)?* Also two pointers moving inward, but there you move the shorter side to look for a bigger area; no water is "trapped".

Part C — Check yourself

Answer each in plain words, from memory. If you cannot, reread the section named.

1. In a heap stored in an array, where are the children and the parent of index `i`? Check your formula on the 7-element example in A1. (A1)
2. Push 45 into `[50, 30, 40, 10, 20, 35, 15]` and write the array after each swap. (A1)
3. Why does a heap pop cost about $\log n$ steps and not n ? (A1)
4. Write the C++ line that declares a priority queue of ints with the smallest on top. What does the same `std::greater` do when you give it to `std::sort` instead? (A1)
5. You want the 3 largest of a million numbers arriving one at a time. Min-heap or max-heap? Why that one? (A2)
6. What does `std::nth_element` promise about the elements before and after the chosen position, and what does it not promise? (A2)
7. In the sliding window maximum, why is it safe to throw out an older, smaller value when a bigger one arrives? Why does the deque store indices and not values? (A3)
8. Run next-greater-element on `[5, 3, 4, 6]` by hand and show the stack after each step. (A3)
9. On the 5-node tree of A4, write the pre-order, in-order and post-order lists. Which one do you use to delete a tree, and why? (A4)
10. Write the level-order loop from memory. What goes wrong if you write `for (int i = 0; i < q.size(); ++i)` instead of taking a snapshot? (A4, Problem 5)
11. In the LCA recursion, what are the three things a node can return, and what does the parent do with each combination? (A5, Problem 6)
12. Why is `root == p` (comparing pointers) right and `root->val == p->val` wrong? (Problem 6)
13. In the LRU cache, why do we need both a hash map and a linked list? What does each one do in one step that the other cannot? (A6)
14. What does `std::list::splice` do, why is it constant time, and why do the bookmarks stored in the map stay valid after it? (A6, Problem 7)
15. In Trapping Rain Water, write the one-line formula for the water above bar `i`, then explain in one sentence why the two-pointer version can settle the shorter side. (Problem 8)

06. Robotics-specific

None of this chapter is on LeetCode. It is the set of small robotics tools that a mobile robot runs every fraction of a second: guessing where it is from a noisy sensor (a Kalman filter), holding a speed or a heading (a PID controller), turning "the camera saw a cup here" into "the cup is there in the room" (transforms), and finding a wall in a cloud of laser points (line fitting and RANSAC). Each of the four is a concept *and* a working C++ file you can put in a public repo. Part B has two LeetCode problems that are literally about robots. Part C turns the work into a portfolio. Rough time: 5 days at about 2 hours a day.

What you need before this chapter: classes, `std::vector`, `struct` and references from chapter 01; DFS and BFS on a grid from chapter 03 (Part B uses both); Dijkstra from chapter 04 (The Maze II is Dijkstra). A tiny bit of secondary-school maths: sine, cosine, and what a mean and a variance are (a variance is "how spread out", a small number means "quite sure").

Compile the plain files with `g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o x && ./x`. The two Eigen files (01b and 03) also need `-isystem /usr/include/eigen3` so the compiler can find the Eigen headers.

Part A — Implement from scratch

Each item below is a story first, then the maths with real numbers, then a hand trace, then the Python you would have written, then the C++ file with every new symbol explained.

b1. 1D Kalman filter in C++

The story. A robot is driving down a corridor. Two voices tell it where it is:

- *Its own dead reckoning* says "I think I am at 10 m, but I am not very sure, give or take 2 m."
- *A range sensor* says "you are at 12 m, give or take 1 m."

Which one do you believe? Neither, fully. You blend them, and you lean toward the one that is more sure. The sensor is twice as precise, so the answer should be closer to 12 than to 10. And here is the surprising part: after combining two opinions you are *more sure than either one alone*, because two independent measurements are better than one.

belief before:	sensor:	belief after:
"about 10, give or take 2"	"about 12, give or take 1"	"about 11.6, give or take 0.9"
----- (=====10=====) -----		wide bell: not sure
----- (==12==) -----		narrower bell: fairly sure
----- (=11.6=) -----		narrowest bell: the blend is surest of all

Doing the blend with actual numbers. "Give or take 2 m" is a standard deviation of 2, so a *variance* (standard deviation squared) of 4. The sensor's variance is 1. The rule is:

```
gain      K = (my variance) / (my variance + sensor variance)
          K = 4 / (4 + 1) = 0.8

estimate  x = my guess + K * (sensor - my guess)
          x = 10 + 0.8 * (12 - 10) = 10 + 1.6 = 11.6

variance  P = (1 - K) * my variance
          P = 0.2 * 4 = 0.8
```

Read the gain $K = 0.8$ as "believe 80 % of the sensor's correction". The sensor was 4 times more certain (variance 1 against 4), so it gets 4 parts out of 5. The new variance 0.8 is smaller than both 4 and 1: you are now surer than either voice on its own.

Try it with the numbers swapped: if *my* variance were 1 and the sensor's 4, $K = 1/5 = 0.2$, $x = 10 + 0.2 * 2 = 10.4$, close to my own guess. Same formula, opposite lean.

In C++ that whole blend is five lines:

```
double x = 10.0, P = 4.0;    // my belief: about 10, variance 4
double z = 12.0, R = 1.0;    // the sensor: says 12, variance 1
double K = P / (P + R);      // 0.8
x = x + K * (z - x);         // 11.6
P = (1.0 - K) * P;          // 0.8
```

The pieces, named. A Kalman filter is just that blend, repeated every time step, with one extra step in between:

1. **Predict.** The robot moves. You shift your guess by how far you think you moved, and you become *less* sure, because wheels slip and the motor is not perfect. Variance goes up.
2. **Update.** The sensor reports. You blend (the five lines above) and become *more* sure. Variance goes down.
3. **The gain K** is the dial between the two, recomputed every update from the two variances. It is never a number you tune by hand.

Two names you will hear: Q is the variance you *add* in each predict, "how much can my motion model be wrong per second". R is the sensor's variance, "how noisy is my sensor". Together with the estimate x and its variance P , that is the whole filter.

The scalar equations, each line matched to the story. State is position x , the control u is the commanded velocity, dt is the time step:

Predict:	$x = x + u * dt$ $P = P + Q * dt$	I moved by roughly $u*dt$...and I am less sure than before
Update:	$y = z - x$ $S = P + R$ $K = P / S$ $x = x + K * y$ $P = (1 - K) * P$	the surprise: what the sensor said minus what I expected how big a surprise is normal (my doubt + the sensor's noise) the gain: how much of the surprise to believe, 0 to 1 move toward the sensor by that fraction ...and I am more sure than before

y is called the *innovation*. If it is much bigger than $\sqrt{S}$ for many steps in a row, something is wrong: the sensor is broken or Q/R are set badly.

A 3-step hand trace. Start at $x = 0$, $P = 1$. Commanded speed $u = 1$ m/s, $dt = 1$ s, $Q = 1$, $R = 1$. The sensor reads 1.4, then 2.6, then 2.9.

step	predict: x, P	sensor z	surprise y	gain K	after update: x, P
1	1.000, 2.000	1.4	+0.400	$2/3 = 0.667$	1.267, 0.667
2	2.267, 1.667	2.6	+0.333	0.625	2.475, 0.625
3	3.475, 1.625	2.9	-0.575	0.619	3.119, 0.619

Check step 1 by hand: predict gives $x = 0 + 1 = 1$, $P = 1 + 1 = 2$. Surprise $y = 1.4 - 1 = 0.4$. $S = 2 + 1 = 3$, $K = 2/3$. $x = 1 + 0.667 * 0.4 = 1.267$. $P = (1 - 0.667) * 2 = 0.667$. Notice how P rises in every predict (2.0, 1.667, 1.625) and falls in every update, and how it settles: with $Q = R = 1$ the after-update variance heads to 0.618 and stays there. K settles too. In a running filter both stop changing after a few dozen steps.

What Q and R mean in plain words. R is "how much I trust my sensor" (small R , big trust). Q is "how much I trust my motion model" (small Q , big trust). You measure R : hold the sensor still, record for a minute, take the variance. You tune Q : it stands for wheel slip, a wrong wheel diameter, bumps in the floor.

- *What if R is tiny?* $K = P / (P + R)$ goes to 1. The estimate jumps straight to every measurement; the filter is just copying the sensor. P collapses to almost 0 after each update.

- *What if Q is huge?* P inflates in every predict, so K stays near 1 and again you are copying the sensor, one step late. No smoothing.
- *What if Q is tiny?* P shrinks and shrinks, K goes to 0, and the filter stops listening to the sensor. If the robot then actually gets bumped, the filter never notices: "the filter fell asleep". This is the most common real bug.
- *Start with a big P .* "I do not know where I am" is a large P_0 , which makes the first few gains close to 1 so the filter locks onto the sensor quickly. $P_0 = 0$ means "I am certain", and the filter will never believe a measurement.

The demo file. The scalar version, tested: the two gain limits above, P shrinking on update to exactly $P \cdot R / (P + R)$, and a 500-step simulation with a fixed random seed where the filtered error must be less than half the raw sensor error.

```
// 1D (scalar-state) Kalman filter with a deterministic simulation.
// State x = position [m]. Control u = commanded velocity [m/s].
// Process model: x_k = x_{k-1} + u*dt + w, w ~ N(0, q*dt)
// Measurement model: z_k = x_k + v, v ~ N(0, r)
#include <cassert>
#include <cmath>
#include <iostream>
#include <random>
#include <vector>

class Kalman1D {
public:
    // x0/p0: initial estimate and its variance. q: process noise variance per second.
    // r: measurement noise variance.
    Kalman1D(double x0, double p0, double q, double r) : x_(x0), p_(p0), q_(q), r_(r) {}

    // Predict step: x' = A x + B u with A = 1, B = dt. P' = A P A^T + Q (A = 1 here).
    void predict(double u, double dt) {
        x_ += u * dt;
        p_ += q_ * dt; // uncertainty only ever grows in predict
    }

    // Update step with z = H x + v, H = 1.
    void update(double z) {
        innovation_ = z - x_; // y = z - H x ("what the sensor says minus what I expected")
        const double s = p_ + r_; // S = H P H^T + R, variance of the innovation
        gain_ = p_ / s; // K = P H^T S^-1, in [0,1]: 0 = trust model, 1 = trust sensor
        x_ += gain_ * innovation_;
        p_ = (1.0 - gain_) * p_; // P = (I - K H) P, always <= previous P
    }

    double state() const { return x_; }
    double variance() const { return p_; }
    double gain() const { return gain_; }
    double innovation() const { return innovation_; }

private:
    double x_, p_, q_, r_;
    double gain_ = 0.0;
    double innovation_ = 0.0;
};

double rms(const std::vector<double>& errors) {
    double sum = 0.0;
    for (double e : errors) sum += e * e;
    return std::sqrt(sum / static_cast<double>(errors.size()));
}

int main() {
    // ---- gain limits: the two questions every interviewer asks ----
    {
        Kalman1D trustSensor(0.0, 1.0, 0.0, 1e-9); // R -> 0
        trustSensor.update(5.0);
        assert(std::fabs(trustSensor.state() - 5.0) < 1e-6); // K -> 1, estimate jumps to z
    }
}
```

```

    assert(trustSensor.gain() > 0.999);

    Kalman1D trustModel(0.0, 1e-9, 0.0, 1.0);    // P -> 0 (or R huge)
    trustModel.update(5.0);
    assert(std::fabs(trustModel.state()) < 1e-6);    // K -> 0, measurement ignored
    assert(trustModel.gain() < 1e-6);
}

// ---- P shrinks on update, grows on predict ----
{
    Kalman1D kf(0.0, 4.0, 0.5, 1.0);
    const double p0 = kf.variance();
    kf.update(0.3);
    const double p1 = kf.variance();
    assert(p1 < p0);
    assert(std::fabs(p1 - (4.0 * 1.0) / (4.0 + 1.0)) < 1e-12);    // P R / (P + R)
    kf.predict(0.0, 0.1);
    assert(kf.variance() > p1);
}

// ---- simulation: target moving at commanded velocity with process noise, noisy sensor ----
{
    const double dt = 0.1, q = 0.01, r = 1.0;    // sensor sigma = 1 m, model drifts slowly
    std::mt19937 rng(42);    // fixed seed: the test is reproducible
    std::normal_distribution<double> processNoise(0.0, std::sqrt(q * dt));
    std::normal_distribution<double> measNoise(0.0, std::sqrt(r));

    double xTrue = 0.0;
    Kalman1D kf(0.0, 1.0, q, r);
    std::vector<double> rawErr, filtErr;
    for (int k = 0; k < 500; ++k) {
        const double u = 1.0;    // commanded 1 m/s
        xTrue += u * dt + processNoise(rng);
        const double z = xTrue + measNoise(rng);

        kf.predict(u, dt);
        kf.update(z);

        rawErr.push_back(z - xTrue);
        filtErr.push_back(kf.state() - xTrue);
    }
    const double rawRms = rms(rawErr), filtRms = rms(filtErr);
    std::cout << "raw RMS = " << rawRms << " m, filtered RMS = " << filtRms
                << " m, steady-state P = " << kf.variance() << ", K = " << kf.gain() << "\n";
    assert(filtRms < 0.5 * rawRms);    // the filter must beat the sensor clearly
    assert(kf.variance() < r);    // posterior tighter than one measurement
}

std::cout << "OK 01_kalman_1d.cpp\n";
return 0;
}

```

What to look at:

- `class Kalman1D` keeps four doubles: `x_` and `p_` (the belief), `q_` and `r_` (the two trust settings). The trailing underscore is a common C++ habit for member variables.
- `Kalman1D(double x0, double p0, double q, double r) : x_(x0), p_(p0), q_(q), r_(r) {}` is a constructor with an *initialiser list*: the part after the colon sets each member directly. Python's `self.x = x0` in `__init__`.
- `predict(u, dt)` is the two predict lines; `update(z)` is the five update lines, exactly as written above. Read them side by side with the table.
- `double state() const { return x_; }` is a *getter*; `const` promises it does not change the object.
- `std::mt19937 rng(42)` is a random number generator with a fixed seed so the test gives the same numbers every run; `std::normal_distribution<double>(0.0, sigma)` draws bell-curve noise.
- The printed line: raw RMS about 1.05 m (the sensor's own noise), filtered RMS about 0.19 m, and a steady-state K of about 0.03. Why so small? The model drifts by only $Q \cdot dt = 0.001$ per step while the

sensor's variance is 1, so the filter trusts the model far more and nudges the estimate by only 3 % of each surprise.

Try it: in the simulation block change $q = 0.01$ to $q = 10.0$ and predict what happens to the filtered RMS and to K before you run it. Then set $r = 1e-6$ and predict again.

The matrix version: position and velocity at once. Now the state is two numbers, position p and velocity v , and the sensor still measures only position. It is *the same blend but with two numbers at once*: x becomes a column of 2, P becomes a 2 by 2 table (how unsure about p , how unsure about v , and how the two errors are *linked*), and the plain multiply and divide become matrix multiply and matrix inverse.

```
state   x = [ p ]      "move" matrix   A = [ 1  dt ]      new p = p + v*dt
        [ v ]          [ 0  1 ]      new v = v

sensor  H = [ 1  0 ]    picks the position out of the state: H * x = p
```

The equations keep their shape. Compare each line with the scalar one:

```
Predict:  x = A x          P = A P A' + Q
Update:   y = z - H x      S = H P H' + R
          K = P H' / S      x = x + K y      P = (I - K H) P
```

(A' means the transpose of A , rows and columns swapped. I is the identity, the matrix version of the number 1.)

Why can the filter recover the velocity when nobody measures it? Two photographs a second apart tell you the speed. In the maths: the predict step says "new position = old position + $v \cdot dt$ ". If the velocity guess is wrong, the predicted position is wrong by $v_error \cdot dt$, so after a predict the position error and the velocity error move together: the off-diagonal entry $P(0,1)$ becomes non-zero. When the sensor then reports a position surprise y , the gain's second entry $K(1) = P(0,1) / S$ says "part of that surprise was the velocity's fault" and corrects the velocity too. The demo asserts exactly this: velocity is never measured, yet the final estimate is within 0.5 m/s of the truth.

```
// Constant-velocity Kalman filter in matrix form with Eigen.
// State x = [p, v]^T. Only position is measured.
// x_k = A x_{k-1} + w,   A = [1 dt; 0 1],   w ~ N(0, Q)
// z_k = H x_k + v,     H = [1 0],         v ~ N(0, R)
// Compile: g++ -std=c++17 -Wall -Wextra -O2 -I/usr/include/eigen3 01b_kalman_cv_eigen.cpp
#include <Eigen/Dense>
#include <cassert>
#include <cmath>
#include <iostream>
#include <random>
#include <vector>

class KalmanCV {
public:
    // accelSigma: standard deviation of the unmodelled acceleration (drives Q).
    // r: position measurement variance.
    KalmanCV(double dt, double accelSigma, double r) : dt_(dt), r_(r) {
        A_ << 1.0, dt,
              0.0, 1.0;
        H_ << 1.0, 0.0;
        // Discrete white-noise-acceleration model: Q = sigma_a^2 * [dt^4/4 dt^3/2; dt^3/2 dt^2]
        const double s2 = accelSigma * accelSigma;
        Q_ << s2 * dt * dt * dt * dt / 4.0, s2 * dt * dt * dt / 2.0,
              s2 * dt * dt * dt / 2.0,   s2 * dt * dt;
        x_.setZero();
        P_ = Eigen::Matrix2d::Identity() * 10.0;    // "I don't know much yet"
    }

    void predict() {
        x_ = A_ * x_;
        P_ = A_ * P_ * A_.transpose() + Q_;
    }

    void update(double z) {
```

```

    const double y = z - (H_ * x_)(0); // innovation (scalar)
    const double S = (H_ * P_ * H_.transpose())(0) + r_; // innovation covariance (1x1)
    const Eigen::Vector2d K = P_ * H_.transpose() / S; // 2x1 gain
    x_ += K * y;
    // Joseph form keeps P symmetric positive-definite under round-off; the
    // textbook (I - K H) P is fine on a whiteboard, this one is what you ship.
    const Eigen::Matrix2d IKH = Eigen::Matrix2d::Identity() - K * H_;
    P_ = IKH * P_ * IKH.transpose() + K * r_ * K.transpose();
}

const Eigen::Vector2d& state() const { return x_; }
const Eigen::Matrix2d& covariance() const { return P_; }

private:
    double dt_, r_;
    Eigen::Matrix2d A_, Q_, P_;
    Eigen::RowVector2d H_;
    Eigen::Vector2d x_;
};

int main() {
    const double dt = 0.1, accelSigma = 0.2, r = 1.0;
    std::mt19937 rng(7);
    std::normal_distribution<double> accelNoise(0.0, accelSigma);
    std::normal_distribution<double> measNoise(0.0, std::sqrt(r));

    double pTrue = 0.0, vTrue = 1.0;
    KalmanCV kf(dt, accelSigma, r);
    double rawSq = 0.0, filtSq = 0.0;
    const int steps = 600;
    for (int k = 0; k < steps; ++k) {
        const double a = accelNoise(rng);
        pTrue += vTrue * dt + 0.5 * a * dt * dt;
        vTrue += a * dt;
        const double z = pTrue + measNoise(rng);

        kf.predict();
        kf.update(z);

        rawSq += (z - pTrue) * (z - pTrue);
        filtSq += (kf.state()(0) - pTrue) * (kf.state()(0) - pTrue);
    }
    const double rawRms = std::sqrt(rawSq / steps), filtRms = std::sqrt(filtSq / steps);
    std::cout << "raw RMS = " << rawRms << ", filtered RMS = " << filtRms
        << ", v_est = " << kf.state()(1) << " (true " << vTrue << ")\n";
    std::cout << "P =\n" << kf.covariance() << "\n";

    assert(filtRms < 0.5 * rawRms);
    assert(std::fabs(kf.state()(1) - vTrue) < 0.5); // velocity is never measured, yet estimated
    assert(kf.covariance().isApprox(kf.covariance().transpose())); // symmetric
    assert(kf.covariance()(0, 0) > 0.0 && kf.covariance()(1, 1) > 0.0);
    assert(kf.covariance()(0, 0) < r); // better than a single measurement

    std::cout << "OK 01b_kalman_cv_eigen.cpp\n";
    return 0;
}

```

What is new (Eigen):

- `#include <Eigen/Dense>` brings in the Eigen matrix library. Compile with `-isystem /usr/include/eigen3`.
- `Eigen::Matrix2d` is a 2 by 2 table of doubles. `Eigen::Vector2d` is a column of 2 doubles. `Eigen::RowVector2d` is a row of 2 doubles (that is H, 1 by 2). The `d` means double.
- `A_ << 1.0, dt, 0.0, 1.0;` fills a matrix row by row. Python numpy: `np.array([[1, dt], [0, 1]])`.
- `A_ * x_` is matrix times vector; `.transpose()` swaps rows and columns; `.inverse()` gives the matrix that undoes another (here `S` is 1 by 1 so the code just divides by it).
- `(H_ * x_)(0)`: `H * x` is a 1 by 1 matrix, `(0)` takes its only element as a plain double.
- `x_.setZero()` fills with zeros; `Eigen::Matrix2d::Identity() * 10.0` is a big starting `P` ("I know nothing yet").
- `kf.state()(1)` is element 1 of the state vector: the velocity. `covariance()(0, 0)` is row 0, column 0 of `P`.

- Q is filled from `accelSigma`, the standard deviation of a random unknown acceleration. The four entries spread that one number into position and velocity uncertainty; you do not need to memorise them, just know that Q is "let the true velocity wander a bit".

Here are the same Eigen types on their own, so you can see what each call does:

```
Eigen::Matrix2d A;           // a 2x2 table of doubles
A << 1.0, 0.1,              // fill it row by row
    0.0, 1.0;
Eigen::Vector2d x(3.0, 2.0); // a column of 2 doubles: position 3, velocity 2
Eigen::Vector2d next = A * x; // matrix times vector: (3 + 0.1*2, 2) = (3.2, 2)
Eigen::Matrix2d At = A.transpose(); // rows become columns
Eigen::Matrix2d Ainv = A.inverse(); // the matrix that undoes A
Eigen::RowVector2d H;           // a row of 2 doubles: picks out the position
H << 1.0, 0.0;
double measured = (H * x)(0); // 1x1 result, take element 0 -> 3.0
```

(You can skip this on a first read.) The update in 01b uses the "Joseph form" $P = (I - KH) P (I - KH)' + K R K'$ instead of the shorter $(I - KH) P$. Both are the same algebra; the long one keeps P symmetric under floating-point round-off over millions of steps, which is why shipped code uses it. Also: always run `predict` every step even when no measurement arrived, and only skip update. Skipping predict is a bug because P must keep growing while the robot moves blind.

EKF and UKF in two sentences each. Real robots are not straight lines: a bearing sensor gives an angle, a car's motion is a curve. The *Extended* Kalman filter (EKF) keeps the same predict/update skeleton but, at every step, replaces the curved motion and sensor functions with the straight line that touches them at the current estimate (a Jacobian), and uses that in place of A and H . The *Unscented* Kalman filter (UKF) does not draw a tangent line at all: it picks a handful of sample points spread around the estimate, pushes each one through the real curved function, and rebuilds the mean and variance from where they landed. The UKF is used when the curve bends hard or the Jacobians are painful to derive.

In your own words: "A Kalman filter keeps a guess and a variance. Predict shifts the guess by the motion and grows the variance by Q ; update blends in the sensor with gain $K = P/(P+R)$ and shrinks the variance. Small R means trust the sensor, small Q means trust the model. With two state numbers the same blend runs on matrices, and the link between position and velocity in P lets the filter recover a velocity it never measures."

b2. PID controller with anti-windup, plus a tiny simulation

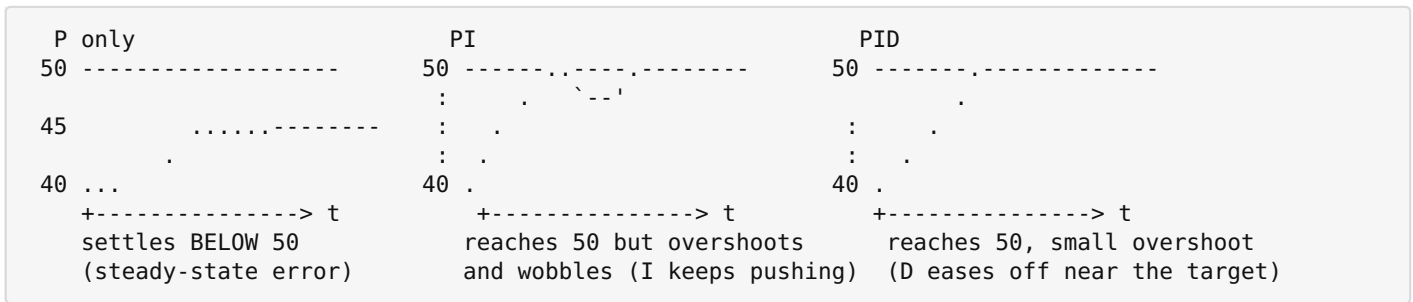
The story: cruise control. You want the car to go 50 km/h. It is going 40. How hard do you press the pedal? The gap ($50 - 40 = 10$) is called the *error*, and a PID controller is three common-sense rules about the pedal, added together:

- **P (proportional):** press the pedal *in proportion to the gap*. Ten under, press fairly hard; one under, press a little; at the target, ease off. $P = k_p * \text{error}$.
- **I (integral):** if you have been *below the target for a long time* (you are climbing a hill and P alone is not enough), press harder and harder, and keep the extra pressure. The integral is the running total of the error over time. $I = k_i * \text{sum}(\text{error} * dt)$.
- **D (derivative):** if you are *closing the gap fast*, ease off before you get there so you do not shoot past. The derivative is how fast the error is changing. $D = k_d * d(\text{error})/dt$.

Pedal position $u = P + I + D$. That is the whole controller.

Why P alone is not enough. On a flat road P works: the gap shrinks, the pedal eases, you settle. On a hill you need *some* pedal just to hold 50, but P gives zero pedal at zero gap. So the car settles a little below 50, where the gap is exactly big enough to hold the hill: a *steady-state error*. I fixes that by remembering: the small gap keeps adding up until the extra pedal holds the hill with the gap at zero.

Rough response curves. Speed against time after setting the target to 50 on a hill:



Integral windup, the story. Now the target jumps from 0 to 100 on a steep hill. P is huge, so the pedal is already *on the floor*. The car cannot accelerate any faster than that, no matter what the controller asks for. But the integral does not know about the floor: for the whole long climb it keeps adding up the big error, growing to an enormous number. When the car finally reaches 100, the pedal should ease, but the integral is still enormous and holds the pedal on the floor. The car sails past 100 and keeps going, and it only comes back once the integral has been "paid back" by an equally long time *above* the target. That huge overshoot and slow wobble is *windup*. It happens on every large target change, at start-up, and whenever the machine is physically stuck.

The three fixes in plain words:

1. **Clamp (stop integrating while the pedal is on the floor).** If the requested output is beyond the limit, do not add to the integral this step, unless the error has the sign that would bring the output back inside the limits. Simplest; it is what the demo does by default.
2. **Back-calculation.** Compute how far over the limit the request was, and pull the integral back toward the value that would put the output *exactly* on the limit. Smoother, one more number to choose.
3. **Clamp the integral term itself** to some fixed range. Crude but common in embedded code.

Derivative kick. When the target jumps from 0 to 1 in one step, the error jumps too, so $d(\text{error})/dt$ is $1/dt$ for one sample: with $kd = 1$ and $dt = 0.01$ that is a spike of 100 into the motor for no good reason. The fix: differentiate the *measurement* instead of the error. When the target is constant, $d(\text{error})/dt = -d(\text{measurement})/dt$, so nothing changes in normal operation, but a target jump produces no spike. The demo does this (note the minus sign on the D term). It also runs the derivative through a small low-pass filter, because a raw difference of a noisy sensor reading is noise multiplied by $1/dt$.

Tuning order on a real motor: set I and D to zero. Raise k_p until the response is fast but starts to ring. Add kd to kill the ringing. Then add just enough k_i to remove the steady-state error. Log target, measurement and command every step; you cannot tune what you cannot plot. Run the loop at least 10 times faster than the machine can respond.

The Python you would have written.

```
class PID:
    def __init__(self, kp, ki, kd, out_min, out_max):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.out_min, self.out_max = out_min, out_max
        self.integral = 0.0
        self.prev_meas = None
    def update(self, setpoint, meas, dt):
        error = setpoint - meas
        d_meas = 0.0 if self.prev_meas is None else (meas - self.prev_meas) / dt
        self.prev_meas = meas
        candidate = self.integral + error * dt
        unsat = self.kp * error + self.ki * candidate - self.kd * d_meas
        out = min(max(unsat, self.out_min), self.out_max)
        if unsat == out:
            # not saturated: allowed to integrate
            self.integral = candidate
        return out
```

The plain (naive) version, without any of the fixes, is five lines and worth knowing by heart:

```
double error = setpoint - measurement;
integral += error * dt;           // I remembers
double derivative = (error - prevError) / dt; // D looks at the trend
prevError = error;
double u = kp * error + ki * integral + kd * derivative;
```

The demo file. A 1 kg mass on a damper, pushed by a weak motor that can give at most 1 N, told to move 10 m. The weak motor forces a long saturated run-up, which is exactly the windup story. The same controller is run three times: no anti-windup, clamping, back-calculation.

```
// Discrete PID controller with output saturation, integral anti-windup and a
// low-pass-filtered derivative on measurement. Plus a mass-damper simulation
// that shows a naive controller winding up and the anti-windup one not.
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <vector>

enum class AntiWindup { None, Clamp, BackCalculation };

class PID {
public:
    struct Gains { double kp, ki, kd; };

    PID(Gains g, double outMin, double outMax, AntiWindup mode, double dFilterTau = 0.0)
        : g_(g), outMin_(outMin), outMax_(outMax), mode_(mode), dTau_(dFilterTau) {}

    // One control step. Returns the saturated command.
    double update(double setpoint, double measurement, double dt) {
        const double error = setpoint - measurement;

        // Derivative on MEASUREMENT, not on error: a setpoint step then produces no
        // "derivative kick" (a one-sample spike of size kd*step/dt).
        double dMeas = 0.0;
        if (hasPrev_) dMeas = (measurement - prevMeas_) / dt;
        // First-order low-pass on the derivative; raw differences amplify sensor noise.
        const double alpha = dTau_ > 0.0 ? dt / (dTau_ + dt) : 1.0;
        dFiltered_ += alpha * (dMeas - dFiltered_);
        prevMeas_ = measurement;
        hasPrev_ = true;

        double integralCandidate = integral_ + error * dt;
        const double p = g_.kp * error;
        const double d = -g_.kd * dFiltered_; // minus: d(error)/dt = -d(meas)/dt for fixed setpoint

        double unsat = p + g_.ki * integralCandidate + d;
        double out = std::clamp(unsat, outMin_, outMax_);

        switch (mode_) {
            case AntiWindup::None:
                integral_ = integralCandidate; // keeps growing while saturated
                break;
            case AntiWindup::Clamp:
                // Conditional integration: only integrate if we are not saturated, or if
                // the error would drive the output back out of saturation.
                if (unsat == out || (unsat > outMax_ && error < 0.0) || (unsat < outMin_ && error > 0.0))
                    integral_ = integralCandidate;
                break;
            case AntiWindup::BackCalculation: {
                // Feed the saturation excess (out - unsat) back into the integrator so it
                // is pulled toward the value that would make unsat == out.
                const double kb = g_.ki > 0.0 ? 1.0 / g_.ki : 0.0; // typical: kb ~ 1/Ti
                integral_ = integralCandidate + kb * (out - unsat) * dt;
                break;
            }
        }
        lastUnsat_ = unsat;
        return out;
    }
};
```

```

}

void reset() { integral_ = 0.0; dFiltered_ = 0.0; hasPrev_ = false; }
double integral() const { return integral_; }
double lastUnsaturated() const { return lastUnsat_; }

private:
    Gains g_;
    double outMin_, outMax_;
    AntiWindup mode_;
    double dTau_;
    double integral_ = 0.0;
    double dFiltered_ = 0.0;
    double prevMeas_ = 0.0;
    double lastUnsat_ = 0.0;
    bool hasPrev_ = false;
};

// Plant: mass on a damper, force input.  $m \cdot a = u - b \cdot v$ . Output = position.
struct MassDamper {
    double m = 1.0, b = 0.8;
    double x = 0.0, v = 0.0;
    void step(double u, double dt) {
        const double a = (u - b * v) / m;
        v += a * dt;
        x += v * dt;
    }
};

struct SimResult {
    double overshoot; // max(x) - setpoint, in metres
    double finalError; // |x - setpoint| at the end
    double settleTime; // first time after which |error| stays < 2 %
};

SimResult simulate(AntiWindup mode, bool print) {
    const double dt = 0.01, tEnd = 80.0, setpoint = 10.0;
    const double uMax = 1.0; // a weak actuator: forces a long saturated run-up
    PID pid({1.0, 0.4, 1.2}, -uMax, uMax, mode, 0.05);
    MassDamper plant;

    SimResult res{0.0, 0.0, tEnd};
    double lastOutside = 0.0;
    const int steps = static_cast<int>(tEnd / dt);
    if (print) std::printf("    t    setpoint    position    command    integral\n");
    for (int k = 0; k <= steps; ++k) {
        const double t = k * dt;
        const double u = pid.update(setpoint, plant.x, dt);
        if (print && k % 800 == 0)
            std::printf("%5.1f %8.2f %9.3f %8.3f %9.3f\n", t, setpoint, plant.x, u, pid.integral());
        plant.step(u, dt);
        res.overshoot = std::max(res.overshoot, plant.x - setpoint);
        if (std::fabs(plant.x - setpoint) > 0.02 * setpoint) lastOutside = t;
    }
    res.finalError = std::fabs(plant.x - setpoint);
    res.settleTime = lastOutside;
    return res;
}

int main() {
    // ---- unit behaviour: no derivative kick on a setpoint step ----
    {
        PID pid({1.0, 0.0, 1.0}, -100.0, 100.0, AntiWindup::None);
        pid.update(0.0, 0.0, 0.1); // prime prevMeas_
        const double out = pid.update(1.0, 0.0, 0.1); // setpoint jumps, measurement does not
        assert(std::fabs(out - 1.0) < 1e-12); // pure P; derivative-on-error would give 11
    }
    // ---- unit behaviour: clamped integral stops growing while saturated ----
    {
        PID pid({1.0, 1.0, 0.0}, -1.0, 1.0, AntiWindup::Clamp);
        for (int i = 0; i < 100; ++i) pid.update(10.0, 0.0, 0.1);
        assert(pid.integral() < 1e-9); // never integrated: always saturated positive
    }
}

```

```

    PID naive({1.0, 1.0, 0.0}, -1.0, 1.0, AntiWindup::None);
    for (int i = 0; i < 100; ++i) naive.update(10.0, 0.0, 0.1);
    assert(std::fabs(naive.integral() - 100.0) < 1e-9); // 100 steps * 10 * 0.1
}

std::cout << "--- naive (no anti-windup) ---\n";
const SimResult naive = simulate(AntiWindup::None, true);
std::cout << "--- integral clamping ---\n";
const SimResult clamp = simulate(AntiWindup::Clamp, true);
const SimResult backCalc = simulate(AntiWindup::BackCalculation, false);

std::printf("overshoot: naive %.3f m, clamp %.3f m, back-calc %.3f m\n",
    naive.overshoot, clamp.overshoot, backCalc.overshoot);
std::printf("settle 2%: naive %.1f s, clamp %.1f s, back-calc %.1f s\n",
    naive.settleTime, clamp.settleTime, backCalc.settleTime);

assert(clamp.finalError < 0.05);
assert(backCalc.finalError < 0.05);
std::printf("final error: naive %.3f, clamp %.3f\n", naive.finalError, clamp.finalError);
assert(naive.finalError < 0.5); // it gets there eventually
assert(clamp.overshoot < 0.5 * naive.overshoot); // anti-windup clearly helps
assert(backCalc.overshoot < 0.5 * naive.overshoot);
assert(clamp.settleTime < naive.settleTime);

std::cout << "OK 02_pid.cpp\n";
return 0;
}

```

What is new:

- `enum class AntiWindup { None, Clamp, BackCalculation };` is a named set of choices, like a Python Enum. You write `AntiWindup::Clamp`. `switch (mode_) { case AntiWindup::None: ... }` picks the branch.
- `struct Gains { double kp, ki, kd; };` bundles the three numbers; `PID pid({1.0, 0.4, 1.2}, ...)` builds one with braces.
- `std::clamp(unsat, outMin_, outMax_)` is Python's `min(max(unsat, lo), hi)`: the pedal cannot go past the floor.
- `double dFilterTau = 0.0` in the constructor is a default argument, like Python's `tau=0.0`.
- `hasPrev_` is a flag so the very first derivative is not computed from garbage; `reset()` clears the integral and the flag, which you must call whenever the controller is re-enabled.
- `std::printf("%5.1f %8.2f ...", ...)` prints a formatted row; `%8.3f` means "a float, 8 characters wide, 3 decimals", the same as Python's `f"{x:8.3f}"`.

The table it prints, and what to look for. Columns are time, target, position, the command actually sent (already limited to plus or minus 1), and the integral. One row every 8 s.

```

--- naive (no anti-windup) ---
  t   setpoint  position  command  integral
  0.0   10.00    0.000    1.000    0.100
  8.0   10.00    8.453    1.000   50.531    <- pedal still on the floor, integral 50
 16.0   10.00   18.410   -0.490   22.882    <- sailed 8.4 m past the target
 24.0   10.00   11.221   -1.000  -23.386    <- now winding up the OTHER way
 32.0   10.00    6.339    1.000    0.351
 40.0   10.00   10.941   -0.391   -0.063
...
--- integral clamping ---
  8.0   10.00    8.434    0.759    1.535    <- integral stayed small
 16.0   10.00   10.097   -0.075   -0.442    <- overshoot 0.1 m, done
overshoot: naive 8.745 m, clamp 0.762 m, back-calc 0.861 m
settle 2%: naive 46.6 s, clamp 15.4 s, back-calc 15.4 s

```

Watch the integral column: 50 at 8 s for the naive run against 1.5 with clamping. That number is the windup, and the 8.7 m overshoot against 0.76 m is its cost. Both runs reach the target eventually; the clamped one gets there three times sooner.

Try it: change `uMax = 1.0` to `uMax = 100.0` (a strong motor) and predict the three overshoot numbers before running. Then put it back and change `ki` from 0.4 to 0.0 and predict the final error.

In your own words: "P presses in proportion to the gap, I keeps pressing while the gap has lasted a long time, D eases off when the gap is closing fast. Windup is the integral growing while the pedal is already on the floor, which causes a big overshoot; I stop integrating while saturated. I differentiate the measurement, not the error, to avoid a spike on a target change. Tune P, then D, then I."

b3. Rotation matrix, quaternion, transform chaining (Eigen)

Start in 2D: rotating a point. Put the point `(1, 0)` on a grid and turn it 90 degrees anticlockwise about the origin. It lands on `(0, 1)`.

```

      y
      |
  2 |
  1 | * (0, 1)  <- after turning 90 degrees
  0 +-----*-----> x
      |
      | 0      (1, 0)  <- before

```

The formula, with `theta` the angle:

```

x' = cos(theta) * x - sin(theta) * y
y' = sin(theta) * x + cos(theta) * y

```

With `theta = 90 degrees`, `cos = 0` and `sin = 1`, so `(1, 0)` gives `x' = 0*1 - 1*0 = 0`, `y' = 1*1 + 0*0 = 1`. Check another: `(2, 1)` gives `x' = 0*2 - 1*1 = -1`, `y' = 1*2 + 0*1 = 2`, so `(2, 1)` lands on `(-1, 2)`. Draw it: yes, that is a quarter turn to the left.

Those two lines are a matrix multiply. The **2 by 2 rotation matrix** is:

```

R(theta) = [ cos(theta)  -sin(theta) ]      R * [x] = [x']
            [ sin(theta)   cos(theta) ]      [y]    [y']

```

Two facts about it that you will use forever: turning by `theta` and then by `-theta` gets you back, so the inverse of `R` is `R(-theta)`, and because `cos(-t) = cos(t)` and `sin(-t) = -sin(t)` that inverse is just `R` with rows and columns swapped, the *transpose*. No division, no solver. And `R` never changes lengths: a 1 m stick is 1 m after rotating.

By hand in C++:

```

void rotate(double px, double py, double theta, double& outX, double& outY) {
    const double c = std::cos(theta), s = std::sin(theta);
    outX = c * px - s * py;
    outY = s * px + c * py;
}

```

A frame is a point of view. "The cup is at `(1, 0)`" means nothing until you say *from where*. From the camera, the cup is 1 m straight ahead. From the robot's base, the camera is half a metre ahead, so the cup is 1.5 m ahead of the base. From the room, the base is standing at `(2, 3)` and has turned 90 degrees to face along `+y`, so "1.5 m ahead of the base" is 1.5 m in the `+y` direction: the cup is at `(2, 4.5)` in the room. Each "from where" is a *frame*, and each hop is "rotate by the frame's heading, then add the frame's position".

```

room (world frame): x to the right, y up

```

```

y
5 |
4 |      C  cup at world (2, 4.5)
  |      |
3 |      B  base at (2, 3), facing +y

```

```

2 |           (camera is 0.5 m ahead of B, at (2, 3.5))
1 |
0 +-----+-----> x
  0      1      2      3

```

```

cup in camera frame: (1, 0)           1 m ahead of the camera
cup in base frame:   (1.5, 0)         = (1, 0) + camera offset (0.5, 0), no turn
cup in world frame:  (2, 4.5)         = rotate (1.5, 0) by 90 degrees -> (0, 1.5), then add (2, 3)

```

The rule for one hop: $p_{\text{parent}} = R(\text{heading}) * p_{\text{child}} + t$, where t is where the child frame sits in the parent. The naming trick that stops sign errors: call a transform T_{a_b} when it takes a point written in frame b and gives it in frame a . Then the chain is

```
p_world = T_world_base * T_base_cam * p_cam
```

and you read it right to left: the cup starts in `cam`, becomes `base`, becomes `world`. The inner subscripts touch (`base, base`), which is how you check you have not chained the wrong way round. The inverse T_{b_a} goes the other way: $R^T * (p - t)$, rotate back after undoing the shift.

The Python you would have written.

```

import math

def rotate(px, py, theta):
    c, s = math.cos(theta), math.sin(theta)
    return c * px - s * py, s * px + c * py

def apply_frame(x, y, theta, px, py):
    """point (px, py) in the child frame -> in the parent frame."""
    rx, ry = rotate(px, py, theta)
    return rx + x, ry + y

cup_cam = (1.0, 0.0)
cup_base = apply_frame(0.5, 0.0, 0.0, *cup_cam)           # camera 0.5 m ahead, same heading
cup_world = apply_frame(2.0, 3.0, math.pi / 2, *cup_base) # base at (2,3), turned 90 degrees
# cup_base == (1.5, 0.0); cup_world == (2.0, 4.5)

```

The same chain with Eigen's 2D types. `Isometry2d` is "a rotation plus a translation in one object", and `*` between two of them chains them:

```

Eigen::Isometry2d T_world_base = Eigen::Translation2d(2.0, 3.0) * Eigen::Rotation2Dd(kPi / 2.0);
Eigen::Isometry2d T_base_cam   = Eigen::Translation2d(0.5, 0.0) * Eigen::Rotation2Dd(0.0);
Eigen::Vector2d cup_cam(1.0, 0.0);
Eigen::Vector2d cup_world = T_world_base * T_base_cam * cup_cam; // read right to left: (2, 4.5)
Eigen::Vector2d back = T_base_cam.inverse() * T_world_base.inverse() * cup_world; // (1, 0) again

```

(`Translation2d(2, 3) * Rotation2Dd(a)` means "rotate first, then shift", which is exactly $R * p + t$.)

Now 3D. Everything above works, but a 3D rotation needs more than one angle. The intuitive way is **three angles** (roll, pitch, yaw: tilt sideways, nose up or down, turn left or right), called *Euler angles*. They are great for reading out to a human and terrible for computing: there are twelve different orderings, and they *break* at one spot:

gimbal lock, in one picture

normal (pitch 30 degrees):

```

yaw axis    : vertical
pitch axis  : sideways
roll axis   : forward

```

three different directions,
three freedoms

pitch = 90 degrees (nose straight up):

```

yaw axis    : vertical
pitch axis  : sideways
roll axis   : ALSO vertical now

```

yaw and roll do the same thing,
one freedom is gone: "gimbal lock"

At pitch 90 degrees, yawing and rolling turn the body about the same line, so infinitely many (yaw, roll) pairs describe one orientation and you cannot tell them apart. The demo asserts that (yaw 0.3, pitch 90, roll 0.5) and (yaw 0, pitch 90, roll 0.2) are the *same* matrix. Interpolating between Euler angles also gives wobbly paths. So: Euler angles for display only.

Rotation matrices in 3D are 3 by 3, and the two rules from 2D still hold: inverse equals transpose, lengths are kept. A matrix is a valid rotation if $R^T * R = I$ and its determinant is +1 (a determinant of -1 is a mirror image, not a turn). The columns of R_{a_b} are the three axes of frame b written in frame a , which is how you build one by hand; the demo builds the camera mount this way.

Quaternions are "one axis and one angle packed into four numbers". Any 3D rotation is a single turn by some angle about some line; a quaternion stores $w = \cos(\text{angle}/2)$ and $(x, y, z) = \sin(\text{angle}/2) * \text{axis}$. You do not derive anything with them; you follow the rules:

- **Keep unit length.** A rotation quaternion must have $w^2 + x^2 + y^2 + z^2 = 1$. After many multiplies round-off drifts it; call `.normalize()`.
- **Multiply to compose.** $q_1 * q_2$ means "do q_2 , then q_1 ", the same right-to-left reading as matrices.
- **Order matters.** $q_1 * q_2$ is not $q_2 * q_1$. Turning 90 degrees about z then 90 about x ends somewhere different from the reverse.
- q and $-q$ are the same rotation (both halves of the angle land on the same place). Compare rotations with `angularDistance`, not by comparing the four numbers.
- Eigen's constructor takes (w, x, y, z) in that order, but stores (x, y, z, w) inside `coeffs()`. A classic bug.

Why use them at all: no gimbal lock, four numbers instead of nine, cheap to compose, cheap to renormalise, and `slerp(q1, q2, t)` gives a smooth constant-speed turn from one orientation to another, which is what a trajectory between waypoints needs.

```
Eigen::Quaterniond q(Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitZ())); // 90 deg about z
Eigen::Vector3d p = q * Eigen::Vector3d(1.0, 0.0, 0.0); // rotate (1,0,0) -> (0,1,0)
Eigen::Quaterniond q2(Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitX()));
Eigen::Quaterniond first_q2_then_q = q * q2; // right one happens first
Eigen::Quaterniond first_q_then_q2 = q2 * q; // a different rotation
Eigen::Quaterniond identity(1.0, 0.0, 0.0, 0.0); // (w, x, y, z): no turn at all
```

Isometry3d is "rotation plus translation in one object". Inside it is a 4 by 4 matrix $[R \ t; 0 \ 1]$ so that rotating and shifting is one multiply; you never touch the 4 by 4 yourself. `T.linear()` is the rotation part, `T.translation()` the shift, `T.inverse()` uses the cheap closed form $(R^T, -R^T t)$, and `T * p` applies it to a 3D point including the shift.

```
Eigen::Isometry3d T = Eigen::Isometry3d::Identity();
T.linear() = Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitZ()).toRotationMatrix();
T.translation() = Eigen::Vector3d(2.0, 3.0, 0.0);
Eigen::Vector3d p = T * Eigen::Vector3d(1.5, 0.0, 0.0); // rotate, then shift: (2, 4.5, 0)
Eigen::Vector3d q = T.inverse() * p; // back to (1.5, 0, 0)
```

The demo file. It has a hand-written 2D `Transform2D` struct (write that from memory before you reach for a library) and then the 3D story: axis-angle to quaternion to matrix and back, the order-matters check, the gimbal lock check, a world-base-camera chain, inverses, `slerp`, and a final block where the hand-written 2D struct is checked against Eigen.

```
// Rotations, quaternions and rigid transforms with Eigen, plus a hand-written 2D
// transform for the whiteboard. Frame convention: T_a_b maps points in frame b
// to frame a, i.e. p_a = T_a_b * p_b. Chains read right-to-left.
// Compile: g++ -std=c++17 -Wall -Wextra -O2 -I/usr/include/eigen3 03_transforms_eigen.cpp
#include <Eigen/Dense>
#include <Eigen/Geometry>
#include <cassert>
#include <cmath>
#include <iostream>
```

```
constexpr double kPi = 3.14159265358979323846;

// ----- the whiteboard version: SE(2) by hand -----
struct Transform2D {
    double x, y, theta;    // translation and heading of frame b expressed in frame a

    // p_a = R(theta) p_b + t
    void apply(double px, double py, double& outX, double& outY) const {
        const double c = std::cos(theta), s = std::sin(theta);
        outX = c * px - s * py + x;
        outY = s * px + c * py + y;
    }

    // T_a_c = T_a_b * T_b_c : rotate the child's translation into our frame, add headings
    Transform2D compose(const Transform2D& other) const {
        Transform2D out{};
        apply(other.x, other.y, out.x, out.y);
        out.theta = std::remainder(theta + other.theta, 2.0 * kPi);    // wrap to (-pi, pi]
        return out;
    }

    // T_b_a = inverse: R^T and -R^T t
    Transform2D inverse() const {
        const double c = std::cos(theta), s = std::sin(theta);
        return Transform2D{-(c * x + s * y), -(s * x + c * y), -theta};
    }
};

bool isRotationMatrix(const Eigen::Matrix3d& R, double tol = 1e-9) {
    // Orthonormal columns and right-handed (det +1, not -1 which is a reflection).
    return (R.transpose() * R).isApprox(Eigen::Matrix3d::Identity(), tol) &&
        std::fabs(R.determinant() - 1.0) < tol;
}

int main() {
    // ---- axis-angle -> quaternion -> matrix and back ----
    const Eigen::AngleAxisd aa(kPi / 3.0, Eigen::Vector3d(1.0, 2.0, 3.0).normalized());
    const Eigen::Quaterniond q(aa);
    const Eigen::Matrix3d R = aa.toRotationMatrix();
    assert(std::fabs(q.norm() - 1.0) < 1e-12);    // unit quaternion
    assert(R.isApprox(q.toRotationMatrix()));
    assert(isRotationMatrix(R));
    assert(R.inverse().isApprox(R.transpose()));    // SO(3): inverse == transpose
    const Eigen::AngleAxisd back(q);
    assert(std::fabs(back.angle() - aa.angle()) < 1e-12);
    assert(back.axis().isApprox(aa.axis()));
    // Double cover: q and -q are the same rotation.
    const Eigen::Quaterniond negQ(-q.w(), -q.x(), -q.y(), -q.z());
    assert(negQ.toRotationMatrix().isApprox(R));
    assert(q.angularDistance(negQ) < 1e-12);

    // ---- composition: matrix product == quaternion product, and order matters ----
    const Eigen::Quaterniond q1(Eigen::AngleAxisd(0.7, Eigen::Vector3d::UnitZ()));
    const Eigen::Quaterniond q2(Eigen::AngleAxisd(-0.4, Eigen::Vector3d::UnitX()));
    const Eigen::Matrix3d R12 = q1.toRotationMatrix() * q2.toRotationMatrix();
    assert(R12.isApprox((q1 * q2).toRotationMatrix()));
    assert(!R12.isApprox((q2 * q1).toRotationMatrix()));    // rotations do not commute
    const Eigen::Vector3d v(1.0, 0.0, 0.0);
    assert((q1 * v).isApprox(q1.toRotationMatrix() * v));    // q * v is "rotate v by q"

    // ---- gimbal lock: at pitch = 90 deg roll and yaw become the same axis ----
    {
        const Eigen::Matrix3d Rlock = (Eigen::AngleAxisd(0.3, Eigen::Vector3d::UnitZ()) *
            Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitY()) *
            Eigen::AngleAxisd(0.5, Eigen::Vector3d::UnitX())).toRotationMatrix();
        const Eigen::Matrix3d Rlock2 = (Eigen::AngleAxisd(0.0, Eigen::Vector3d::UnitZ()) *
            Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitY()) *
            Eigen::AngleAxisd(0.2, Eigen::Vector3d::UnitX())).toRotationMatrix();
        assert(Rlock.isApprox(Rlock2, 1e-9));    // (yaw 0.3, roll 0.5) == (yaw 0, roll 0.2): a DOF is lost
    }

    // ---- SE(3) chain: world <- base <- camera ----
    Eigen::Isometry3d T_world_base = Eigen::Isometry3d::Identity();
    T_world_base.rotate(Eigen::AngleAxisd(kPi / 2.0, Eigen::Vector3d::UnitZ()));    // robot faces +y
    T_world_base.pretranslate(Eigen::Vector3d(2.0, 1.0, 0.0));    // at (2,1,0)

    // Camera optical frame (x right, y down, z forward) expressed in body axes
}
```

```

// (x forward, y left, z up). Columns of R_base_cam are the camera axes in base coords.
Eigen::Matrix3d R_base_cam;
R_base_cam.col(0) = Eigen::Vector3d(0.0, -1.0, 0.0); // optical x -> body -y
R_base_cam.col(1) = Eigen::Vector3d(0.0, 0.0, -1.0); // optical y -> body -z
R_base_cam.col(2) = Eigen::Vector3d(1.0, 0.0, 0.0); // optical z -> body +x
assert(isRotationMatrix(R_base_cam));
Eigen::Isometry3d T_base_cam = Eigen::Isometry3d::Identity();
T_base_cam.linear() = R_base_cam;
T_base_cam.translation() = Eigen::Vector3d(0.3, 0.0, 0.5); // 30 cm ahead, 50 cm up

const Eigen::Isometry3d T_world_cam = T_world_base * T_base_cam; // chain reads right-to-left
const Eigen::Vector3d p_cam(0.0, 0.0, 2.0); // 2 m straight ahead (optical z)
const Eigen::Vector3d p_world = T_world_cam * p_cam;
// Camera looks along the robot's +x, robot's +x is world +y, camera sits at base (0.3,0,0.5):
// point is at base (2.3, 0, 0.5) -> world (2 - 0, 1 + 2.3, 0.5)
assert(p_world.isApprox(Eigen::Vector3d(2.0, 3.3, 0.5)));

// Round trip through the inverse, and inverse of a chain is the reversed chain of inverses.
assert((T_world_cam.inverse() * p_world).isApprox(p_cam));
assert(T_world_cam.inverse().isApprox(T_base_cam.inverse() * T_world_base.inverse()));
assert(isRotationMatrix(T_world_cam.rotation()));
// Inverse by hand: [R^T, -R^T t]
Eigen::Isometry3d invByHand = Eigen::Isometry3d::Identity();
invByHand.linear() = T_world_cam.rotation().transpose();
invByHand.translation() = -T_world_cam.rotation().transpose() * T_world_cam.translation();
assert(invByHand.isApprox(T_world_cam.inverse()));

// ---- slerp: constant angular velocity between two orientations ----
{
    const Eigen::Quaterniond qa(Eigen::AngleAxisd(0.0, Eigen::Vector3d::UnitZ()));
    const Eigen::Quaterniond qb(Eigen::AngleAxisd(1.0, Eigen::Vector3d::UnitZ()));
    const Eigen::Quaterniond qHalf = qa.slerp(0.5, qb);
    const Eigen::AngleAxisd aaHalf(qHalf);
    assert(std::fabs(aaHalf.angle() - 0.5) < 1e-12);
    assert(qa.slerp(0.0, qb).isApprox(qa) && qa.slerp(1.0, qb).isApprox(qb));
    // Nlerp (normalize(lerp)) hits the same endpoints but is not constant-speed in between.
    Eigen::Quaterniond nlerp(qa.coeffs() * 0.5 + qb.coeffs() * 0.5);
    nlerp.normalize();
    assert(std::fabs(Eigen::AngleAxisd(nlerp).angle() - 0.5) < 1e-12); // symmetric case: equal
}

// ---- 2D whiteboard version agrees with Eigen ----
{
    const Transform2D T_a_b{1.0, 2.0, kPi / 4.0};
    const Transform2D T_b_c{0.5, -0.5, kPi / 6.0};
    const Transform2D T_a_c = T_a_b.compose(T_b_c);
    double px, py;
    T_a_c.apply(1.0, 1.0, px, py);

    const Eigen::Isometry2d E_a_b = Eigen::Translation2d(1.0, 2.0) * Eigen::Rotation2Dd(kPi / 4.0);
    const Eigen::Isometry2d E_b_c = Eigen::Translation2d(0.5, -0.5) * Eigen::Rotation2Dd(kPi / 6.0);
    const Eigen::Vector2d pe = (E_a_b * E_b_c) * Eigen::Vector2d(1.0, 1.0);
    assert(std::fabs(px - pe.x()) < 1e-12 && std::fabs(py - pe.y()) < 1e-12);

    const Transform2D ident = T_a_c.compose(T_a_c.inverse());
    assert(std::fabs(ident.x) < 1e-12 && std::fabs(ident.y) < 1e-12 && std::fabs(ident.theta) < 1e-12);
}

std::cout << "OK 03_transforms_eigen.cpp\n";
return 0;
}

```

What is new:

- `#include <Eigen/Geometry>` adds the rotation and transform types.
- `Eigen::AngleAxisd(angle, axis)` is the "one axis, one angle" object; `Eigen::Quaterniond q(aa)` converts it; `aa.toRotationMatrix()` gives the 3 by 3 `Eigen::Matrix3d`.
- `Eigen::Vector3d::UnitZ()` is `(0, 0, 1)`; `.normalized()` scales a vector to length 1.
- `.isApprox(other)` compares with a tolerance; never use `==` on doubles.
- `R.determinant()` and `R.transpose()` are what `isRotationMatrix` checks.

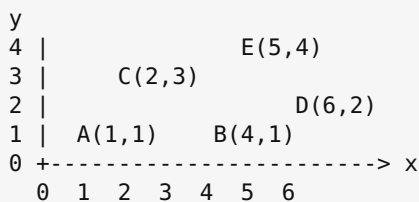
- `T_world_base.rotate(...)` then `.pretranslate(...)`: build the rotation, then put the frame at a position. `pretranslate` adds the shift *after* the rotation, which is what "the frame is at `t` with heading `R`" means. Setting `.linear()` and `.translation()` directly, as the camera mount does, is the safer habit.
- `R_base_cam.col(0) = ...` writes one column: the camera's `x` axis expressed in base axes. A camera's optical frame is `x` right, `y` down, `z` forward, while a robot body is `x` forward, `y` left, `z` up, so the three columns are $(0, -1, 0)$, $(0, 0, -1)$, $(1, 0, 0)$.
- `qa.slerp(0.5, qb)` is the orientation half way from `qa` to `qb`.
- `std::remainder(a, 2*kPi)` wraps an angle into $(-\pi, \pi]$.
- `double& outX in Transform2D::apply` is a reference parameter: the function writes the answer into the caller's variable, the C++ way of returning two values.

Try it: in the SE(3) block change `p_cam` to $(0, 0, 1)$ (a point 1 m ahead of the camera instead of 2 m) and predict `p_world` before running. Then swap the order to `T_base_cam * T_world_base` and see which assert fires.

In your own words: "A rotation is a matrix with inverse equal to its transpose. A frame is a point of view; `T_a_b` takes points from frame `b` into frame `a`, chains read right to left, and the inner names must match. Euler angles are for display only because of gimbal lock; I compute with quaternions (keep them unit length, multiply to compose, order matters) or with `Isometry3d`, which holds a rotation and a translation together."

b4. Closest pair of 2D points / line fitting (RANSAC discussion)

Closest pair, brute force. Five points; which two are nearest?



Compare every pair, keep the smallest distance. A-C is $\sqrt{1 + 4} = 2.24$, B-D is $\sqrt{4 + 1} = 2.24$, A-B is 3, and so on; the answer is 2.24 (a tie between A-C and B-D). With `n` points that is $n*(n-1)/2$ comparisons, so time grows with `n` squared. For 2,000 points that is 2 million distances, fine. For 2 million points it is not.

```
double closestPairBrute(const std::vector<Point>& pts) {
    double best = std::numeric_limits<double>::infinity();
    for (std::size_t i = 0; i < pts.size(); ++i)
        for (std::size_t j = i + 1; j < pts.size(); ++j)
            best = std::min(best, std::hypot(pts[i].x - pts[j].x, pts[i].y - pts[j].y));
    return best;
}
```

`std::hypot(dx, dy)` is $\sqrt{dx*dx + dy*dy}$; `infinity()` is Python's `float("inf")`.

Closest pair, divide and conquer. The faster idea: split, solve the halves, then check only the narrow strip around the split.

1. sort by `x`, cut at the middle

<pre> </pre>		<pre> </pre>
--	--	--
2. solve each half on its own

<pre> d_left = 2 </pre>		<pre> d_right = 3 </pre>
<pre> </pre>		
<pre> </pre>		
<pre> d = min(2, 3) = 2 </pre>		
3. the only pairs still unchecked straddle the cut. Both points of such a pair must be within `d` of the cut line, so look only in this strip:

```

      [ | ]      width 2d, points sorted by y
    . [ . | . ]
      [ | . ]    inside the strip, each point is compared with the
    . [ . | ]    few points just above it (at most 7): the strip is
      [ | ]      cheap. Any closer pair found here lowers d.

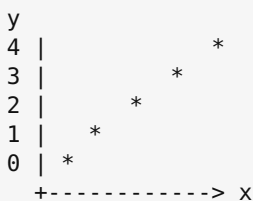
```

Why only 7? Points on one side are already at least d apart from each other, so only a handful can fit in a d by $2d$ box. That makes the strip pass linear, and the whole thing is sort-and-recurse: $n \log n$. In robotics the practical answer is often a third one: drop the points into grid cells of side d_0 (your sensor's spacing) and compare only within a cell and its 8 neighbours. It is expected linear time and it also answers "all pairs within radius r ", which is what clustering a laser scan actually needs.

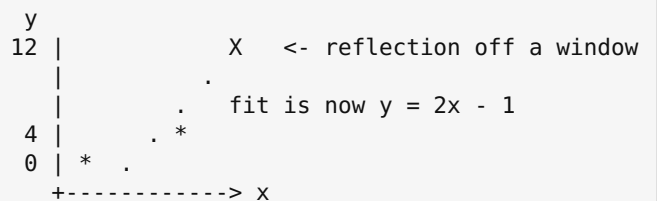
Line fitting: least squares in plain words. You have laser points along a wall and want the wall as a line. *Least squares* picks the line that makes the sum of the squared distances from the points to the line as small as possible: nudge the line up, some distances grow, some shrink; the best line is where the total squared miss is smallest. Two flavours: ordinary least squares measures the *vertical* miss (it fits $y = m x + c$ and cannot represent a vertical wall, the slope would be infinite), total least squares measures the *perpendicular* miss and handles every direction. The demo uses the perpendicular version and writes a line as $n_x * x + n_y * y = d$ with (n_x, n_y) a unit normal, so a vertical wall is nothing special.

Why one bad point wrecks it. Squaring punishes big misses enormously, so one far-away point drags the whole line toward itself:

five points on $y = x$, fit is $y = x$ (perfect):



add ONE outlier at (4, 12):



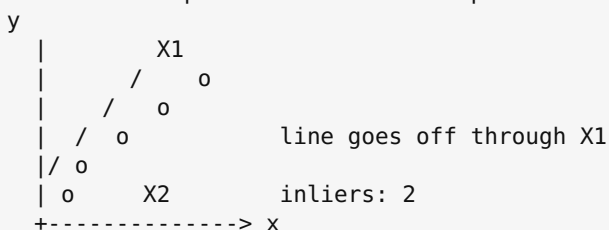
The slope doubled and the intercept moved from 0 to -1 because of a single point. Laser scans are full of such points: a reflection, a person walking past, a chair leg.

RANSAC is the fix, and it is nothing more than "guess, count, keep the best":

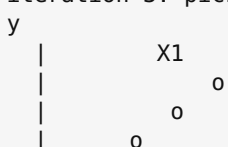
1. Pick 2 points at random. Draw the line through them. (2 is the *minimum* that defines a line.)
2. Count how many of *all* the points lie within a small distance t of that line: the *inliers*.
3. Repeat N times. Keep the line with the most inliers.
4. Finally, refit the line by least squares using only the inliers of the winner (the 2-point line was rough; the refit on 100 clean points is precise).

Three iterations drawn, on a wall with two bad points:

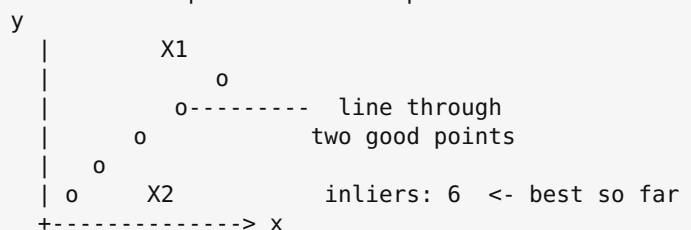
iteration 1: picked X1 and a wall point



iteration 3: picked X2 and a wall point



iteration 2: picked two wall points



after N tries keep the iteration-2 line, then refit it on its 6 inliers. Notice X1 and X2 were never "removed"; they simply never got a majority.

inliers: 2

How many tries? If a fraction e of the points are outliers and you need s points per guess, one guess is all-good with probability $(1 - e)^s$. To be p sure of at least one all-good guess:

$$N = \log(1 - p) / \log(1 - (1 - e)^s)$$

Worked number: 30 % outliers, a 2-point line, $p = 0.99$. $(1 - 0.3)^2 = 0.49$. $\log(0.01) = -4.605$, $\log(1 - 0.49) = \log(0.51) = -0.673$. $N = 4.605 / 0.673 = 6.84$, so 7 tries. The demo asserts `ransacIterations(0.3, 2) == 7`. For 50 % outliers and a 3-point plane it is 35. In practice run 3 to 5 times more, because "both points are good" does not mean "the line is good" when the two points are right next to each other. Pick t from the sensor's noise, about 2 to 3 standard deviations of its range error.

The Python you would have written (brute force and RANSAC; the refit is left to the C++):

```
import math, random

def closest_pair_brute(pts):
    best = float("inf")
    for i in range(len(pts)):
        for j in range(i + 1, len(pts)):
            best = min(best, math.dist(pts[i], pts[j]))
    return best

def line_through(a, b):
    nx, ny = -(b[1] - a[1]), b[0] - a[0]    # normal is perpendicular to a->b
    L = math.hypot(nx, ny)
    nx, ny = nx / L, ny / L
    return nx, ny, nx * a[0] + ny * a[1]    # nx*x + ny*y = d

def point_line_distance(line, p):
    nx, ny, d = line
    return abs(nx * p[0] + ny * p[1] - d)

def ransac_line(pts, threshold, iterations, rng):
    best_line, best_inliers = None, []
    for _ in range(iterations):
        a, b = rng.sample(pts, 2)
        cand = line_through(a, b)
        inliers = [p for p in pts if point_line_distance(cand, p) < threshold]
        if len(inliers) > len(best_inliers):
            best_line, best_inliers = cand, inliers
    return best_line, best_inliers

def ransac_iterations(outlier_ratio, sample_size, p=0.99):
    all_inlier = (1 - outlier_ratio) ** sample_size
    return math.ceil(math.log(1 - p) / math.log(1 - all_inlier))    # (0.3, 2) -> 7
```

The demo file. No Eigen. Brute force and divide-and-conquer closest pair checked against each other on 20 random sets, the perpendicular least-squares fit checked on a vertical line, and RANSAC on 140 good points plus 60 random outliers.

```
// Closest pair of 2D points (brute force and divide & conquer) and line fitting:
// total least squares via the 2x2 covariance eigenvector, and RANSAC on top of it.
#include <algorithm>
#include <cassert>
#include <cmath>
#include <iostream>
#include <limits>
#include <random>
#include <utility>
#include <vector>

struct Point { double x, y; };
```

```

double dist(const Point& a, const Point& b) { return std::hypot(a.x - b.x, a.y - b.y); }

// ----- closest pair:  $O(n^2)$  reference -----
double closestPairBrute(const std::vector<Point>& pts) {
    double best = std::numeric_limits<double>::infinity();
    const int n = static_cast<int>(pts.size());
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j) best = std::min(best, dist(pts[i], pts[j]));
    return best;
}

// ----- closest pair:  $O(n \log n)$  divide & conquer -----
// px sorted by x; py the same points sorted by y (maintained through the recursion).
double closestRec(const std::vector<Point>& px, const std::vector<Point>& py) {
    const int n = static_cast<int>(px.size());
    if (n <= 3) return closestPairBrute(px);

    const int mid = n / 2;
    const double midX = px[mid].x;
    std::vector<Point> lx(px.begin(), px.begin() + mid), rx(px.begin() + mid, px.end());
    std::vector<Point> ly, ry; // split py by side, keeping y order
    for (const auto& p : py) {
        if (p.x < midX || (p.x == midX && ly.size() < lx.size())) ly.push_back(p);
        else ry.push_back(p);
    }
    double d = std::min(closestRec(lx, ly), closestRec(rx, ry));

    // Strip: points within d of the split line, in y order. Any pair closer than d
    // inside the strip is at most 7 positions apart in y order (packing argument).
    std::vector<Point> strip;
    for (const auto& p : py)
        if (std::fabs(p.x - midX) < d) strip.push_back(p);
    const int m = static_cast<int>(strip.size());
    for (int i = 0; i < m; ++i)
        for (int j = i + 1; j < m && strip[j].y - strip[i].y < d; ++j)
            d = std::min(d, dist(strip[i], strip[j]));
    return d;
}

double closestPairFast(std::vector<Point> pts) {
    if (pts.size() < 2) return std::numeric_limits<double>::infinity();
    std::vector<Point> px = pts, py = pts;
    std::sort(px.begin(), px.end(), [](const Point& a, const Point& b) { return a.x < b.x; });
    std::sort(py.begin(), py.end(), [](const Point& a, const Point& b) { return a.y < b.y; });
    return closestRec(px, py);
}

// ----- line fitting -----
// Line in normal form:  $nx \cdot x + ny \cdot y = d$  with  $(nx, ny)$  unit. Works for vertical lines,
// unlike  $y = m x + c$ .
struct Line { double nx, ny, d; };

double pointLineDistance(const Line& l, const Point& p) {
    return std::fabs(l.nx * p.x + l.ny * p.y - l.d);
}

// Total least squares: minimise perpendicular distances. The direction is the
// eigenvector of the  $2 \times 2$  covariance with the LARGEST eigenvalue; the normal is the
// one with the smallest. Closed form for a symmetric  $2 \times 2$ , no Eigen needed.
Line fitLineTLS(const std::vector<Point>& pts) {
    const double n = static_cast<double>(pts.size());
    double mx = 0.0, my = 0.0;
    for (const auto& p : pts) { mx += p.x; my += p.y; }
    mx /= n; my /= n;
    double sxx = 0.0, sxy = 0.0, syy = 0.0;
    for (const auto& p : pts) {
        const double dx = p.x - mx, dy = p.y - my;
        sxx += dx * dx; sxy += dx * dy; syy += dy * dy;
    }
    // Smallest eigenvalue of  $\begin{bmatrix} sxx & sxy \\ sxy & syy \end{bmatrix}$ 
    const double tr = sxx + syy, det = sxx * syy - sxy * sxy;
    const double lamMin = tr / 2.0 - std::sqrt(std::max(0.0, tr * tr / 4.0 - det));
    // Eigenvector for lamMin:  $(A - \lambda I) v = 0 \rightarrow$  pick the better-conditioned row.
    double nx, ny;
    if (std::fabs(sxy) > 1e-12) { nx = sxy; ny = lamMin - sxx; }
    else if (sxx < syy) { nx = 1.0; ny = 0.0; } // points spread along y: vertical line
}

```

```

else { nx = 0.0; ny = 1.0; }
const double len = std::hypot(nx, ny);
nx /= len; ny /= len;
return Line{nx, ny, nx * mx + ny * my}; // passes through the centroid
}

Line lineThrough(const Point& a, const Point& b) {
    double nx = -(b.y - a.y), ny = b.x - a.x; // normal = perpendicular to the direction
    const double len = std::hypot(nx, ny);
    nx /= len; ny /= len;
    return Line{nx, ny, nx * a.x + ny * a.y};
}

struct RansacResult { Line line; std::vector<int> inliers; };

RansacResult ransacLine(const std::vector<Point>& pts, double threshold, int iterations, std::mt19937& rng) {
    const int n = static_cast<int>(pts.size());
    std::uniform_int_distribution<int> pick(0, n - 1);
    RansacResult best;
    for (int it = 0; it < iterations; ++it) {
        int i = pick(rng), j = pick(rng);
        if (i == j) continue; // degenerate minimal set
        const Line cand = lineThrough(pts[i], pts[j]);
        std::vector<int> inl;
        for (int k = 0; k < n; ++k)
            if (pointLineDistance(cand, pts[k]) < threshold) inl.push_back(k);
        if (inl.size() > best.inliers.size()) { best.line = cand; best.inliers = std::move(inl); }
    }
    // Refit on the consensus set: the 2-point hypothesis is noisy, the TLS fit is not.
    std::vector<Point> inlierPts;
    for (int k : best.inliers) inlierPts.push_back(pts[k]);
    if (inlierPts.size() >= 2) best.line = fitLineTLS(inlierPts);
    return best;
}

// Iterations needed so that with probability p at least one sample is all-inlier.
int ransacIterations(double outlierRatio, int sampleSize, double p = 0.99) {
    const double allInlier = std::pow(1.0 - outlierRatio, sampleSize);
    return static_cast<int>(std::ceil(std::log(1.0 - p) / std::log(1.0 - allInlier)));
}

int main() {
    std::mt19937 rng(2024);

    // ---- closest pair: fast == brute on random sets, plus edge cases ----
    {
        std::uniform_real_distribution<double> u(0.0, 100.0);
        for (int trial = 0; trial < 20; ++trial) {
            const int n = 2 + trial * 25;
            std::vector<Point> pts;
            for (int i = 0; i < n; ++i) pts.push_back({u(rng), u(rng)});
            const double a = closestPairBrute(pts), b = closestPairFast(pts);
            assert(std::fabs(a - b) < 1e-9);
        }
        std::vector<Point> dup = {{1, 1}, {5, 5}, {1, 1}}; // duplicates -> distance 0
        assert(closestPairFast(dup) == 0.0);
        std::vector<Point> vertical = {{3, 0}, {3, 10}, {3, 4}, {3, 7}}; // all same x
        assert(std::fabs(closestPairFast(vertical) - 3.0) < 1e-12);
        assert(std::isinf(closestPairFast({{1, 2}}))); // single point: no pair
    }

    // ---- TLS handles a vertical line, where y = mx + c cannot ----
    {
        std::vector<Point> v = {{2, 0}, {2, 1}, {2, 2}, {2, 3}};
        const Line l = fitLineTLS(v);
        assert(std::fabs(std::fabs(l.nx) - 1.0) < 1e-12 && std::fabs(l.ny) < 1e-12);
        assert(std::fabs(std::fabs(l.d) - 2.0) < 1e-12);
        std::vector<Point> diag = {{0, 0}, {1, 1}, {2, 2}, {3, 3.1}};
        const Line ld = fitLineTLS(diag);
        for (const auto& p : diag) assert(pointLineDistance(ld, p) < 0.1);
    }

    // ---- RANSAC: 70 % inliers on a known line, 30 % uniform outliers ----
    {
        const Line truth = lineThrough({0.0, 1.0}, {10.0, 4.0}); // y = 0.3 x + 1
        std::normal_distribution<double> noise(0.0, 0.05);
        std::uniform_real_distribution<double> ux(0.0, 10.0), uy(-5.0, 10.0);
    }
}

```

```

std::vector<Point> pts;
const int nIn = 140, nOut = 60;
for (int i = 0; i < nIn; ++i) {
    const double x = ux(rng);
    pts.push_back({x, 0.3 * x + 1.0 + noise(rng)});
}
for (int i = 0; i < nOut; ++i) pts.push_back({ux(rng), uy(rng)});
std::shuffle(pts.begin(), pts.end(), rng);

const int iters = ransacIterations(0.3, 2); // = 7 for p = 0.99
assert(iters == 7);
const RansacResult res = ransacLine(pts, 0.2, 50, rng); // use a margin over the minimum

const double cosAngle = std::fabs(res.line.nx * truth.nx + res.line.ny * truth.ny);
assert(cosAngle > 0.9999); // same direction (sign-free)
const double dSigned = res.line.d * (res.line.nx * truth.nx + res.line.ny * truth.ny > 0 ? 1.0 : -1.0);
assert(std::fabs(dSigned - truth.d) < 0.05);
const int nInl = static_cast<int>(res.inliers.size());
assert(nInl >= nIn - 5 && nInl <= nIn + 15); // ~all inliers, few lucky outliers
std::cout << "RANSAC: " << nInl << " inliers of " << pts.size()
    << ", line normal (" << res.line.nx << ", " << res.line.ny << "), d = " << res.line.d << "\n";
}

std::cout << "OK 04_closest_pair_and_ransac.cpp\n";
return 0;
}

```

What is new:

- `struct Point { double x, y; }; and pts.push_back({u(rng), u(rng)}): braces build a Point in place.`
- `std::sort(px.begin(), px.end(), [](const Point& a, const Point& b) { return a.x < b.x; });` sorts with a *lambda*, an unnamed function written inline. Python: `sorted(pts, key=lambda p: p[0])`.
- `std::vector<Point> lx(px.begin(), px.begin() + mid)` copies a slice, Python's `px[:mid]`.
- `closestRec(px, py)` carries two copies of the points, one sorted by `x` and one by `y`, and splits the `y`-sorted copy by side at every level so the strip never needs re-sorting. That is the detail that keeps it $n \log n$.
- `fitLineTLS` finds the normal of the best perpendicular fit from a 2 by 2 table (`sxx`, `sxy`, `syy`) with a closed formula. You do not need to reproduce the formula; know that the normal is the direction in which the points spread the *least*.
- `std::uniform_int_distribution<int> pick(0, n - 1)` draws a random index; if `(i == j)` continue; skips a useless pair.
- `best.inliers = std::move(inl)` hands the vector over without copying.
- The final asserts compare the found normal with the true one using `fabs(dot) > 0.9999` because `(n, d)` and `(-n, -d)` describe the same line.

Try it: change `nOut = 60` to `nOut = 190` (about 58 % outliers) and predict whether 50 iterations are still enough, using the formula. Then run it.

In your own words: "Closest pair: compare all pairs, or sort by `x`, split, solve the halves, and check only the strip around the cut. Least squares picks the line with the smallest total squared miss, and one outlier can drag it far off. RANSAC fixes that: pick two points, draw the line, count the points close to it, repeat, keep the best, then refit on its inliers. The number of tries comes from the outlier fraction."

Part B — The problems

Both problems are LeetCode premium, so the statements are given in full here. Read the statement, close the file, and try each in C++ for 25 minutes before reading on.

b5. Robot Room Cleaner (LeetCode 489, hard)

The problem in plain words. A robot vacuum is in a room. The room is a grid of cells, each either open (1) or blocked (0). You are *not* given the grid, and you are not told where the robot is. All you have is the robot itself, which understands four commands:

- `move()`: try to go one cell forward. Returns `true` if it moved, `false` if there was a wall (then it stays where it was).
- `turnLeft()`, `turnRight()`: turn 90 degrees on the spot.
- `clean()`: clean the cell it is standing on.

The robot starts facing up. Write `cleanRoom(robot)` so that every open cell the robot can reach gets cleaned.

the real room (the robot never sees this picture):

	c0	c1	c2	c3	c4	c5	c6	c7	
r0	1	1	1	1	1	0	1	1	
r1	1	1	1	R	1	0	1	1	R = robot at row 1, column 3, facing up
r2	1	0	1	1	1	1	1	1	
r3	0	0	0	1	0	0	0	0	
r4	1	1	1	1	1	1	1	1	

every 1 is connected to R through other 1s, so all 30 open cells must be cleaned.

Edge cases: a room with a single open cell (just clean it); the robot standing on an island of one cell surrounded by walls; long corridors with dead ends (the robot must physically come back). The grid is at most 100 by 200.

By hand. Imagine you are the robot with a notebook, blindfolded. You cannot see the room, but you can count your own steps. Write "I started at (0, 0), facing up" in the notebook. Clean. Try to go forward: if it works, you are now at (-1, 0) (one row up). Clean that, and from there try all four directions in turn. Whenever you reach a cell you have already written down, do not enter it. When every direction from a cell is done, walk back to where you came from, and carry on there. Eventually every cell in the notebook has all four directions tried.

The idea. This is DFS from chapter 03, with two twists.

Twist 1, the relative-coordinate trick. You do not know the real row and column, so you invent your own: the start is (0, 0) and "up" is row -1. Negative coordinates are fine because it is your notebook, not the real grid. A set of visited (r, c) pairs replaces the grid you do not have.

real room	what the robot writes down (its own frame)
r1 1 1 1 R 1	(0,-3) (0,-2) (0,-1) (0,0) (0,1)
r2 1 0 1 1 1	(1,-3) # (1,-1) (1,0) (1,1)
	start = (0,0), up = row -1, right = column +1

Twist 2, physical backtracking. In an ordinary DFS the recursion "returns" for free. Here the robot's body is at the far cell and must actually drive back: turn around (two right turns), `move()`, turn around again. That leaves the robot in the parent cell facing the way it faced before, which is what the parent's loop relies on.

Directions are stored clockwise, up, right, down, left, so that `turnRight()` is simply `dir = (dir + 1) % 4`. In each cell the robot tries the direction it is facing, then turns right, four times. After four right turns it is facing the original way again.

The Python you would have written.

```
def cleanRoom(robot):
    dr = [-1, 0, 1, 0]          # up, right, down, left (clockwise)
    dc = [0, 1, 0, -1]
    visited = set()
    def go_back():
        robot.turnRight(); robot.turnRight()
```

```

robot.move()
robot.turnRight(); robot.turnRight()
def dfs(r, c, d):
    visited.add((r, c))
    robot.clean()
    for i in range(4):
        nd = (d + i) % 4 # the way the robot faces right now
        nr, nc = r + dr[nd], c + dc[nd]
        if (nr, nc) not in visited and robot.move():
            dfs(nr, nc, nd)
            go_back()
            robot.turnRight()
dfs(0, 0, 0)

```

In C++. The file also contains a `GridRobot` simulator that implements the four commands over a real grid, and a BFS that computes which cells are truly reachable, so the test can check "every reachable cell was cleaned and nothing else".

```

// LeetCode 489. Robot Room Cleaner (hard, premium).
// The robot only knows: move() (true if it moved), turnLeft(), turnRight(), clean().
// It does not know the grid or its own position. Clean every reachable cell.
#include <cassert>
#include <stdint>
#include <iostream>
#include <queue>
#include <unordered_set>
#include <utility>
#include <vector>

// The interface exactly as LeetCode declares it.
class Robot {
public:
    virtual ~Robot() = default;
    virtual bool move() = 0;
    virtual void turnLeft() = 0;
    virtual void turnRight() = 0;
    virtual void clean() = 0;
};

class Solution {
public:
    void cleanRoom(Robot& robot) {
        visited_.clear();
        dfs(robot, 0, 0, 0); // our own frame: start at (0,0) facing "up" (direction 0)
    }

private:
    // Clockwise order matters: turnRight() advances the direction index by one.
    static constexpr int dr[4] = {-1, 0, 1, 0};
    static constexpr int dc[4] = {0, 1, 0, -1};

    static std::int64_t key(int r, int c) {
        return (static_cast<std::int64_t>(r) << 32) ^ static_cast<std::uint32_t>(c);
    }

    void dfs(Robot& robot, int r, int c, int dir) {
        visited_.insert(key(r, c));
        robot.clean();
        for (int i = 0; i < 4; ++i) {
            const int nd = (dir + i) % 4; // the direction the robot is facing right now
            const int nr = r + dr[nd], nc = c + dc[nd];
            if (!visited_.count(key(nr, nc)) && robot.move()) {
                dfs(robot, nr, nc, nd);
                goBack(robot); // physical backtrack: the robot must return
            }
            robot.turnRight(); // after 4 turns we face `dir` again
        }
    }

    // Turn around, step back, turn around again: heading is preserved.
    static void goBack(Robot& robot) {
        robot.turnRight();
        robot.turnRight();
        robot.move();
        robot.turnRight();
        robot.turnRight();
    }
}

```

```

std::unordered_set<std::int64_t> visited;
};

// ----- test-side simulator -----
class GridRobot : public Robot {
public:
    GridRobot(std::vector<std::vector<int>> grid, int row, int col)
        : grid_(std::move(grid)), cleaned_(grid_.size(), std::vector<bool>(grid_[0].size(), false)),
          r_(row), c_(col) {}

    bool move() override {
        const int nr = r_ + dr[dir_], nc = c_ + dc[dir_];
        if (nr < 0 || nc < 0 || nr >= static_cast<int>(grid_.size()) ||
            nc >= static_cast<int>(grid_[0].size()) || grid_[nr][nc] == 0)
            return false;
        r_ = nr; c_ = nc;
        return true;
    }

    void turnLeft() override { dir_ = (dir_ + 3) % 4; }
    void turnRight() override { dir_ = (dir_ + 1) % 4; }
    void clean() override { cleaned_[r_][c_] = true; }

    bool cleaned(int r, int c) const { return cleaned_[r][c]; }

private:
    static constexpr int dr[4] = {-1, 0, 1, 0};
    static constexpr int dc[4] = {0, 1, 0, -1};
    std::vector<std::vector<int>> grid_;
    std::vector<std::vector<bool>> cleaned_;
    int r_, c_, dir_ = 0; // 0 = up, matching the problem statement
};

// Ground truth: which cells are reachable from the start (plain BFS in the test).
std::vector<std::vector<bool>> reachable(const std::vector<std::vector<int>>& grid, int row, int col) {
    const int R = static_cast<int>(grid.size()), C = static_cast<int>(grid[0].size());
    std::vector<std::vector<bool>> seen(R, std::vector<bool>(C, false));
    std::queue<std::pair<int, int>> q;
    seen[row][col] = true;
    q.push({row, col});
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nc < 0 || nr >= R || nc >= C || grid[nr][nc] == 0 || seen[nr][nc]) continue;
            seen[nr][nc] = true;
            q.push({nr, nc});
        }
    }
    return seen;
}

void check(const std::vector<std::vector<int>>& grid, int row, int col) {
    GridRobot robot(grid, row, col);
    Solution().cleanRoom(robot);
    const auto truth = reachable(grid, row, col);
    for (int r = 0; r < static_cast<int>(grid.size()); ++r)
        for (int c = 0; c < static_cast<int>(grid[0].size()); ++c)
            assert(robot.cleaned(r, c) == truth[r][c]); // every reachable cell, nothing else
}

int main() {
    // LeetCode example 1
    check({{1, 1, 1, 1, 1, 0, 1, 1},
           {1, 1, 1, 1, 1, 0, 1, 1},
           {1, 0, 1, 1, 1, 1, 1, 1},
           {0, 0, 0, 1, 0, 0, 0, 0},
           {1, 1, 1, 1, 1, 1, 1, 1}}, 1, 3);
    // LeetCode example 2: single open cell
    check({{1}}, 0, 0);
    // Corridor with dead ends: forces deep backtracking
    check({{1, 1, 1, 1, 1},
           {1, 0, 0, 0, 1},
           {1, 0, 1, 0, 1},
           {1, 0, 1, 0, 1},
           {1, 1, 1, 1, 1}}, 2, 2); // start on the inner pillar; it joins the ring via row 4, so everything is cleaned
    check({{1, 1, 1, 1, 1},
           {1, 0, 0, 0, 1},
           {1, 0, 1, 0, 1},
           {1, 0, 1, 0, 1},
           {1, 1, 1, 1, 1}}, 0, 0); // start on the ring: ring cleaned, island not

    std::cout << "OK 05_robot_room_cleaner.cpp\n";
}

```

```
    return 0;
}
```

What is new:

- `class Robot { virtual bool move() = 0; ... }` is an *interface*: a class with methods that have no body (`= 0`), so any class that inherits from it must provide them. `GridRobot : public Robot` does. Python would use `abc.ABC` and `@abstractmethod`, or simply duck typing.
- `virtual ~Robot() = default;` is a virtual destructor, needed on any class you inherit from.
- `bool move()` override in `GridRobot` says "this replaces the base class version"; the compiler checks the signature matches.
- `static constexpr int dr[4] = {-1, 0, 1, 0};` inside the class: one shared array, fixed at compile time, the same `dr/dc` arrays as chapter 03.
- `std::unordered_set<std::int64_t> visited_` is Python's `set()`. C++ cannot hash a `std::pair` without extra code, so `key(r, c)` packs the two ints into one 64-bit integer: `r` shifted into the top 32 bits, `c` in the bottom 32. The cast through `std::uint32_t` makes negative `c` values pack cleanly.
- `if (!visited_.count(key(nr, nc)) && robot.move()):` `count` is "is it in the set". Because `&&` stops at the first false, `move()` is only attempted for unvisited cells; the robot does not waste a move.
- `Solution().cleanRoom(robot);` builds a temporary `Solution` and calls the method on it.

Walk through a small example. A 3 by 3 room with a wall in the middle, robot at the top left facing up:

```
R 1 1
1 0 1
1 1 1
```

`dfs(0,0,up)`: clean (0,0). Facing up: (-1,0) unvisited, `move()` fails (edge of the room). Turn right, now facing right: (0,1) unvisited, `move()` succeeds. `dfs(0,1,right)`: clean. Facing right: (0,2) unvisited, `move` succeeds. `dfs(0,2,right)`: clean; right is out (move fails); turn, facing down: (1,2) succeeds; `dfs(1,2,down)` ... and so on around the ring: (2,2), (2,1), (2,0), (1,0). From (1,0) facing up: (0,0) is visited, skip without moving. Every direction there is done, so `go_back()` drives to (2,0), and each `dfs` returns in turn, each followed by a `go_back()`, until the robot is standing at (0,0) again facing up. All 8 open cells cleaned, the middle 0 never entered.

Complexity. Each open cell is entered once and gets a constant number of robot commands (4 turns, up to 4 move attempts, and one 5-command drive back per successful move), so time is proportional to the number of open cells. The visited set and the recursion depth are also proportional to the number of cells.

Robotics. This is coverage without a map, exactly what a cheap robot vacuum does: keep a notebook of where you have been in your own coordinates, and explore. The `Robot` interface is a hardware abstraction: the test plugs in a simulator, the real product plugs in the motor driver. The weak spot is that "visited by coordinates" assumes the odometry never drifts; a real robot adds localisation, and the DFS becomes a planner to the nearest uncovered cell.

Follow-ups you may get. - *What must every `dfs` call promise about the robot's body?* That it returns with the robot in the same cell facing the same way it started. Forget the second turn-around in `go_back()` and the whole notebook drifts. - *Why not `std::unordered_set<std::pair<int,int>>`?* It does not compile; there is no standard hash for pairs. Pack into one integer or write a hasher. - *Recursion depth?* The longest DFS path, fine at 100 by 200. For a warehouse costmap switch to an explicit stack and a recorded path to drive back along.

b6. The Maze II (LeetCode 505, medium)

The problem in plain words. A maze is a grid of 0 (empty) and 1 (wall). A ball sits on an empty cell. You can push it up, down, left or right, and it *rolls until it hits a wall or the edge of the maze*; it cannot stop half way along a corridor. Given the start and a destination, return the smallest *distance* the ball can travel to come to rest exactly on the destination, where distance is the number of empty cells passed through (the start does not count, the destination does). Return -1 if the ball can never stop there.

rolling: push the ball right from column 0

```
0 0 0 1 0
0 -> -> |      the ball passes column 1 and column 2, the wall at column 3 stops it
stops at column 2, distance 2
```

The example maze, start (0, 4) (top right), destination (4, 4) (bottom right):

	c0	c1	c2	c3	c4
r0	0	0	1	0	S
r1	0	0	0	0	0
r2	0	0	0	1	0
r3	1	1	0	1	1
r4	0	0	0	0	D

one best route, seven pushes, distance 12:

left	from (0,4): rolls 1 cell to (0,3)	(wall at (0,2))	1
down	from (0,3): rolls 1 cell to (1,3)	(wall at (2,3))	1
left	from (1,3): rolls 3 cells to (1,0)	(edge)	3
down	from (1,0): rolls 1 cell to (2,0)	(wall at (3,0))	1
right	from (2,0): rolls 2 cells to (2,2)	(wall at (2,3))	2
down	from (2,2): rolls 2 cells to (4,2)	(edge)	2
right	from (4,2): rolls 2 cells to (4,4)	= D	2
		total	12

Same maze with destination (3, 2): the ball can roll *through* that cell but there is no way to stop on it, so the answer is -1. Edge cases: start equals destination (answer 0); a 1 by 1 maze; a straight corridor (one push, answer is its length). The grid is at most 100 by 100. *The Maze* (LeetCode 490) is the same setup asking only "can it stop there at all?"

By hand. Look at the example route. Each push is a whole roll of some length, and different rolls have different lengths. Fewest pushes is *not* the same as shortest distance: a route of 2 long rolls can be longer than a route of 4 short ones. So you cannot use plain BFS (which counts pushes). You want the cheapest total, where each roll costs its length: that is Dijkstra from chapter 04, on a graph whose nodes are the *stopping cells* and whose edges are rolls.

The idea. 1. `roll(r, c, direction)`: step in that direction until the next cell is a wall or outside; return where you stopped and how many cells you passed. 2. Dijkstra: a min-heap of (distance so far, r, c). Pop the cheapest. If it is the destination, that distance is final; return it. Otherwise try the four rolls; if a roll reaches a cell with a smaller total than recorded, record it and push it. 3. Skip a popped entry whose distance is bigger than the recorded one (a stale copy). 4. For LeetCode 490 replace the heap with a queue and the distance table with a visited table.

The Python you would have written.

```
import heapq

def roll(maze, r, c, dr, dc):
    steps = 0
    while True:
        nr, nc = r + dr, c + dc
        if not (0 <= nr < len(maze) and 0 <= nc < len(maze[0])) or maze[nr][nc] == 1:
            return r, c, steps
        r, c, steps = nr, nc, steps + 1
```

```
def shortestDistance(maze, start, dest):
    INF = float("inf")
    dist = [[INF] * len(maze[0]) for _ in maze]
    dist[start[0]][start[1]] = 0
    pq = [(0, start[0], start[1])]
    while pq:
        d, r, c = heapq.heappop(pq)
        if d > dist[r][c]:
            continue # stale entry
        if [r, c] == dest:
            return d
        for dr, dc in ((-1,0),(1,0),(0,-1),(0,1)):
            nr, nc, length = roll(maze, r, c, dr, dc)
            if length == 0:
                continue # wall right next to us
            if d + length < dist[nr][nc]:
                dist[nr][nc] = d + length
                heapq.heappush(pq, (d + length, nr, nc))
    return -1
```

In C++.

```
// LeetCode 505. The Maze II (medium, premium). Ball rolls until it hits a wall;
// return the minimum number of empty cells travelled to stop at the destination, or -1.
// Also LeetCode 490. The Maze: same rolling rule, just "can it stop there?" -> BFS.
#include <cassert>
#include <iostream>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

constexpr int kDr[4] = {-1, 1, 0, 0};
constexpr int kDc[4] = {0, 0, -1, 1};

// Roll from (r,c) in direction k until the next cell is a wall or outside.
// Returns the stopping cell and how many cells were traversed.
std::tuple<int, int, int> roll(const std::vector<std::vector<int>>& maze, int r, int c, int k) {
    const int R = static_cast<int>(maze.size()), C = static_cast<int>(maze[0].size());
    int steps = 0;
    while (true) {
        const int nr = r + kDr[k], nc = c + kDc[k];
        if (nr < 0 || nc < 0 || nr >= R || nc >= C || maze[nr][nc] == 1) break;
        r = nr; c = nc; ++steps;
    }
    return {r, c, steps};
}

// 505: Dijkstra. Nodes are cells the ball can stop at; an edge is one roll, weight = its length.
int shortestDistance(std::vector<std::vector<int>>& maze, std::vector<int>& start, std::vector<int>& destination) {
    const int R = static_cast<int>(maze.size()), C = static_cast<int>(maze[0].size());
    const int INF = 1 << 30;
    std::vector<std::vector<int>> dist(R, std::vector<int>(C, INF));
    using Item = std::tuple<int, int, int>; // (dist, r, c)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;
    dist[start[0]][start[1]] = 0;
    pq.push({0, start[0], start[1]});
    while (!pq.empty()) {
        auto [d, r, c] = pq.top();
        pq.pop();
        if (d > dist[r][c]) continue; // stale entry (lazy deletion)
        if (r == destination[0] && c == destination[1]) return d; // first pop of the target is optimal
        for (int k = 0; k < 4; ++k) {
            auto [nr, nc, len] = roll(maze, r, c, k);
            if (len == 0) continue; // wall right next to us
            if (d + len < dist[nr][nc]) {
                dist[nr][nc] = d + len;
                pq.push({d + len, nr, nc});
            }
        }
    }
    return -1;
}

// 490: plain BFS over stopping cells (edge weights irrelevant, only reachability).
```

```

bool hasPath(std::vector<std::vector<int>>& maze, std::vector<int>& start, std::vector<int>& destination) {
    const int R = static_cast<int>(maze.size()), C = static_cast<int>(maze[0].size());
    std::vector<std::vector<bool>> seen(R, std::vector<bool>(C, false));
    std::queue<std::pair<int, int>> q;
    seen[start[0]][start[1]] = true;
    q.push({start[0], start[1]});
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        if (r == destination[0] && c == destination[1]) return true;
        for (int k = 0; k < 4; ++k) {
            auto [nr, nc, len] = roll(maze, r, c, k);
            if (seen[nr][nc]) continue;
            seen[nr][nc] = true;
            q.push({nr, nc});
        }
    }
    return false;
}

int main() {
    std::vector<std::vector<int>> maze1 = {{0, 0, 1, 0, 0},
                                           {0, 0, 0, 0, 0},
                                           {0, 0, 0, 1, 0},
                                           {1, 1, 0, 1, 1},
                                           {0, 0, 0, 0, 0}};

    std::vector<int> s1 = {0, 4}, d1 = {4, 4};
    assert(shortestDistance(maze1, s1, d1) == 12); // LeetCode example 1
    assert(hasPath(maze1, s1, d1));
    std::vector<int> d2 = {3, 2};
    assert(shortestDistance(maze1, s1, d2) == -1); // example 2: cannot stop there
    assert(!hasPath(maze1, s1, d2));

    std::vector<std::vector<int>> maze3 = {{0, 0, 0, 0, 0},
                                           {1, 1, 0, 0, 1},
                                           {0, 0, 0, 0, 0},
                                           {0, 1, 0, 0, 1},
                                           {0, 1, 0, 0, 0}};

    std::vector<int> s3 = {4, 3}, d3 = {0, 1};
    assert(shortestDistance(maze3, s3, d3) == -1); // example 3

    std::vector<std::vector<int>> one = {{0}};
    std::vector<int> z = {0, 0};
    assert(shortestDistance(one, z, z) == 0); // start == destination
    assert(hasPath(one, z, z));

    std::vector<std::vector<int>> corridor = {{0, 0, 0, 0}};
    std::vector<int> cs = {0, 0}, cd = {0, 3};
    assert(shortestDistance(corridor, cs, cd) == 3); // one roll to the end wall

    std::cout << "OK 06_the_maze_ii.cpp\n";
    return 0;
}

```

What is new:

- `std::tuple<int, int, int> roll(...)` returns three values at once; return `{r, c, steps}`; builds the tuple, and `auto [nr, nc, len] = roll(...)` unpacks it, like Python's `nr, nc, len = roll(...)`.
- `std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;` is a min-heap of tuples. The default `priority_queue` is a `max-heap`, so `std::greater` is what turns it round; forget it and you silently get the longest path. Tuples compare element by element, so distance is the sort key. Python's `heapq` is a min-heap already.
- `pq.top()` then `pq.pop()`: look, then remove, as with `queue`.
- `const int INF = 1 << 30;` is "a very big int" (2 to the power 30); adding a roll length to it cannot overflow an `int`.
- `constexpr int kDr[4]` at file scope: the direction arrays as compile-time constants.

Walk through the example. Push `(0, (0,4))`. Pop it. Rolls: up, 0 cells (skip); down, 4 cells to `(4,4)`? No: `(1,4)`, `(2,4)` are empty but `(3,4)` is a wall, so it stops at `(2,4)` after 2 cells, record `dist(2,4) = 2`; left, 1 cell to `(0,3)`, record 1; right, edge, skip. Heap: `(1, (0,3))`, `(2, (2,4))`. Pop `(0,3)`: down rolls 1 to `(1,3)` giving 2; left is a wall. Pop `(2, (1,3))`: left 3 to `(1,0)` gives 5; down is a wall; right 1 to `(1,4)` gives 3. Pop `(2,`

(2,4)), pop (3,(1,4)), ... the heap keeps popping the cheapest stop cell, and each stop cell gets its best total. Following the route drawn above: (1,0) at 5, (2,0) at 6, (2,2) at 8, (4,2) at 10, (4,4) at 12. When (12, (4,4)) is popped no cheaper entry for it exists, so 12 is returned. For destination (3,2) no roll ever stops there (the ball passes through it from (2,2) to (4,2)), dist(3,2) stays infinite, the heap empties, return -1.

Complexity. There are at most rows times columns stop cells, each with 4 rolls, and a roll can be as long as the longer side of the maze. With the heap's log factor: $O(\text{rows} * \text{cols} * \max(\text{rows}, \text{cols}) * \log(\text{rows} * \text{cols}))$ time, $O(\text{rows} * \text{cols})$ memory.

Robotics. A planner that commits to whole motion primitives (drive straight until something stops you, or a fixed arc for a car that cannot turn on the spot) has exactly this shape: the graph's nodes are *where a primitive ends*, not every grid cell, and primitives have different costs. Lattice planners such as Hybrid A are Dijkstra or A over such a graph. The lesson "BFS counts moves, Dijkstra counts distance" matters for a battery-limited robot: it wants the short route, not the one with the fewest decisions.

Follow-ups you may get. - *Why not mark a cell as done when it is pushed, as in BFS?* Because a cheaper route to it may arrive later. Mark on pop, or use the stale-entry check. - *Could you use A*?* Yes: the Manhattan distance to the destination never over-estimates, because every roll moves at least that far along one axis. - *The Maze III (499)?* Same Dijkstra, but the ball can drop into a hole mid-roll and ties are broken by the shortest instruction string: carry the string in the heap item and compare (distance, string).

Part C — Portfolio for the CV

The four files in Part A are proof you can write this code. Part C is about making that proof visible to someone who spends five minutes on your GitHub and thirty seconds on your CV. Each piece below is explained with *why a reviewer cares* in one sentence.

b7. Rewrite one Python robotics project in C++ (public repo)

Choosing the project. Pick something you already understand end to end in Python, that has a speed reason to be in C++ (a particle filter, an occupancy-grid mapper, an EKF localiser, a path tracker with a small simulator), and that can be shown working in 60 seconds without any hardware: a simulation with a plot or a terminal picture. Avoid anything that needs ROS to run, because a reviewer will not install it. Two to four source files and about a thousand lines is plenty.

Repo layout. *Why it matters:* a reviewer who recognises the shape in two seconds spends the remaining minutes reading your code instead of hunting for it.

```
ekf-localiser/
├── CMakeLists.txt
├── README.md
├── .clang-format
├── .github/workflows/ci.yml
├── include/ekf/
│   ├── ekf.hpp           # public headers only
│   └── motion_model.hpp
├── src/
│   ├── ekf.cpp
│   ├── motion_model.cpp
│   └── main.cpp          # the demo executable
├── tests/
│   ├── CMakeLists.txt
│   └── test_ekf.cpp      # GoogleTest or Catch2
└── docs/
    └── architecture.md   # the diagram from b9
```

Top-level CMakeLists.txt. *Why it matters:* "clone, one build command, it works" is the first thing a reviewer tries, and a failed build ends the review.

```
cmake_minimum_required(VERSION 3.16)
project(ekf_localiser LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
find_package(Eigen3 3.3 REQUIRED)

add_library(ekf src/ekf.cpp src/motion_model.cpp)
target_include_directories(ekf PUBLIC include)
target_link_libraries(ekf PUBLIC Eigen3::Eigen)
target_compile_options(ekf PRIVATE -Wall -Wextra -Wpedantic)

add_executable(ekf_demo src/main.cpp)
target_link_libraries(ekf_demo PRIVATE ekf)

enable_testing()
add_subdirectory(tests)
```

Reading it: `add_library` builds your code as a library, `add_executable` builds the demo that uses it, `target_compile_options` turns warnings on, `enable_testing` lets `ctest` find the tests.

tests/CMakeLists.txt with GoogleTest (installed as `libgtest-dev`). *Why it matters:* tests that run by themselves are the difference between "I think it works" and "it works".

```
find_package(GTest REQUIRED)
add_executable(test_ekf test_ekf.cpp)
target_link_libraries(test_ekf PRIVATE ekf GTest::gtest_main)
include(GoogleTest)
gtest_discover_tests(test_ekf)
```

.clang-format. *Why it matters:* consistent formatting reads as care, and a reviewer notices messy indentation before they notice a clever algorithm.

```
BasedOnStyle: Google
IndentWidth: 4
ColumnLimit: 100
```

.github/workflows/ci.yml. *Why it matters:* the green badge at the top of the README is the first thing a reviewer sees and it says "this builds and passes its tests on a clean machine, not just on mine".

```
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt-get update && sudo apt-get install -y cmake libeigen3-dev libgtest-dev
      - run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
      - run: cmake --build build --parallel
      - run: ctest --test-dir build --output-on-failure
```

Reading it: on every push, GitHub starts a fresh Ubuntu machine, installs the three packages, builds, and runs the tests. If any step fails the badge turns red.

What a reviewer looks for in the code, in the order they notice it. Each row is one habit; the right column is what makes them close the tab.

Habit	Looks like	Makes them close the tab
Ownership is obvious	values, references, <code>std::unique_ptr</code> ; no <code>new/delete</code> in application code	raw owning pointers, manual <code>delete</code>
<code>const</code> where nothing changes	<code>const</code> on parameters and methods that only read	<code>Eigen::MatrixXd&</code> parameters that are never written
Types carry meaning	<code>struct Pose2D { double x, y, theta; }, enum class</code>	<code>std::vector<double></code> meaning "a pose"
Headers are clean	<code>#pragma once</code> , include only what is used, no <code>using namespace</code> in headers	<code>bits/stdc++.h</code> , <code>using namespace std</code> ; in a header
Tests exist and run in CI	GoogleTest, <code>cctest</code> green in Actions	a <code>main()</code> that prints numbers you eyeball
Warnings on and zero	<code>-Wall -Wextra -Wpedantic</code> in CMake	warnings ignored
One-command build	<code>cmake -S . -B build && cmake --build build</code> from a clean clone	a Makefile with hard-coded paths
Range-for and algorithms	<code>for (const auto& p : points),</code> <code>std::min_element</code>	index loops with casts everywhere
No showing off	templates only where two types actually exist	a template hierarchy for one motion model

Commit history counts too: small commits with real messages, not one "initial commit" holding everything, because the history shows how you work.

b8. Official project written as 3 CV bullets with numbers

The formula. *Why it matters:* a reviewer reads a bullet in three seconds and the number at the end is what they remember, and what they ask about.

action verb + what you built + the tech that proves you can do it + a measured result

Past tense, no "I", no "responsible for", one line per bullet, the number at the end where the eye lands. Good test: if a reviewer could read the bullet and ask a technical follow-up, it is a good bullet.

Weak: "Worked on the navigation stack for a mobile robot using ROS."

Three bullets for a typical navigation project:

- Implemented a lattice-based global planner in C++17 (A* over 16 motion primitives, Eigen, ROS 2) replacing the grid planner; cut median planning time from 410 ms to 35 ms on 50 by 50 m warehouse maps.
- Built an EKF fusing wheel odometry, IMU and 2D LiDAR scan matching for localisation; reduced pose drift from 1.2 m to 0.15 m per 100 m of travel, measured against motion-capture ground truth over 40 runs.
- Wrote a GoogleTest suite and GitHub Actions CI for the planner and controller packages (180 tests, 91 % line coverage); caught 3 regressions in costmap inflation before they reached the robot.

Three for a perception pipeline:

- Developed a LiDAR obstacle-detection pipeline in C++ (RANSAC ground removal, Euclidean clustering, PCL) running at 20 Hz on a Jetson Xavier, up from 6 Hz for the Python prototype.
- Trained and deployed a YOLOv8 detector (TensorRT, FP16) for pallet detection; 94 % mAP@0.5 on a 2,000-image in-house set, 18 ms inference at 640 px.
- Designed the camera-to-LiDAR calibration procedure (checkerboard target, PnP, Kabsch refinement); reprojection error 0.8 px, adopted across 12 robots.

What to measure if you have no numbers yet. *Why it matters:* a rough number with a stated method ("over 20 runs on the office loop") beats no number, and an invented number is found out by the first follow-up question. Measuring takes an afternoon.

If you built...	Measure
Any planner	Median and worst-case planning time on N maps; path length against the old planner; success rate
A localiser or SLAM	Drift per 100 m against ground truth (mocap, RTK GPS, or a known loop); time to recover after being moved
A controller	Settling time, overshoot, RMS tracking error on a reference path; loop rate achieved
A perception model	Precision, recall or mAP on a held-out set; frames per second on the target hardware; dataset size
A port to C++	Speed-up against the Python version on the same input and machine; memory use
Infrastructure or CI	Number of tests, coverage, build time, bugs caught before hardware
Anything on a real robot	Hours of operation, robots deployed, distance travelled, failures per hour

b9. Each repo has README, architecture diagram, 60-second demo video

README template. *Why it matters:* the first screen must let a reviewer see what it does (the GIF), build it (one line), and run it (one line) without scrolling, because that is all the time you get.

```
# ekf-localiser
![[ci]](https://github.com/<you>/ekf-localiser/actions/workflows/ci.yml/badge.svg)

One sentence: what it does and for what robot or dataset.

![[demo]](docs/demo.gif)

## Quick start
git clone ... && cd ekf-localiser
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build
./build/ekf_demo data/sample_run.csv

## What it does
Three or four bullets: inputs, outputs, the algorithm, the numbers (from b8).

## Architecture
The mermaid diagram (below) and two paragraphs on the data flow.

## Results
A table: drift / runtime / test count. Say how you measured it.

## Project layout
One line per top-level directory.

## Tests
ctest --test-dir build

## Roadmap / limitations
Honest, three bullets. Reviewers trust a repo that knows its limits.

## License
MIT.
```

Architecture diagram. *Why it matters:* a data-flow picture tells a reviewer in ten seconds that you understand the system as a whole, not just the function you wrote. GitHub draws mermaid diagrams straight from markdown. Draw the *data flow* (what goes where, at what rate), not the class hierarchy, and keep it to 6 to 10 boxes.

```

flowchart LR
    ODOM[Wheel odometry<br/>50 Hz] --> PRED[EKF predict]
    IMU[IMU yaw rate<br/>200 Hz] --> PRED
    LIDAR[2D LiDAR<br/>10 Hz] --> SM[Scan matcher<br/>ICP vs local map]
    SM -->|pose + covariance| UPD[EKF update]
    PRED --> UPD
    UPD -->|state x, P| MAP[Map integrator]
    UPD -->|/pose 50 Hz| CTRL[Path tracker]
    MAP --> SM

```

If the project has a control loop, draw that as a second small diagram: setpoint, controller, plant, sensor, and back. A photo of a hand drawing beats no diagram; mermaid beats the photo because you will actually update it.

Recording the demo. *Why it matters:* the reviewer watches it on a phone with the sound off, and if they cannot tell what the project does in 60 seconds they move on. For terminal projects: `asciinema rec demo.cast` then `agg demo.cast demo.gif`; text stays crisp and the file is small. For anything with a window (a matplotlib animation, OpenCV, RViz): record a 1280 by 720 region with OBS Studio, then `ffmpeg -i demo.mkv -vf "fps=12,scale=960:-1" -loop 0 docs/demo.gif`, and keep the GIF under 5 MB so GitHub shows it inline. Put the full-quality MP4 on YouTube as unlisted and link it. No narration needed; a caption overlay is enough.

Seconds	Show
0 to 5	Title card: project name, one line, your name
5 to 15	The one-command build and the tests going green (ctest output)
15 to 45	The thing working: the plot or visualisation, raw against filtered side by side
45 to 55	The numbers on screen: the same ones as in your CV bullets
55 to 60	Repo URL

Part D — Check yourself

Answer each in plain words, without looking, before marking the phase done.

- Your guess is 10 m with variance 4; the sensor says 12 m with variance 1. What are the gain, the new estimate and the new variance? Which way does the answer lean, and why is the new variance smaller than both?
- Write the scalar predict and update lines from memory. Which line makes you less sure, which makes you more sure?
- R is tiny: what happens to K, to the estimate, to P? Now Q is huge?
- Q is tiny and the robot gets bumped. What does the filter do, and what is that bug called?
- Why must you call `predict` on a step where no measurement arrived, and why is skipping `update` on that step fine?
- In the position-and-velocity filter nobody measures velocity. Explain in one sentence why it is still recovered, and name the matrix entry that makes it work.
- What does an EKF do differently from a plain Kalman filter? When would you reach for a UKF?
- Cruise control on a hill with P only: does the car reach 50 km/h? Which term fixes that, and what new problem does that term bring?
- Tell the windup story with the pedal on the floor, then name three ways to prevent it.

10. Why differentiate the measurement rather than the error, and why low-pass the derivative?
11. Rotate $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$ by 90 degrees anticlockwise by hand. Write the 2 by 2 matrix. What is its inverse, and why is that free?
12. The camera sees a cup at $(1, 0)$; the camera is 0.5 m ahead of the base; the base is at $(2, 3)$ turned 90 degrees. Where is the cup in the room? Write the chain as $T_{\dots\dots}$ products and explain why it reads right to left.
13. What is gimbal lock in one picture? Name the three quaternion rules (length, composing, order), and why q and $-q$ are the same rotation.
14. Why does one outlier wreck a least-squares line? Describe RANSAC in four steps and compute the number of tries for 30 % outliers, a 2-point model, $p = 0.99$.
15. In Robot Room Cleaner, what must every `dfs` call promise about the robot's body, and which five commands restore it? In The Maze II, why is it Dijkstra and not BFS when the same maze with "can it stop there?" is BFS?

07. Interview day

The other chapters teach you what to know. This one teaches you how to perform it in a live C++ round at a UK robotics or AI company, and gives you the oral question bank to drill in the last week. Rough time: 2 days of drills, then 20 minutes a day until the interview.

Part A — How the round actually runs

A typical robotics software round is 45–60 minutes in a shared editor (CoderPad, HackerRank, Google Docs, or a screen-shared VS Code). The usual shape:

Minutes	What happens	What they are scoring
0–5	Intro, "tell me about a project"	Can you describe a system clearly
5–35	One coding problem, usually grid/graph/array	Approach first, then clean C++, then tests
35–45	Follow-ups: complexity, edge cases, "what if the grid is 10^6 cells"	Do you understand your own code
45–55	C++ fundamentals: pointers, const, smart pointers, STL, <code>size_t</code>	Do you actually write C++
55–60	Your questions	Are you curious about their stack

Some companies replace the LeetCode-style problem with a small robotics task: "implement a PID", "write a 1D Kalman filter", "transform a point through two frames", "find the closest obstacle in a costmap". Chapter 06 covers these. The scoring is the same.

Part B — The protocol for the coding problem

Follow this every time. It is what separates "solved it" from "hire".

1. **Restate the problem in one sentence** and confirm the input format. Ask the two questions that matter: sizes ("how big can the grid be?") and edge cases ("can the start be blocked?").
2. **Say the brute force and its complexity** in one breath, then say why it is not enough.
3. **State the approach and the complexity before writing any code.** "Multi-source BFS from all rotten cells, $O(\text{rows} \times \text{cols})$ time and space." Wait for a nod.
4. **Write the signature and the tests first.** A `main()` with two `asserts`. It shows you think about correctness and gives you something to run.
5. **Write the code top-down, narrating.** Say what each block does as you type it. Silence is the most common reason a correct solution gets a weak score.
6. **Trace one small example by hand** before saying "done". Pick the example that exercises the boundary you are least sure of.
7. **Then volunteer the follow-ups:** what breaks at scale, what you would change for a real robot (fixed-size buffers, no heap allocation in the control loop, `std::array` over `std::vector`).

Things to say that cost nothing and score well:

- "I'll use `int` here because the constraints say $\leq 10^5$; if this could overflow I'd use `long long`."
- "I'll take this by `const&` to avoid the copy."
- "I'll compile this with `-Wall -Wextra`; the `size()` comparison would warn otherwise, so I'll cast."
- "This recursion is fine for 100×100 but on a 10^6 -cell costmap I'd switch to an explicit stack."

- "The priority queue has no decrease-key, so I'll push duplicates and skip stale entries."

Part C — The oral question bank

Every question below has been asked in a robotics C++ round. Answer each out loud in under 30 seconds. The short answer is here; the long explanation is in the chapter given.

Memory and objects (chapter 02)

What is the difference between a pointer and a reference? A reference is an alias that must be bound at creation and cannot be null or reseated. A pointer is a variable holding an address; it can be null, reassigned and used for arithmetic. Prefer references for parameters, pointers when "no object" is a valid state or for non-owning links.

void f(int a) { ++a; } — what does the caller see? Nothing. `a` is a copy. To modify the caller's variable, take `int&`.

What does `const` after a member function mean? The function promises not to modify the object, so it can be called on a `const` object or through a `const&`. Getters should be `const`.

Why pass `std::string` by `const&`? Passing by value copies the heap buffer. `const&` avoids the copy and documents that the function does not modify it. Pass small trivially-copyable types like `int` by value.

What is RAII? Resource Acquisition Is Initialisation: a resource is acquired in a constructor and released in the destructor, so it is released deterministically when the object goes out of scope, including when an exception unwinds the stack. `std::vector`, `std::unique_ptr`, `std::lock_guard` and `std::ofstream` all work this way.

unique_ptr vs shared_ptr? `unique_ptr` is sole ownership, zero overhead, move-only. `shared_ptr` is shared ownership with an atomic reference count, and objects die when the last owner goes. Default to `unique_ptr`; use `shared_ptr` only when lifetime is genuinely shared; use `weak_ptr` to break cycles.

Why avoid `new/delete`? Every `new` needs exactly one `delete` on every path, including exceptions and early returns. Containers and smart pointers do that for you. Manual `new` is a leak or double-free waiting to happen.

Stack vs heap? Stack memory is per-function, allocated by moving the stack pointer, freed automatically, fast, limited in size (~8 MB). Heap memory is allocated with `new/malloc`, lives until freed, is slower to allocate and can fragment. Never return a reference or pointer to a local variable.

Why does `while (--n >= 0)` with `size_t n` loop forever? `size_t` is unsigned. Decrementing zero wraps to the largest value, so the condition is always true. Use a signed index or loop `for (size_t i = n; i-- > 0;)`.

What is the rule of zero? If your class manages no raw resource, do not write a destructor, copy or move operations; let the compiler generate them from the members. If you must write one of the five, you probably need all five.

What is a virtual destructor for? Deleting a derived object through a base pointer is undefined behaviour unless the base destructor is virtual. Any class meant to be inherited from and deleted polymorphically needs one.

What is undefined behaviour? Give three examples. The standard places no requirement on what happens. Out-of-bounds indexing, signed integer overflow, dereferencing a dangling pointer, reading an uninitialised variable, data races.

STL (chapter 02, 05)

std::map vs std::unordered_map? `map` is a red-black tree: ordered keys, $O(\log n)$, supports `lower_bound`. `unordered_map` is a hash table: $O(1)$ average, $O(n)$ worst, no order, needs a hash for the key type. Default to `unordered_map`; use `map` when you need order or range queries.

What does `m[key]` do if the key is absent? Inserts a default-constructed value and returns a reference to it. Use `find` or `count` when you only want to look.

How do you make a min-heap with `std::priority_queue`? `std::priority_queue<T, std::vector<T>, std::greater<T>>`. The default is a max-heap.

What must a comparator for `std::sort` satisfy? Strict weak ordering: irreflexive, asymmetric, transitive. Returning `true` for equal elements (`<=`) is undefined behaviour and can crash.

What is iterator invalidation? Operations that change a container's storage make existing iterators dangling. `vector::push_back` may reallocate and invalidate everything; `erase` invalidates from the erased element onward. `std::list` and `std::map` keep other iterators valid on `erase`.

vector::reserve vs resize? `reserve` allocates capacity without changing size. `resize` changes size and value-initialises new elements. Use `reserve` before a loop of `push_back` when you know the count.

When would you use `std::array`? Fixed size known at compile time, stored inline (on the stack or inside the object), no heap allocation. Ideal for control loops and embedded code.

What is `std::string_view`? A non-owning pointer-plus-length to characters. Cheap to pass, but it dangles if the underlying string dies.

Algorithms (chapters 03, 04, 05)

Why does BFS find shortest paths in an unweighted graph? It expands nodes in order of distance from the source, so the first time a node is dequeued it is at its minimum distance. That fails with weights because a longer-hop path can be cheaper.

Dijkstra in one breath. Distances initialised to infinity except the source. Min-heap of (distance, node). Pop the smallest; if it is stale, skip; otherwise relax every edge and push improved neighbours. Works only for non-negative weights. $O((V+E) \log V)$.

Why can Dijkstra not handle negative edges? Once a node is popped it is assumed final. A later negative edge could give a shorter path to it. Use Bellman-Ford, or reweight.

Dijkstra vs A*? A* adds an admissible heuristic to the priority: $f = g + h$. With $h = 0$ it is Dijkstra. An admissible heuristic never over-estimates, so the first time the goal is popped the path is optimal; a consistent heuristic also lets you close nodes on first pop.

What heuristic for an 8-connected grid? Octile distance: $dx + dy + (\sqrt{2} - 2) \cdot \min(dx, dy)$. Manhattan is not admissible with diagonals.

BFS vs DFS: when each? BFS for shortest paths and level structure, DFS for connectivity, cycle detection, topological order, backtracking. DFS recursion depth equals the longest path; large grids need an explicit stack.

What is a topological sort and how do you detect a cycle? An order of a DAG where every edge goes forward. Kahn's algorithm repeatedly removes in-degree-zero nodes; if fewer than n nodes come out, there is a cycle.

Union-find: what does it do and how fast? Maintains disjoint sets with `find` and `union`. With path compression and union by rank each operation is amortised $O(\alpha(n))$, effectively constant.

Kth largest element: options? Sort $O(n \log n)$; min-heap of size k $O(n \log k)$; quickselect / `std::nth_element` $O(n)$ average.

How does an LRU cache get $O(1)$ for both operations? Hash map from key to a node in a doubly linked list. `get` moves the node to the front with `list::splice`; `put` evicts the back. Both are pointer operations.

Sliding window maximum in $O(n)$? A deque of indices kept decreasing by value. Pop the back while smaller than the new element, pop the front when it leaves the window; the front is the maximum.

Robotics (chapter 06)

What does the Kalman gain mean? How much to trust the measurement versus the prediction. Gain near 1: trust the sensor. Gain near 0: trust the model. It is computed from the prediction uncertainty P and the measurement noise R .

What happens if you set R very small? Q very large? Small R : the filter follows measurements almost exactly and becomes noisy. Large Q : the filter distrusts its model, P grows, and again it follows measurements. Both remove the smoothing.

Why does the integral term cause overshoot, and what is windup? The integral keeps accumulating while the error is positive, so it is still large when the error crosses zero. Windup is when the actuator saturates and the integral keeps growing anyway; fix with clamping or back-calculation.

Why quaternions instead of Euler angles? No gimbal lock, cheap composition, numerically stable renormalisation, and slerp for interpolation. Euler angles are for humans; quaternions and rotation matrices are for computation.

How do you check a 3×3 matrix is a rotation? $R^T R = I$ within tolerance and $\det R = +1$.

`T_world_base * T_base_cam * p_cam` — what is it doing? Expressing a point measured in the camera frame in the world frame by chaining transforms. Read the subscripts right to left; the inner frames cancel.

What is RANSAC and when do you use it? Repeatedly fit a model to a minimal random sample, count inliers within a threshold, keep the best, refit on its inliers. Use it when outliers would wreck least squares: plane fitting in point clouds, line detection, ICP correspondences.

How would you make this code safe for a real-time control loop? No heap allocation after initialisation (`reserve`, `std::array`, fixed pools), no locks in the loop, no exceptions on the hot path, bounded worst-case time, measure jitter.

Part D — The last week

- **Day 7-5.** Re-solve, from blank files: Number of Islands, Rotting Oranges, Network Delay Time, LRU Cache, K Closest Points, Merge Intervals. Time each at 25 minutes. Compile with `-Wall -Wextra`.
- **Day 4.** Dijkstra and A* from scratch, no references, 15 minutes each. If either takes longer, do it again the next day.
- **Day 3.** Kalman 1D and PID from scratch, 20 minutes each. Then read Part C aloud once.
- **Day 2.** One mock: pick a problem you have not seen, share your screen with a friend or record yourself, narrate for 35 minutes. Watch it back. Note the silences.
- **Day 1.** Read Part C aloud once. Prepare your three project bullets with numbers and the three questions you will ask them. Sleep.

Part E — Questions worth asking them

- What does the planning stack look like: grid, lattice, sampling-based, optimisation-based?
- How much of the codebase is C++17 or newer, and is there a style guide?
- How do you test perception and planning code: simulation, replay of logged bags, hardware-in-the-loop?
- What does a typical week look like for someone in this role?

08. UK companies to apply to

Grouped by sector. "Fit" says which chapters of this tutorial matter most for that company's interviews. Check each careers page directly: openings change monthly, and a few of these hire in waves after funding rounds. Status notes are as of September 2026.

Autonomous driving and mobile autonomy

Company	Where	What they do	Fit
Wayve	London	End-to-end AI driving, robotaxi trials with Uber in London	03, 04, 06 (Kalman, transforms)
Oxa (formerly Oxbotica)	Oxford	Autonomy software for industrial vehicles, SHIFFT JV in Dubai	03, 04, 06
Five AI (part of Bosch)	Cambridge, Bristol	Simulation and safety assurance for AVs	02, 05
Aurrigo	Coventry	Autonomous airport and shuttle vehicles	03, 04
Academy of Robotics / Kar-go	Wales	Autonomous delivery vehicles	03, 04
Conigital	Birmingham	Autonomous fleet software	03, 04

Humanoids, manipulation, general-purpose robots

Company	Where	What they do	Fit
Humanoid	London	Humanoid robots, \$152m Series A in 2026, Bosch factory partnership	02, 06 (transforms, control)
Shadow Robot	London	Dexterous robot hands, teleoperation	02, 06
Automata	London	Lab automation robotics, \$45m Series C in 2026	02, 05
Extend Robotics	Reading	VR teleoperation of robot arms	02, 06
Kinematic Trees	UK	Robot-agnostic control software, pre-seed 2026	06
All3	London	Construction robotics	03, 06

Warehouse, logistics, agriculture

Company	Where	What they do	Fit
Ocado Technology	Hatfield, London	Grid-based warehouse robots, one of the largest robotics software teams in the UK	03 (grids, BFS, A*), 04, 05
Dexory	Oxford	Autonomous warehouse inventory robots	03, 04
BotsAndUs	London	Warehouse data robots	03
Dogtooth Technologies	Cambridge	Fruit-picking robots, £14m raised July 2026	06 (perception, RANSAC)

Company	Where	What they do	Fit
Small Robot Company	Salisbury	Farming robots	03, 06
Wootzano	County Durham	Fruit packing robots	06
Tharsus	Blyth	Robotics engineering for other companies	02, 06

Medical and surgical

Company	Where	What they do	Fit
CMR Surgical	Cambridge	Versius surgical robot, \$131m Series D in 2025	02 (safety-critical C++), 06
Proximie	London	Surgical telepresence software	02, 05
Bioliberty	Edinburgh	Rehabilitation robotics	06

Drones, marine, inspection, space

Company	Where	What they do	Fit
Skyports	London	Drone delivery and vertiports	03, 04
Sees.ai	London	Beyond-line-of-sight drone autonomy	03, 06
Rovco / Vaarst	Bristol	Subsea robotics and 3D vision	06
HydroSurv	Exeter	Uncrewed surface vessels	06
Createc	Cumbria	Nuclear inspection robotics	03, 06
Q-Bot	London	Underfloor insulation robots	03
Open Cosmos	Harwell	Satellite platforms with autonomy	02, 06
Space Forge	Cardiff	In-space manufacturing	02

Big tech and research labs with robotics teams in the UK

Company	Where	What they do	Fit
Google DeepMind	London	Robotics research, Gemini Robotics	04, 05, 06
Dyson	Bristol, Malmesbury, Hullavington	Home robots, robotic vacuum navigation	03, 06
Arm	Cambridge	Embedded compute for robotics	02
Amazon (Robotics, Scout)	Cambridge, London	Warehouse and delivery robotics	03, 04, 05
Toyota Research / TRI partners	Cambridge	Perception and manipulation research	06
Sony AI / Sony Europe	Stuttgart, UK partners	Robotics research	05, 06
Nvidia	Reading, Cambridge	Isaac robotics platform, simulation	02, 06

Company	Where	What they do	Fit
Samsung AI Centre	Cambridge	Perception research	05, 06

Defence and aerospace (UK citizenship or right-to-work checks often apply)

Company	Where	What they do	Fit
BAE Systems	Multiple	Autonomous systems	02, 03
QinetiQ	Farnborough	Autonomy and robotics R&D	02, 03
MBDA	Stevenage	GNC software for autonomous platforms	02, 06
Airbus Defence and Space	Stevenage	Rover autonomy (Mars rover heritage)	03, 06
Leonardo	Edinburgh, Yeovil	Sensors and autonomy	02
Helsing	London	Defence AI	04, 05

Robotics software, tooling, consultancies

Company	Where	What they do	Fit
Som3	London	Control systems for ground and aerial robots, hiring C++ now	02, 06
Cambridge Consultants	Cambridge	Product development consultancy with robotics teams	02, 06
TTP	Melbourn	Technology consultancy	02, 06
Plexal / Digital Catapult	London	Not employers, but run robotics hiring events	

Where to look every week

- roboticsjobs.co.uk is UK-only and robotics-only.
- Glassdoor UK and LinkedIn with the search "robotics software engineer C++ ROS2".
- The careers pages of the companies above; most use Greenhouse, Lever or Workable, which let you set alerts.
- Cambridge is the densest cluster (Ocado is not there, but CMR, Dogtooth, Arm, Amazon, Five AI, Cambridge Consultants and TTP are). London second, then Oxford and Bristol.

How to prioritise

1. Apply first where your existing Python robotics project maps onto their product: mobile autonomy companies if it was navigation, manipulation companies if it was arms, perception companies if it was vision.
2. Ocado, Wayve, Oxa, Dexory, CMR and Humanoid run the most structured C++ coding rounds. Those are the ones this tutorial was built for. Finish chapters 01 to 04 before applying to them.
3. Consultancies and defence primes usually interview on fundamentals (chapter 02) and a system design chat rather than LeetCode-style problems.